璞致 Kintex UltraScalePlus
PZ-KU5P FPGA 开发指南

璞致电子科技（上海）有限公司

文档修订记录：

时间	版本	说明
2024.5.26	V1.0	初始版本

本教程会根据实际的开发经验不断的修正、优化以及增加教程内容，为大家提供更多更好的例程，帮助大家成为行业大牛，尽绵薄之力。

目录

文档修订记录：.....2
第一章：FPGA 开发流程介绍.....6
第二章：Vivado 等软件安装.....8
2.1 Vivado 安装说明.....8
2.2 VS Code 安装说明.....14
第三章：FPGA 例程.....19
3.1 LED 流水灯例程.....19
3.1.1 功能概述.....19
3.1.2 管脚配置.....19
3.1.3 工程搭建.....20
3.1.4 运行结果.....26
3.2 LED PWM 控制例程.....27
3.2.1 功能概述.....27
3.2.2 管脚配置.....27
3.2.3 工程搭建.....28
3.2.4 运行结果.....34
3.3 KEY 例程.....35
3.3.1 功能概述.....35
3.3.2 管脚配置.....35
3.3.3 工程搭建.....36
3.3.4 运行结果.....42
3.4 FPGA 片内 RAM 读写例程.....43
3.4.1 功能概述.....43
3.4.2 管脚配置.....43
3.4.3 工程搭建.....44
3.4.4 运行结果.....53
3.5 FPGA 片内 ROM 读写例程.....54
3.5.1 功能概述.....54
3.5.2 管脚配置.....54
3.5.3 工程搭建.....55
3.5.4 运行结果.....63
3.6 FPGA 片内 PLL 配置例程.....64
3.6.1 功能概述.....64
3.6.2 管脚配置.....64
3.6.3 工程搭建.....65
3.6.4 运行结果.....72
3.7 FPGA 片内 FIFO 读写例程.....73
3.7.1 功能概述.....73
3.7.2 管脚配置.....73
3.7.3 工程搭建.....74
3.7.4 运行结果.....82
3.8 FPGA 侧 DDR4 控制读写例程.....83

3.8.1 功能概述 83
3.8.2 管脚配置 83
3.8.3 工程搭建 86
3.8.4 运行结果 97
3.9 I2C 读写 E2PROM 例程 98
3.9.1 功能概述 98
3.9.2 管脚配置 98
3.9.3 工程搭建 99
3.9.4 运行结果 103
3.10 串口收发例程 104
3.10.1 功能概述 104
3.10.2 管脚配置 104
3.10.3 工程搭建 105
3.10.4 运行结果 109
3.11 SD 卡读写例程 110
3.11.1 功能概述 110
3.11.2 管脚配置 112
3.11.3 工程搭建 113
3.11.4 运行结果 117
3.12 AD_DA 转换例程 118
3.12.1 功能概述 118
3.12.2 管脚配置 119
3.12.3 工程搭建 121
3.12.4 运行结果 126
3.13 LCD 彩条显示例程 127
3.13.1 功能概述 127
3.13.2 管脚配置 127
3.13.3 工程搭建 129
3.13.4 运行结果 133
3.14 LCD 输出图片例程 134
3.14.1 功能概述 134
3.14.2 管脚配置 134
3.14.3 工程搭建 136
3.14.4 运行结果 140
3.15 SD 卡读 BMP 图片 LCD 显示例程 141
3.15.1 功能概述 141
3.15.2 管脚配置 142
3.15.3 工程搭建 146
3.15.4 运行结果 152
3.16 基于 RTL8211 实现千兆 UDP 例程 153
3.16.1 千兆以太网、UDP 协议简介及功能概述 153
3.16.2 管脚配置 155
3.16.3 工程搭建 157
3.16.4 运行结果 161

3.17 QSFP 误码率例程 162
3.17.1 功能概述 162
3.17.2 约束配置 162
3.17.3 工程搭建 164
3.17.4 运行结果 168
3.18 PCIE 测试例程 170
3.18.1 功能概述 170
3.18.2 管脚配置 171
3.18.3 工程搭建 173
3.18.4 运行结果 180
3.19 Aurora 通信例程 184
3.19.1 功能概述 184
3.19.2 管脚配置 185
3.19.3 工程搭建 193
3.19.4 运行结果 197
3.20 高速 ADC—AD9643 采集例程(250MSPS) 198
3.20.1 功能概述 198
3.20.2 管脚配置 199
3.20.3 工程搭建 200
3.20.4 运行结果 205
3.21 UDP 传输高速 ADC 采集数据例程 206
3.21.1 功能概述 206
3.21.2 管脚配置 206
3.21.3 工程搭建 209
3.21.4 运行结果 213
3.22 高速 ADC—AD9643 的 LCD 显示例程 214
3.22.1 功能概述 214
3.22.2 管脚配置 214
3.22.3 工程搭建 217
3.22.4 运行结果 221
3.23 OV5640 图像的 UDP 传输电脑端显示 222
3.23.1 功能概述 222
3.23.2 管脚配置 222
3.23.3 工程搭建 225
3.23.4 运行结果 229

第一章：FPGA 开发流程介绍

FPGA（现场可编程门阵列）开发是一个复杂且系统的过程，以下简单介下绍其开发流程：

1. 需求分析与设计规划

明确设计目标：与项目团队、客户等沟通，清晰了解要实现的功能，如设计一个视频图像处理系统，需明确处理的视频分辨率、帧率及处理算法等。

制定技术方案：依据设计目标，确定实现的技术途径。例如实现数字信号处理功能，可采用流水线技术提升处理速度或用并行处理结构满足实时性要求。

进行架构设计：将整体功能分解为多个子模块，确定各模块功能、接口及相互连接关系。如设计微处理器，可划分成运算器、控制器、寄存器组等模块，并定义各模块数据交互方式。

2. 硬件描述语言编程

选择编程语言：常用硬件描述语言有 VHDL 和 Verilog HDL，根据项目需求、团队熟悉程度选择。如航天领域项目，因 VHDL 语法严谨、可读性好常被选用；而在 ASIC 设计中，Verilog HDL 因灵活性和与 C 语言相似性应用广泛。

编写代码：依据架构设计，使用选定语言对各模块进行编码。以 4 位加法器为例，用 Verilog HDL 可如下编写：

```
module adder_4bit (
    input [3:0] a,
    input [3:0] b,
    input cin,
    output [3:0] sum,
    output cout);
    assign {cout, sum} = a + b + cin;
endmodule
```

代码调试与优化：编写过程中，利用仿真工具对代码功能进行验证，修正逻辑错误。同时，从资源利用、速度等方面优化代码，如减少不必要逻辑门、采用优化算法等。

3. 功能仿真

创建测试平台：为验证设计功能，编写测试平台激励待测试模块。如测试上述 4 位加法器，测试平台如下：

```
module tb_adder_4bit;
    reg [3:0] a;
    reg [3:0] b;
    reg cin;
    wire [3:0] sum;
    wire cout;
    adder_4bit uut (
```

```
    .a(a),
    .b(b),
    .cin(cin),
    .sum(sum),
    .cout(cout) );
  initial begin
      a = 4'b0000; b = 4'b0000; cin = 1'b0; #10;
      a = 4'b0010; b = 4'b0011; cin = 1'b1; #10;
      // 更多测试向量
  endendmodule
```

执行仿真：使用 ModelSim 或者 Vivado 自带的等仿真工具运行测试平台，观察输出结果波形。对比预期与实际结果，若不符则检查代码逻辑并修正，重复此过程直至功能正确。

4. 综合

选择综合工具：如 Xilinx 的 Vivado。不同工具针对不同 FPGA 器件和设计需求有不同优势，需根据实际情况选择。

设置综合约束：对设计的时序、面积等进行约束。如设定系统时钟频率为 50MHz，综合工具依据此优化设计满足时序要求。

执行综合：将 HDL 代码转化为门级网表，综合工具依据约束条件优化逻辑结构，选择合适逻辑门实现功能，并生成报告展示资源利用、时序等信息。

5. 布局布线

导入综合结果：将综合生成的门级网表导入布局布线工具，如 Xilinx 的 Vivado。

设置布局布线约束：指定某些模块在 FPGA 芯片中的位置，或约束引脚位置。如将高速数据接口引脚分配到特定 Bank 满足电气特性要求。

执行布局布线：工具根据约束和芯片资源，将逻辑单元放置在芯片合适位置并完成布线连接。完成后生成报告，含布线资源使用、时序分析结果等。

6. 时序仿真与验证

生成仿真模型：布局布线后，工具生成包含实际布线延迟信息的仿真模型。

进行时序仿真：使用与功能仿真类似测试平台，对模型仿真。此时仿真结果包含实际延迟，更准确反映设计在 FPGA 上的运行情况。检查关键路径时序，若不满足要求，需调整设计或约束，重复综合、布局布线及时序仿真过程。

7. 编程与调试

生成编程文件：布局布线满足要求后，生成可下载到 FPGA 芯片的编程文件，如 Xilinx 的.bit 文件。

下载到 FPGA 芯片：通过 JTAG 接口等，使用下载工具将编程文件烧录到 FPGA 芯片。

硬件调试：借助逻辑分析仪、示波器等工具，对实际硬件系统测试调试。检查芯片工作状态、信号时序等，若有问题，分析是硬件连接还是设计本身问题，针对性修改设计、重新综合、布局布线及下载调试。

第二章：Vivado 等软件安装

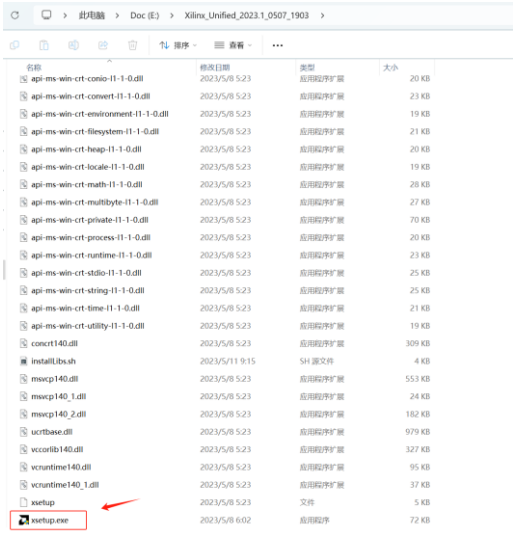
2.1 Vivado 安装说明

软件可在 Xilinx 官方网站下载,链接如下：

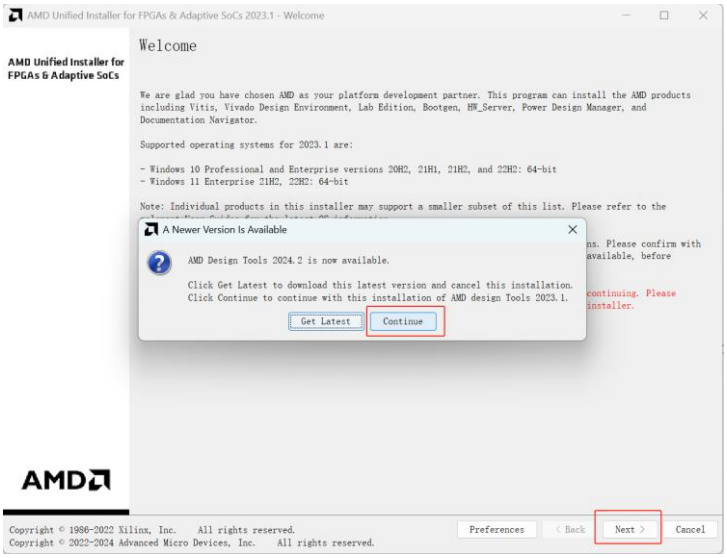
https://china.xilinx.com/support/download.html

我们使用的 Vitis 版本是 2023.1,建议下载 Vitis 2023.1,如果软件下载过慢,资料软件包里面有提供或者联系客服索取。

将压缩包解压出来（ 注意：解压目录的路径名称只能够包含字母、数字、下划线，不要出现中文与特殊字符，否则安装程序有可能出问题），为避免在安装过程中出错，在开始安装之前，请先关闭安全或杀毒软件。双击解压出来的文件夹下的“xsetup.exe”，开始安装 Vivado 软件，如下图所示：



双击“xsetup.exe”后，需要等待一会才会进入 Vivado 的安装引导页面，进入引导页面后如上图所示，最前面会有一个小的弹框提示，是在联网检测 Vivado 最新版本，我们选择“Continue”，如下图所示：



欢迎界面主要是介绍哪些操作系统可以支持 Vivado 2023.1，详细操作系统版本描述由如下：

1）Windows 10 专业版和企业版 20H2，21H1，21H2 和 22H2：64 位；

2）Windows 11 企业版 21H2，22H2：64 位；

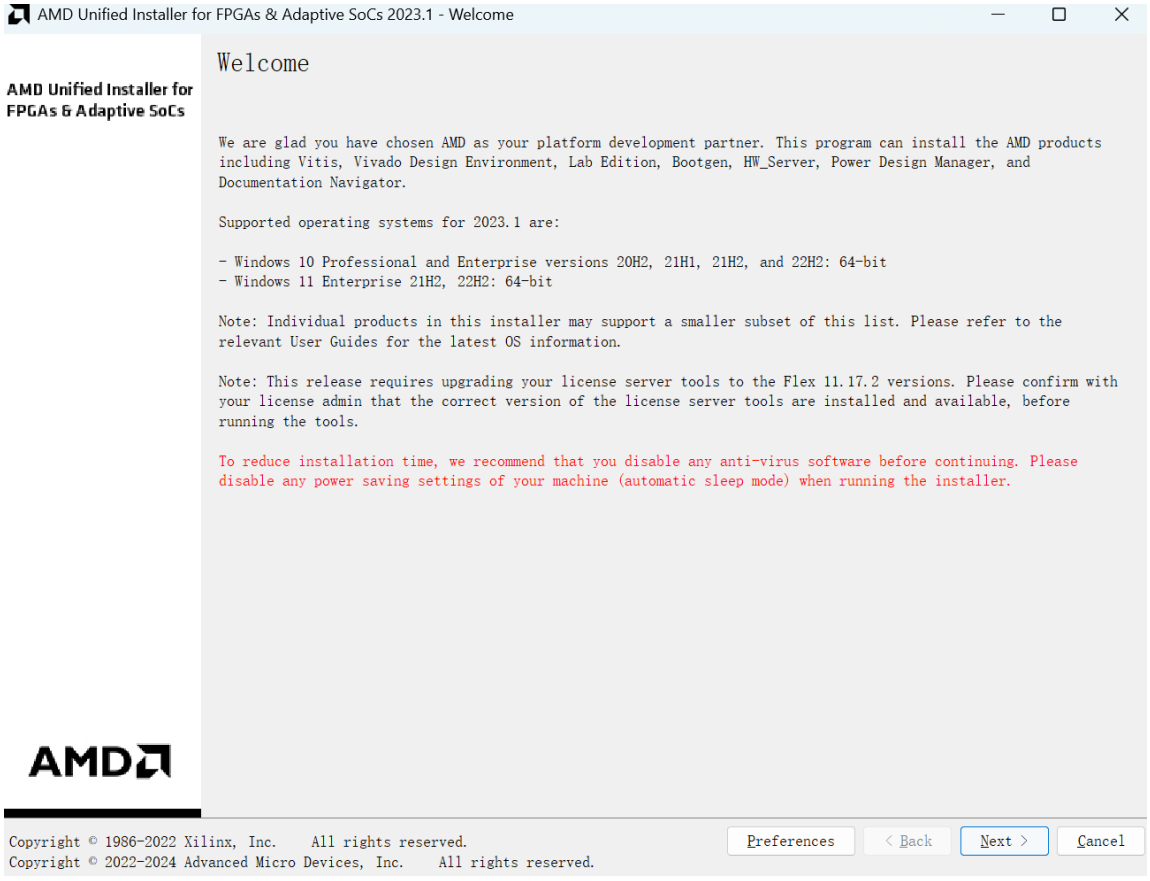
接下来是两个注意项：

1）注意：此安装程序中的个别产品可能支持此列表中的较小子集。请参阅相关用户指南以获取最新的操作系统信息。

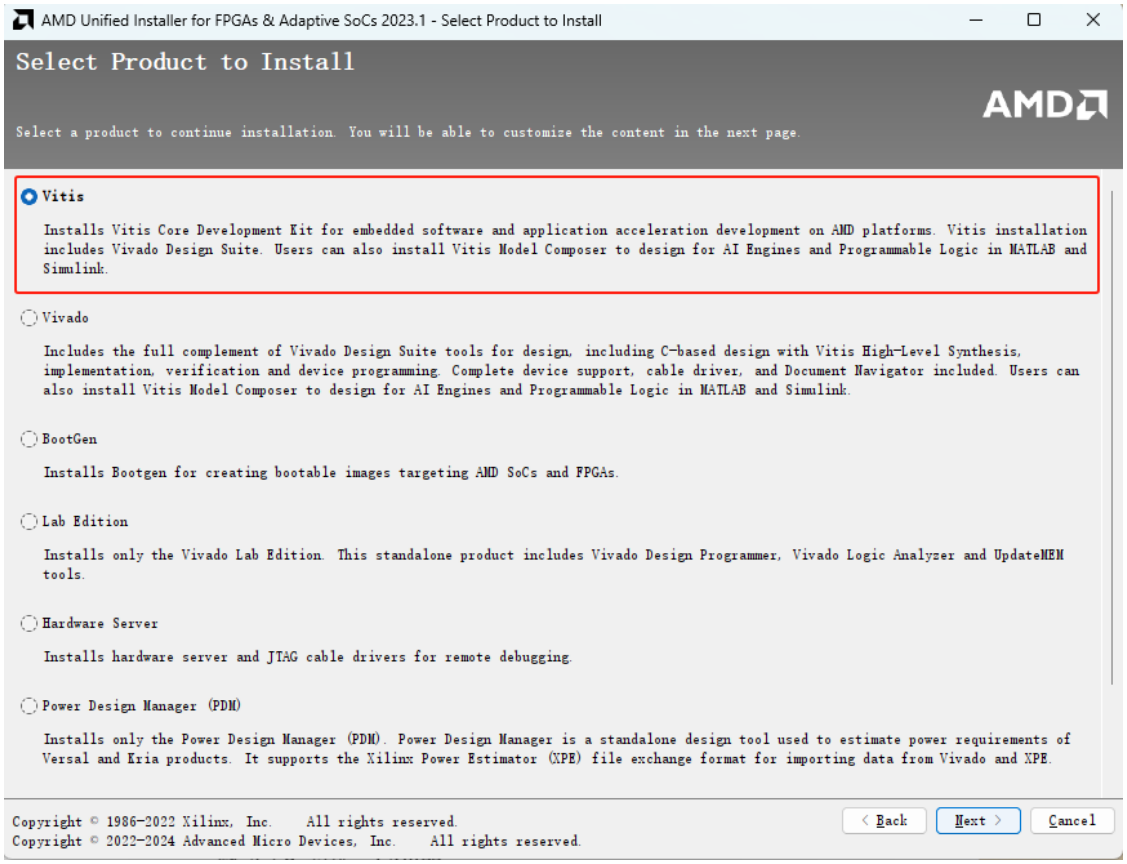
2）注意：此版本要求将您的许可证服务器工具升级到 Flex 11.17.2 版本。在运行这些工具之前，请使用正确版本的许可证服务器工具。

下图中的红色文字是提醒我们，为了节约安装时间，在进行下一个步骤之前建议我们关闭电脑的杀毒软件，还有关闭电脑的任何节能设置（例如自动休眠）。因为软件安装过程比较长，防止杀毒软件中断安装过程或者电脑的长时间不操作进入休眠模式导致 Vivado 软件安装失败。

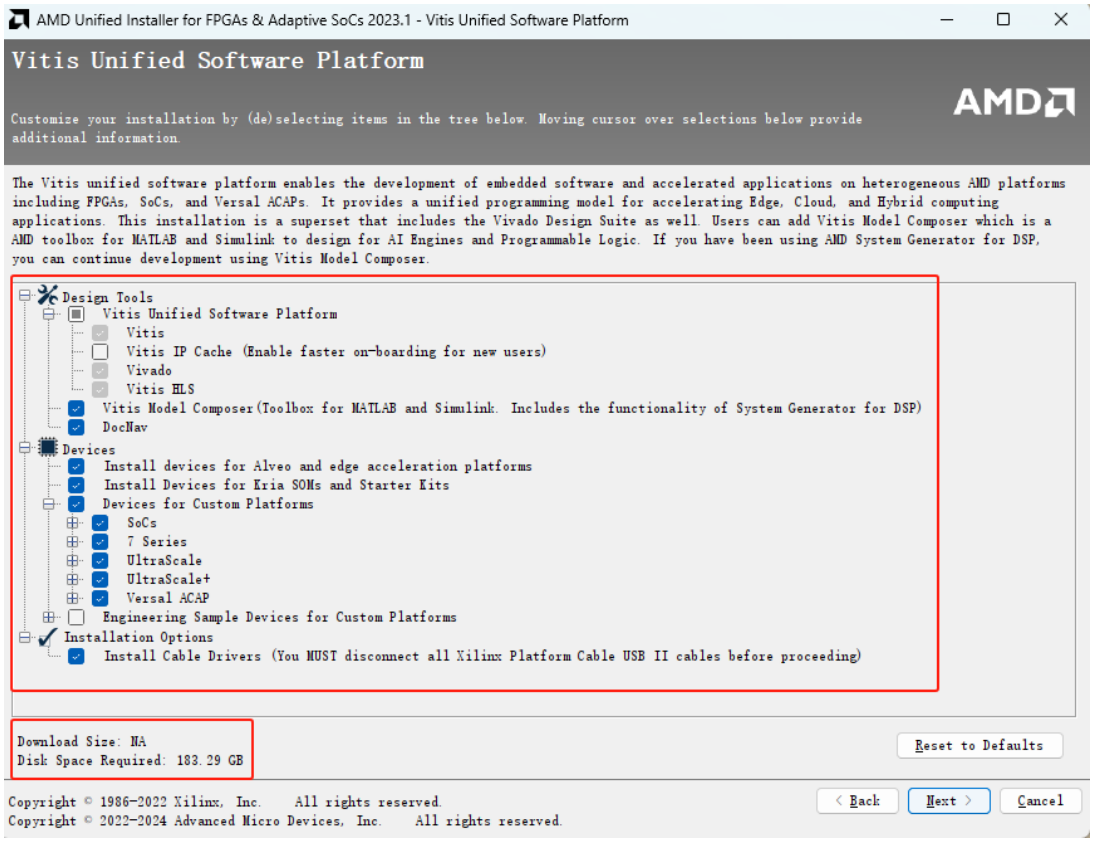
这里没有关闭自身电脑杀毒软件，我们先关闭电脑的杀毒软件并且解除电脑的节能操作后，再点击“Next”继续安装。如下图所示：



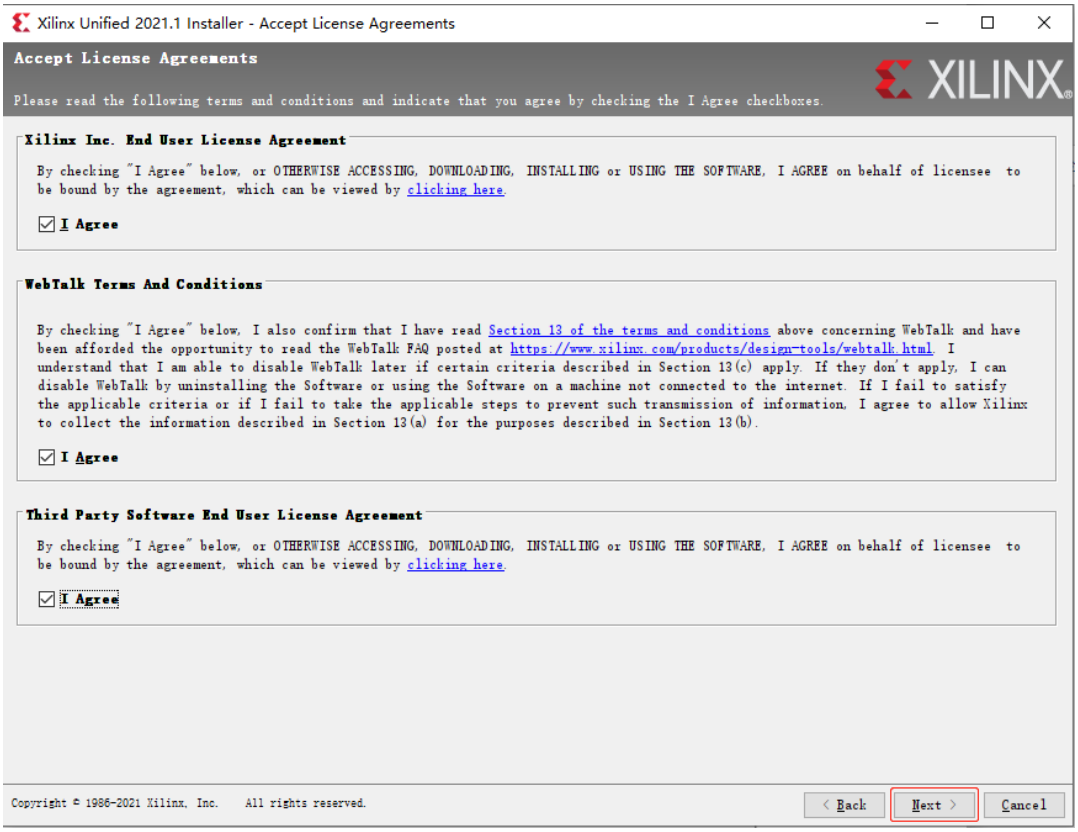
选择 Vitis(选择该选项后，会安装对应的 Vivado 2023.1、Vitis 2023.1 与 Vitis HLS2023.1 等后续开发中我们所使用的全部软件),然后点击“Next”



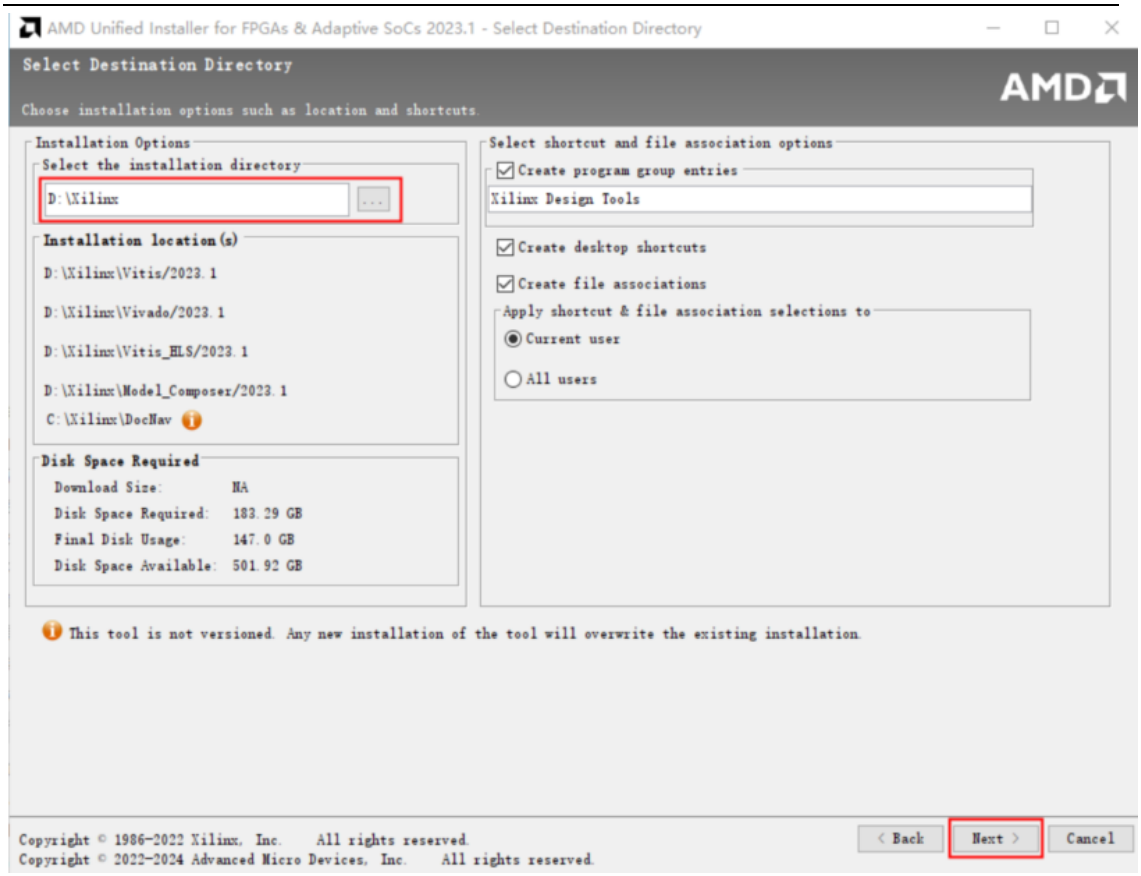
按需求勾选所需要的工具和芯片平台，然后点击“Next”



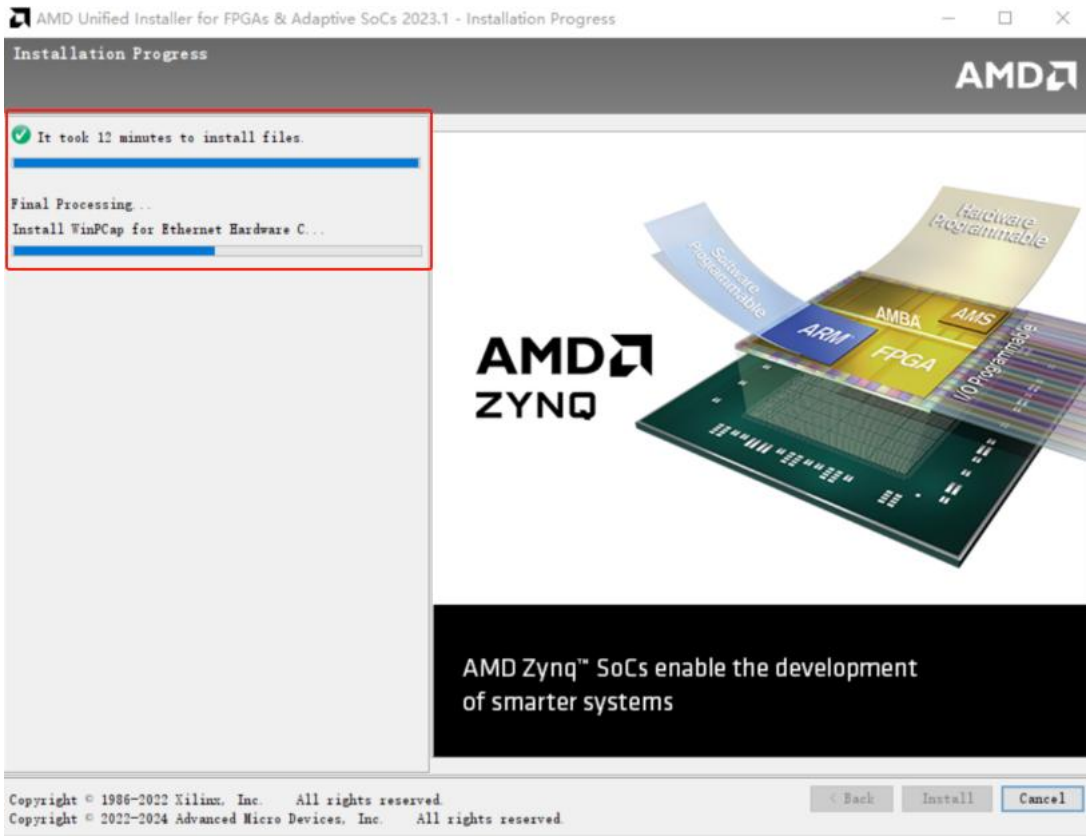
全部勾选，然后点击“Next”



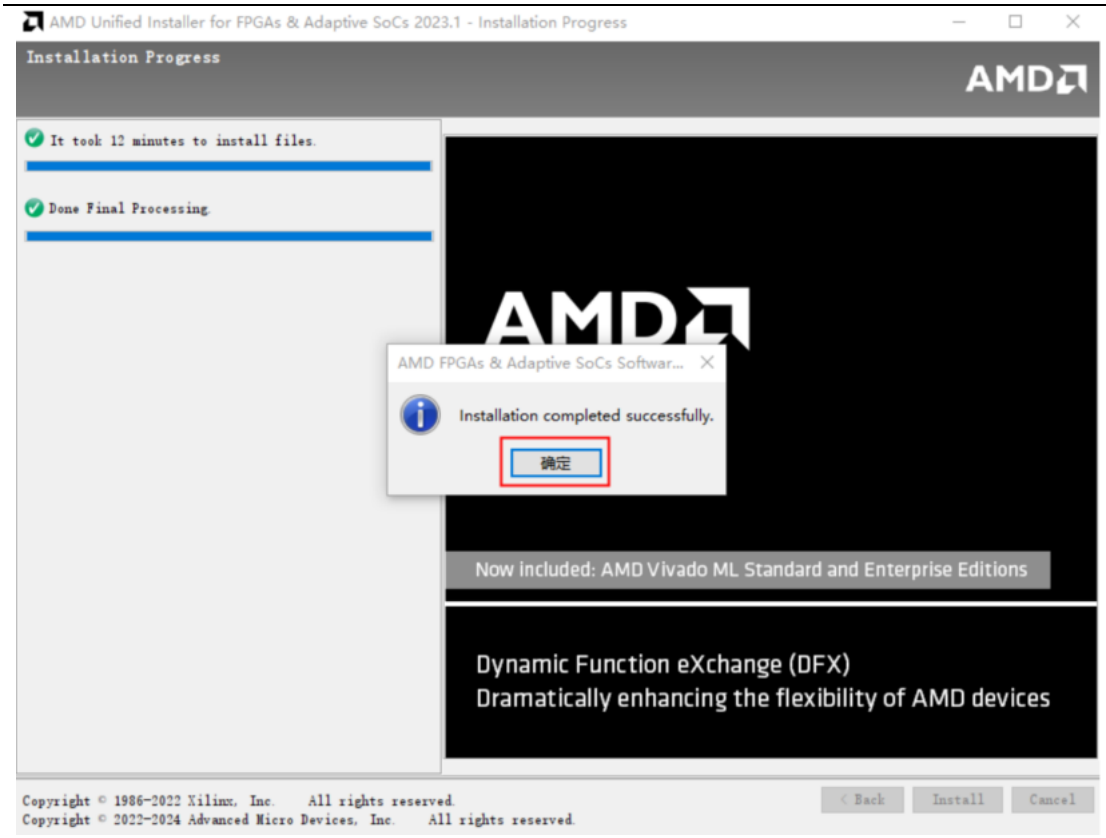
选择安装路径，然后点击“Next”，然后再点击“Intall”


AMD Unified Installer for FPGAs & Adaptive SoCs 2023.1 - Select Destination Directory
Select Destination Directory
Choose installation options such as location and shortcuts.
AMD
Installation Options
Select the installation directory
D:\Xilinx
Installation location(s)
D:\Xilinx\Vitis\2023.1
D:\Xilinx\Vivado\2023.1
D:\Xilinx\Vitis_HLS\2023.1
D:\Xilinx\Model_Composer\2023.1
C:\Xilinx\DocNav
Disk Space Required
Download Size: NA
Disk Space Required: 183.29 GB
Final Disk Usage: 147.0 GB
Disk Space Available: 501.92 GB
Select shortcut and file association options
Create program group entries
Xilinx Design Tools
Create desktop shortcuts
Create file associations
Apply shortcut & file association selections to
Current user
All users
This tool is not versioned. Any new installation of the tool will overwrite the existing installation.
Copyright © 1986-2022 Xilinx, Inc. All rights reserved.
Copyright © 2022-2024 Advanced Micro Devices, Inc. All rights reserved.
< Back
Next >
Cancel

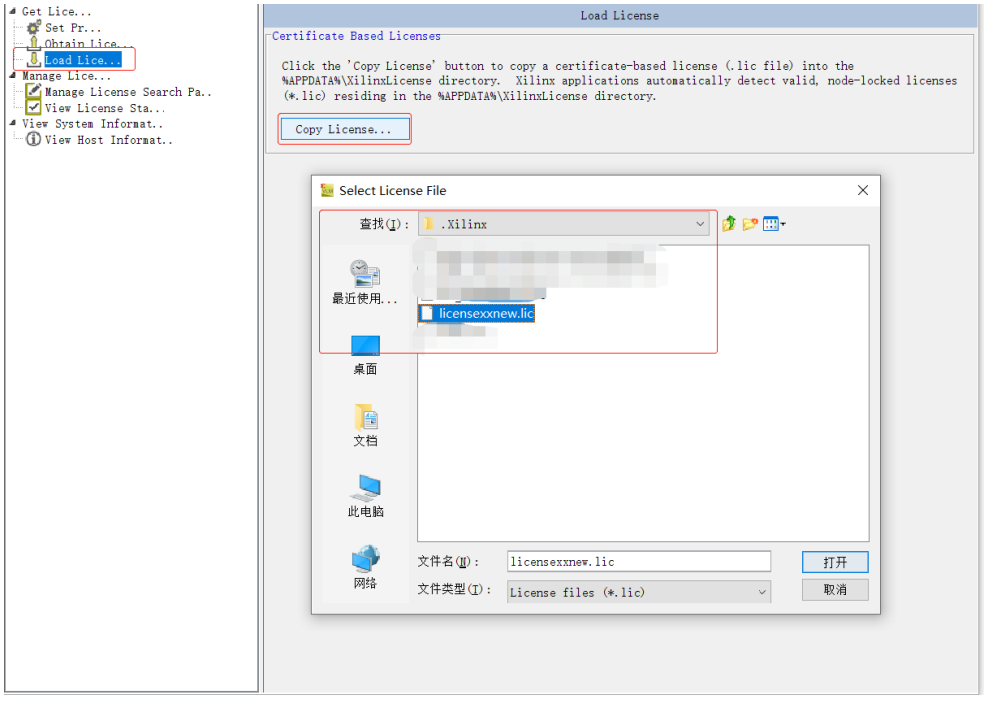
等待安装完成


AMD Unified Installer for FPGAs & Adaptive SoCs 2023.1 - Installation Progress
Installation Progress
AMD
It took 12 minutes to install files.
Final Processing...
Install WinPCap for Ethernet Hardware C...
AMD
ZYNQ
Hardware Programmable
Software Programmable
ARM
AMBA
AMS
FPGA
I/O Programmable
AMD Zynq™ SoCs enable the development of smarter systems
Copyright © 1986-2022 Xilinx, Inc. All rights reserved.
Copyright © 2022-2024 Advanced Micro Devices, Inc. All rights reserved.
< Back
Install
Cancel

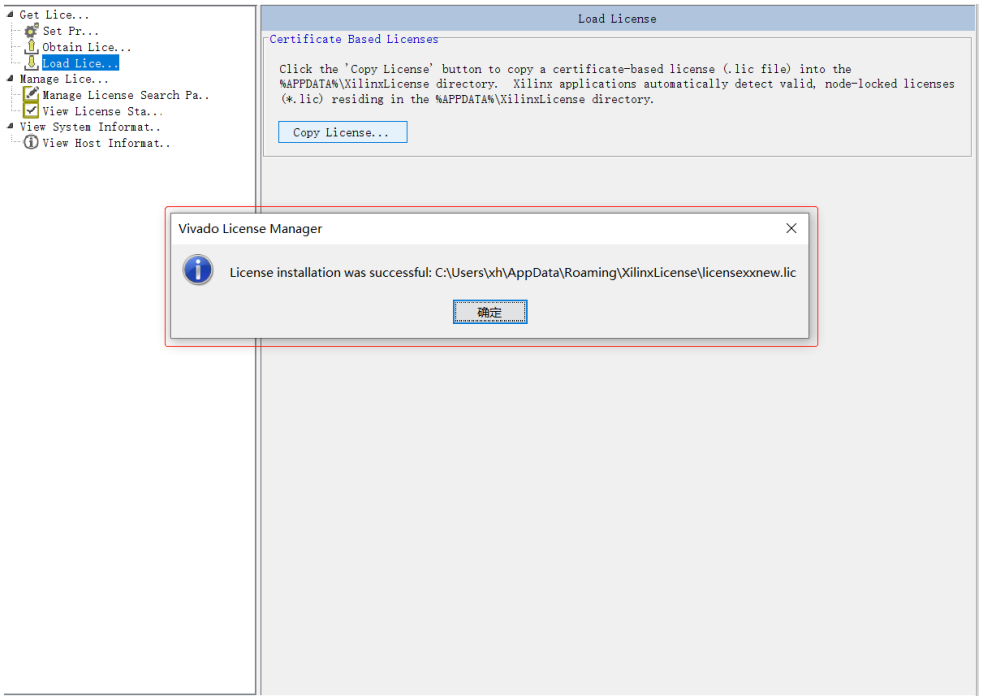
安装完成



添加 License（涉及版权问题，建议网上查找下或者联系客服索取）

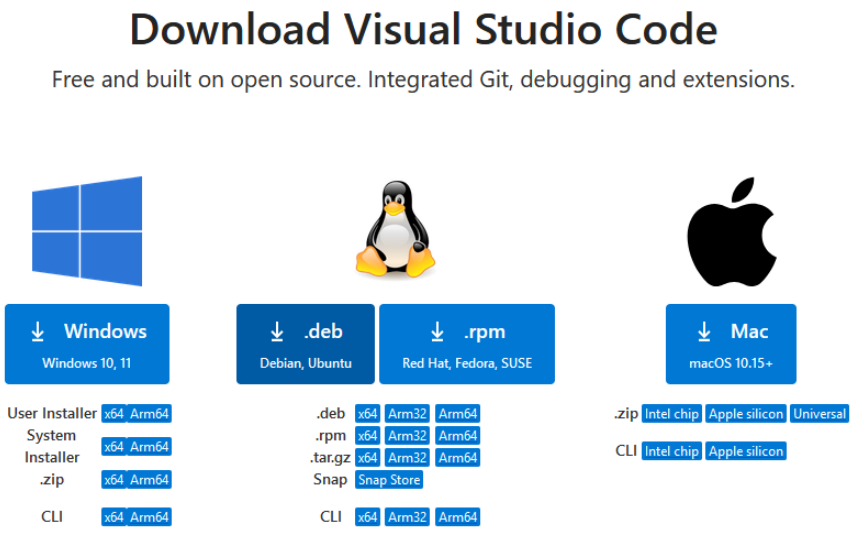


添加成功，然后退出界面，安装到此完成



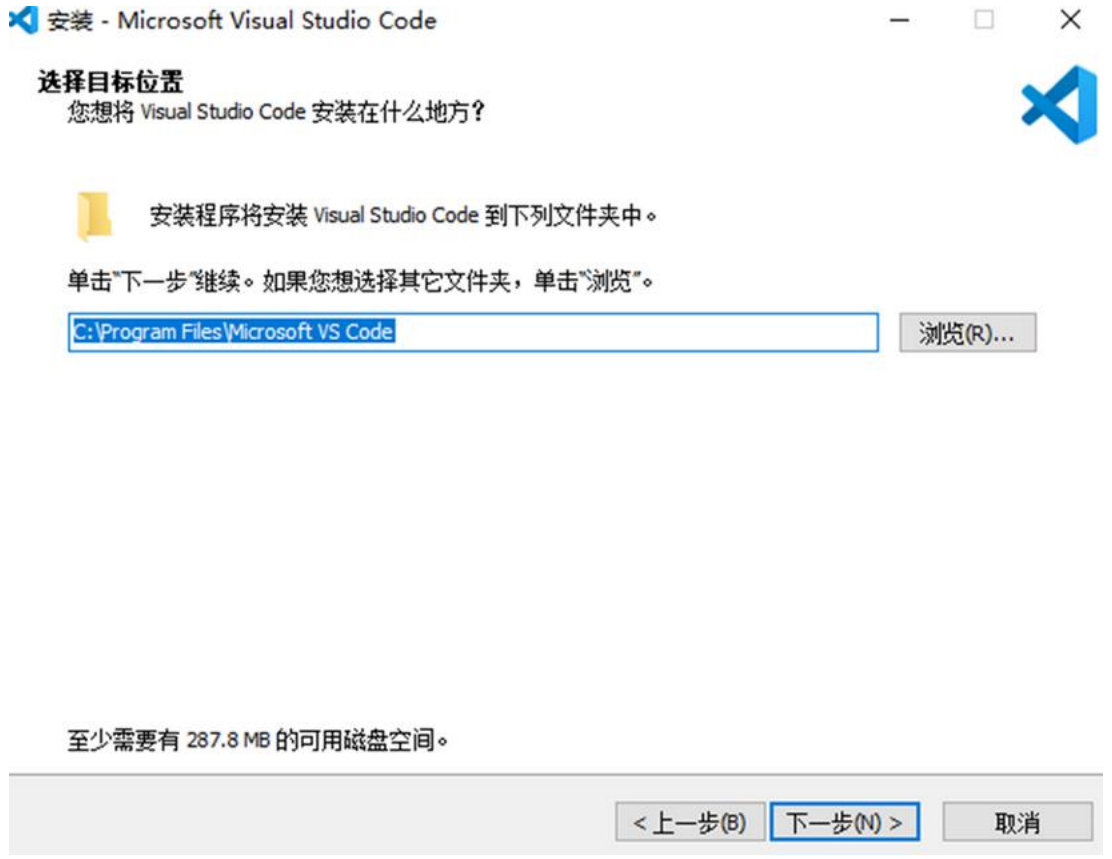
2.2 VS Code 安装说明

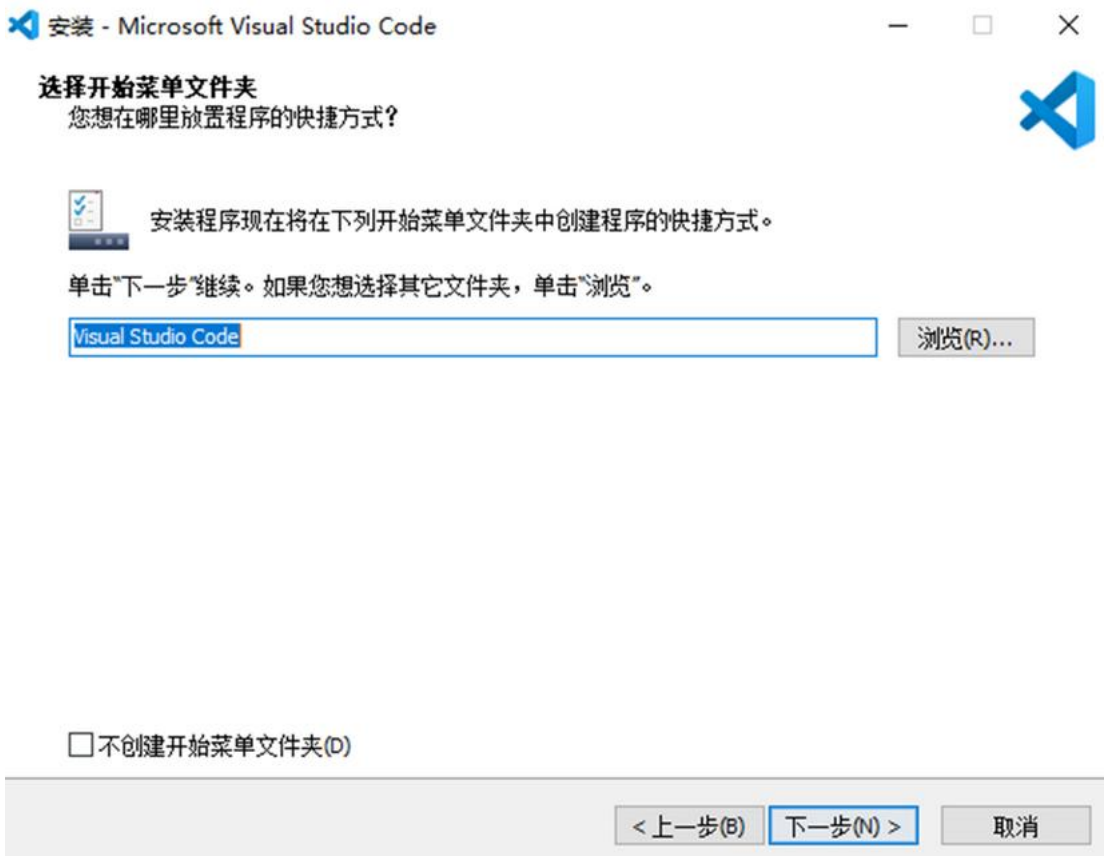
软件可在官方网站下载,链接如下：
https://code.visualstudio.com/Download

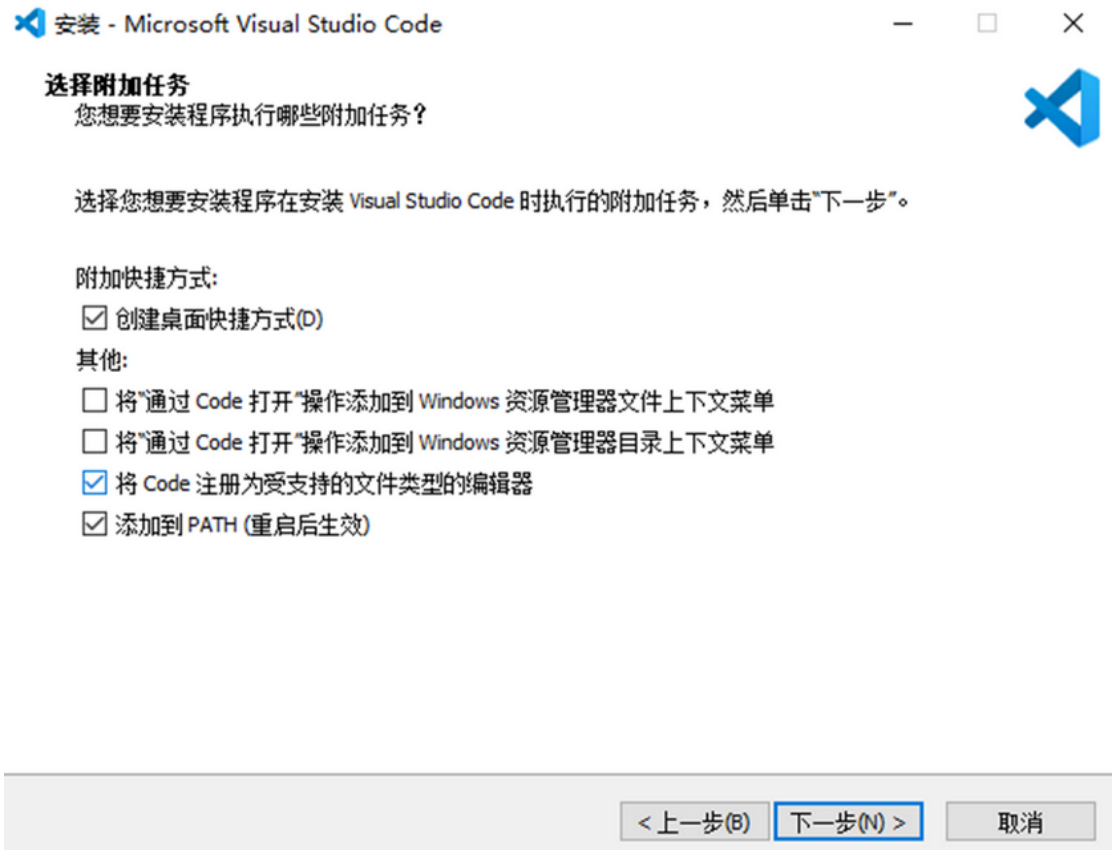


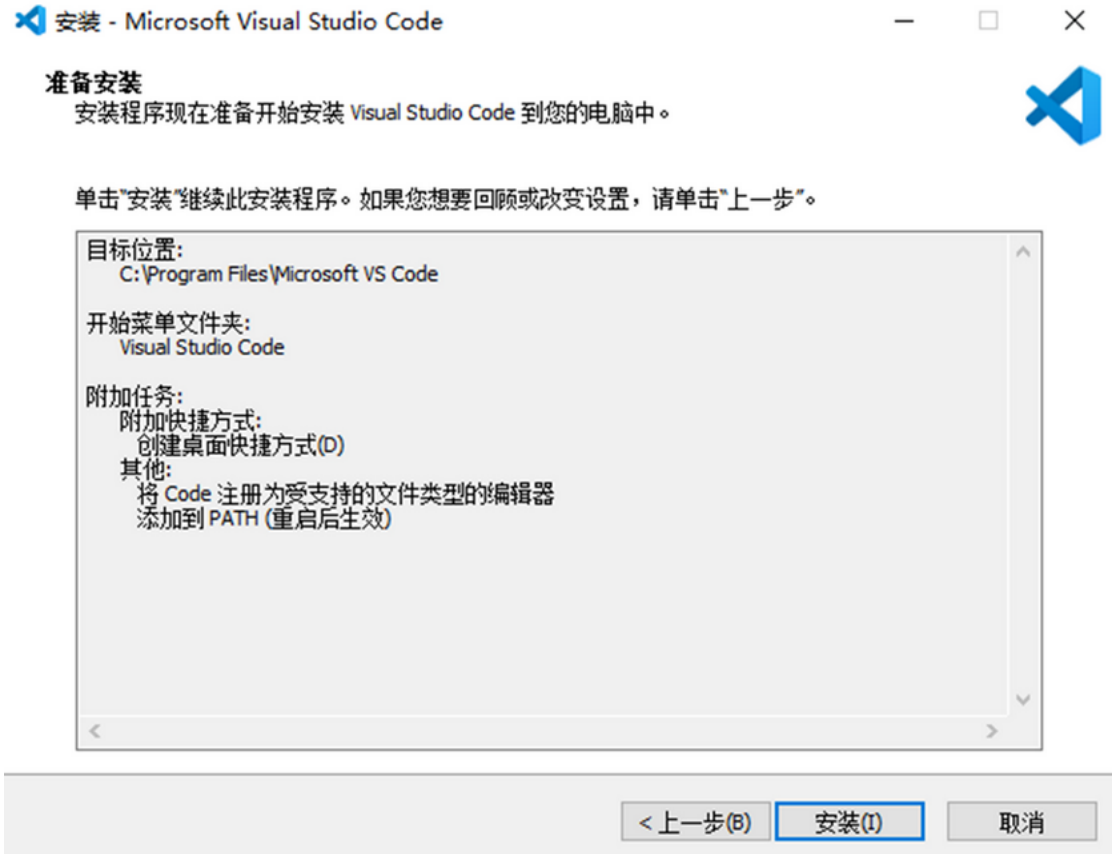
我们使用的是 Windows 版本，用户自定义路径，其余部分默认，点击下一步即可

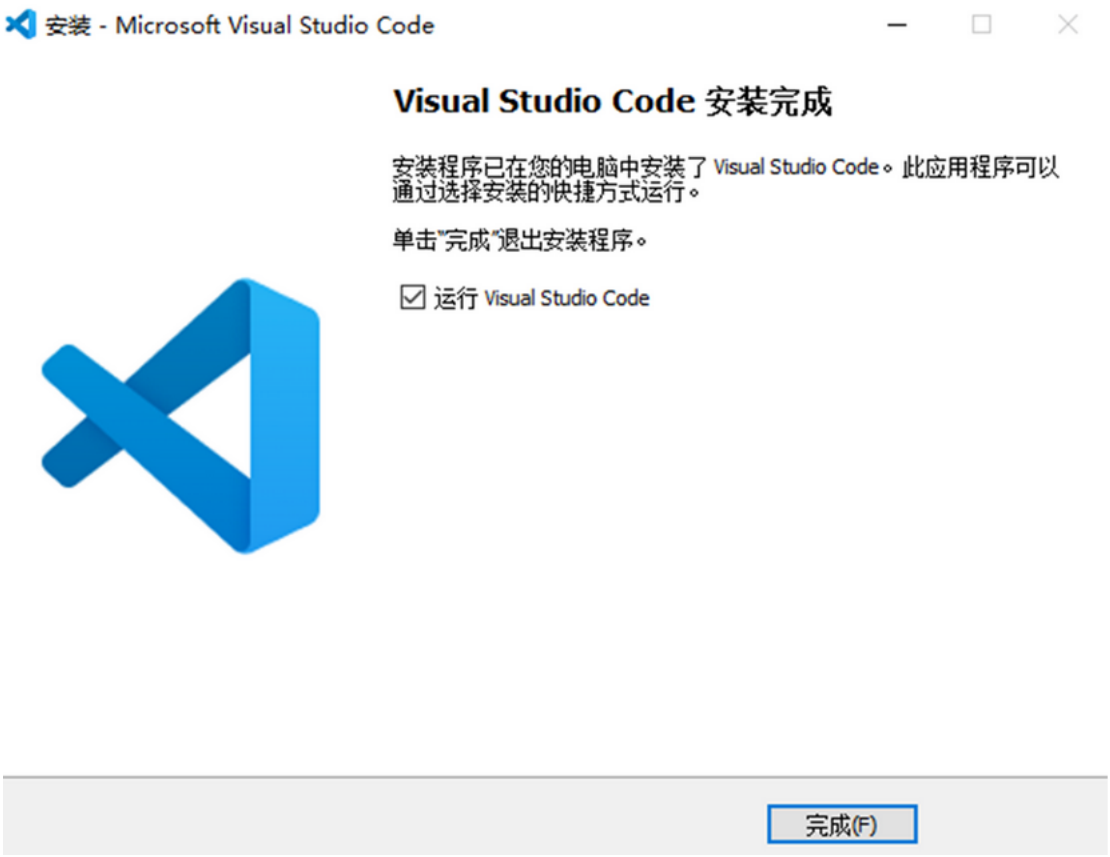




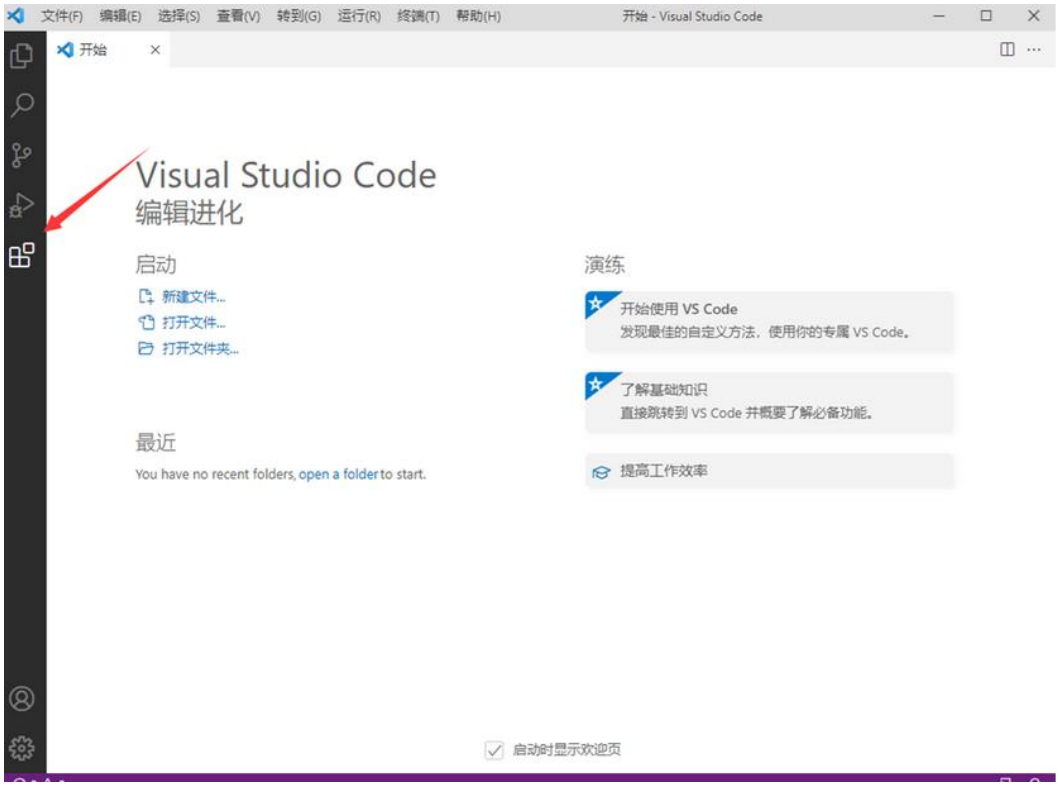

安装 - Microsoft Visual Studio Code
选择开始菜单文件夹
您想在哪里放置程序的快捷方式？
安装程序现在将在下列开始菜单文件夹中创建程序的快捷方式。
单击"下一步"继续。如果您想选择其它文件夹，单击"浏览"。
Visual Studio Code
浏览(R)...
不创建开始菜单文件夹(D)
< 上一步(B)
下一步(N) >
取消


安装 - Microsoft Visual Studio Code
选择附加任务
您想要安装程序执行哪些附加任务？
选择您想要安装程序在安装 Visual Studio Code 时执行的附加任务，然后单击"下一步"。
附加快捷方式:
创建桌面快捷方式(D)
其他:
将"通过 Code 打开"操作添加到 Windows 资源管理器文件上下文菜单
将"通过 Code 打开"操作添加到 Windows 资源管理器目录上下文菜单
将 Code 注册为受支持的文件类型的编辑器
添加到 PATH (重启后生效)
< 上一步(B)
下一步(N) >
取消


安装 - Microsoft Visual Studio Code
准备安装
安装程序现在准备开始安装 Visual Studio Code 到您的电脑中。
单击"安装"继续此安装程序。如果您想要回顾或改变设置，请单击"上一步"。
目标位置:
C:\Program Files\Microsoft VS Code
开始菜单文件夹:
Visual Studio Code
附加任务:
附加快捷方式:
创建桌面快捷方式(D)
其他:
将 Code 注册为受支持的文件类型的编辑器
添加到 PATH (重启后生效)
< 上一步(B)
安装(I)
取消



软件安装完成后，我们可以继续安装插件，用来支持 Verilog 等开发，读者可以根据实际情况，点击箭头的地方，自行添加。



第三章：FPGA 例程

3.1 LED 流水灯例程

3.1.1 功能概述

本实例使用 200MHZ PL 侧的外部差分时钟做计数器的循环计数，计数器的周期为 250ms，每过 250ms 依次点亮一个 LED 灯，如此达到流水灯的效果。

3.1.2 管脚配置

为方便调试，底板上放置了 2 个 LED 灯,当管脚输出 1 时 LED 灯点亮。

管脚约束如下：

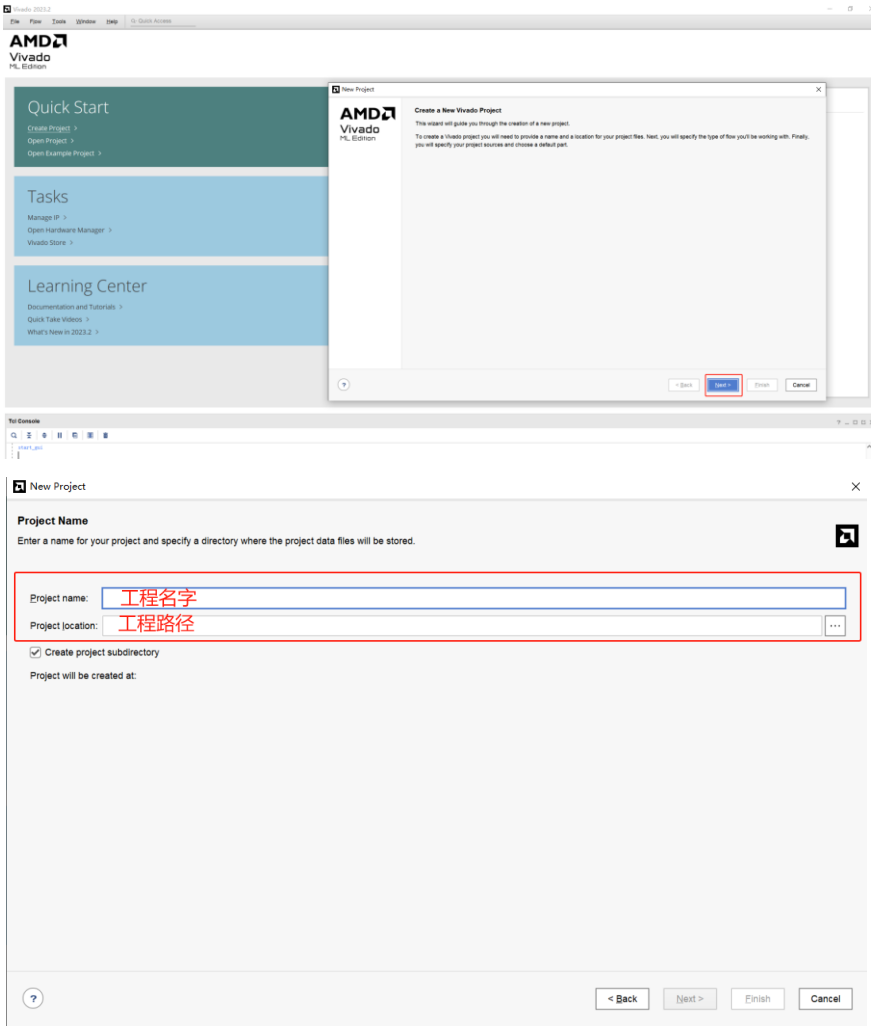
```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]

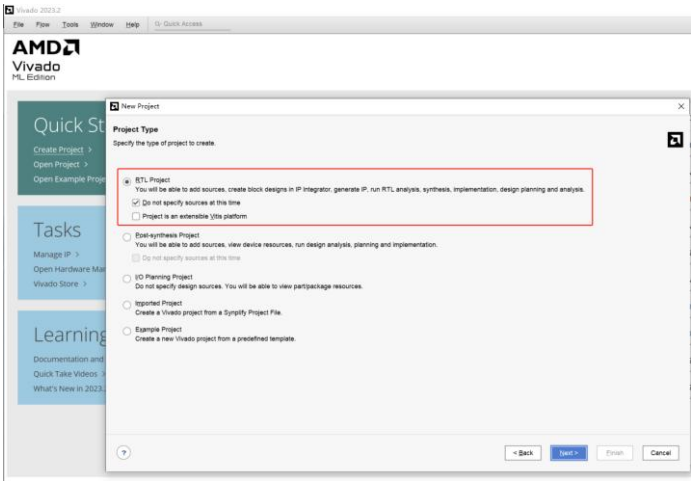
set_property -dict {PACKAGE_PIN AA14 IOSTANDARD LVCMOS33} [get_ports
{led[0]}]
set_property -dict {PACKAGE_PIN AB14 IOSTANDARD LVCMOS33} [get_ports
{led[1]}]
```

3.1.3 工程搭建

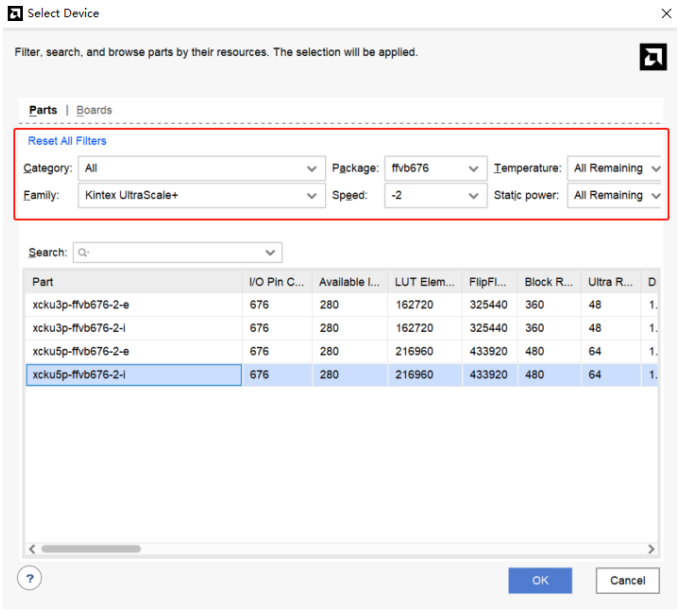
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



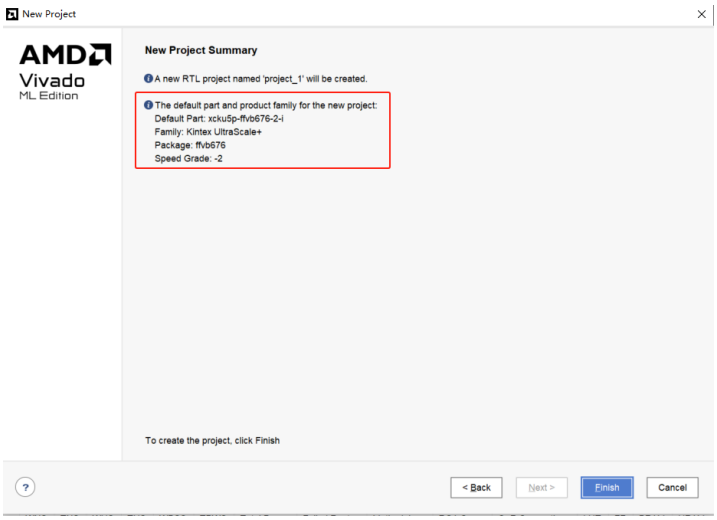
选择 RTL Project 并且勾选复选框，之后单击 NEXT



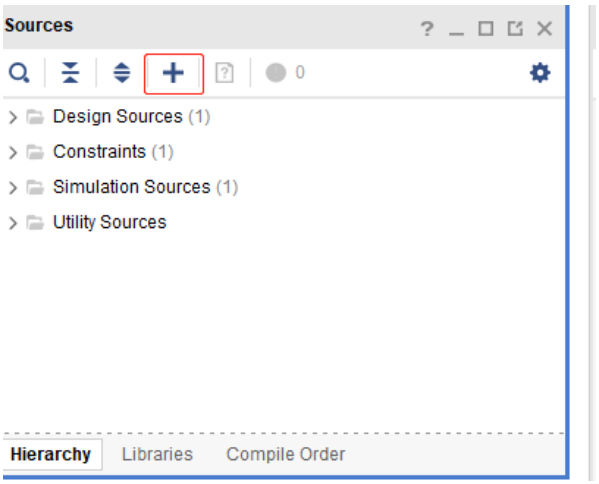
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



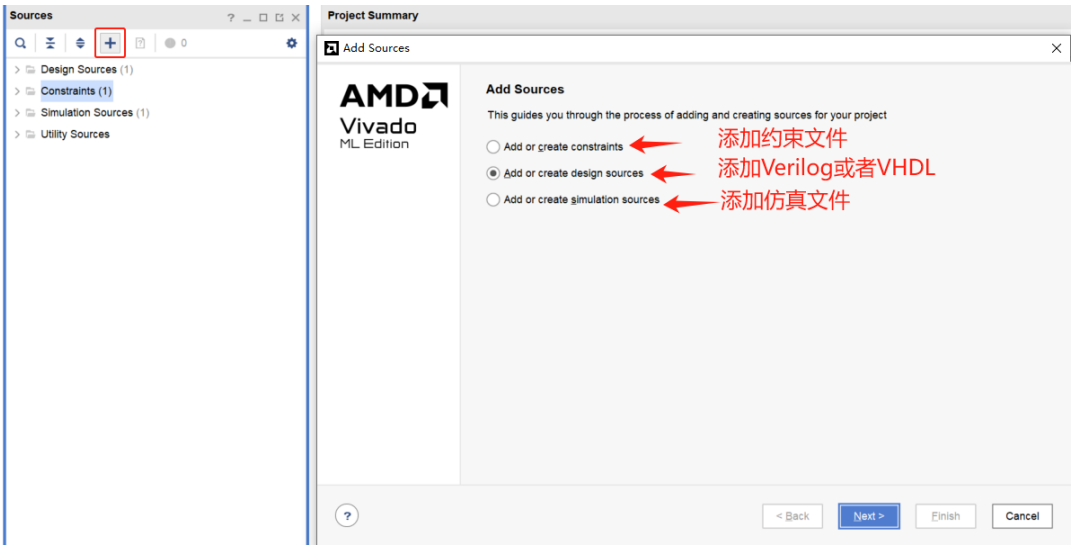
单击 Finish 完成工程创建

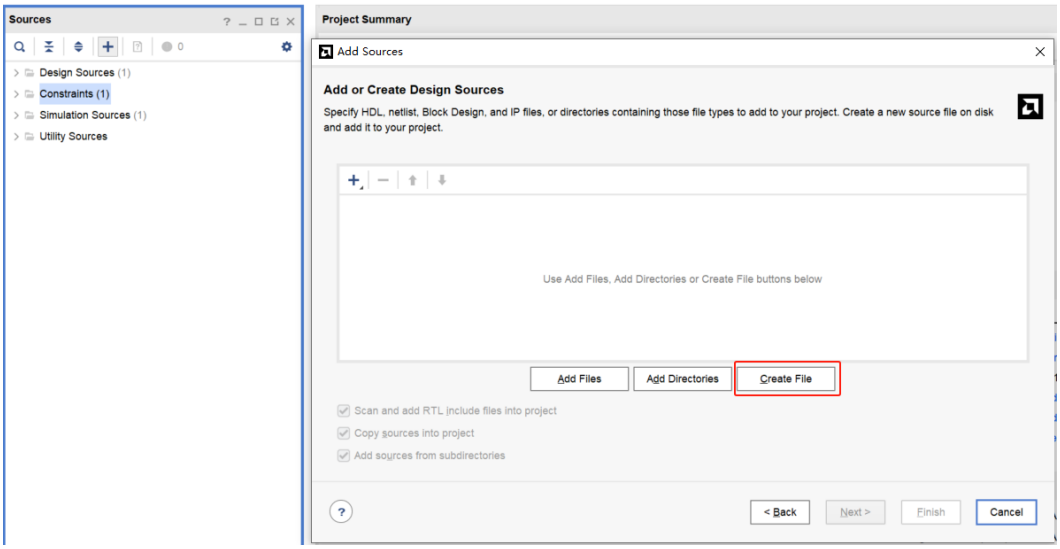


添加代码，显示界面如下

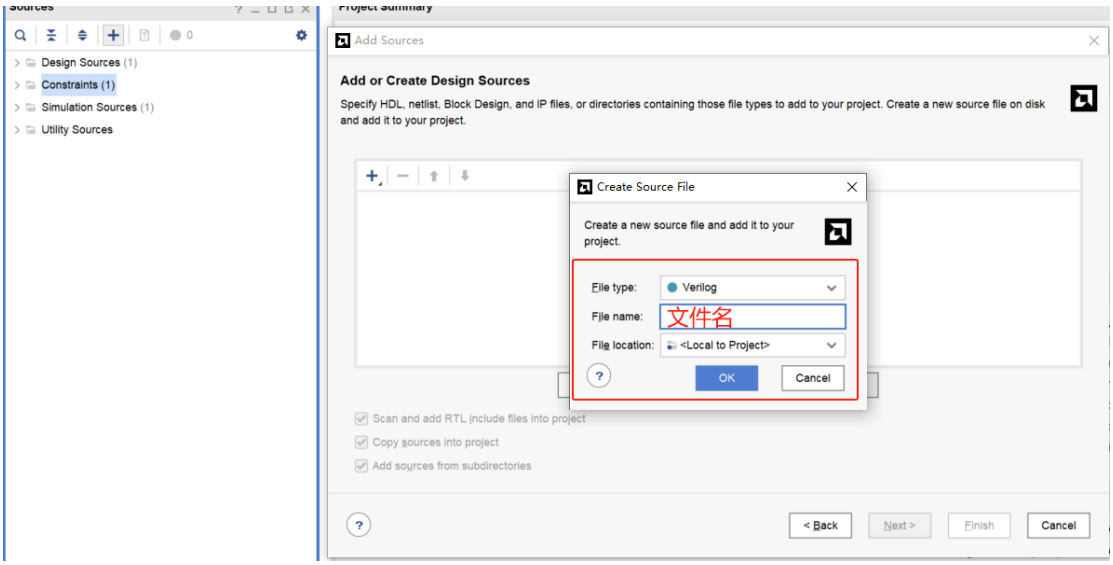


添加代码

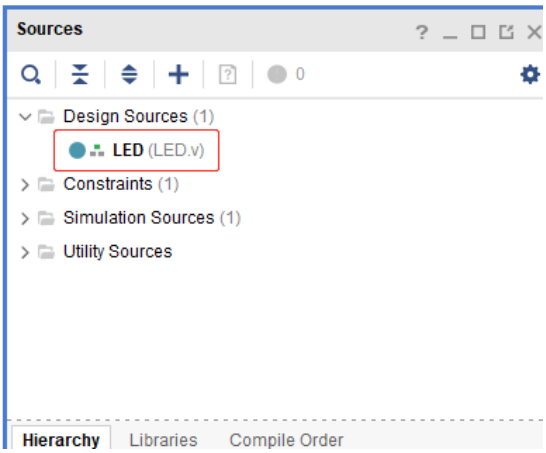




创建一个 LED 文件，并且选择 Verilog 格式



创建完成后，可以看到 Design Sources 文件夹中有 LED.v 文件，这个文件就是我们可以编写 verilog 程序的文件



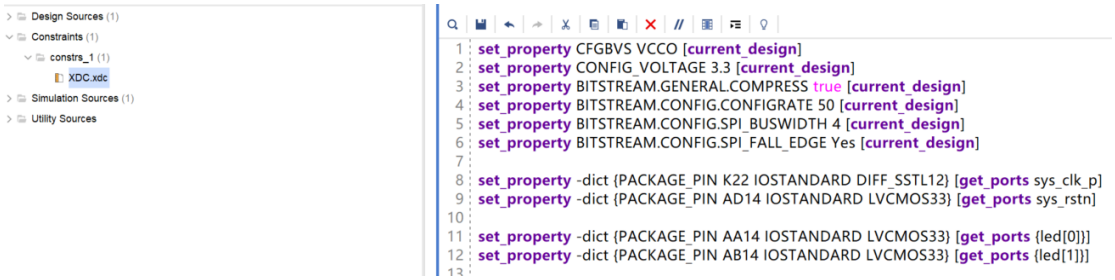
双击 LED 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps
/**********************************************************************
*   例程名称 ：LED
*   例程版本 ：V1.0
*   例程说明 ：
*   开发环境 ：Vitis 2023.2
*   发布日期 ：2024/1/21
*   修改作者 ：璞致电子科技（上海）有限公司
*   公司名称 ：璞致电子科技（上海）有限公司 ，Copyright (C), 2018-2025
*   公司网址 ：www.puzhitech.com
**********************************************************************/
module LED(
input sys_clk_p,
input sys_clk_n,
input sys_rstn,
output wire   [1:0]led
    );
   wire clk;
   IBUFDS #(
      .DIFF_TERM("FALSE"),
      .IBUF_LOW_PWR("TRUE"),
      .IOSTANDARD("DEFAULT")
   ) IBUFDS_inst (
      .O(clk),
      .I(sys_clk_p),
      .IB(sys_clk_n)
   );
assign led=(cnt<pwm_cnt)?2'b11:2'b00;
reg [31:0]cnt;
```

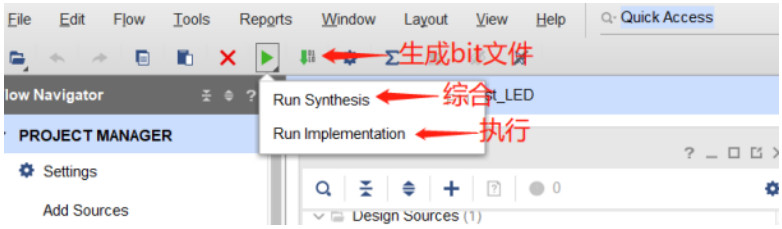
```
always@(posedge clk)
    begin
    if(!sys_rstn)begin
        cnt<=0;
    end
    else begin
        if(cnt<=200000)begin
            cnt<=cnt+1;
        end
        else begin
            cnt<=0;
        end
    end
 end
reg [31:0]pwm_cnt;
reg [1:0]pwm_state;
always@(posedge clk)
    begin
    if(!sys_rstn)begin
        pwm_cnt<=0;
        pwm_state<=0;
    end
    else begin
        case(pwm_state)
        0:begin
             if(pwm_cnt<=200000)begin
                if(cnt==200000)begin
                    pwm_cnt<=pwm_cnt+100;
                end
                else begin
                    pwm_cnt<=pwm_cnt;
                end
            end
            else begin
                pwm_state<=1;
            end
        end
        1:begin
             if(pwm_cnt!=0)begin
                if(cnt==200000)begin
                    pwm_cnt<=pwm_cnt-100;
                end
                else begin
                    pwm_cnt<=pwm_cnt;
```

```
            end
        end
        else begin
            pwm_state<=0;
        end
    end
    default:begin
        pwm_cnt<=0;
        pwm_state<=0;
    end
    endcase
```

添加管脚约束文件



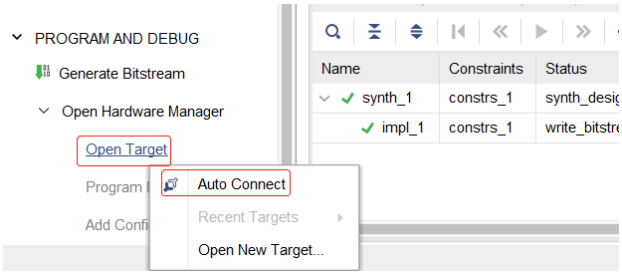
编译并且产生 bit 文件
1:单击综合
2:单击执行
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

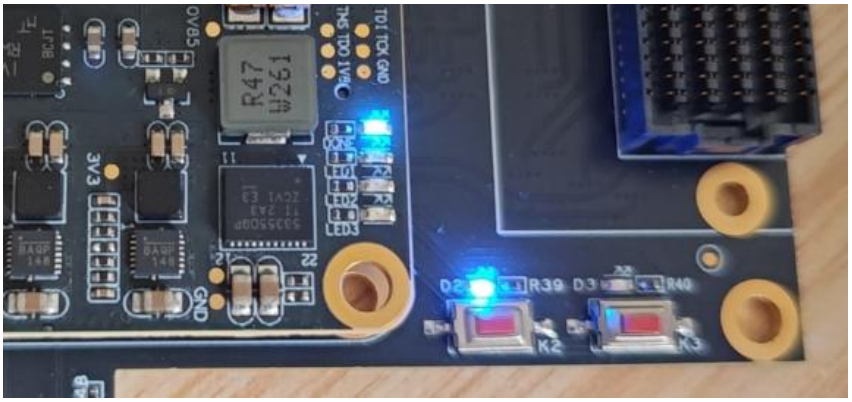


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.1.4 运行结果

下载完成后，LED 流水灯就运行起来了



3.2 LED PWM 控制例程

3.2.1 功能概述

PWM（Pulse Width Modulation），即脉冲宽度调制，它利用微处理器输出的 PWM 信号，实现对模拟电路控制的技术。

本实例使用 200MHZ PL 侧的外部差分时钟做计数器的循环计数，在由计数器产生的固定周期的 PWM 信号下，如果其占空比为 0，则 LED 灯不亮；如果其占空比为 100%，则 LED 灯最亮，由此对 LED 进行 PWM 调节闪烁。

3.2.2 管脚配置

为方便调试，底板上放置了 2 个 LED 灯,当管脚输出 1 时 LED 灯点亮。

管脚约束如下：

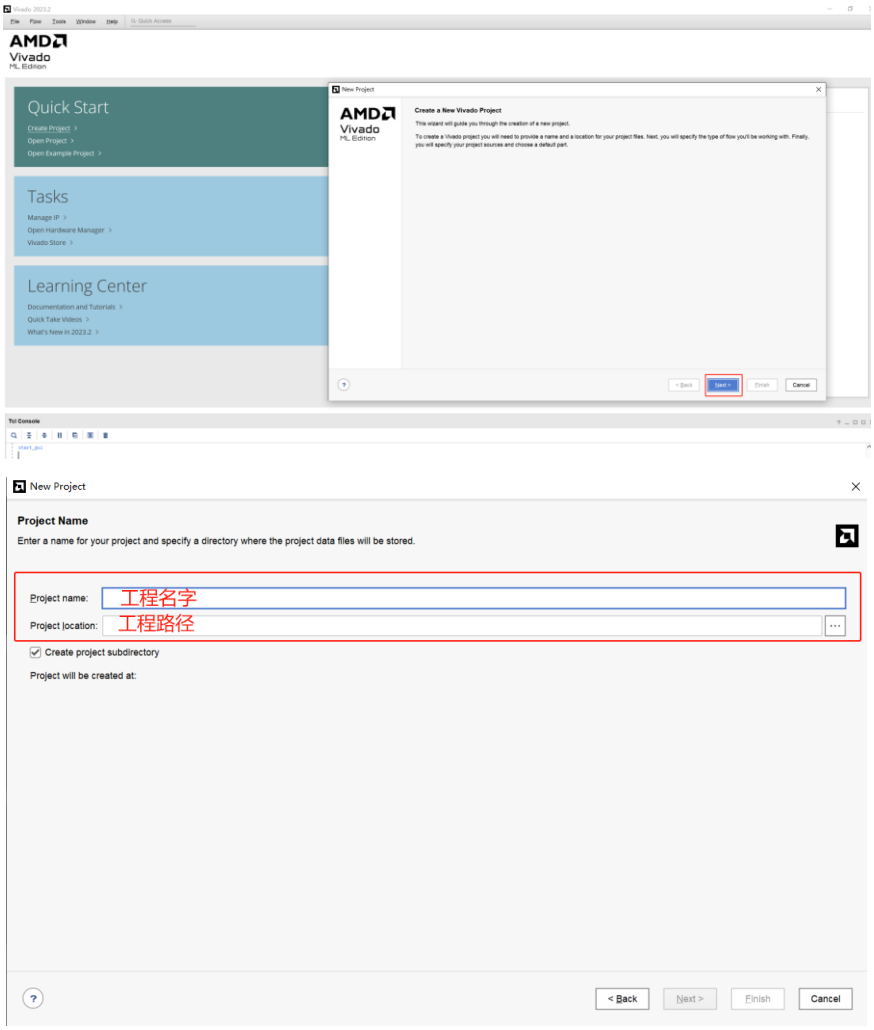
```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]

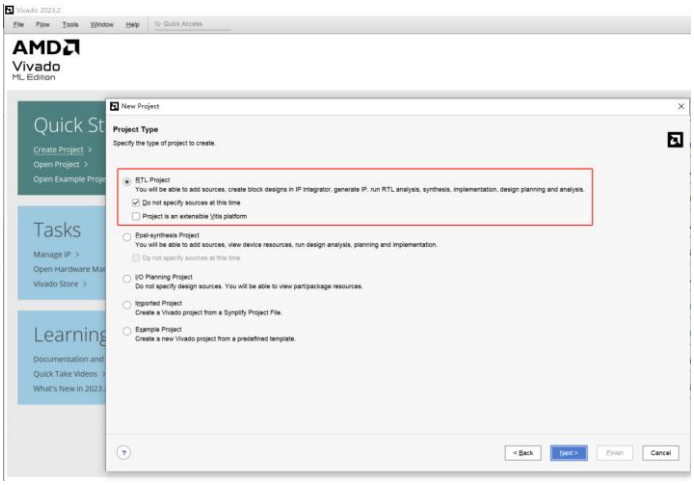
set_property -dict {PACKAGE_PIN AA14 IOSTANDARD LVCMOS33} [get_ports
{led[0]}]
set_property -dict {PACKAGE_PIN AB14 IOSTANDARD LVCMOS33} [get_ports
{led[1]}]
```

3.2.3 工程搭建

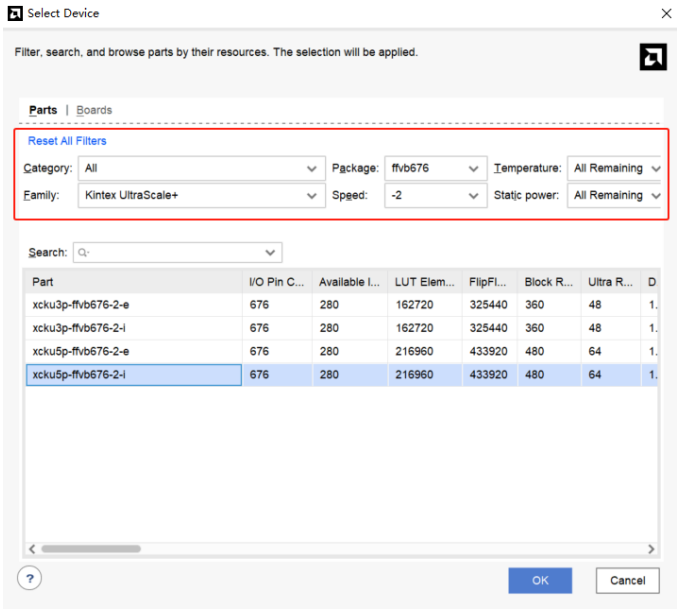
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



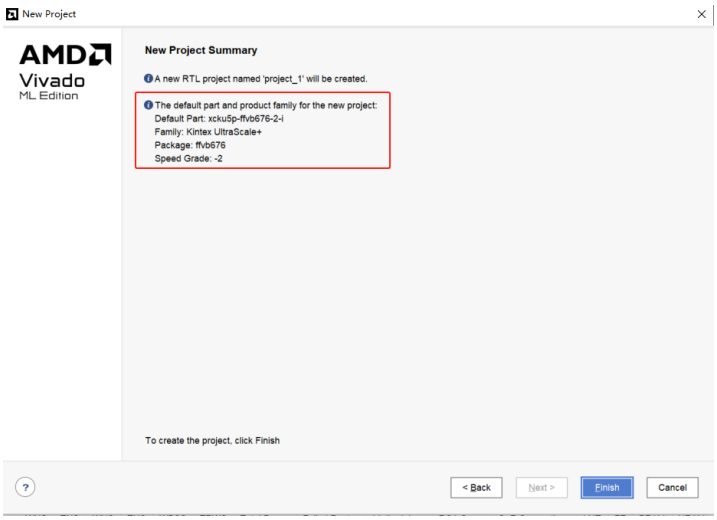
选择 RTL Project 并且勾选复选框，之后单击 NEXT



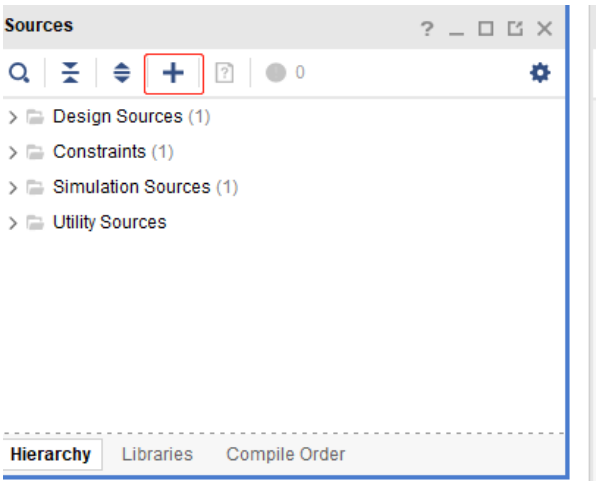
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



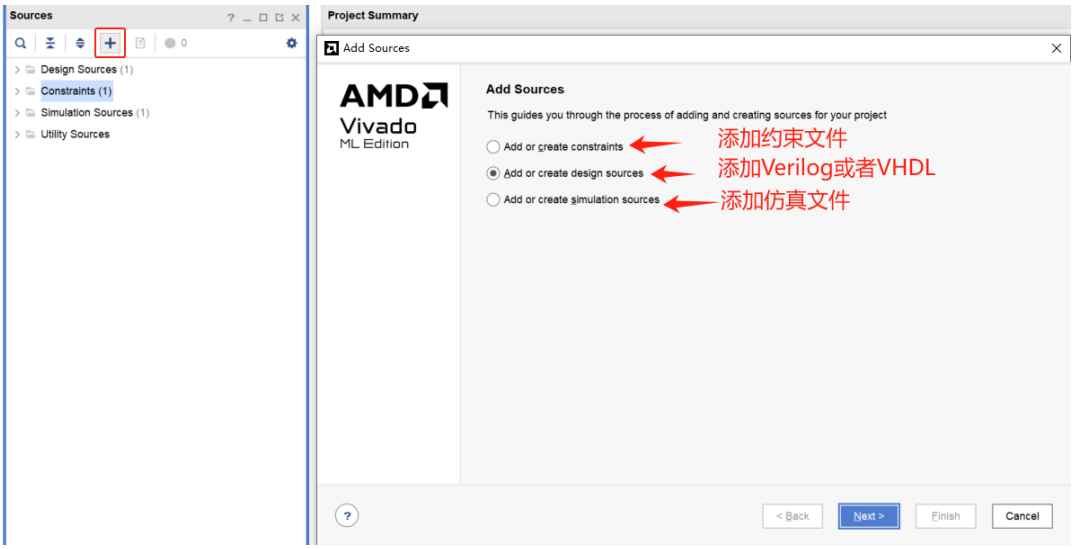
单击 Finish 完成工程创建

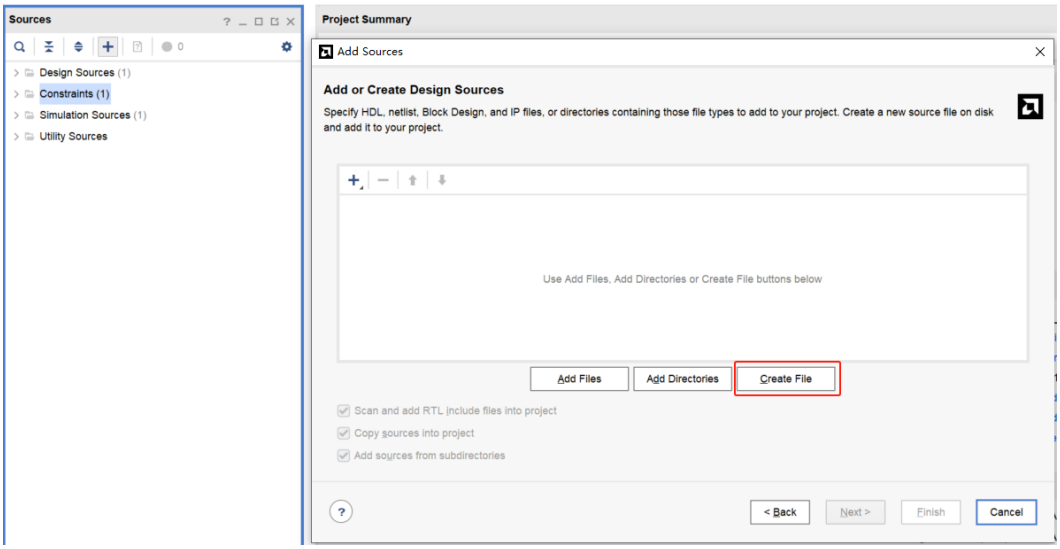


添加代码，显示界面如下

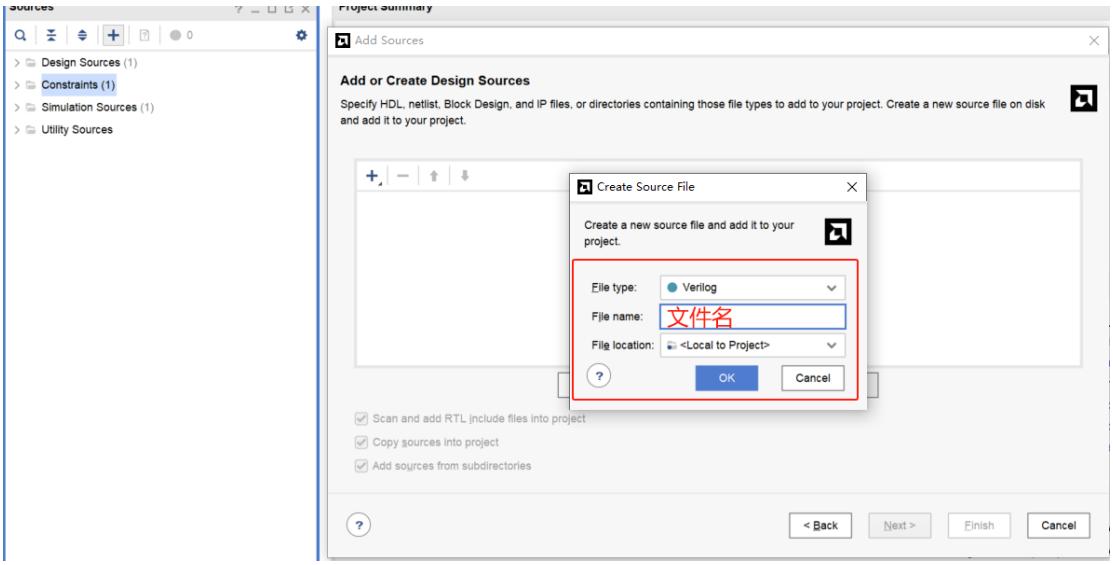


添加代码

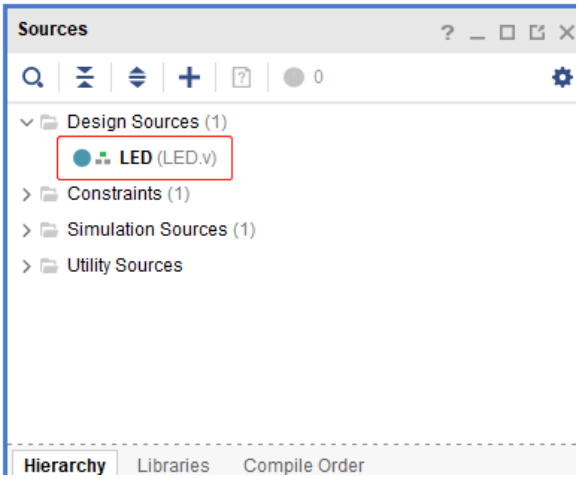




创建一个 LED 文件，并且选择 Verilog 格式



创建完成后，可以看到 Design Sources 文件夹中有 LED.v 文件，这个文件就是我们可以编写 verilog 程序的文件



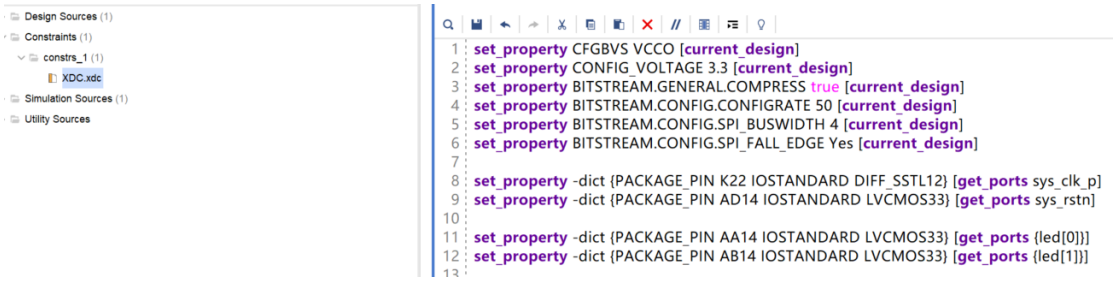
双击 LED 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps
/************************************************************************
*   例程名称 ： LED_PWM
*   例程版本 ： V1.0
*   例程说明 ：
*   开发环境 ： Vitis 2023.2
*   发布日期 ： 2024/1/21
*   修改作者 ： 璞致电子科技（上海）有限公司
*   公司名称 ： 璞致电子科技（上海）有限公司 ，Copyright (C), 2018-2025
*   公司网址 ： www.puzhitech.com
************************************************************************/
module LED(
input sys_clk_p,
input sys_clk_n,
input sys_rstn,
output wire   [1:0]led
    );
   wire clk;
   IBUFDS #(
      .DIFF_TERM("FALSE"),
      .IBUF_LOW_PWR("TRUE"),
      .IOSTANDARD("DEFAULT")
   ) IBUFDS_inst (
      .O(clk),
      .I(sys_clk_p),
      .IB(sys_clk_n)
   );
assign led=(cnt<pwm_cnt)?2'b11:2'b00;
```

```
reg [31:0]cnt;
always@(posedge clk)
    begin
    if(!sys_rstn)begin
        cnt<=0;
    end
    else begin
        if(cnt<=200000)begin
            cnt<=cnt+1;
        end
        else begin
            cnt<=0;
        end
    end
 end
reg [31:0]pwm_cnt;
reg [1:0]pwm_state;
always@(posedge clk)
    begin
    if(!sys_rstn)begin
        pwm_cnt<=0;
        pwm_state<=0;
    end
    else begin
        case(pwm_state)
        0:begin
             if(pwm_cnt<=200000)begin
                if(cnt==200000)begin
                    pwm_cnt<=pwm_cnt+100;
                end
                else begin
                    pwm_cnt<=pwm_cnt;
                end
            end
            else begin
                pwm_state<=1;
            end
        end
        1:begin
             if(pwm_cnt!=0)begin
                if(cnt==200000)begin
                    pwm_cnt<=pwm_cnt-100;
                end
                else begin
```

```
                    pwm_cnt<=pwm_cnt;
                end
            end
            else begin
                pwm_state<=0;
            end
        end
        default:begin
             pwm_cnt<=0;
             pwm_state<=0;
        end
        endcase
    end
 end//*/
endmodule
```

添加管脚约束文件

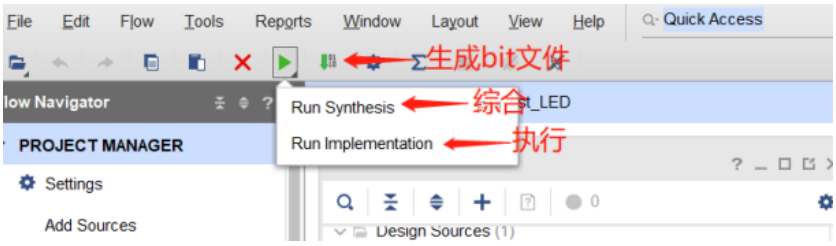


编译并且产生 bit 文件

1:单击综合

2:单击执行

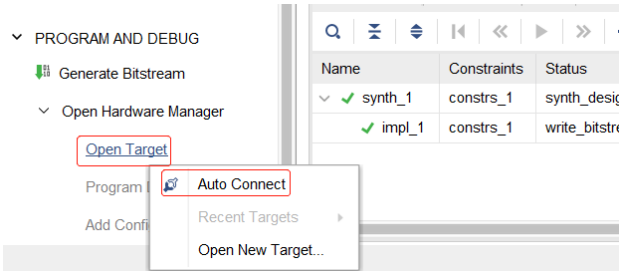
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

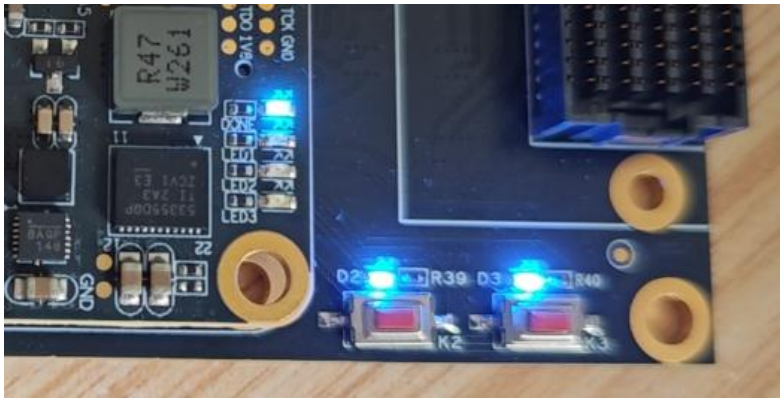


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.2.4 运行结果

下载完成后，LED 呼吸灯就运行起来了



3.3 KEY 例程

3.3.1 功能概述

本实例使用 200MHZ PL 侧的外部差分时钟来检测外部按键是否按下，当按键被按下时，对应的灯会点亮。当松开按键时，对应的灯会熄灭。

3.3.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

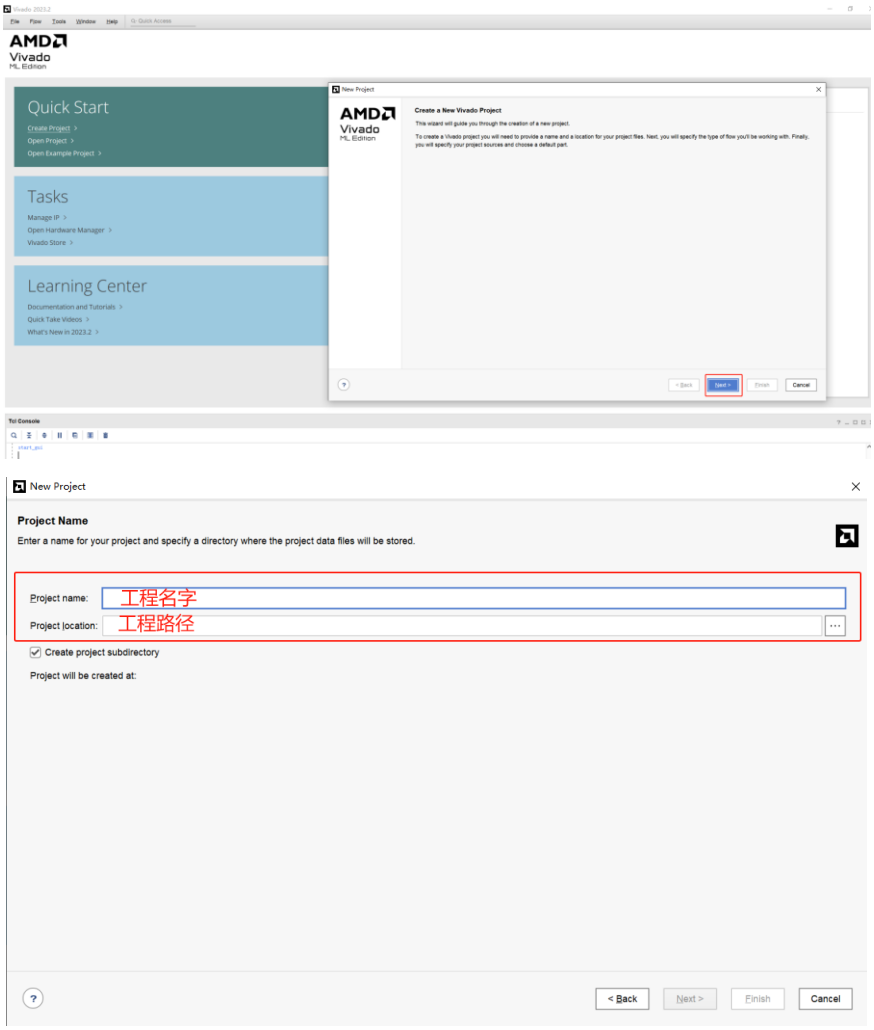
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]

set_property -dict {PACKAGE_PIN AA14 IOSTANDARD LVCMOS33} [get_ports
{led[0]}]
set_property -dict {PACKAGE_PIN AB14 IOSTANDARD LVCMOS33} [get_ports
{led[1]}]

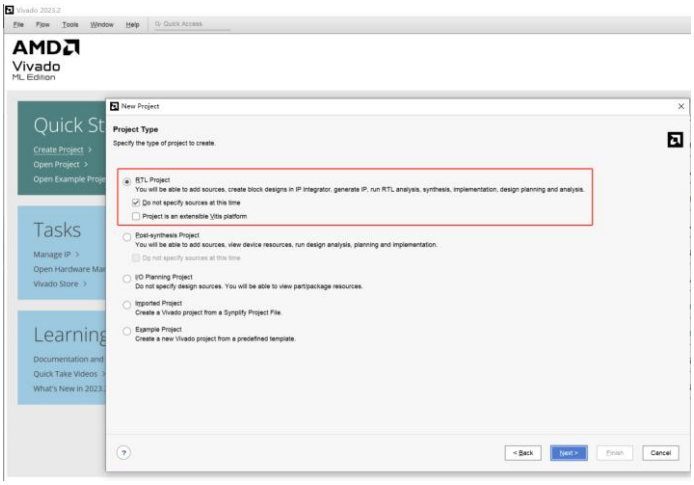
set_property -dict {PACKAGE_PIN AE13 IOSTANDARD LVCMOS33} [get_ports
{key[0]}]
set_property -dict {PACKAGE_PIN AF13 IOSTANDARD LVCMOS33} [get_ports
{key[1]}]
```

3.3.3 工程搭建

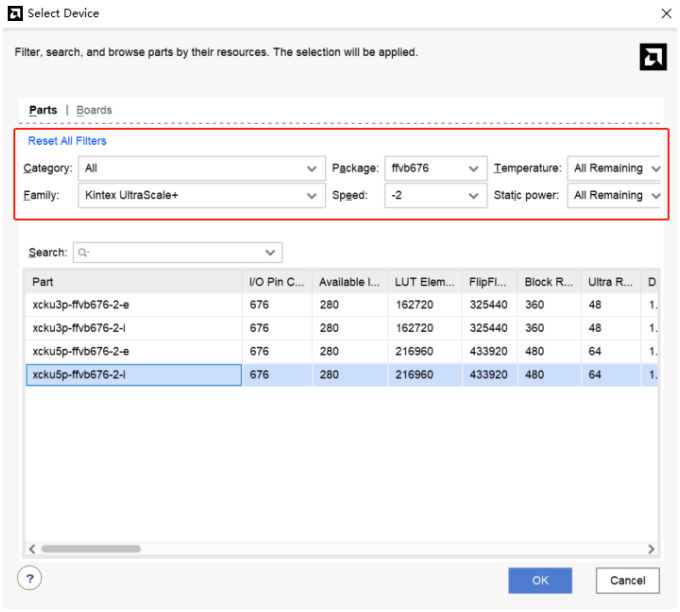
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



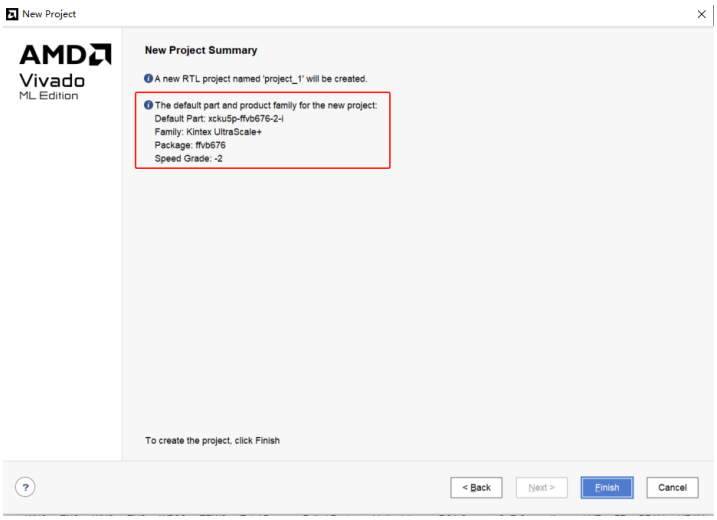
选择 RTL Project 并且勾选复选框，之后单击 NEXT



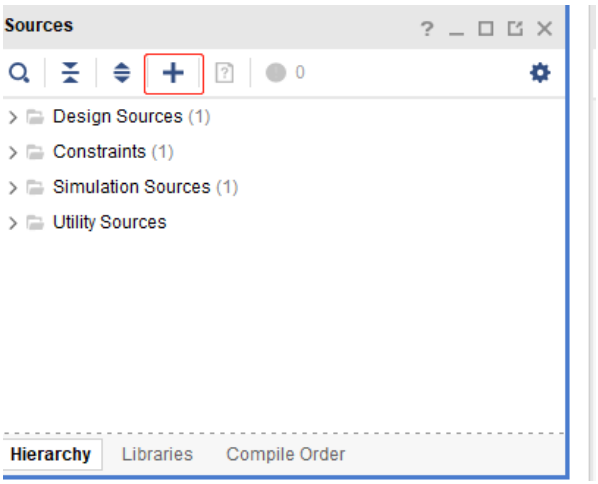
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



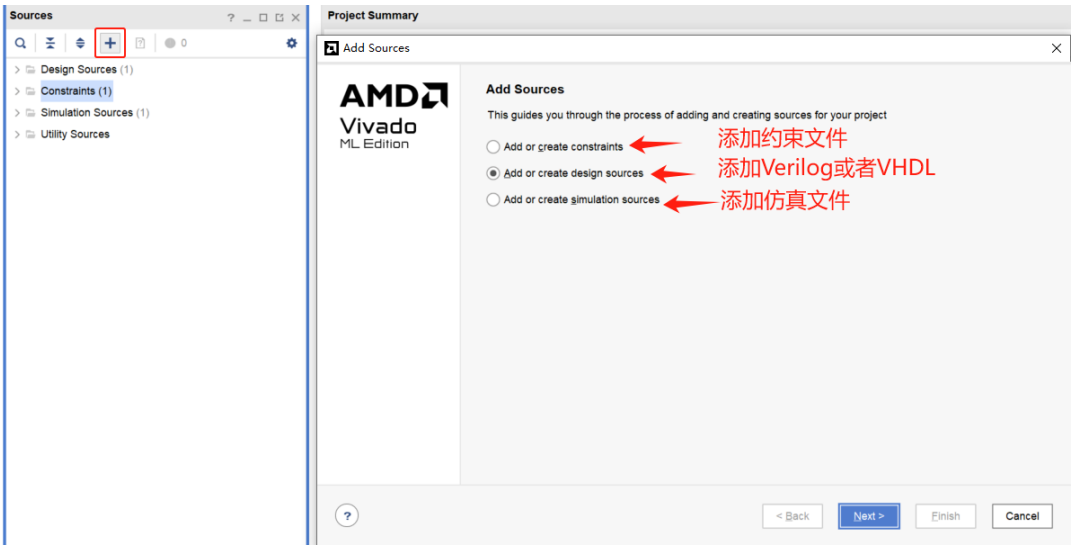
单击 Finish 完成工程创建

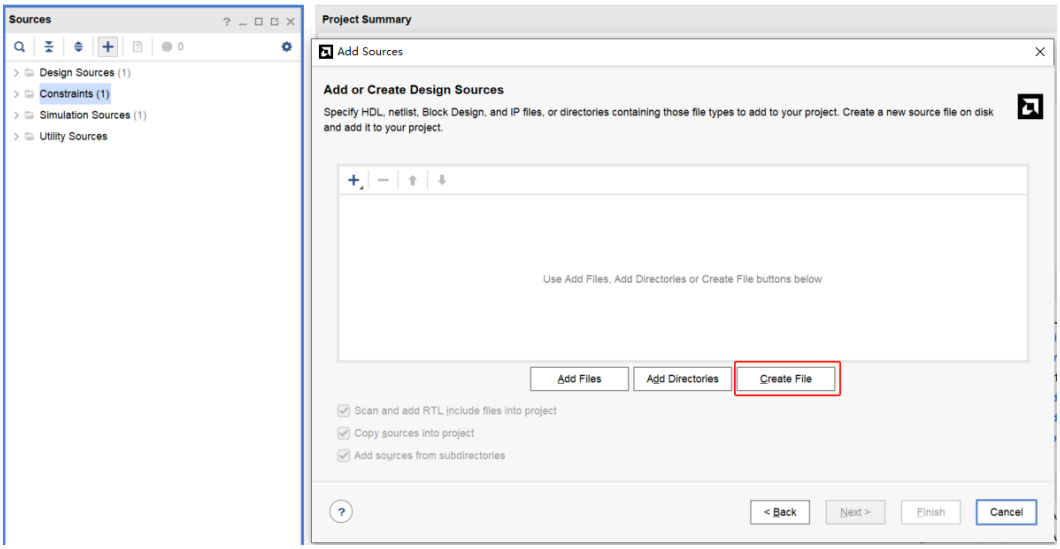


添加代码，显示界面如下

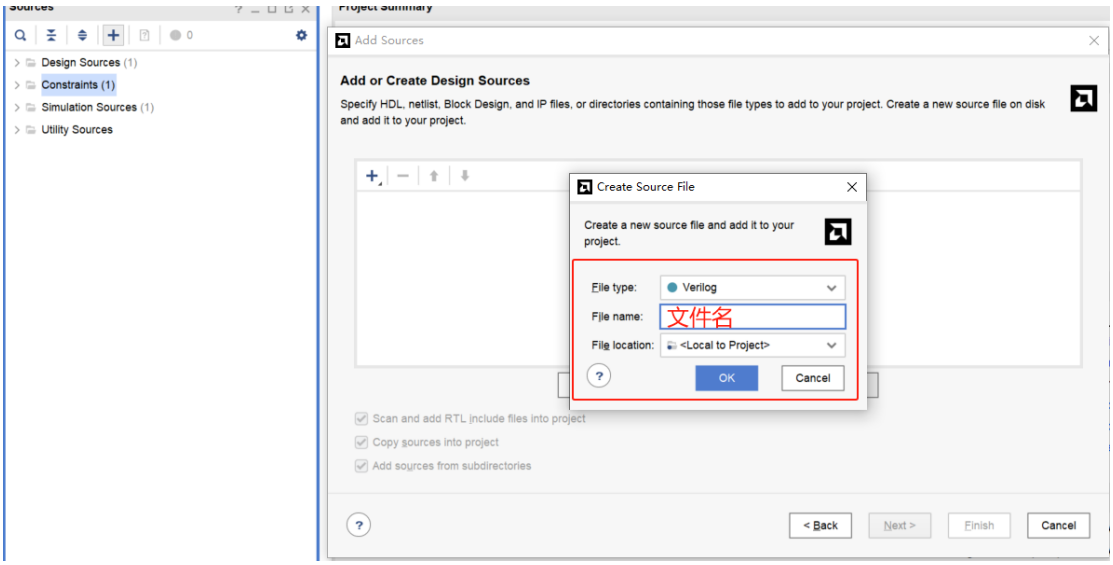


添加代码

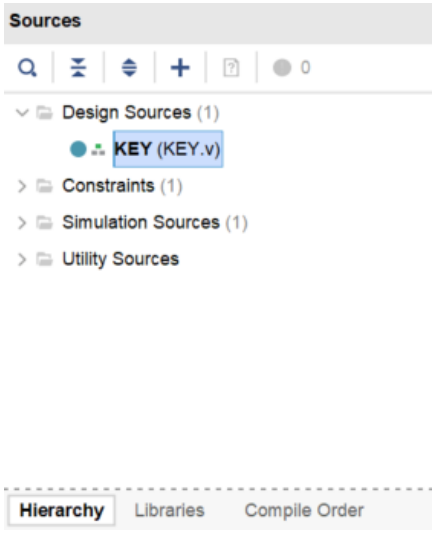




创建一个 KEY 文件，并且选择 Verilog 格式



创建完成后，可以看到 Design Sources 文件夹中有 KEY.v 文件，这个文件就是我们可以编写 verilog 程序的文件



双击 KEY 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps
/**********************************************************************
*   例程名称 ：KEY
*  例程版本 :V1.0
*   例程说明 ：
*  开发环境 : Vitis 2023.2
*   发布日期 ：2024/1/21
*   修改作者 ：璞致电子科技（上海）有限公司
*   公司名称 ：璞致电子科技（上海）有限公司 ，Copyright (C), 2018-2025
*   公司网址 ：www.puzhitech.com
**********************************************************************/
    module KEY(
    input sys_clk_p,
    input sys_clk_n,
    input sys_rstn,
    input    [1:0]key,
    output  reg [1:0]led
        );
    parameter delay_time=20;

       wire clk;
       IBUFDS #(
          .DIFF_TERM("FALSE"),
          .IBUF_LOW_PWR("TRUE"),
          .IOSTANDARD("DEFAULT")
       ) IBUFDS_inst (
          .O(clk),
          .I(sys_clk_p),
```

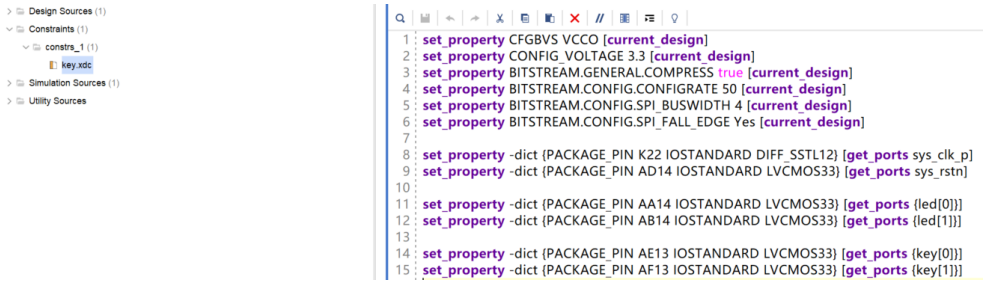
```
        .IB(sys_clk_n)
    );

reg [31:0]cnt;
always@(posedge clk)
    begin
    if(!sys_rstn)begin
        cnt<=0;
    end
    else begin
        if(!key[0])begin
            if(cnt<delay_time)begin
                cnt<=cnt+1;
            end
            else begin
               if(!key[0])begin
                    led[0]<=1;
               end
               else begin
                    cnt<=0;
                    led[0]<=0;
               end
            end
        end
        else begin
             cnt<=0;
             led[0]<=0;
        end
    end
 end

 reg [31:0]cnt1;
always@(posedge clk)
    begin
    if(!sys_rstn)begin
        cnt1<=0;
    end
    else begin
        if(!key[1])begin
            if(cnt1<delay_time)begin
                cnt1<=cnt1+1;
            end
            else begin
               if(!key[1])begin
```

```
                led[1]<=1;
            end
            else begin
                cnt1<=0;
                led[1]<=0;
            end
        end
    end
    else begin
        cnt1<=0;
        led[1]<=0;
    end
  end
 end
endmodule
```

添加管脚约束文件

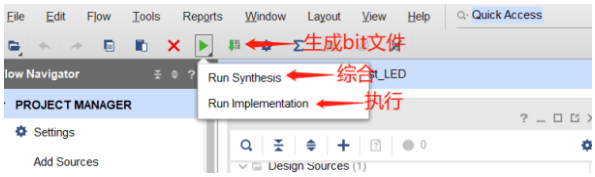


编译并且产生 bit 文件

1:单击综合

2:单击执行

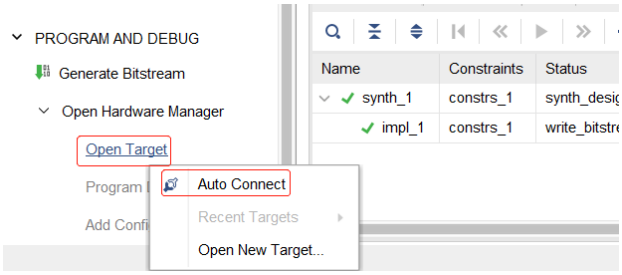
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

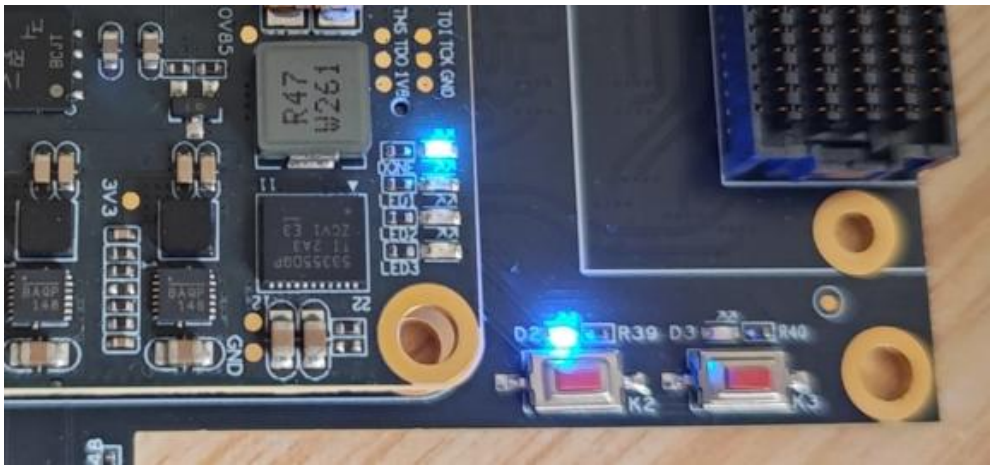


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.3.4 运行结果

下载完成后，按下按键，对应的灯就会被点亮，如下图：



3.4 FPGA 片内 RAM 读写例程

3.4.1 功能概述

RAM 的英文全称是 Random Access Memory，即随机存取存储器，它可以随时把数据写入任一指定地址的存储单元，也可以随时从任一指定地址中读出数据，其读写速度是由时钟频率决定的。

本实例实现往 RAM 里面写 1024 个数据，然后再读取出来，通过 ILA（在线逻辑分析仪）进行查看。

3.4.2 管脚配置

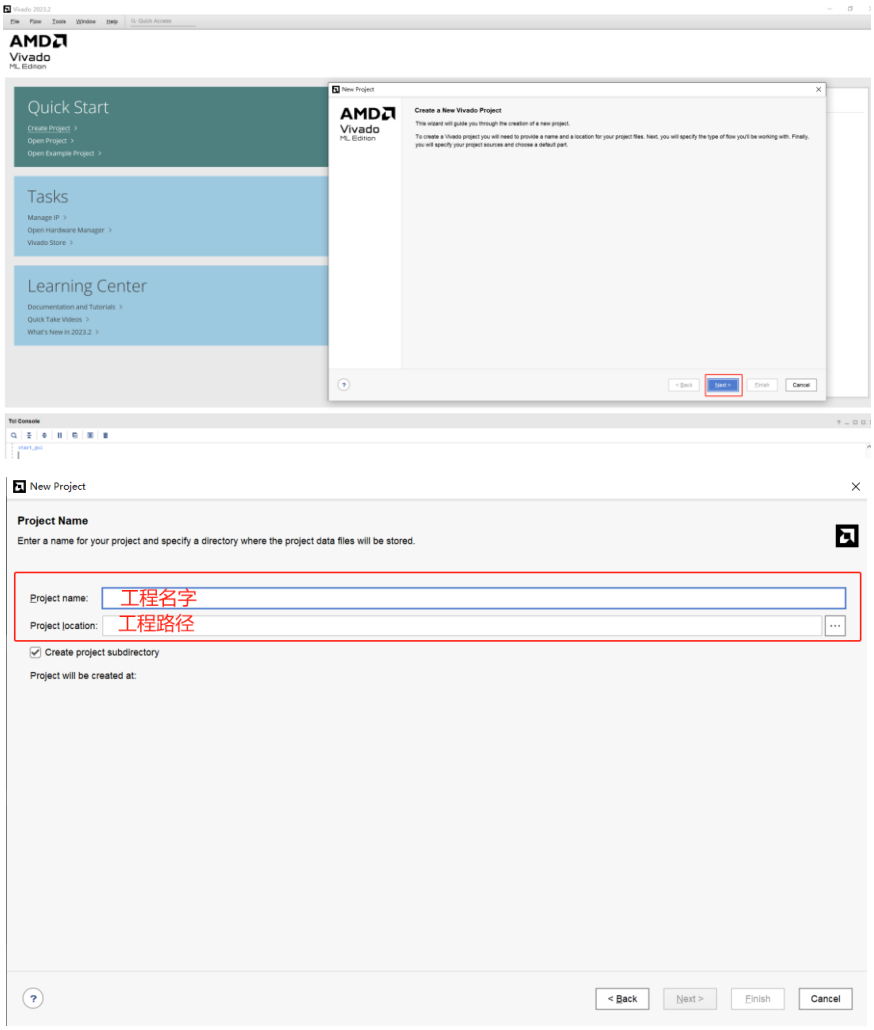
管脚配置如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

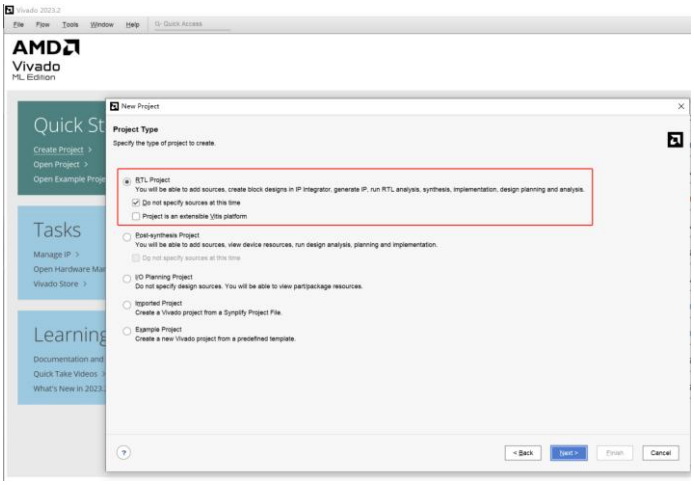
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]
```

3.4.3 工程搭建

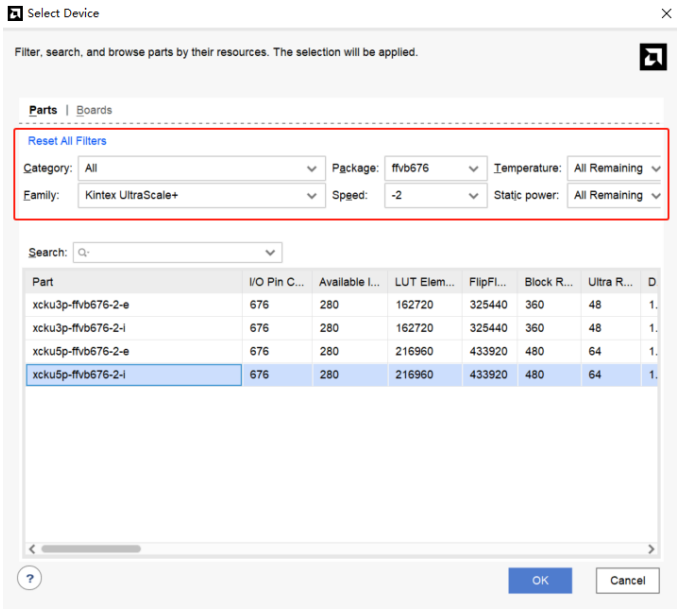
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



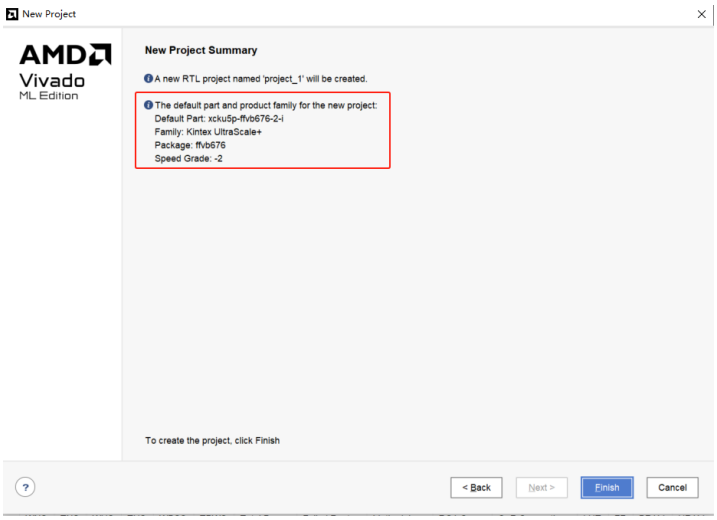
选择 RTL Project 并且勾选复选框，之后单击 NEXT



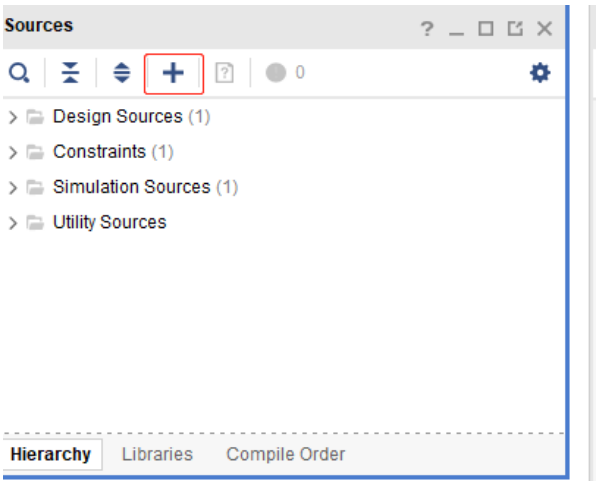
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



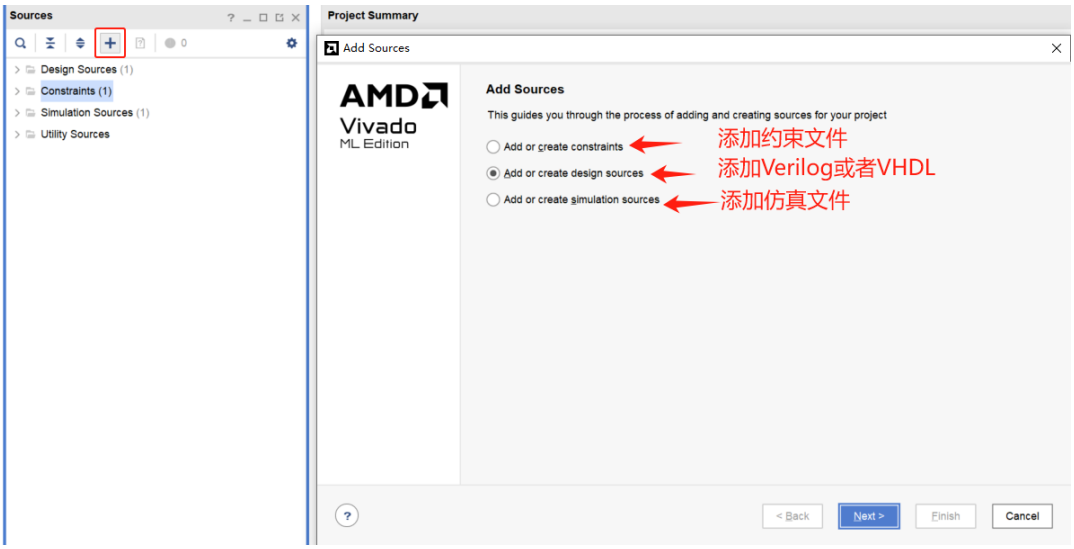
单击 Finish 完成工程创建

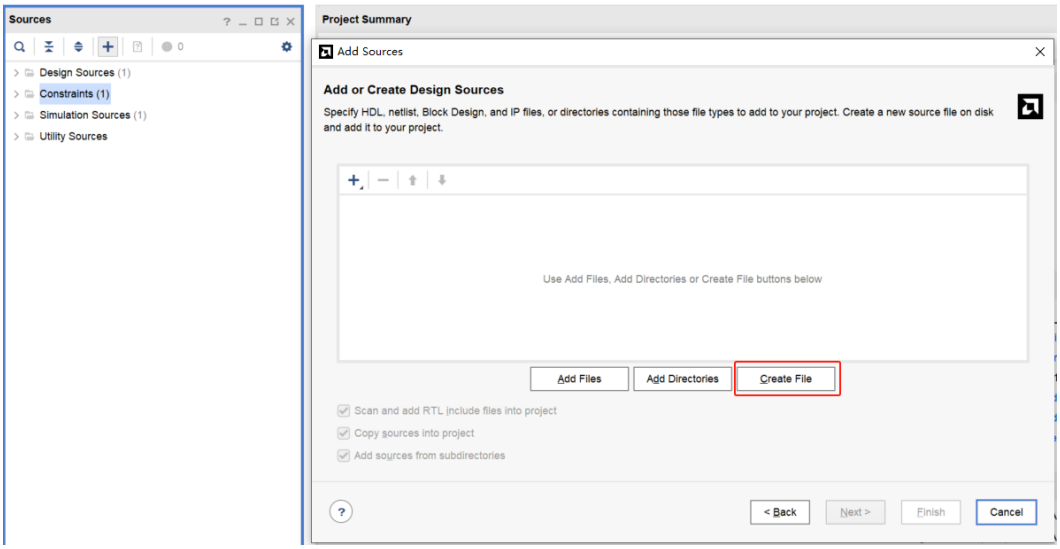


添加代码，显示界面如下

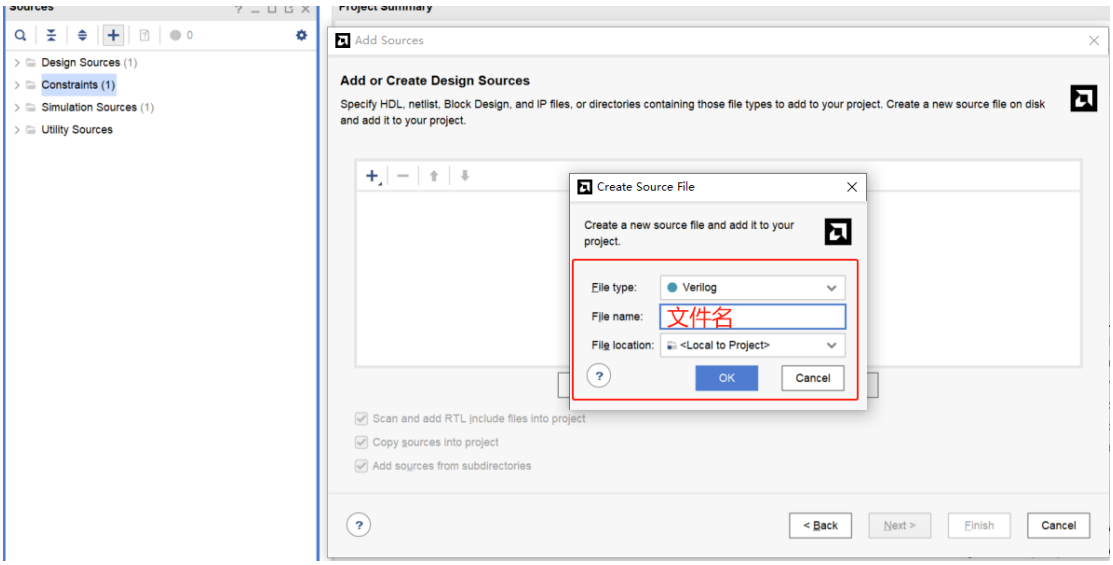


添加代码

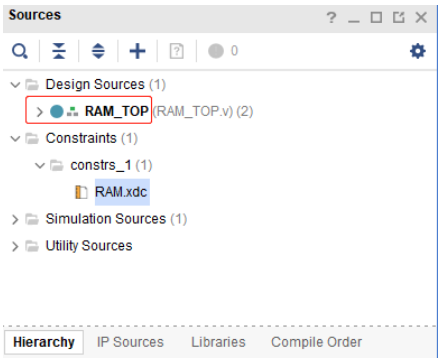




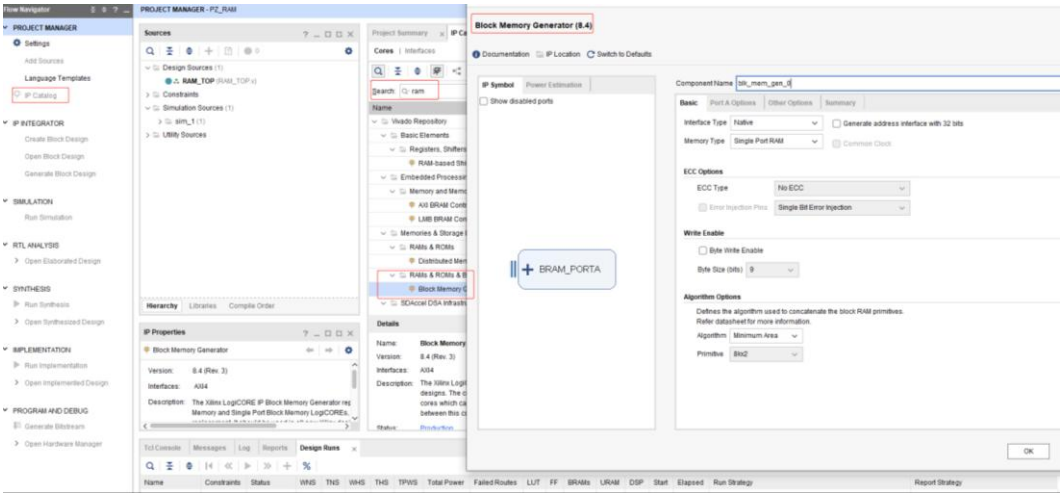
创建一个 RAM_TOP 文件，并且选择 Verilog 格式



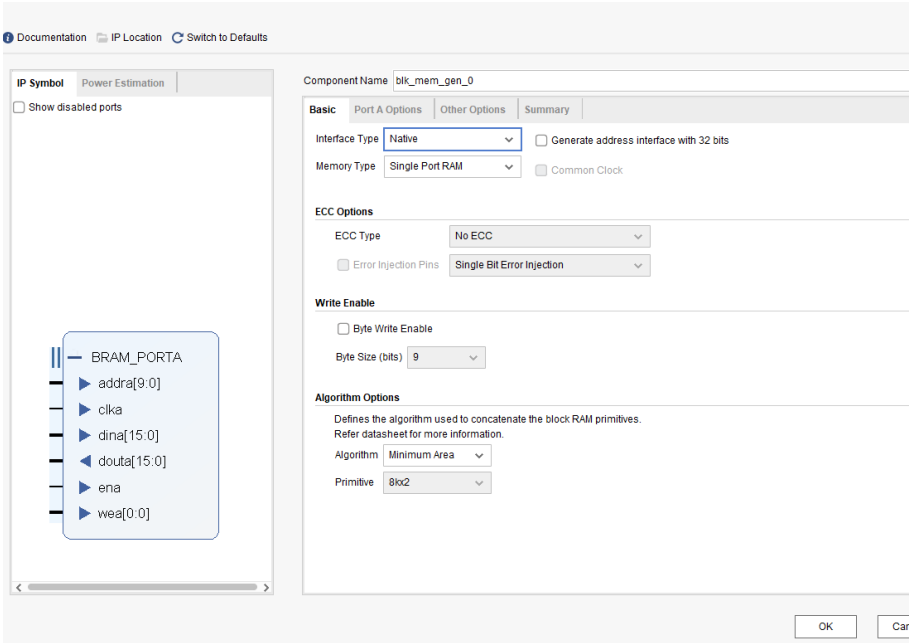
创建完成后，可以看到 Design Sources 文件夹中有 RAM_TOP.v 文件，这个文件就是我们可以编写 verilog 程序的文件

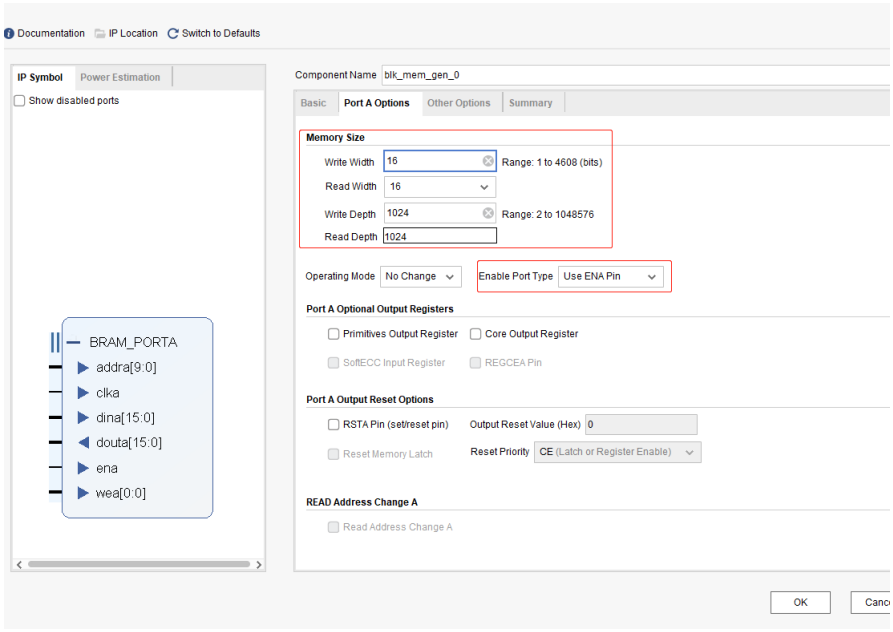


添加 RAM IP 到工程，如下图：

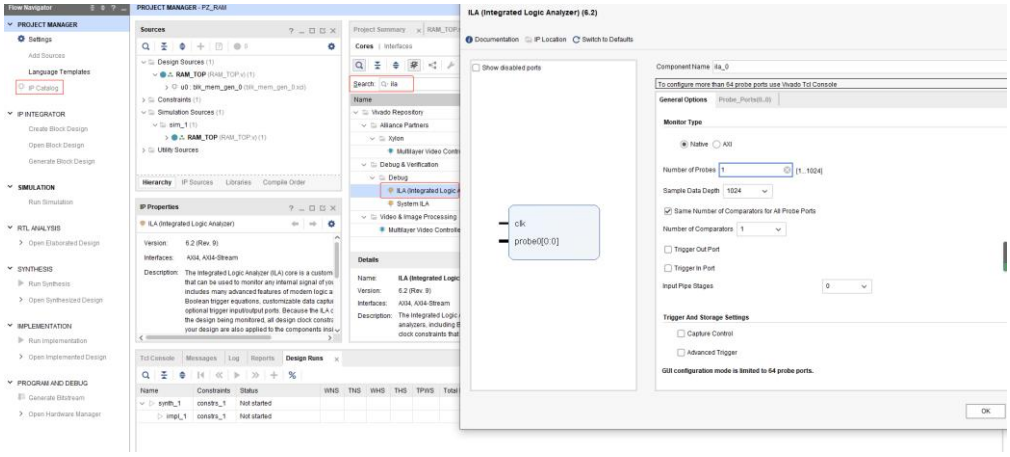


RAM IP 配置如下（其他默认）：

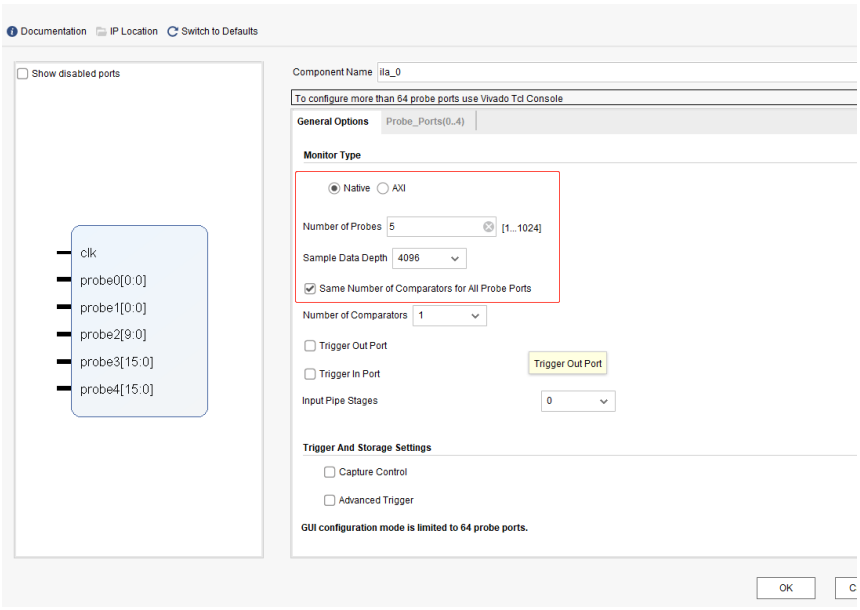


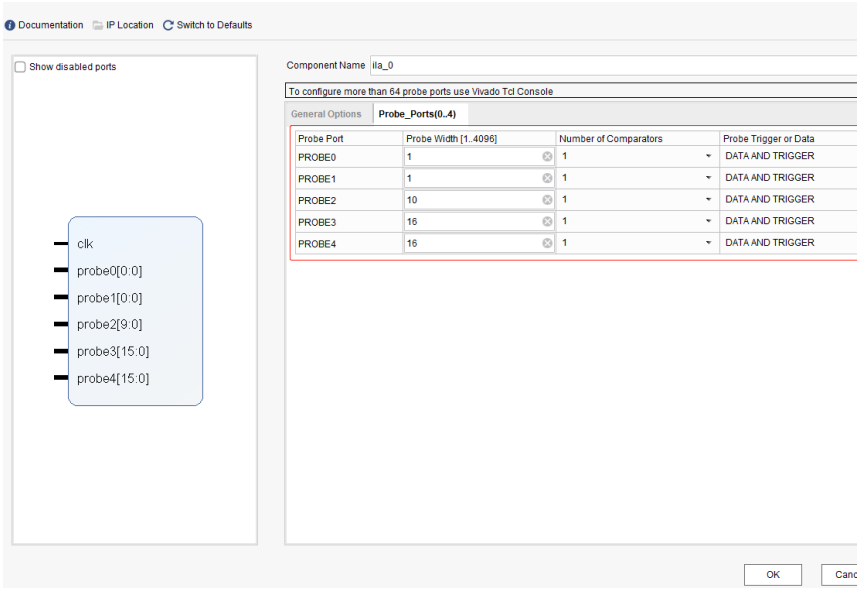


添加 ILA IP 到工程，如下图：



ILA IP 配置如下（其他默认）：





双击 RAM_TOP 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2024/01/24 13:40:59
// Design Name:
// Module Name: RAM_TOP
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//////////////////////////////////////////////////////////////////////////////////
module RAM_TOP(
input sys_clk_p,
input sys_clk_n,
input sys_rstn

    );

   wire clk;
   IBUFDS #(
```

```
      .DIFF_TERM("FALSE"),
      .IBUF_LOW_PWR("TRUE"),
      .IOSTANDARD("DEFAULT")
   ) IBUFDS_inst (
      .O(clk),
      .I(sys_clk_p),
      .IB(sys_clk_n)
   );
   wire ena;
   reg [0 : 0] wea;
   reg [9 : 0] addra;
   reg [15 : 0] dina;
   wire [15 : 0] douta;

assign ena=sys_rstn;



reg [2:0]ram_state;
always@(posedge clk )
    if(!sys_rstn)begin
        ram_state<=0;
        wea<=1;
        addra<=0;
        dina<=0;
    end
    else begin
        case(ram_state)
        0:begin
            if(addra<1023)begin
                wea<=1;
                addra<=addra+1;
                dina<=dina+1;
             end
             else begin
                ram_state<=1;
                wea<=0;
                addra<=0;
                dina<=0;
             end
        end
        1:begin
            if(addra<1023)begin
                wea<=0;
```

```
                addra<=addra+1;
            end
            else begin
                ram_state<=0;
                wea<=1;
                addra<=0;
                dina<=0;
            end
        end
        default:begin
            ram_state<=0;
            wea<=0;
            addra<=0;
            dina<=0;
        end
        endcase
    end

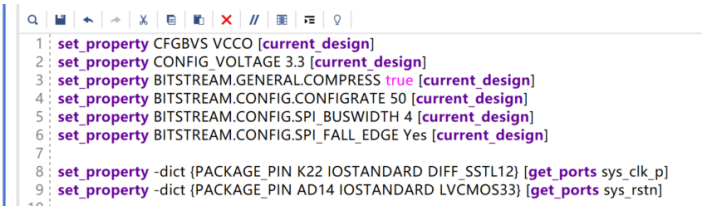


blk_mem_gen_0 u0(
  .clka(clk),    // input wire clka
  .ena(ena),      // input wire ena
  .wea(wea),      // input wire [0 : 0] wea
  .addra(addra),  // input wire [9 : 0] addra
  .dina(dina),    // input wire [15 : 0] dina
  .douta(douta)  // output wire [15 : 0] douta
);

ila_0 u1 (
    .clk(clk), // input wire clk
    .probe0(ena), // input wire [0:0]  probe0
    .probe1(wea), // input wire [0:0]  probe1
    .probe2(addra), // input wire [9:0]  probe2
    .probe3(dina), // input wire [15:0]  probe3
    .probe4(douta) // input wire [15:0]  probe4
);
endmodule
```

添加管脚约束文件





```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

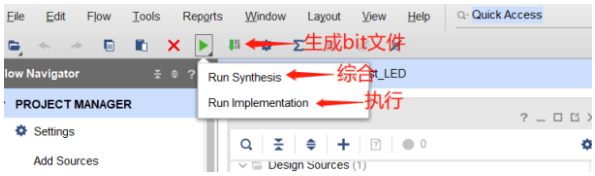
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]
```

编译并且产生 bit 文件

1:单击综合

2:单击执行

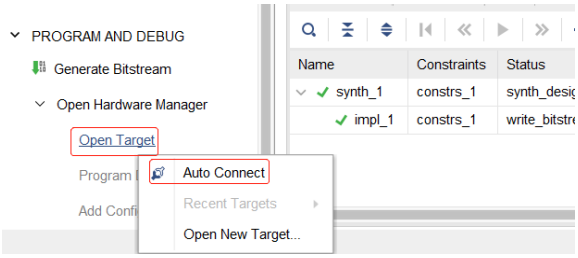
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



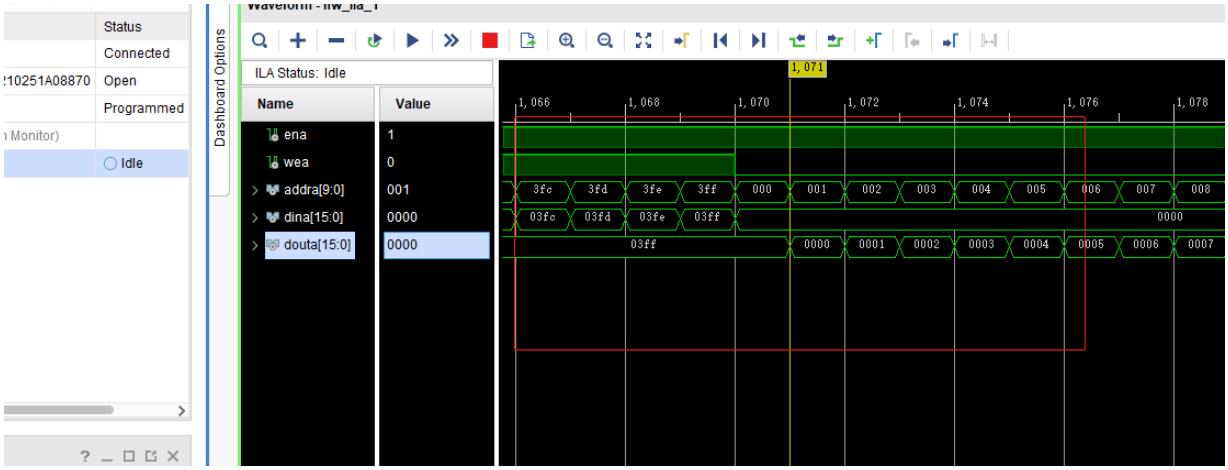
单击 Program Device（也可以从顶部单击 Program device）

弹出的对话框中有我们要下载的 Bit 文件



3.4.4 运行结果

下载完成后，自动跳出 ILA 界面，然后进行信号抓取，如下图



3.5 FPGA 片内 ROM 读写例程

3.5.1 功能概述

只读存储器（Read-Only Memory，ROM）以非破坏性读出方式工作，只能读出无法写入信息。信息一旦写入后就固定下来，即使切断电源，信息也不会丢失，所以又称为固定存储器。ROM 所存数据通常是装入整机前写入的，整机工作过程中只能读出，不像随机存储器能快速方便地改写存储内容。ROM 所存数据稳定 ，断电后所存数据也不会改变，并且结构较简单，使用方便，因而常用于存储各种固定程序和数据。

本实例实现往 ROM 里面写 1024 个数据，然后再读取出来，通过 ILA（在线逻辑分析仪）进行查看。

3.5.2 管脚配置

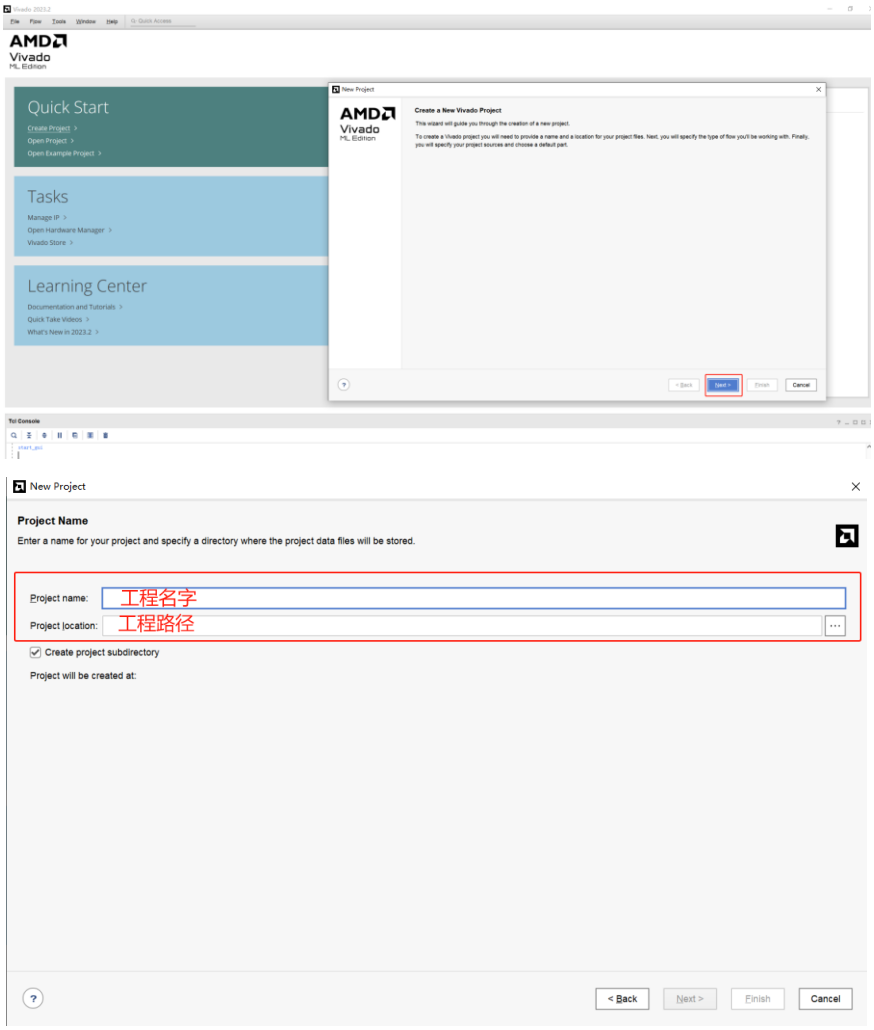
管脚配置如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

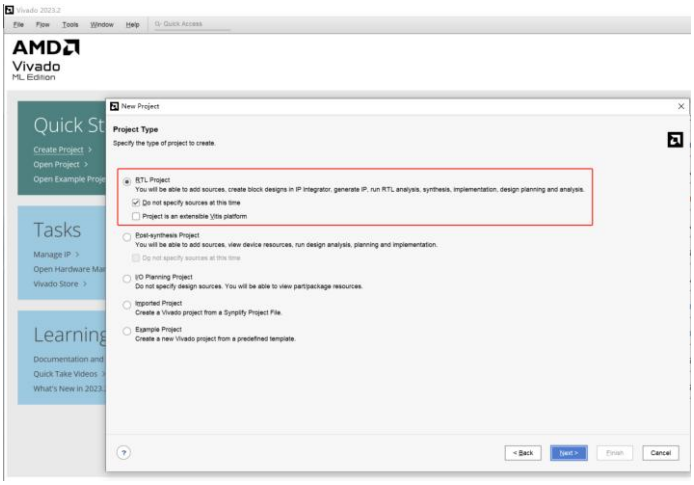
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]
```

3.5.3 工程搭建

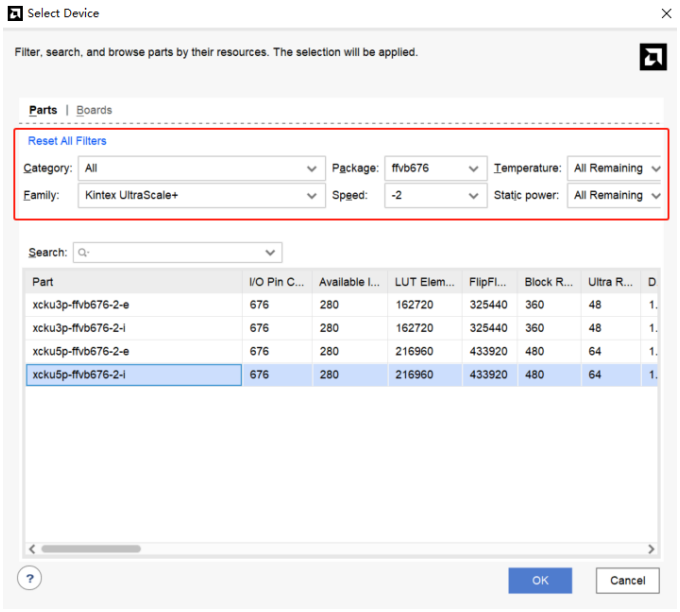
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



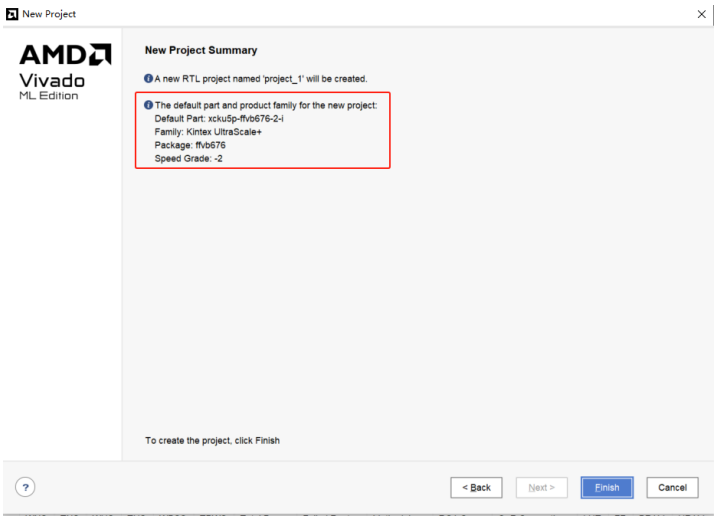
选择 RTL Project 并且勾选复选框，之后单击 NEXT



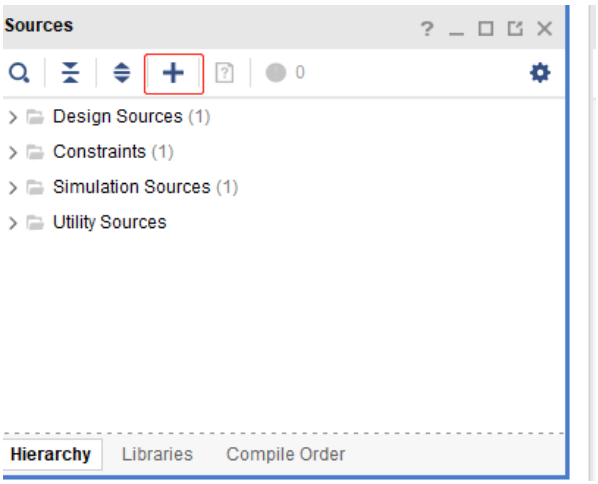
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



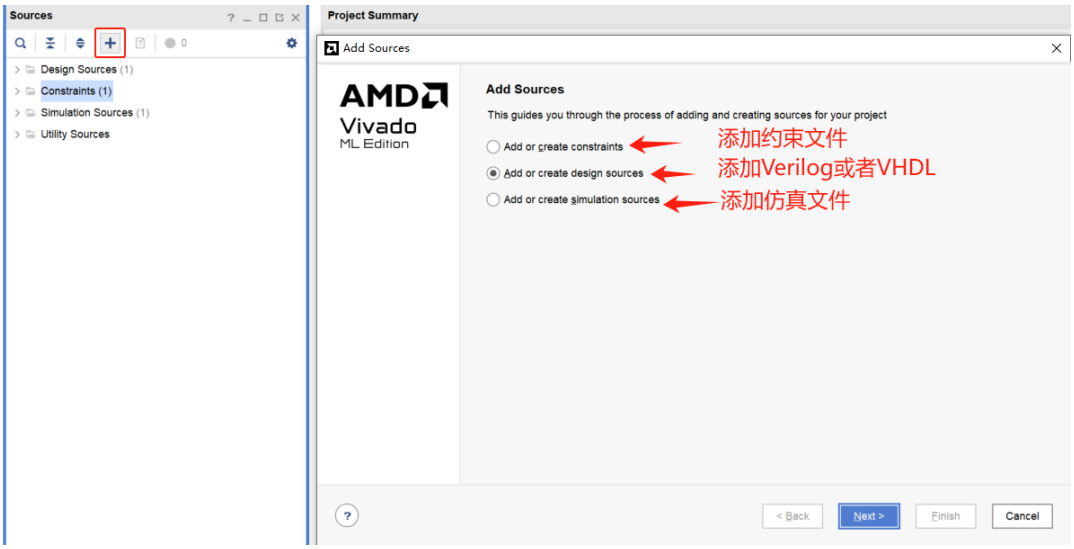
单击 Finish 完成工程创建

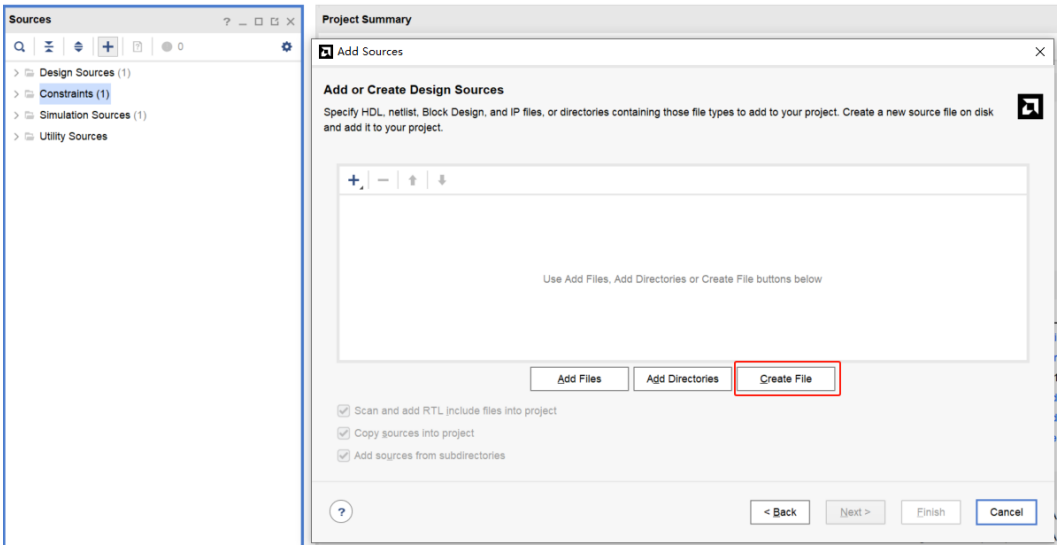


添加代码，显示界面如下

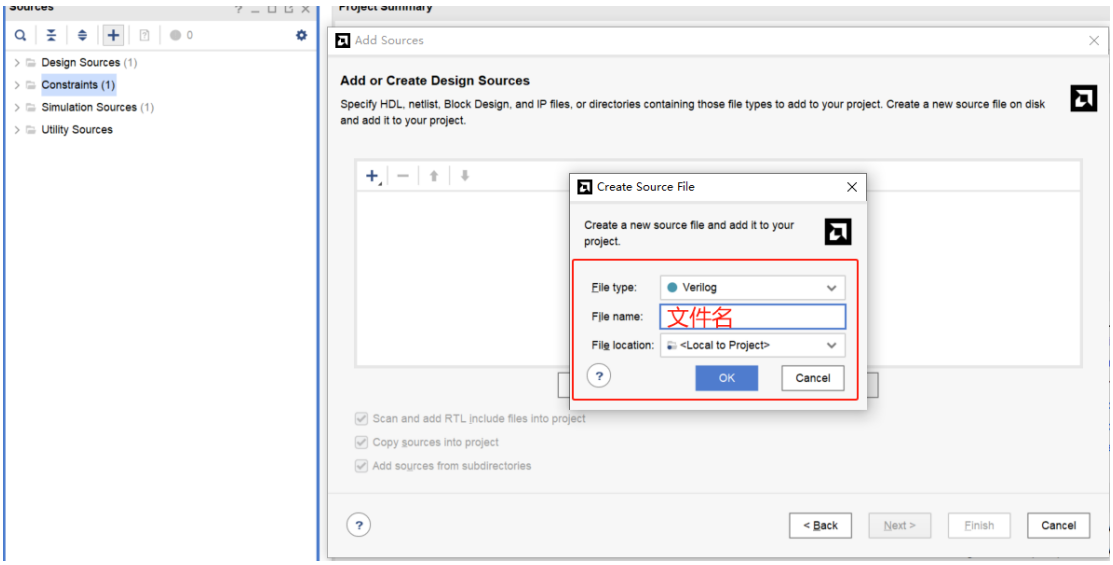


添加代码

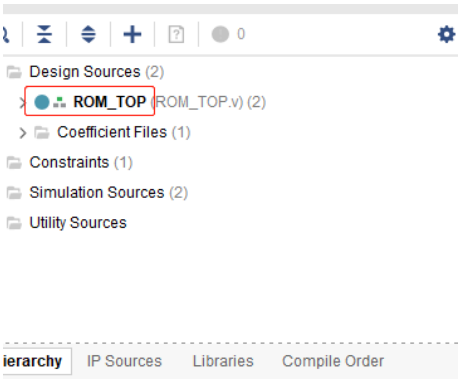




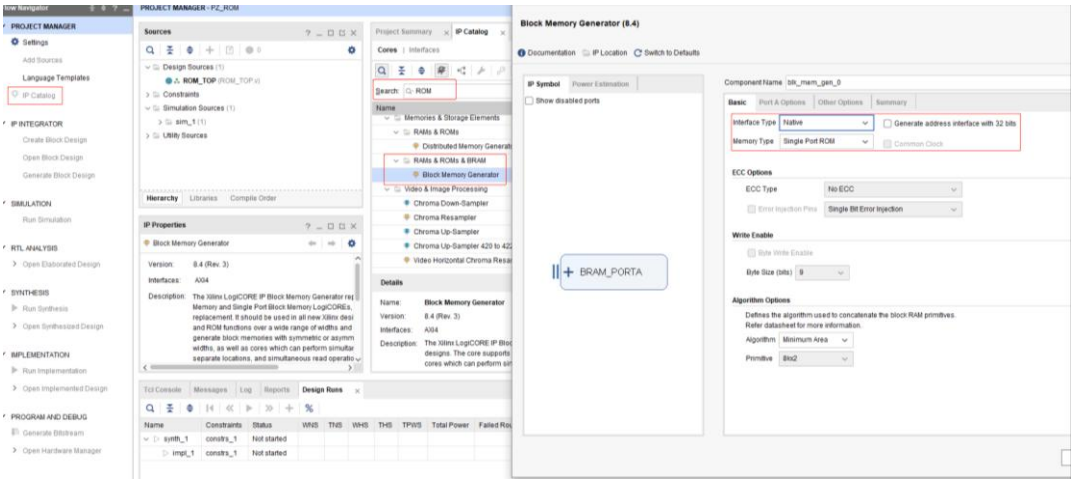
创建一个 ROM_TOP 文件，并且选择 Verilog 格式



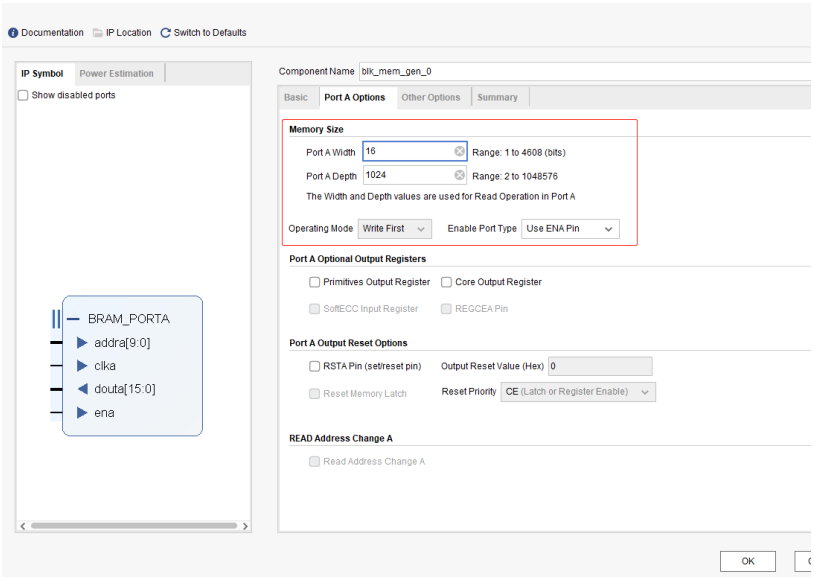
创建完成后，可以看到 Design Sources 文件夹中有 ROM_TOP.v 文件，这个文件就是我们可以编写 verilog 程序的文件



添加 ROM IP 到工程，如下图：



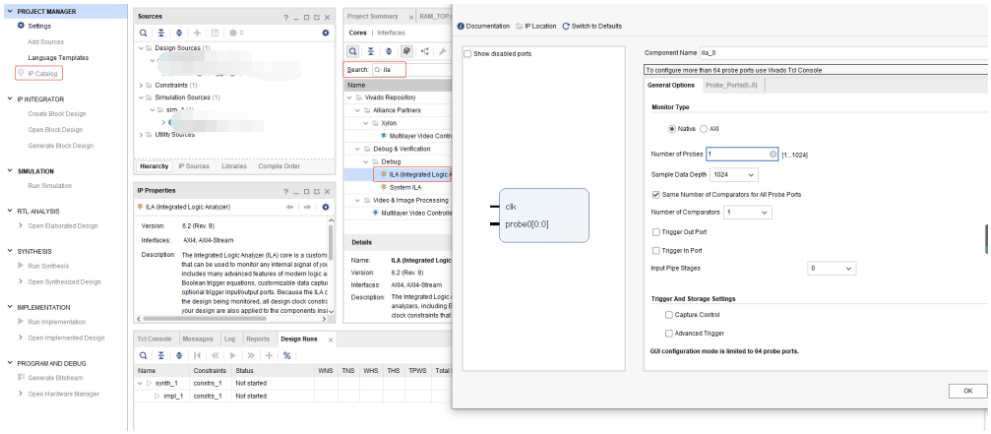
ROM IP 配置如下（其他默认）：



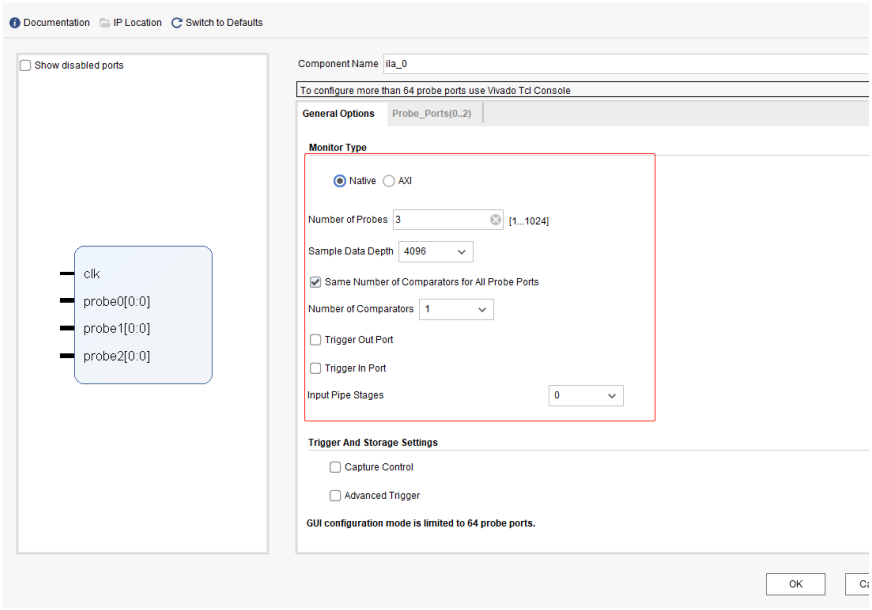
因为是 ROM，所以需要我们准备好数据，我们可以创建一个文件，名字为 rom_data.coe 的文件，注意后缀一定是“.coe”（该文件在工程文件夹下）。

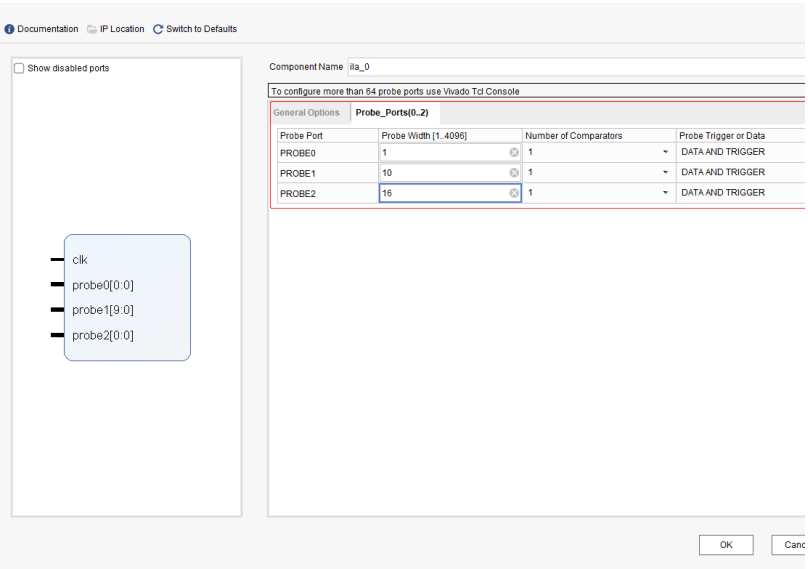
```
MEMORY_INITIALIZATION_RADIX=16;
MEMORY_INITIALIZATION_VECTOR=
FFFF,
FFC0,
FF80,
FF40,
FF00,
FEC0,
FE80,
FE40,
FE00,
FDC0,
FD80,
FD40,
FD00,
FCC0,
FC80,
FC40,
FC00,
FBC0,
FB80,
FB40,
FB00,
FAC0,
FA80,
FA40,
FA00,
F9C0,
F980,
F940,
F900,
F8C0,
F880,
F840,
F800,
F7C0,
F780,
F740,
F700,
F6C0,
F680,
```

添加 ILA IP 到工程，如下图：



ILA IP 配置如下（其他默认）：





双击 ROM_TOP 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2024/01/23 15:11:22
// Design Name:
// Module Name: ROM_TOP
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module ROM_TOP(
input sys_clk_p,
input sys_clk_n,
input sys_rstn
    );

   wire clk;
   IBUFDS #(
      .DIFF_TERM("FALSE"),
```

```
      .IBUF_LOW_PWR("TRUE"),
      .IOSTANDARD("DEFAULT")
   ) IBUFDS_inst (
      .O(clk),
      .I(sys_clk_p),
      .IB(sys_clk_n)
   );

wire ena;
reg  [9:0]addra;
wire [15:0]douta;

assign ena=sys_rstn;


always@(posedge clk)
    begin
    if(!sys_rstn)begin
        addra<=0;
    end
    else begin
        if(addra<1023)begin
            addra<=addra+1;
        end
        else begin
            addra<=0;
        end
    end
 end

blk_mem_gen_0 u0 (
  .clka(clk),    // input wire clka
  .ena(ena),      // input wire ena
  .addra(addra),  // input wire [9 : 0] addra
  .douta(douta)  // output wire [15 : 0] douta
);
ila_0 u1 (
    .clk(clk), // input wire clk
    .probe0(ena), // input wire [0:0]  probe0
    .probe1(addra), // input wire [9:0]  probe1
    .probe2(douta) // input wire [15:0]  probe2
);
endmodule
```

添加管脚约束文件




```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

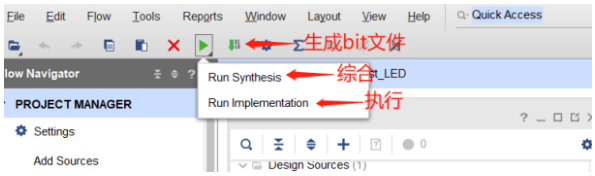
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]
```

编译并且产生 bit 文件

1:单击综合

2:单击执行

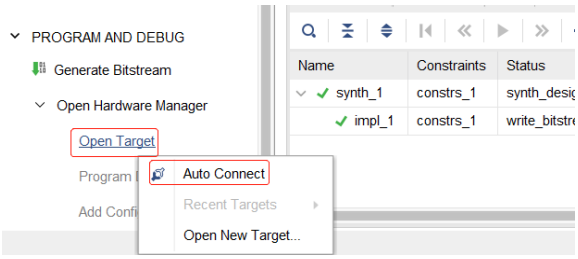
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



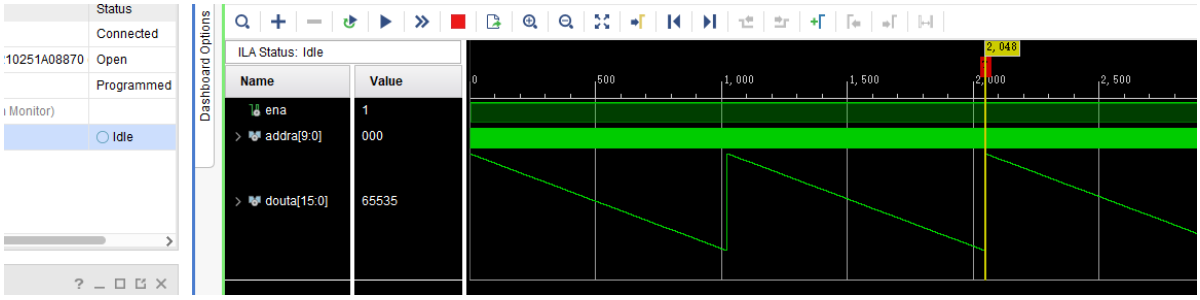
单击 Program Device（也可以从顶部单击 Program device）

弹出的对话框中有我们要下载的 Bit 文件



3.5.4 运行结果

下载完成后，自动跳出 ILA 界面，然后进行信号抓取，如下图



3.6 FPGA 片内 PLL 配置例程

3.6.1 功能概述

PLL 的英文全称是 Phase Locked Loop，即锁相环，是一种反馈控制电路。PLL 对时钟网络进行系统级的时钟管理和偏移控制，具有时钟倍频、分频、相位偏移和可编程占空比的功能。

本实例实现 PLL 输出 100M、20M 和 10M 的频率，然后通过 ILA（在线逻辑分析仪）进行抓取 20M 和 10M 的信，进行查看（100M 提供给 ILA 去抓取信号）。

3.6.2 管脚配置

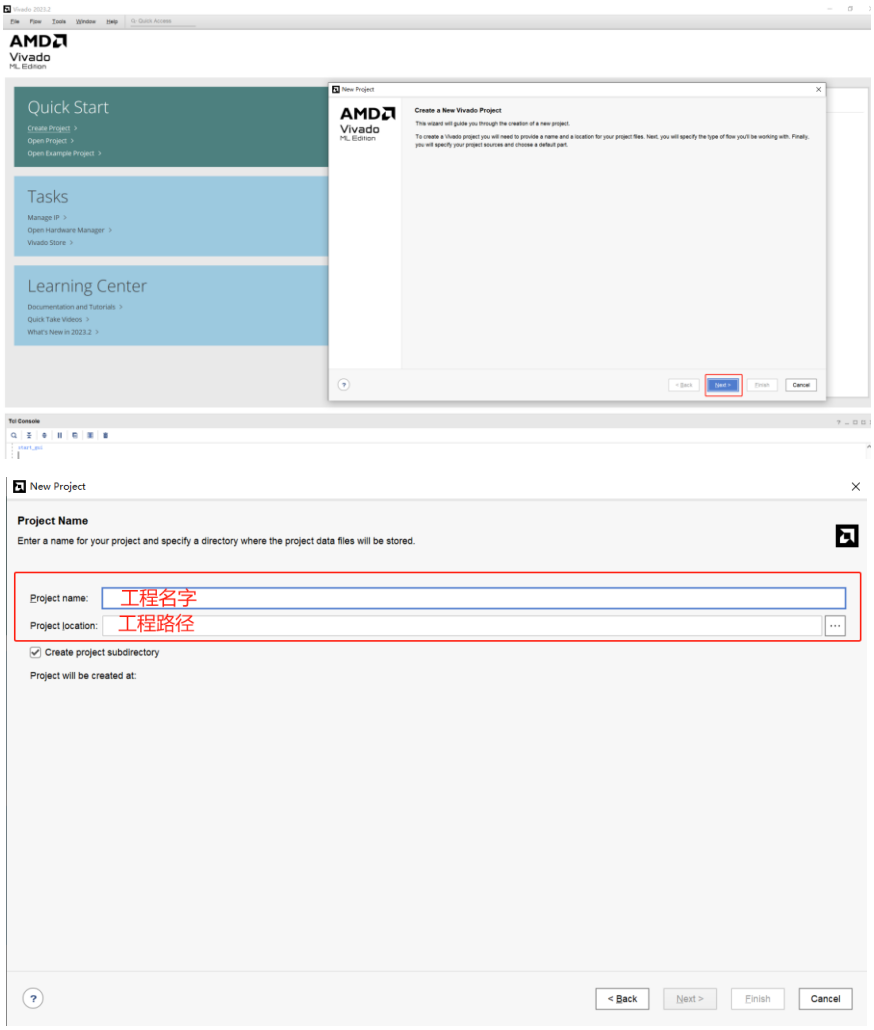
管脚配置如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

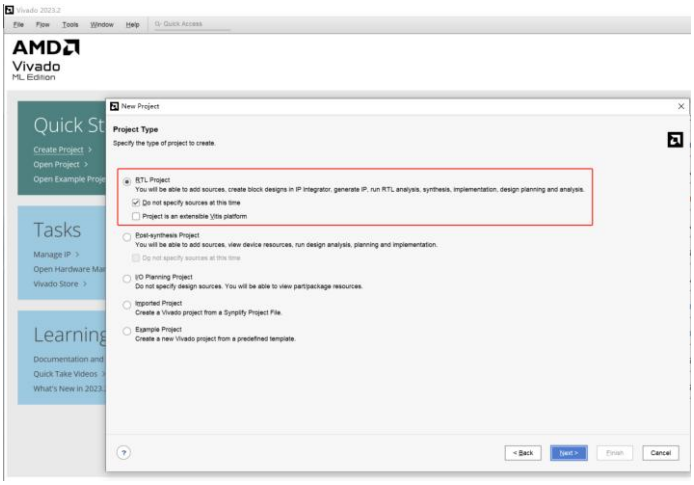
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]
```

3.6.3 工程搭建

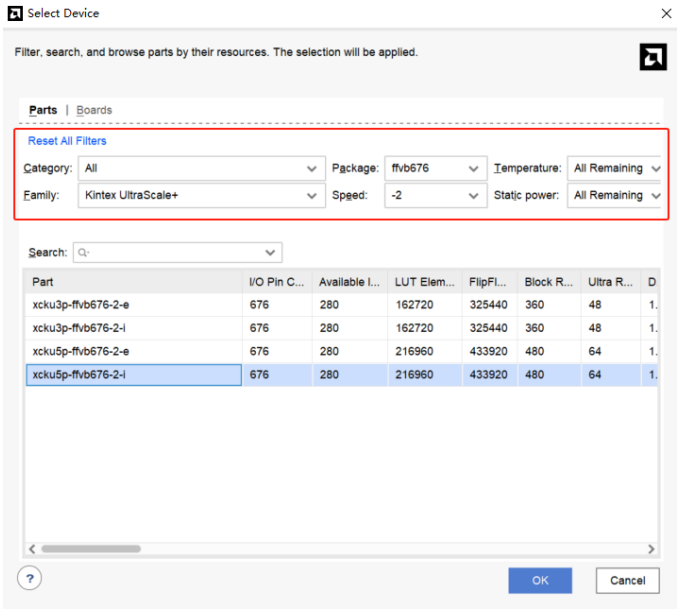
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



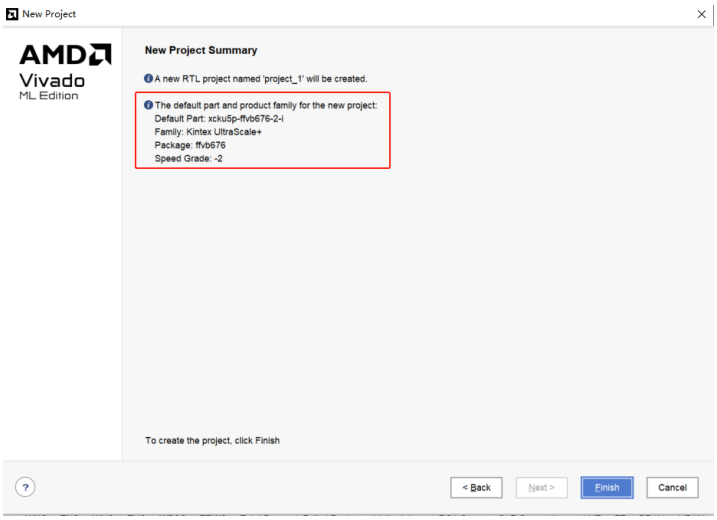
选择 RTL Project 并且勾选复选框，之后单击 NEXT



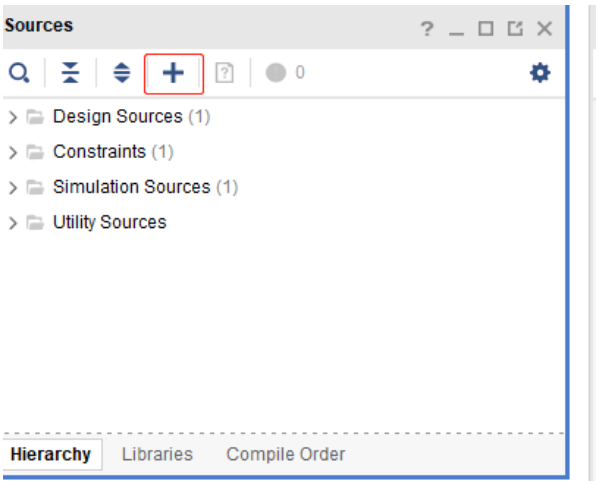
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



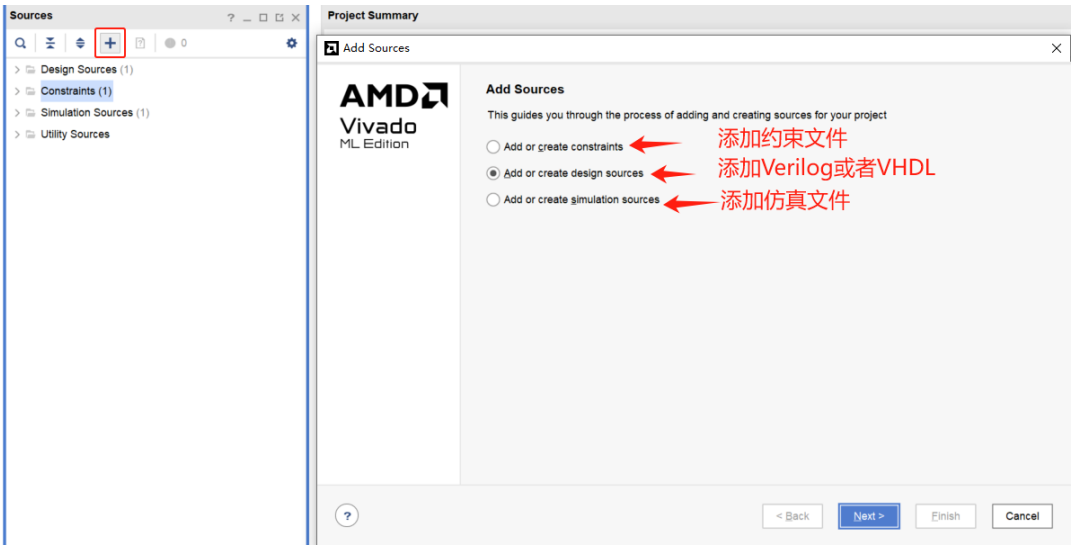
单击 Finish 完成工程创建

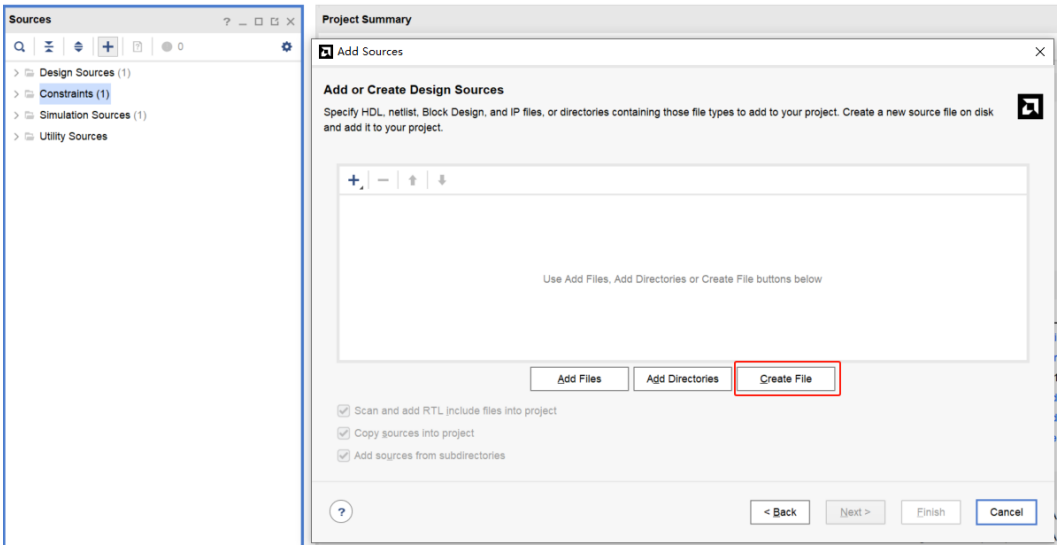


添加代码，显示界面如下

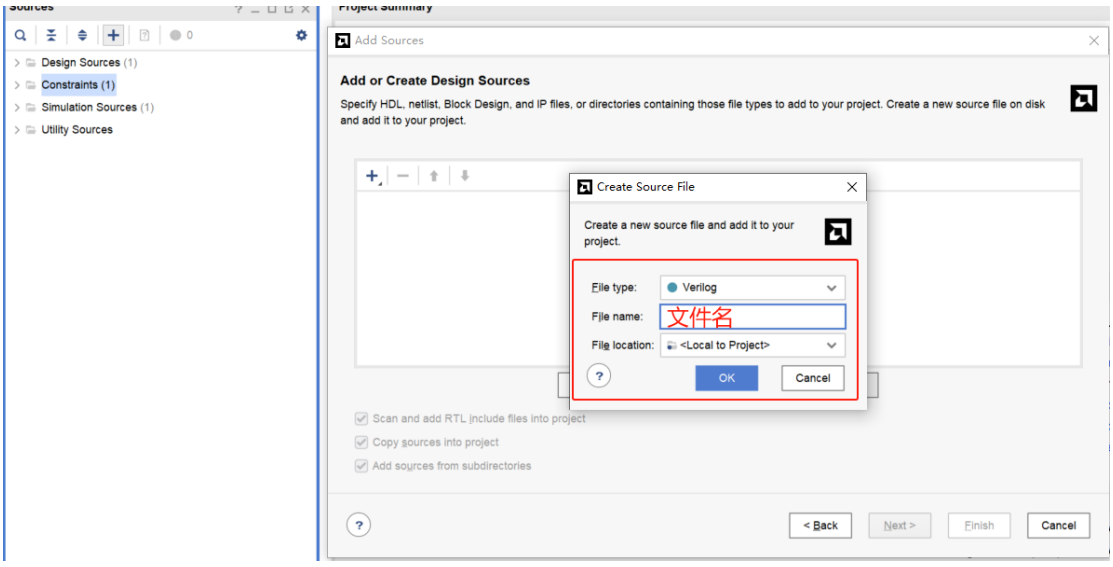


添加代码

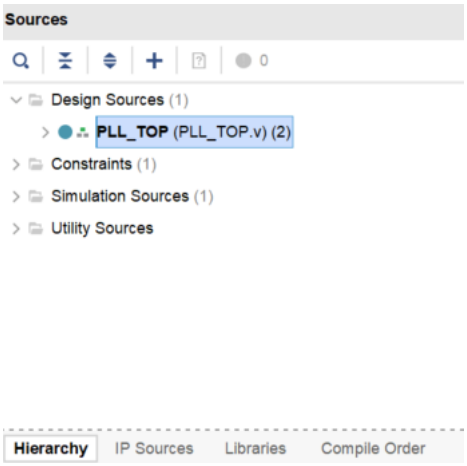




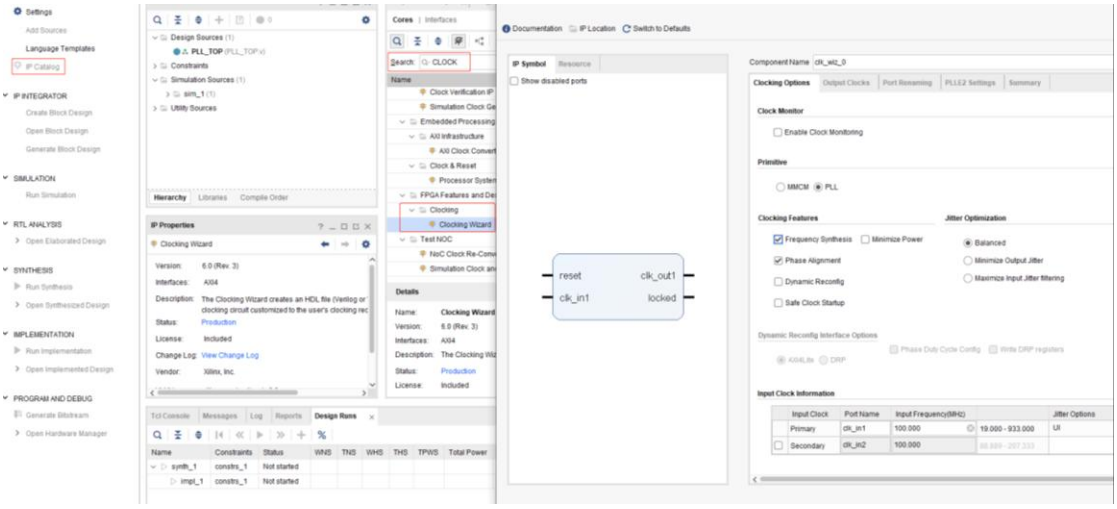
创建一个 PLL_TOP 文件，并且选择 Verilog 格式



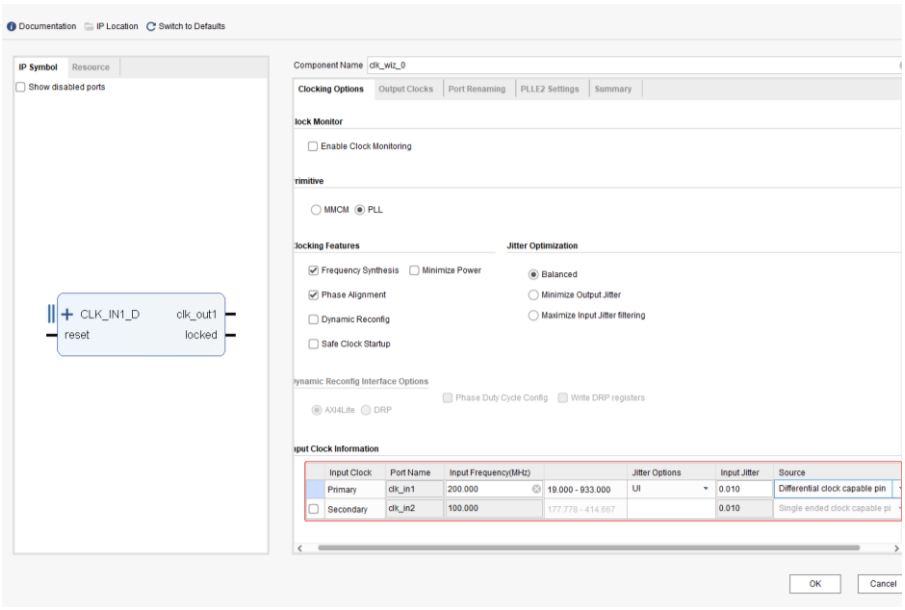
创建完成后，可以看到 Design Sources 文件夹中有 PLL_TOP.v 文件，这个文件就是我们可以编写 verilog 程序的文件

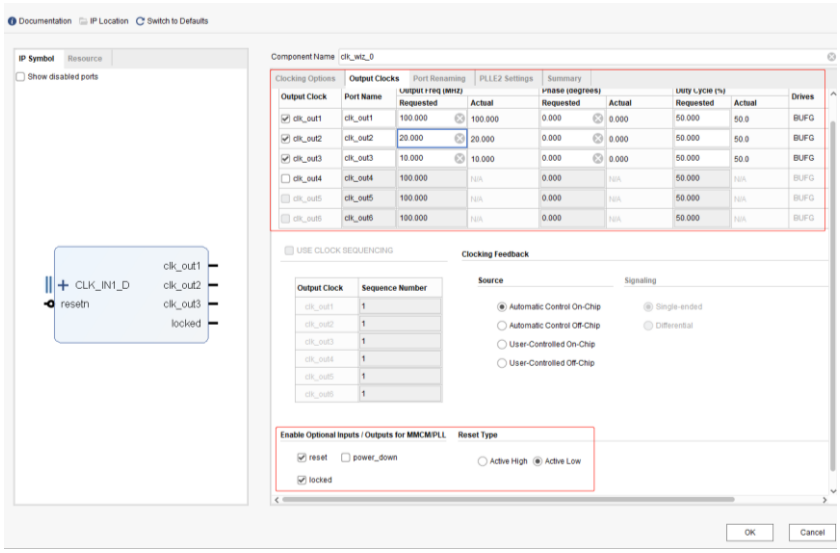


添加 CLOCKING WIZARD IP 到工程，如下图：

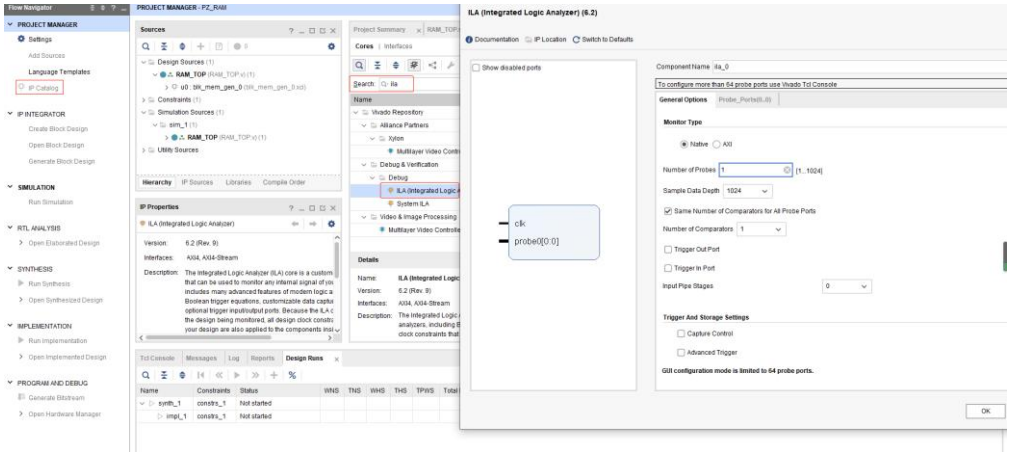


CLOCKING WIZARD IP 配置如下（其他默认）：

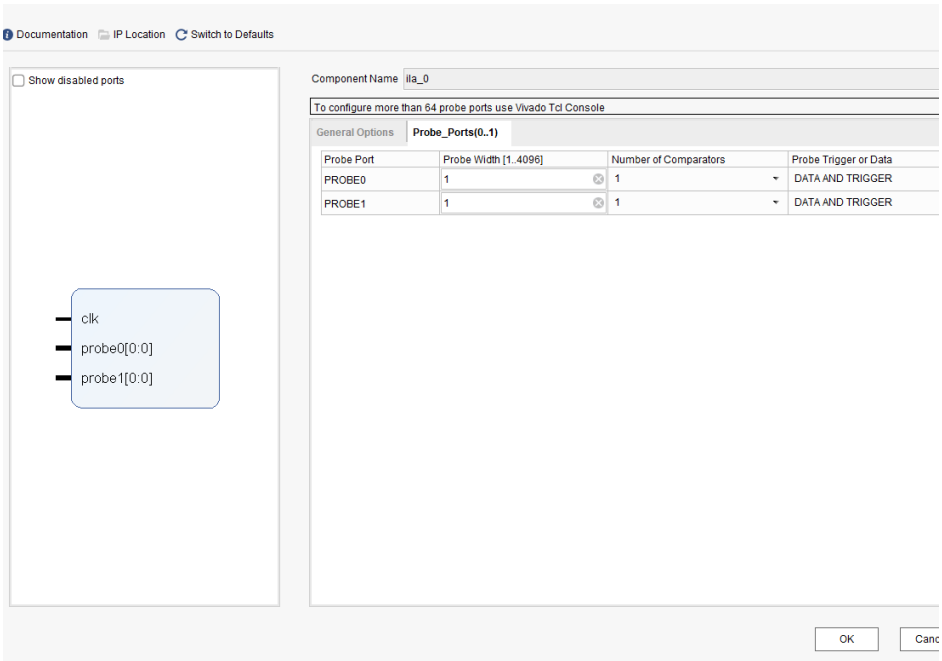




添加 ILA IP 到工程，如下图：



ILA IP 配置如下（其他默认）：



双击 PLL_TOP 文件，添加代码，代码如下：

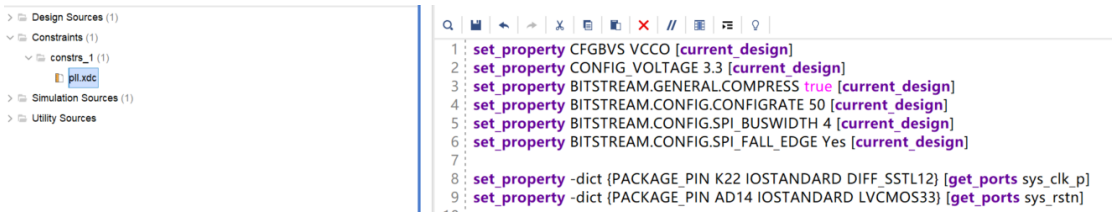
```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2024/01/23 17:22:18
// Design Name:
// Module Name: PLL_TOP
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module PLL_TOP(
input sys_clk_p,
input sys_clk_n,
input sys_rstn
    );

  wire sys_clk_100m;
  wire sys_clk_20m;
  wire sys_clk_10m;

  clk_wiz_0 u0
   (
    // Clock out ports
    .clk_out1(sys_clk_100m),     // output sys_clk_out1
    .clk_out2(sys_clk_20m),     // output sys_clk_out2
    .clk_out3(sys_clk_10m),     // output sys_clk_out3
    // Status and control signals
    .resetn(sys_rstn), // input resetn
    .locked(),       // output locked
   // Clock in ports
    .clk_in1_p(sys_clk_p),    // input sys_clk_in1_p
    .clk_in1_n(sys_clk_n));    // input sys_clk_in1_n
```

```
ila_0 u1(
    .clk(sys_clk_100m), // input wire clk
    .probe0(sys_clk_20m), // input wire [0:0]  probe0
    .probe1(sys_clk_10m)  // input wire [0:0]  probe1
);
endmodule
```

添加管脚约束文件

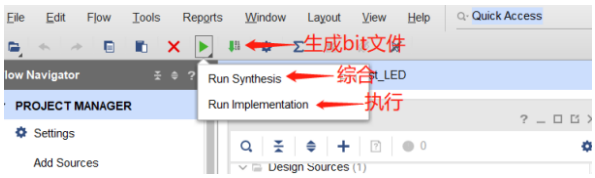


编译并且产生 bit 文件

1:单击综合

2:单击执行

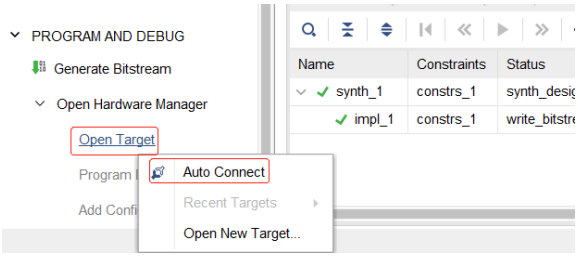
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

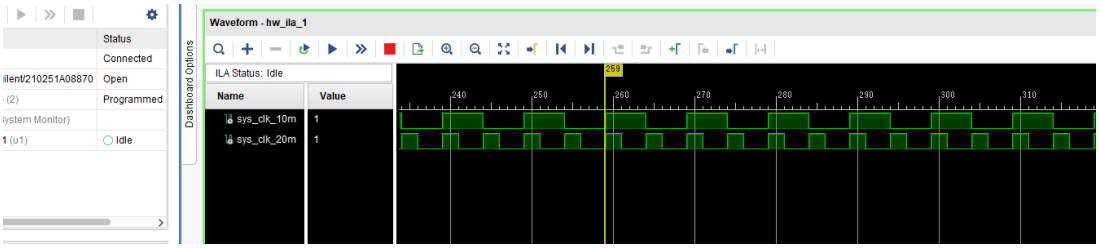


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.6.4 运行结果

下载完成后，自动跳出 ILA 界面，然后进行信号抓取，如下图



3.7 FPGA 片内 FIFO 读写例程

3.7.1 功能概述

FIFO 的英文全称是 First In First Out，即先进先出。FPGA 使用的 FIFO 一般指的是对数据的存储具有先进先出特性的一个缓存器，常被用于数据的缓存，或者高速异步数据的交互也即所谓的跨时钟域信号传递。

本实例实现往 FIFO 里面写满，然后再读取出来，通过 ILA（在线逻辑分析仪）进行查看。

3.7.2 管脚配置

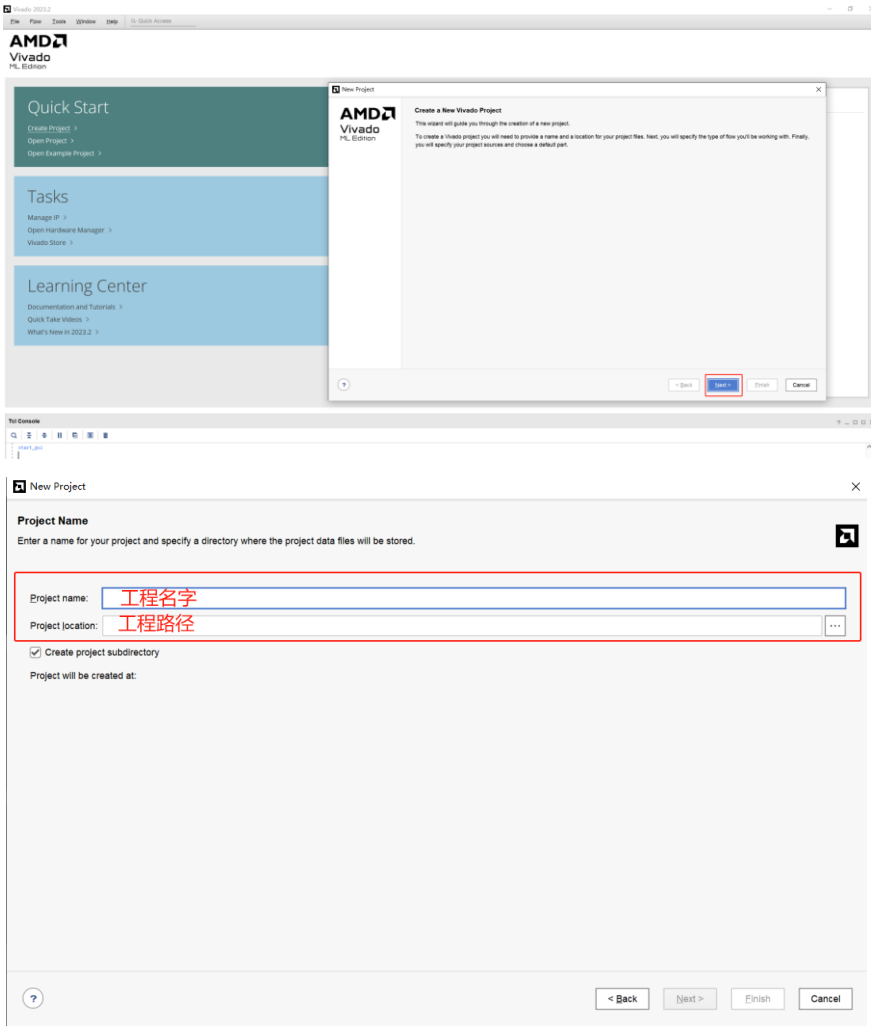
管脚配置如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

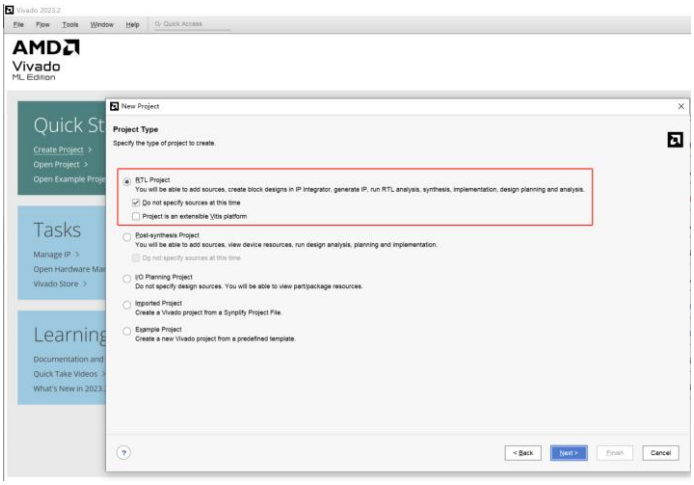
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]
```

3.7.3 工程搭建

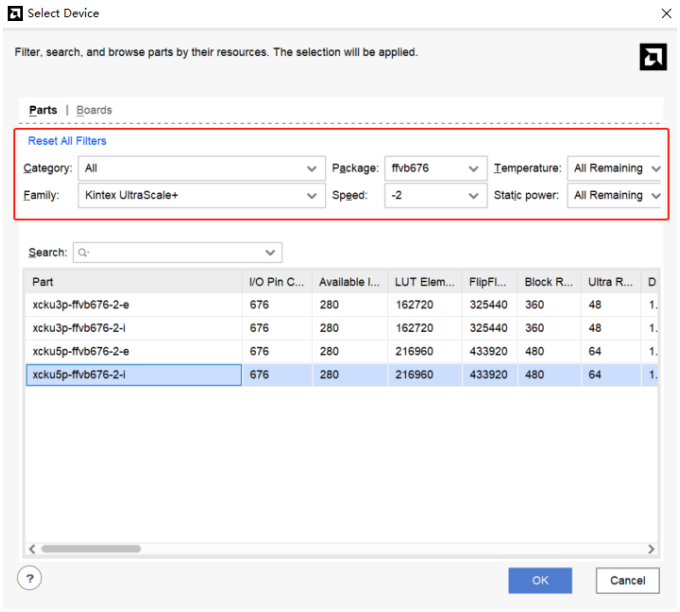
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



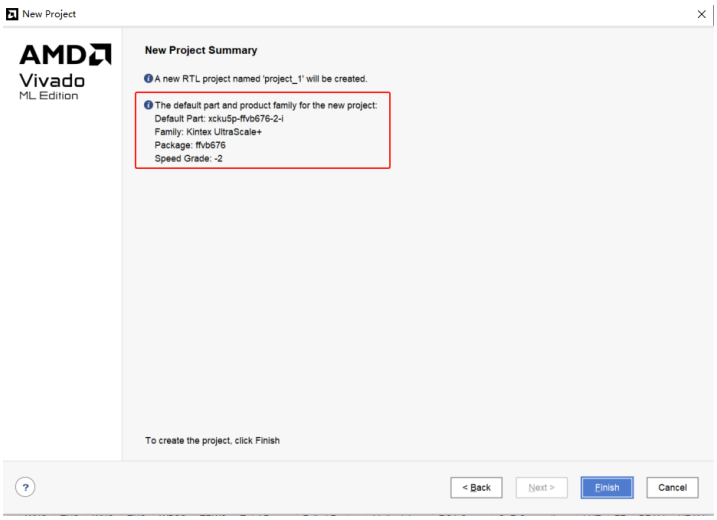
选择 RTL Project 并且勾选复选框，之后单击 NEXT



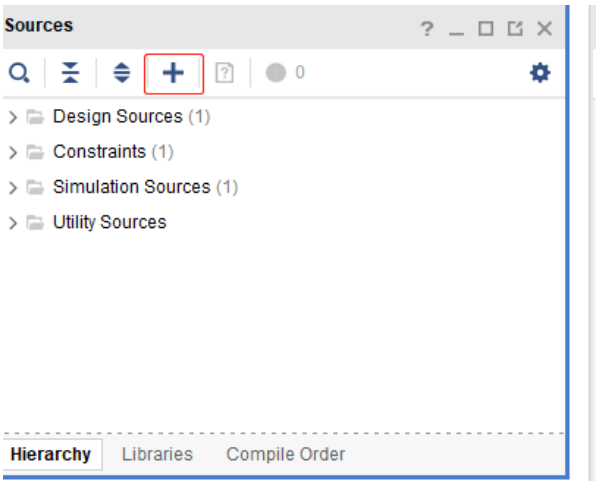
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



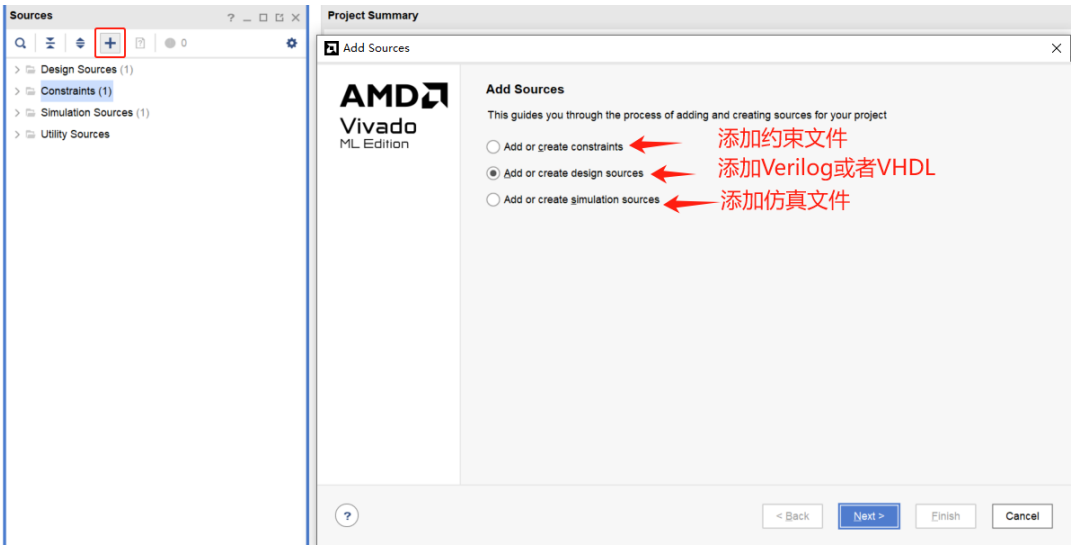
单击 Finish 完成工程创建

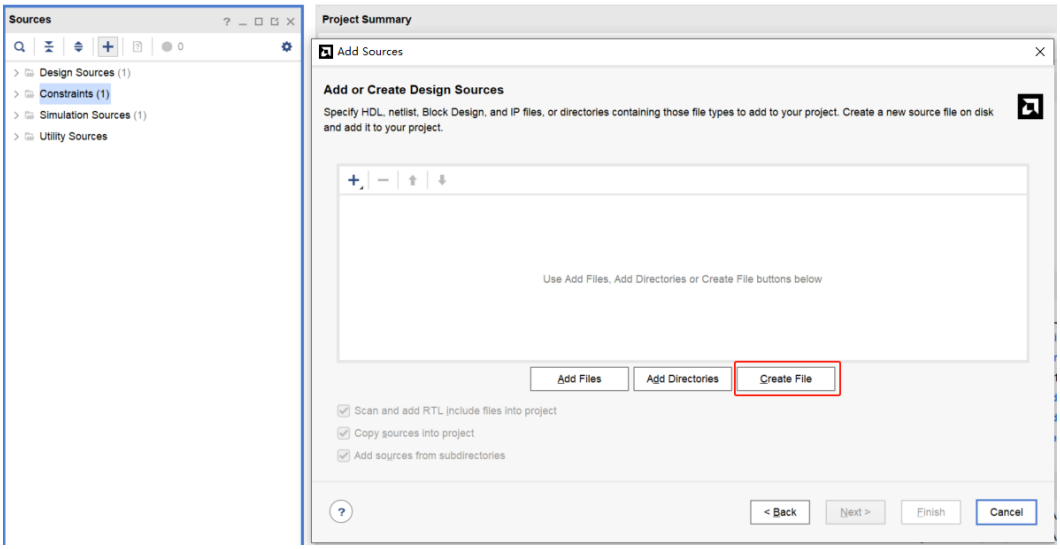


添加代码，显示界面如下

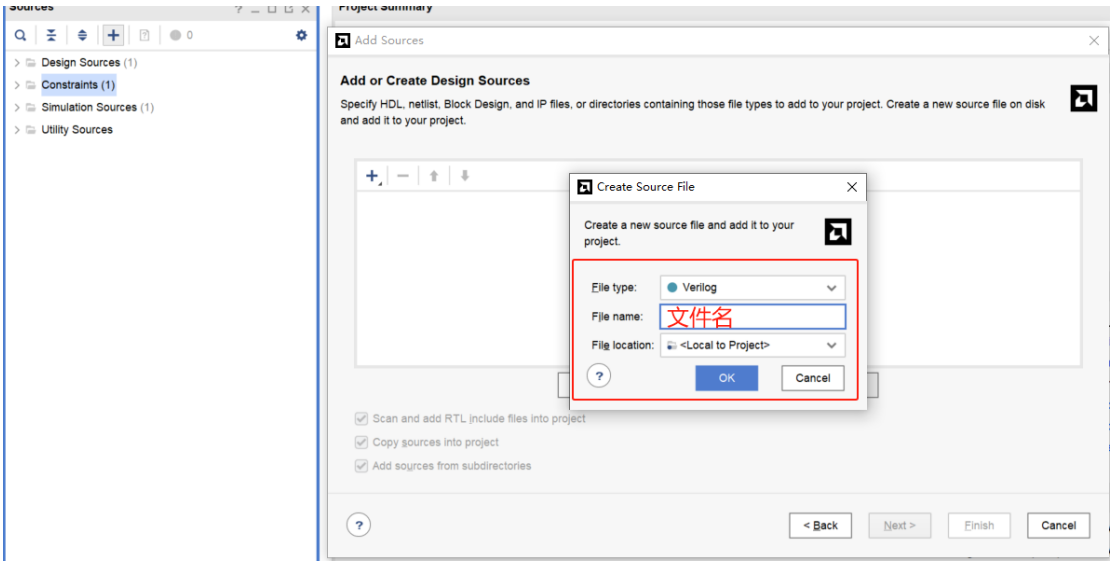


添加代码

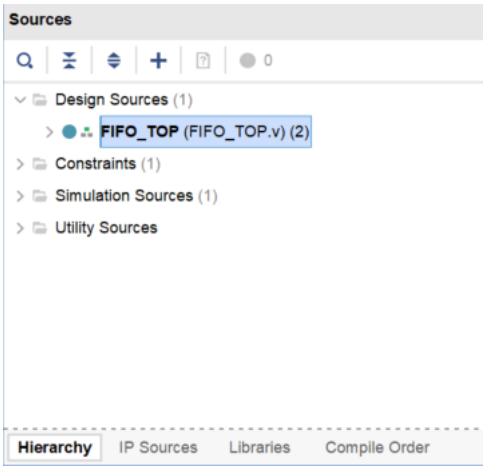




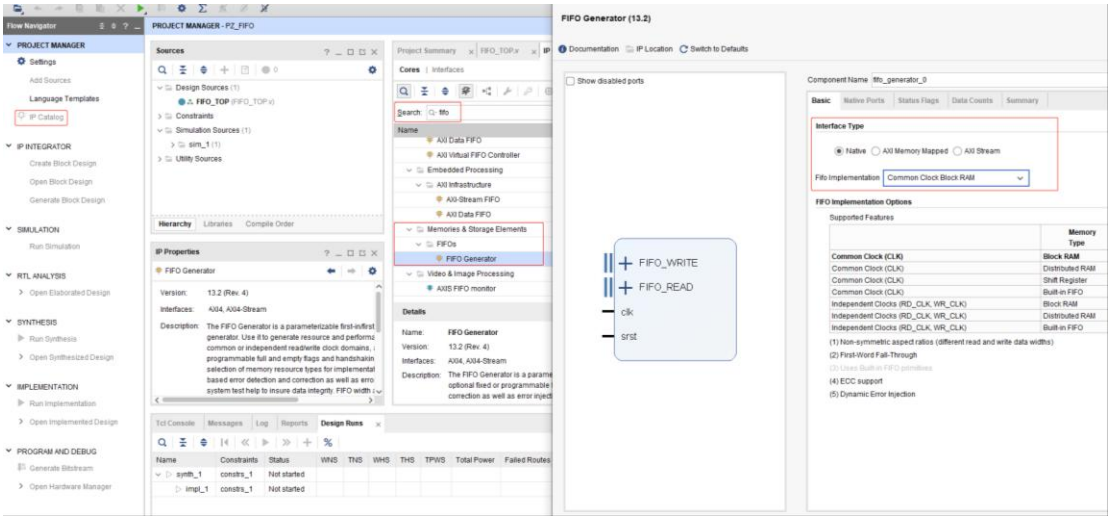
创建一个 FIFO_TOP 文件，并且选择 Verilog 格式



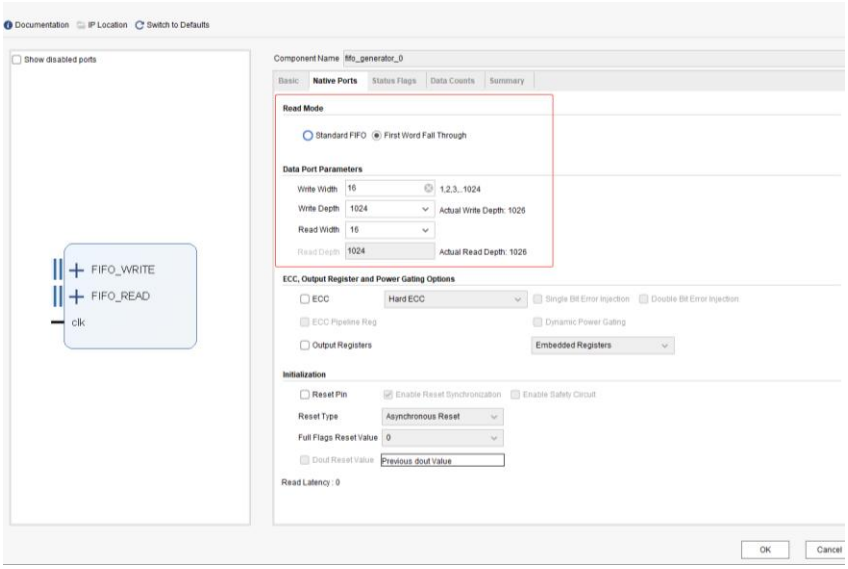
创建完成后，可以看到 Design Sources 文件夹中有 FIFO_TOP.v 文件，这个文件就是我们可以编写 verilog 程序的文件



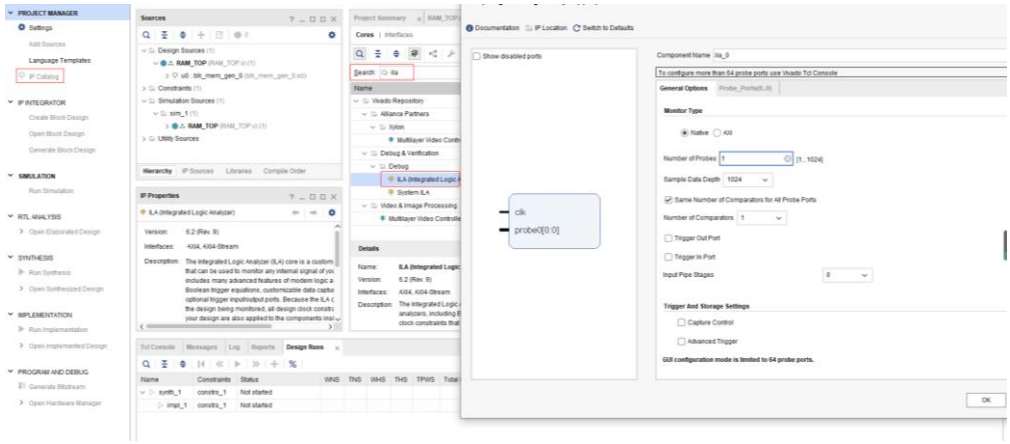
添加 FIFO IP 到工程，如下图：



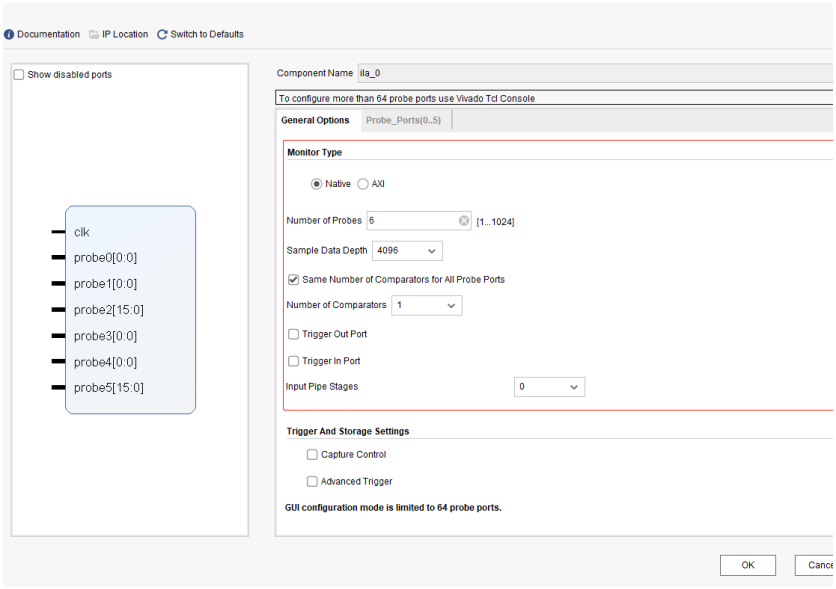
FIFO IP 配置如下（其他默认）：



添加 ILA IP 到工程，如下图：



ILA IP 配置如下（其他默认）：



双击 FIFO_TOP 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2020/09/03 16:09:43
// Design Name:
// Module Name: FIFO_TOP
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
```

```
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
    module FIFO_TOP(
    input sys_clk_p,
    input sys_clk_n,
    input sys_rstn
        );

       wire clk;
       IBUFDS #(
          .DIFF_TERM("FALSE"),
          .IBUF_LOW_PWR("TRUE"),
          .IOSTANDARD("DEFAULT")
       ) IBUFDS_inst (
          .O(clk),
          .I(sys_clk_p),
          .IB(sys_clk_n)
       );

     reg [15 : 0] din;
     reg wr_en;
     reg rd_en;
     wire [15 : 0] dout;
     wire full;
     wire empty;
     reg  [1:0]fifo_state;

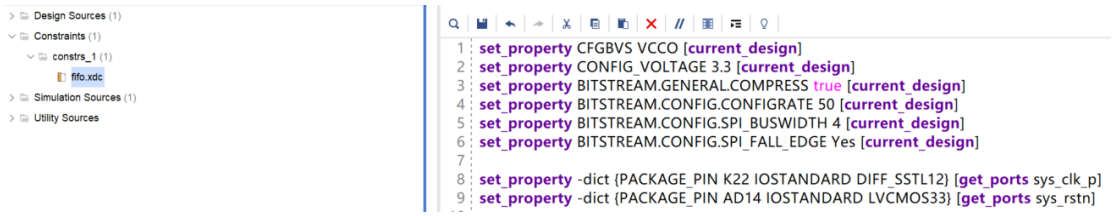
       always@(posedge clk)
            if(!sys_rstn)begin
                din<=0;
                wr_en<=0;
                rd_en<=0;
                fifo_state<=0;
            end
            else begin
                case(fifo_state)
                0:begin
                    if(!full)begin
                        wr_en<=1;
                        din<=din+1;
                    end
```

```
                else begin
                    wr_en<=0;
                    din<=0;
                    fifo_state<=1;
                end
            end
            1:begin
                if(!empty)begin
                    rd_en<=1;
                end
                else begin
                    rd_en<=0;
                    fifo_state<=0;
                end
            end
            default:begin
                 din<=0;
                 wr_en<=0;
                 rd_en<=0;
                 fifo_state<=0;
            end
            endcase
        end

fifo_generator_0 u0(
  .clk(clk),      // input wire clk
  .din(din),      // input wire [15 : 0] din
  .wr_en(wr_en),  // input wire wr_en
  .rd_en(rd_en),  // input wire rd_en
  .dout(dout),    // output wire [15 : 0] dout
  .full(full),    // output wire full
  .empty(empty)  // output wire empty
);

ila_0 u1(
   .clk(clk), // input wire clk
   .probe0(din), // input wire [15:0]  probe0
   .probe1(wr_en), // input wire [0:0]  probe1
   .probe2(rd_en), // input wire [0:0]  probe2
   .probe3(dout), // input wire [15:0]  probe3
   .probe4(full), // input wire [0:0]  probe4
   .probe5(empty) // input wire [0:0]  probe5
);
endmodule
```

添加管脚约束文件

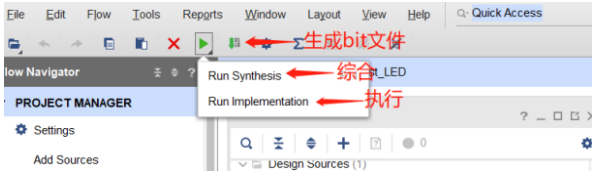


编译并且产生 bit 文件

1:单击综合

2:单击执行

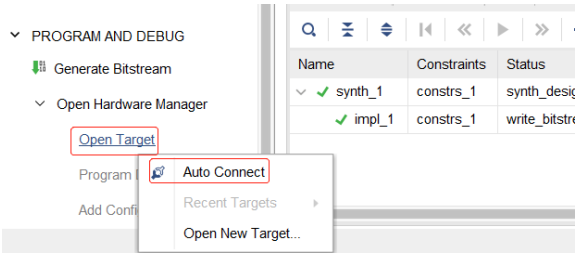
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

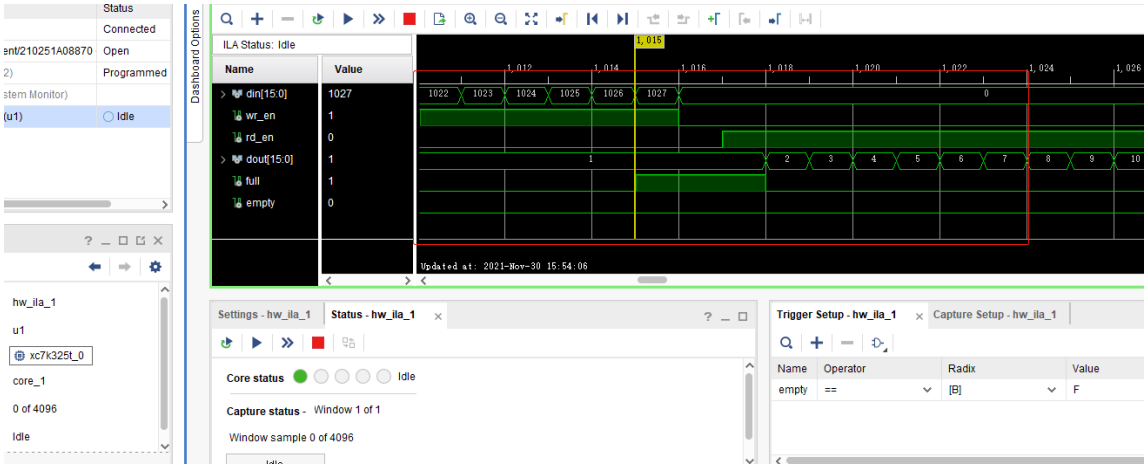


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.7.4 运行结果

下载完成后，自动跳出 ILA 界面，然后进行信号抓取，如下图



3.8 FPGA 侧 DDR4 控制读写例程

3.8.1 功能概述

DDR=Double Data Rate 双倍速率,DDR SDRAM=双倍速率同步动态随机存储器，人们习惯称为 DDR，其中 SDRAM 是 Synchronous Dynamic Random Access Memory 的缩写，即同步动态随机存取存储器。

DDR SDRAM 是 Double Data Rate SDRAM 的缩写，是双倍速率同步动态随机存储器的意思。

DDR 内存是在 SDRAM 内存基础上发展而来的，仍然沿用 SDRAM 生产体系，因此对于内存厂商而言，只需对制造普通 SDRAM 的设备稍加改进，即可实现 DDR 内存的生产，可有效的降低成本。

本实例实现按照测试的长度往 DDR4 里面写值，然后再读取出来进行对比，通过 ILA（在线逻辑分析仪）进行查看。

3.8.2 管脚配置

管脚配置如下：

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property IOSTANDARD DIFF_SSTL12 [get_ports c0_sys_clk_p]
set_property PACKAGE_PIN K22 [get_ports c0_sys_clk_p]

set_property PACKAGE_PIN AD14  [get_ports sys_rstn]
set_property IOSTANDARD LVCMOS33 [get_ports sys_rstn]


set_property PACKAGE_PIN H23 [get_ports {c0_ddr4_cs_n[0]}]
set_property PACKAGE_PIN G24 [get_ports {c0_ddr4_ck_t[0]}]
set_property PACKAGE_PIN G25 [get_ports {c0_ddr4_ck_c[0]}]
set_property PACKAGE_PIN H24 [get_ports {c0_ddr4_cke[0]}]
set_property PACKAGE_PIN L25 [get_ports c0_ddr4_reset_n]
set_property PACKAGE_PIN L24 [get_ports c0_ddr4_act_n]
set_property PACKAGE_PIN H21 [get_ports {c0_ddr4_ba[1]}]
```

```
set_property PACKAGE_PIN J25 [get_ports {c0_ddr4_ba[0]}]
set_property PACKAGE_PIN M24 [get_ports {c0_ddr4_odt[0]}]
set_property PACKAGE_PIN G26 [get_ports {c0_ddr4_bg[0]}]

set_property PACKAGE_PIN  F23 [get_ports {c0_ddr4_adr[0]}]
set_property PACKAGE_PIN  D25 [get_ports {c0_ddr4_adr[1]}]
set_property PACKAGE_PIN  D23 [get_ports {c0_ddr4_adr[2]}]
set_property PACKAGE_PIN  B26 [get_ports {c0_ddr4_adr[3]}]
set_property PACKAGE_PIN  E23 [get_ports {c0_ddr4_adr[4]}]
set_property PACKAGE_PIN  C26 [get_ports {c0_ddr4_adr[5]}]
set_property PACKAGE_PIN  F24 [get_ports {c0_ddr4_adr[6]}]
set_property PACKAGE_PIN  D26 [get_ports {c0_ddr4_adr[7]}]
set_property PACKAGE_PIN  H26 [get_ports {c0_ddr4_adr[8]}]
set_property PACKAGE_PIN  E25 [get_ports {c0_ddr4_adr[9]}]
set_property PACKAGE_PIN  C24 [get_ports {c0_ddr4_adr[10]}]
set_property PACKAGE_PIN  F22 [get_ports {c0_ddr4_adr[11]}]
set_property PACKAGE_PIN  B25 [get_ports {c0_ddr4_adr[12]}]
set_property PACKAGE_PIN  E26 [get_ports {c0_ddr4_adr[13]}]
set_property PACKAGE_PIN  D24 [get_ports {c0_ddr4_adr[14]}]
set_property PACKAGE_PIN  J23 [get_ports {c0_ddr4_adr[15]}]
set_property PACKAGE_PIN  G22 [get_ports {c0_ddr4_adr[16]}]


set_property PACKAGE_PIN C21 [get_ports {c0_ddr4_dqs_t[0]}]
set_property PACKAGE_PIN B21 [get_ports {c0_ddr4_dqs_c[0]}]
set_property PACKAGE_PIN A17 [get_ports {c0_ddr4_dqs_t[1]}]
set_property PACKAGE_PIN A18 [get_ports {c0_ddr4_dqs_c[1]}]
set_property PACKAGE_PIN F20 [get_ports {c0_ddr4_dqs_t[2]}]
set_property PACKAGE_PIN E20 [get_ports {c0_ddr4_dqs_c[2]}]
set_property PACKAGE_PIN E16 [get_ports {c0_ddr4_dqs_t[3]}]
set_property PACKAGE_PIN E17 [get_ports {c0_ddr4_dqs_c[3]}]

set_property PACKAGE_PIN  A22 [get_ports {c0_ddr4_dm_dbi_n[0]}]
set_property PACKAGE_PIN  C18 [get_ports {c0_ddr4_dm_dbi_n[1]}]
set_property PACKAGE_PIN  H18 [get_ports {c0_ddr4_dm_dbi_n[2]}]
set_property PACKAGE_PIN  G15 [get_ports {c0_ddr4_dm_dbi_n[3]}]



set_property PACKAGE_PIN C23  [get_ports {c0_ddr4_dq[0]}]
set_property PACKAGE_PIN B22  [get_ports {c0_ddr4_dq[1]}]
set_property PACKAGE_PIN A24  [get_ports {c0_ddr4_dq[2]}]
set_property PACKAGE_PIN C22  [get_ports {c0_ddr4_dq[3]}]
set_property PACKAGE_PIN B24  [get_ports {c0_ddr4_dq[4]}]
```

```
set_property PACKAGE_PIN D21  [get_ports {c0_ddr4_dq[5]}]
set_property PACKAGE_PIN A25  [get_ports {c0_ddr4_dq[6]}]
set_property PACKAGE_PIN E21  [get_ports {c0_ddr4_dq[7]}]

set_property PACKAGE_PIN A19  [get_ports {c0_ddr4_dq[8]}]
set_property PACKAGE_PIN B17  [get_ports {c0_ddr4_dq[9]}]
set_property PACKAGE_PIN A20  [get_ports {c0_ddr4_dq[10]}]
set_property PACKAGE_PIN C17  [get_ports {c0_ddr4_dq[11]}]
set_property PACKAGE_PIN B20  [get_ports {c0_ddr4_dq[12]}]
set_property PACKAGE_PIN B15  [get_ports {c0_ddr4_dq[13]}]
set_property PACKAGE_PIN B19  [get_ports {c0_ddr4_dq[14]}]
set_property PACKAGE_PIN A15  [get_ports {c0_ddr4_dq[15]}]

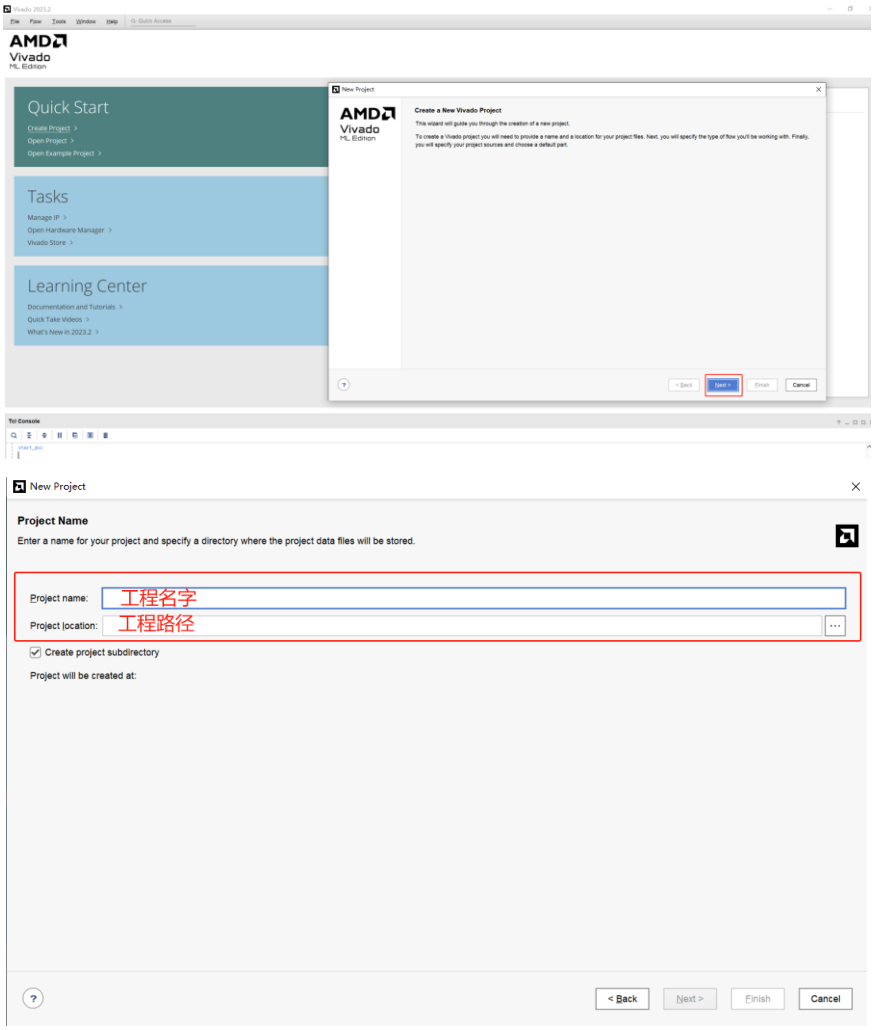
set_property PACKAGE_PIN D19 [get_ports {c0_ddr4_dq[16]}]
set_property PACKAGE_PIN F19 [get_ports {c0_ddr4_dq[17]}]
set_property PACKAGE_PIN D20 [get_ports {c0_ddr4_dq[18]}]
set_property PACKAGE_PIN G20 [get_ports {c0_ddr4_dq[19]}]
set_property PACKAGE_PIN G21 [get_ports {c0_ddr4_dq[20]}]
set_property PACKAGE_PIN F18 [get_ports {c0_ddr4_dq[21]}]
set_property PACKAGE_PIN D18 [get_ports {c0_ddr4_dq[22]}]
set_property PACKAGE_PIN E18 [get_ports {c0_ddr4_dq[23]}]

set_property PACKAGE_PIN E15 [get_ports {c0_ddr4_dq[24]}]
set_property PACKAGE_PIN G16 [get_ports {c0_ddr4_dq[25]}]
set_property PACKAGE_PIN D15 [get_ports {c0_ddr4_dq[26]}]
set_property PACKAGE_PIN G17 [get_ports {c0_ddr4_dq[27]}]
set_property PACKAGE_PIN D16 [get_ports {c0_ddr4_dq[28]}]
set_property PACKAGE_PIN H17 [get_ports {c0_ddr4_dq[29]}]
set_property PACKAGE_PIN C16 [get_ports {c0_ddr4_dq[30]}]
set_property PACKAGE_PIN H16 [get_ports {c0_ddr4_dq[31]}]

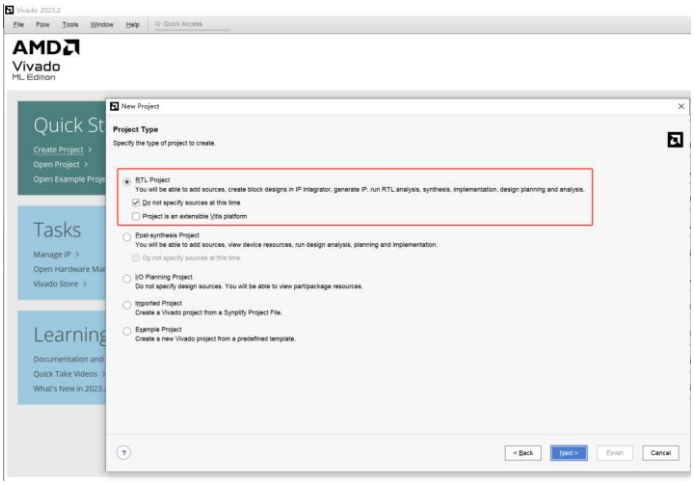

set_property INTERNAL_VREF 0.6 [get_iobanks 64]
```

3.8.3 工程搭建

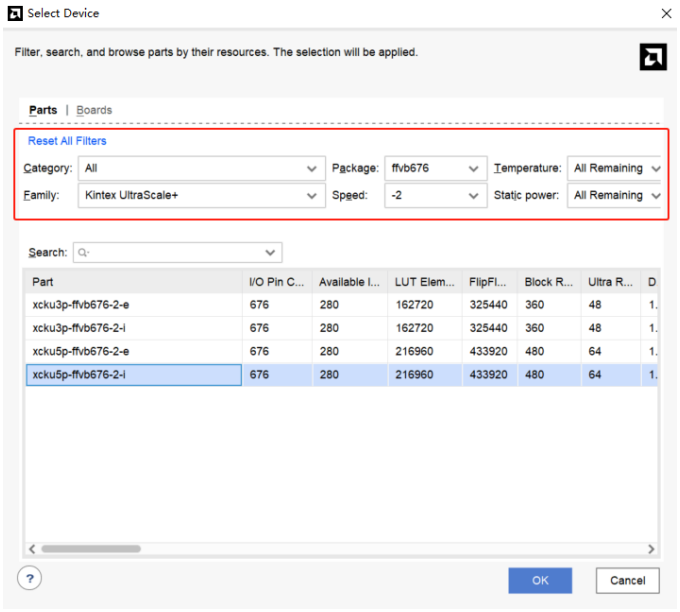
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



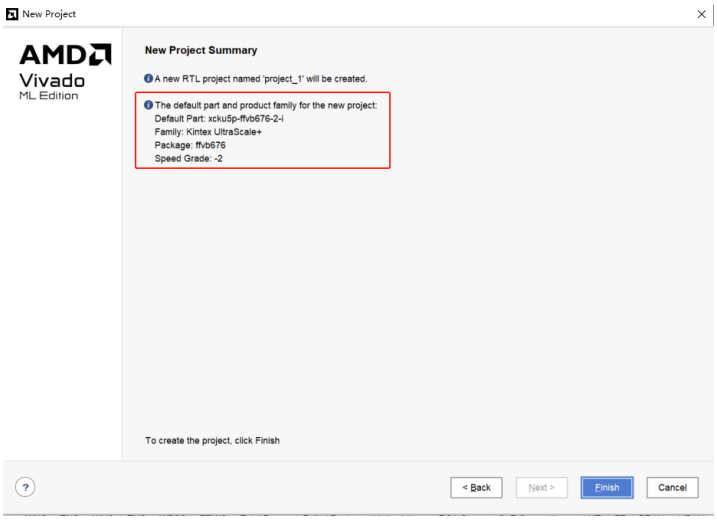
选择 RTL Project 并且勾选复选框，之后单击 NEXT



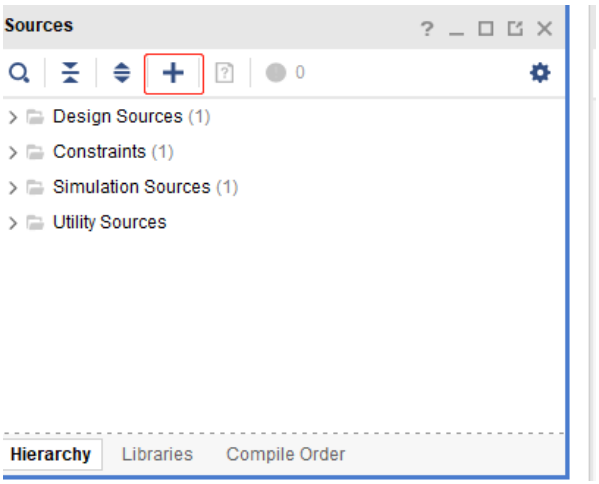
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



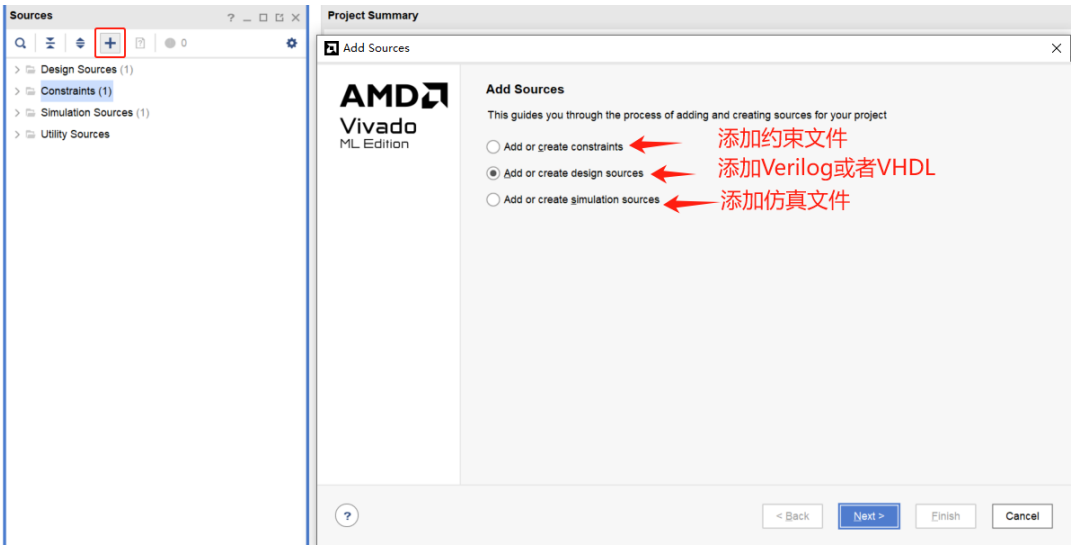
单击 Finish 完成工程创建

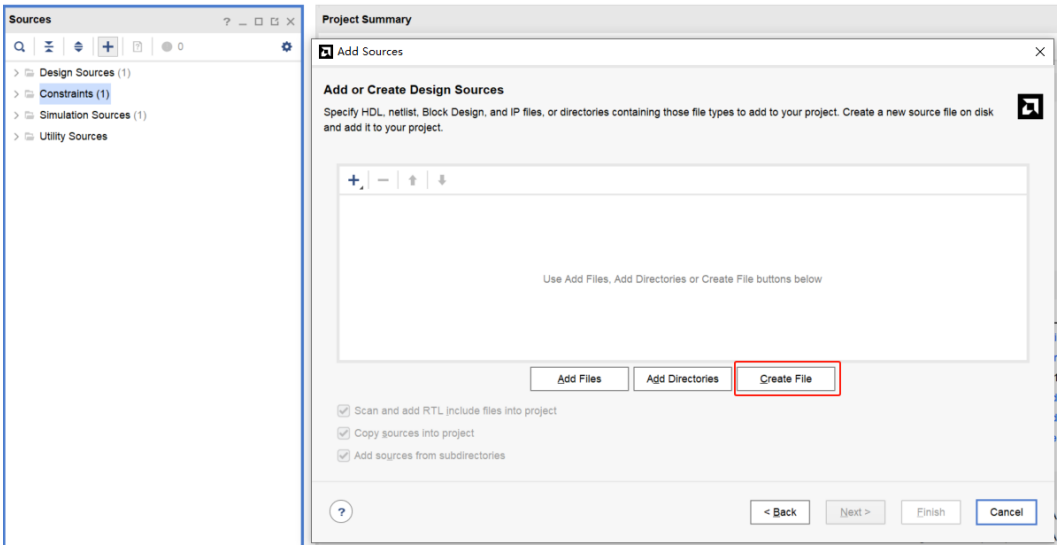


添加代码，显示界面如下

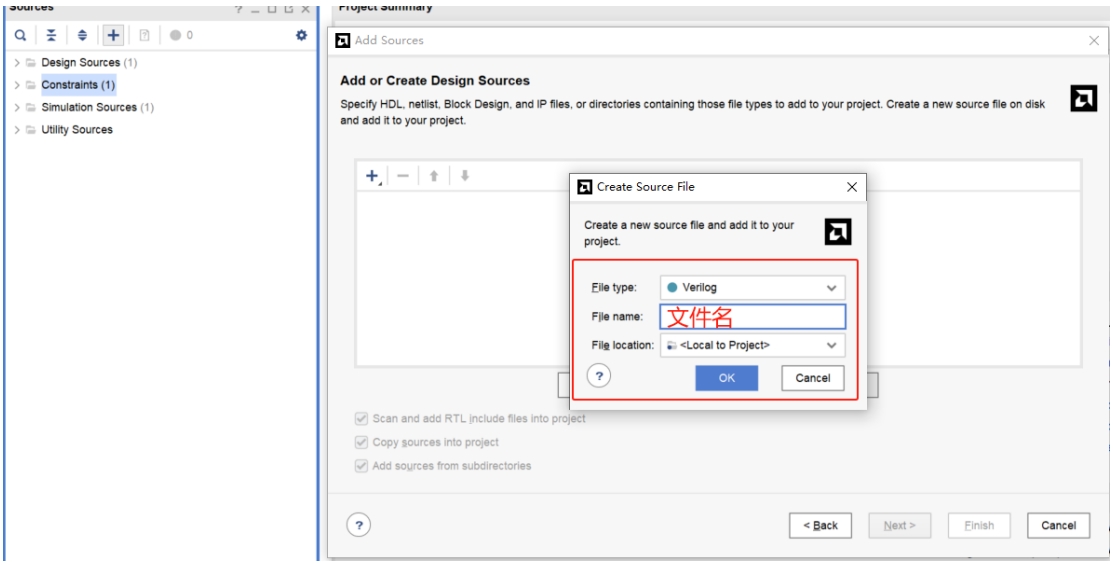


添加代码

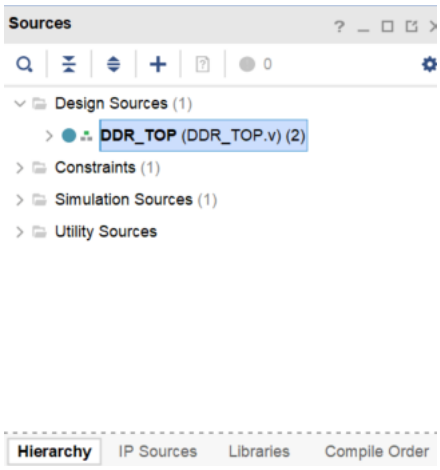




创建一个 DDR_TOP 文件，并且选择 Verilog 格式



创建完成后，可以看到 Design Sources 文件夹中有 DDR_TOP.v 文件，这个文件就是我们可以编写 verilog 程序的文件



双击 DDR_TOP 文件，添加代码，代码如下：

```
`timescale 1ns / 1ps

module DDR_TOP(
    output                          c0_ddr4_act_n   ,
    output [16:0]                   c0_ddr4_adr     ,
    output [1:0]                    c0_ddr4_ba      ,
    output [0:0]                    c0_ddr4_bg      ,
    output [0:0]                    c0_ddr4_cke     ,
    output [0:0]                    c0_ddr4_odt     ,
    output [0:0]                    c0_ddr4_cs_n    ,
    output [0:0]                    c0_ddr4_ck_t    ,
    output [0:0]                    c0_ddr4_ck_c    ,
    output                          c0_ddr4_reset_n ,
    inout [3:0]                    c0_ddr4_dm_dbi_n,
    inout [31:0]                   c0_ddr4_dq      ,
    inout [3:0]                    c0_ddr4_dqs_c   ,
    inout [3:0]                    c0_ddr4_dqs_t   ,

    input                           c0_sys_clk_p,
    input                           c0_sys_clk_n,
    input                           sys_rstn
    );



localparam nCK_PER_CLK                = 4;
localparam ADDR_WIDTH                 = 28;
localparam MEM_IF_ADDR_BITS           = 29;
localparam PAYLOAD_WIDTH              = 32;
```

```
    localparam APP_DATA_WIDTH                = 2 * nCK_PER_CLK *
PAYLOAD_WIDTH;
    localparam APP_MASK_WIDTH                = APP_DATA_WIDTH / 8;
    // Wire declarations
    wire [MEM_IF_ADDR_BITS-1:0]              app_addr;
    wire [2:0]                               app_cmd;
    wire                                     app_en;
    wire                                     app_rdy;
    wire [APP_DATA_WIDTH-1:0]                app_rd_data;
    wire                                     app_rd_data_end;
    wire                                     app_rd_data_valid;
    wire [APP_DATA_WIDTH-1:0]                app_wdf_data;
    wire                                     app_wdf_end;
    wire [APP_MASK_WIDTH-1:0]                app_wdf_mask;
    wire                                     app_wdf_rdy;
    wire                                     app_wdf_wren;
    wire                                     clk;
    wire                                     rst;
    wire                                     init_calib_complete;


    ddr4_0 u_ddr4
       (
       .sys_rst                    (~sys_rstn                 ),
       .c0_sys_clk_p               (c0_sys_clk_p              ),
       .c0_sys_clk_n               (c0_sys_clk_n              ),
       .c0_ddr4_act_n              (c0_ddr4_act_n             ),
       .c0_ddr4_adr                (c0_ddr4_adr               ),
       .c0_ddr4_ba                 (c0_ddr4_ba                ),
       .c0_ddr4_bg                 (c0_ddr4_bg                ),
       .c0_ddr4_cke                (c0_ddr4_cke               ),
       .c0_ddr4_odt                (c0_ddr4_odt               ),
       .c0_ddr4_cs_n               (c0_ddr4_cs_n              ),
       .c0_ddr4_ck_t               (c0_ddr4_ck_t              ),
       .c0_ddr4_ck_c               (c0_ddr4_ck_c              ),
       .c0_ddr4_reset_n            (c0_ddr4_reset_n           ),
       .c0_ddr4_dm_dbi_n           (c0_ddr4_dm_dbi_n          ),
       .c0_ddr4_dq                 (c0_ddr4_dq                ),
       .c0_ddr4_dqs_c              (c0_ddr4_dqs_c             ),
       .c0_ddr4_dqs_t              (c0_ddr4_dqs_t             ),
       .c0_init_calib_complete     (init_calib_complete       ),
       .c0_ddr4_ui_clk             (clk                   ),
       .c0_ddr4_ui_clk_sync_rst    (rst                       ),
       .dbg_clk                    (                          ),
```

```
        .c0_ddr4_app_addr               (app_addr                   ),
        .c0_ddr4_app_cmd                (app_cmd                    ),
        .c0_ddr4_app_en                 (app_en                     ),
        .c0_ddr4_app_hi_pri             (1'b0                       ),
        .c0_ddr4_app_wdf_data           (app_wdf_data               ),
        .c0_ddr4_app_wdf_end            (app_wdf_end                ),
        .c0_ddr4_app_wdf_mask           (app_wdf_mask               ),
        .c0_ddr4_app_wdf_wren           (app_wdf_wren               ),
        .c0_ddr4_app_rd_data            (app_rd_data                ),
        .c0_ddr4_app_rd_data_end        (app_rd_data_end            ),
        .c0_ddr4_app_rd_data_valid      (app_rd_data_valid          ),
        .c0_ddr4_app_rdy                (app_rdy                    ),
        .c0_ddr4_app_wdf_rdy            (app_wdf_rdy                ),
        .dbg_bus                        (                           )
        );



    //读写测试
        parameter    [1:0]IDLE  =2'd0;
        parameter    [1:0]WRITE =2'd1;
        parameter    [1:0]WAIT  =2'd2;
        parameter    [1:0]READ  =2'd3;
        parameter    [2:0]CMD_WRITE    =3'd0;
        parameter    [2:0]CMD_READ     =3'd1;
        parameter    TEST_DATA_RANGE   =32'd1000;

         reg    [1 :0]state=0;
        reg    [31:0]Count_32=0;
        reg    [ADDR_WIDTH-1:0]app_addr_begin=0;

        assign                                          app_wdf_mask
=64'h0000_0000_0000_0000;
        assign    app_wdf_end                   =app_wdf_wren;
        assign     app_en                          =(state==WRITE) ?
(app_rdy&&app_wdf_rdy) : ((state==READ)&&app_rdy);
        assign     app_wdf_wren                    =(state==WRITE) ?
(app_rdy&&app_wdf_rdy) : 1'b0;
        assign     app_cmd                         =(state==WRITE) ?
CMD_WRITE : CMD_READ;
        assign    app_addr                      =app_addr_begin;
        assign    app_wdf_data                  ={

Count_32,Count_32,Count_32,Count_32,
```

```
Count_32,Count_32,Count_32,Count_32
                                                    };

        always@(posedge clk)
           if(rst&!init_calib_complete)
               begin
               state                              <=IDLE;
               app_addr_begin                     <=29'd0;
               Count_32                           <=32'd0;
               end
        else case(state)
            IDLE:    begin
               state                               <=WRITE;
               Count_32                            <=32'd0;
               app_addr_begin                      <=29'd0;
              end
            WRITE:    begin//写 DDR4
               state
<=(Count_32==TEST_DATA_RANGE)&&app_rdy&&app_wdf_rdy ? WAIT:state;//最
后一个地址写完之后跳出状态
               Count_32
<=app_rdy&&app_wdf_rdy?(Count_32+1):Count_32;
               app_addr_begin
<=app_rdy&&app_wdf_rdy?(app_addr_begin+29'd8):app_addr_begin;//跳到下
一个（8*16=128）bit 数据地址
               end
            WAIT:    begin
               state                              <=READ;
               Count_32                           <=32'd0;
               app_addr_begin                     <=29'd0;
               end
            READ:    begin//读 DDR4
               state
<=(Count_32==TEST_DATA_RANGE)&&app_rdy? IDLE:state;
               Count_32
<=app_rdy?(Count_32+1):Count_32;
               app_addr_begin
<=app_rdy?(app_addr_begin+29'd8):app_addr_begin;
               end
        default:begin
               state                              <=IDLE;
               app_addr_begin                     <=29'd0;
               Count_32                           <=32'd0;
```

```
                end
            endcase

        (* KEEP = "TRUE" *) reg [511:0]app_rd_data_r;
        (* KEEP = "TRUE" *) reg app_rd_data_valid_r;
        always @(posedge clk) begin
            app_rd_data_r <=  app_rd_data;
            app_rd_data_valid_r <= app_rd_data_valid;
        end


        reg     [31:0] count_temp=0;//读计数
        always @(posedge clk) begin
            if(state==WAIT) count_temp <=32'd0;
            else  if(app_rd_data_valid_r)    count_temp<=  count_temp  +
32'b1;

         end

        wire [255:0]cm_data;
        assign cm_data = {count_temp,count_temp,count_temp,count_temp,

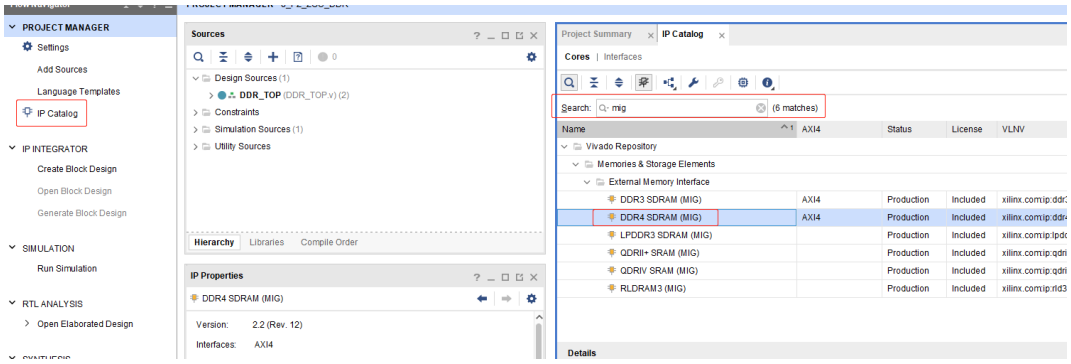
count_temp,count_temp,count_temp,count_temp  };
        assign
tg_compare_error=(app_rd_data_valid_r&&(cm_data!=app_rd_data_r));



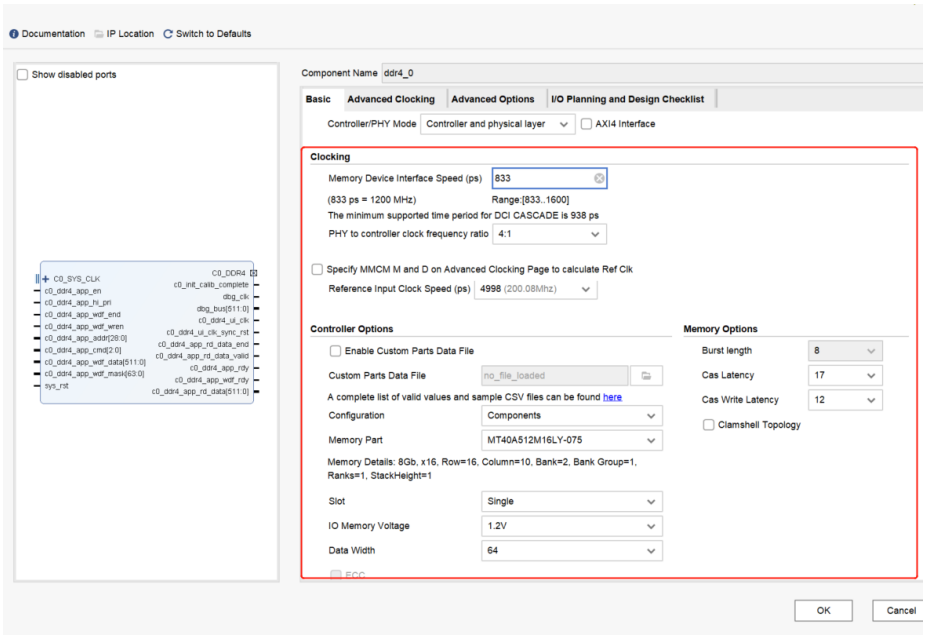
        ila_0 degbug_0 (
            .clk(clk), // input wire clk
            .probe0(cm_data), // input wire [127:0]  probe0
            .probe1(app_rd_data_r), // input wire [127:0]  probe1
            .probe2(tg_compare_error), // input wire [0:0]  probe2
            .probe3(init_calib_complete) // input wire [0:0]  probe3
        );

    endmodule
```

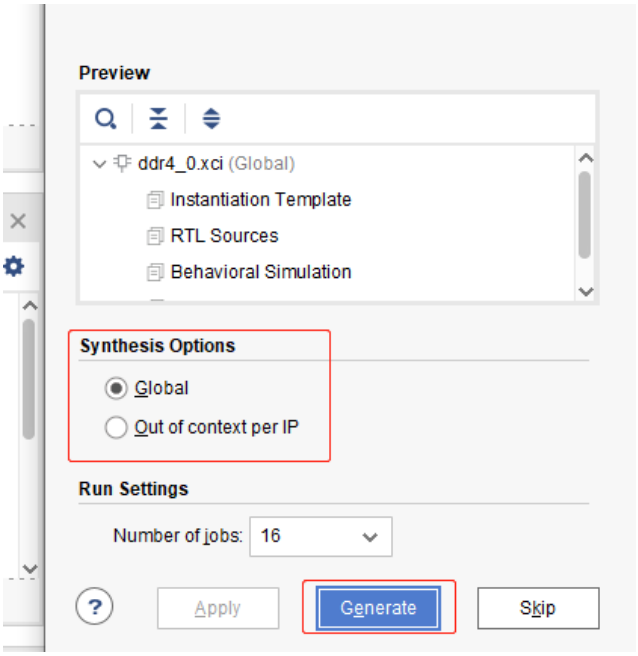
添加 MIG IP 到工程，如下图：



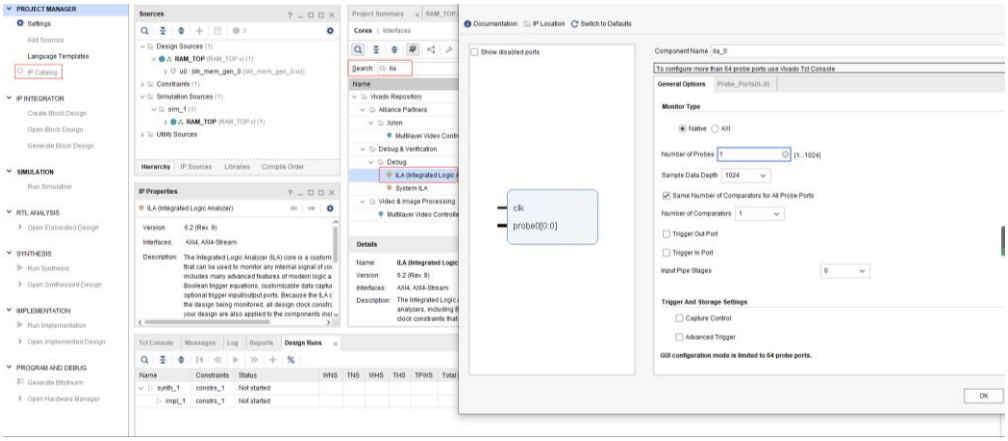
MIG IP 配置如下，然后点击 OK（其他配置项，全部默认）：



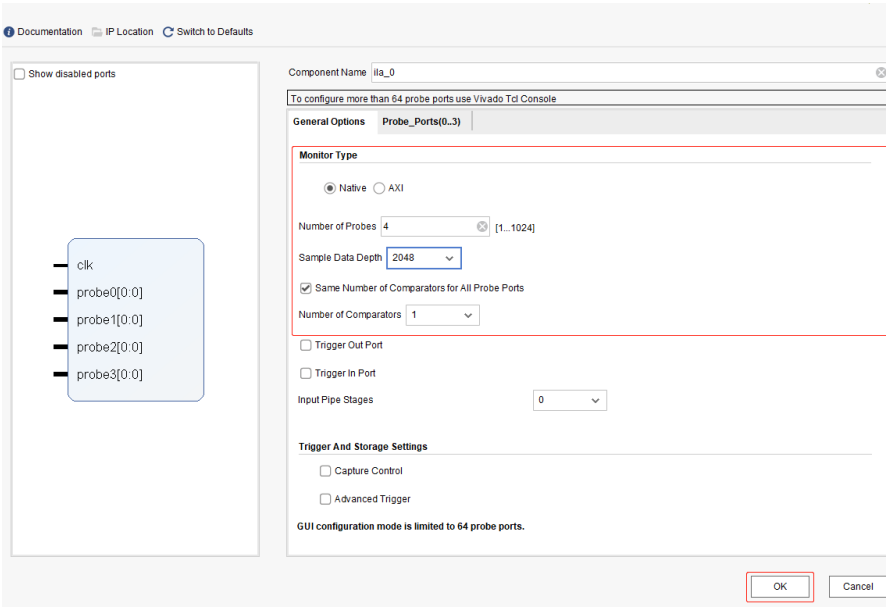
点击 Generate，如下：

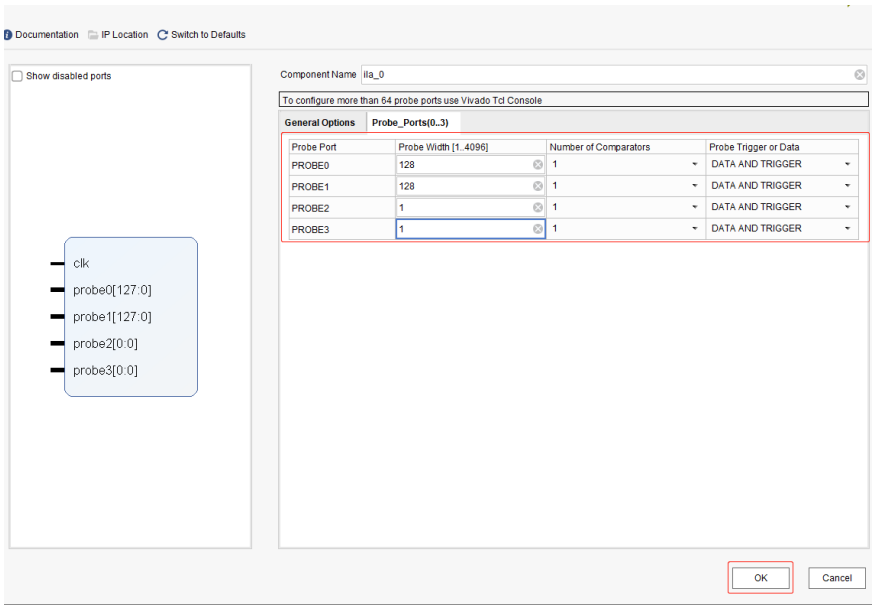


添加 ILA IP 到工程，如下图：

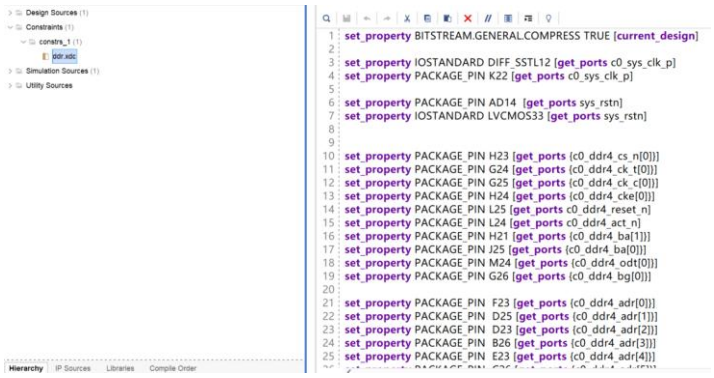


ILA IP 配置如下（其他默认）：





添加管脚约束文件

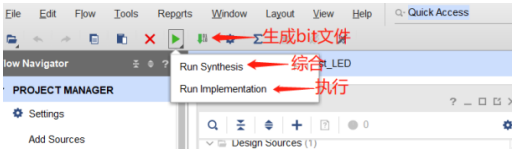


编译并且产生 bit 文件

1:单击综合

2:单击执行

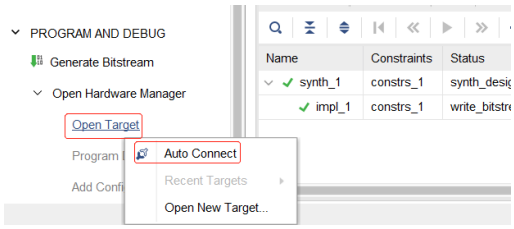
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

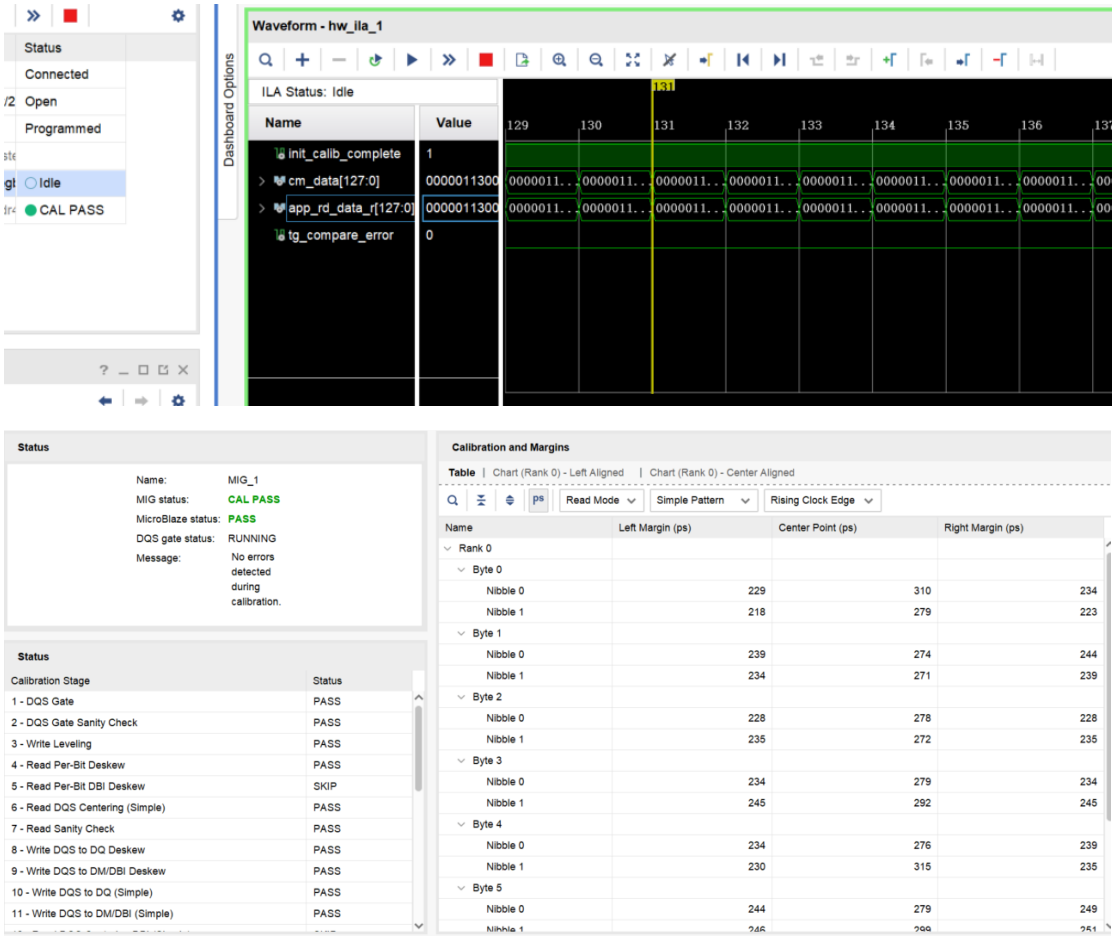


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.8.4 运行结果

下载完成后，自动跳出 ILA 界面，然后进行信号抓取，如下图



3.9 I2C 读写 E2PROM 例程

3.9.1 功能概述

EEPROM (Electrically Erasable Progammable Read Only Memory，E2PROM)即电可擦除可编程只读存储器，是一种常用的非易失性存储器(掉电数据不丢失)，EEPROM 有多种类型的产品。

我们 K7325T 开发板上使用的是 AT24C64。AT24C64 具有高可靠性，可对所存数据保存 100 年，并可进行多次擦写，擦写次数达一百万次。

本实例往 E2PROM 里面写数，然后读取出来对比，数据对比期间，LED 常灭，如果数据对比正确，LED 灯常亮;如果数据对比错误，LED 灯闪烁。

3.9.2 管脚配置

管脚约束如下：

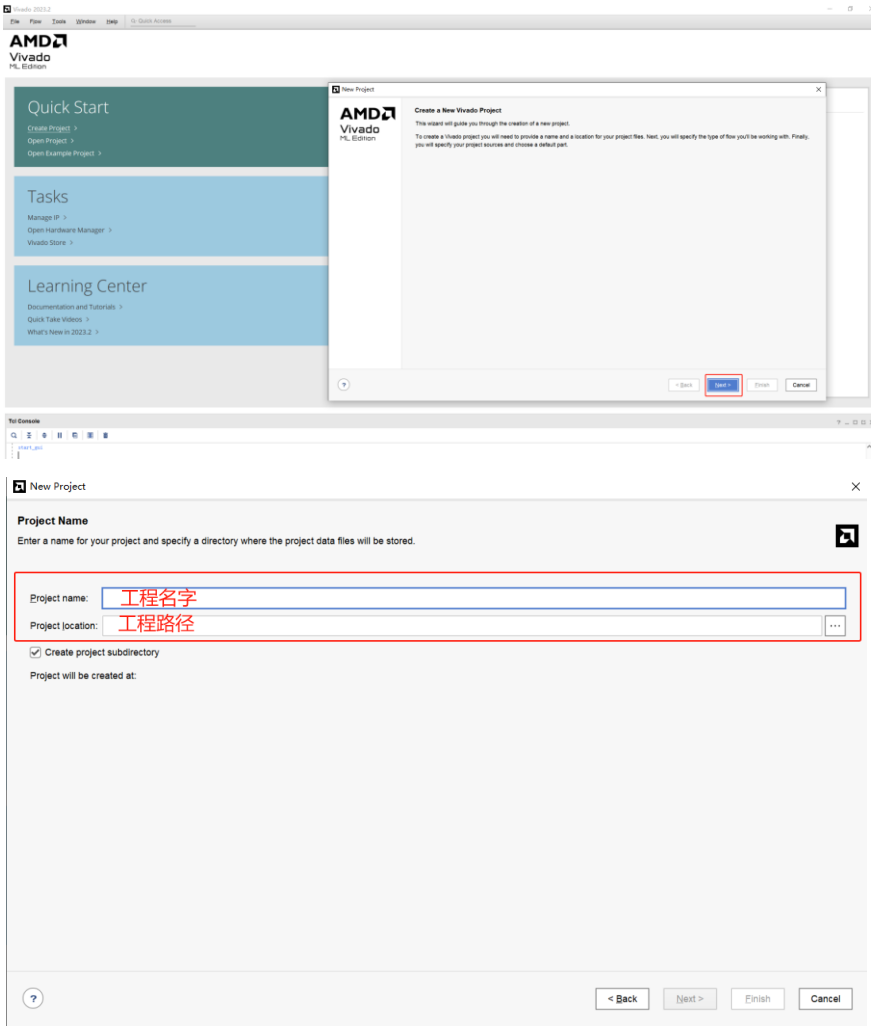
```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]

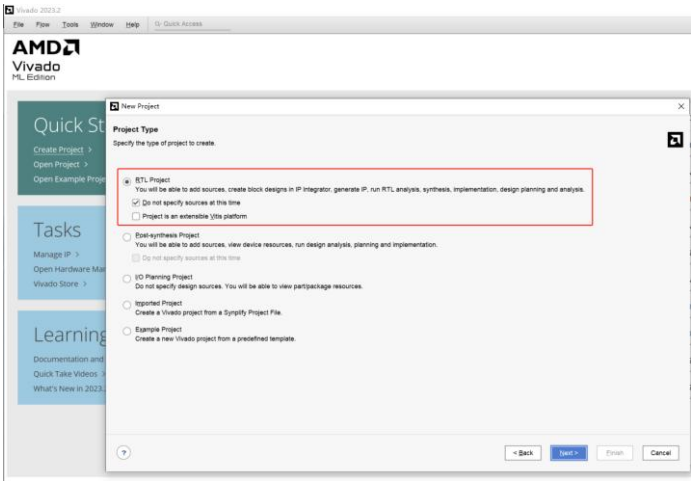
set_property -dict {PACKAGE_PIN AA14  IOSTANDARD LVCMOS33} [get_ports led]
set_property -dict {PACKAGE_PIN AB15 IOSTANDARD LVCMOS33} [get_ports iic_scl]
set_property -dict {PACKAGE_PIN AB16 IOSTANDARD LVCMOS33} [get_ports iic_sda]
```

3.9.3 工程搭建

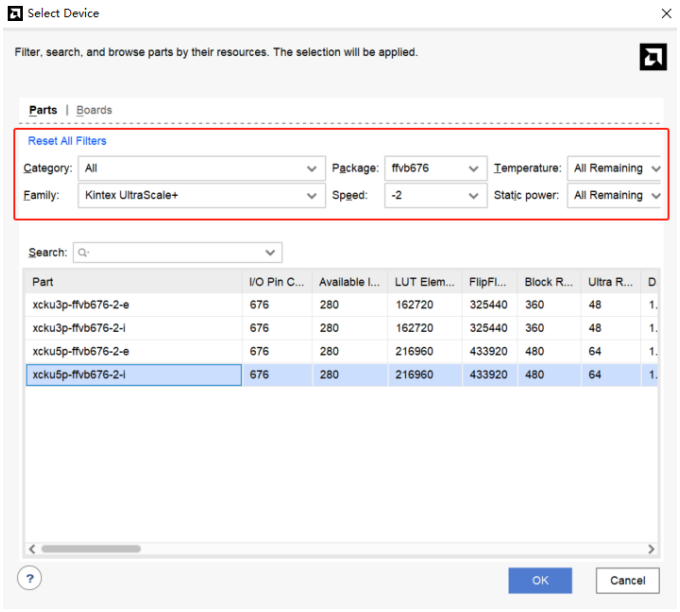
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



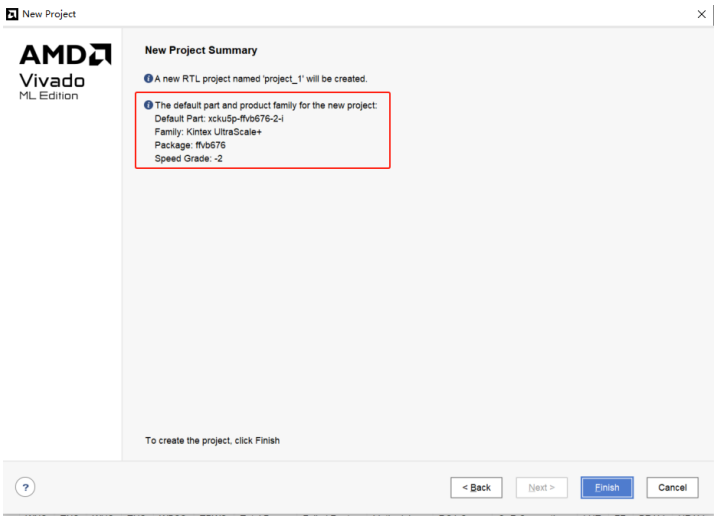
选择 RTL Project 并且勾选复选框，之后单击 NEXT



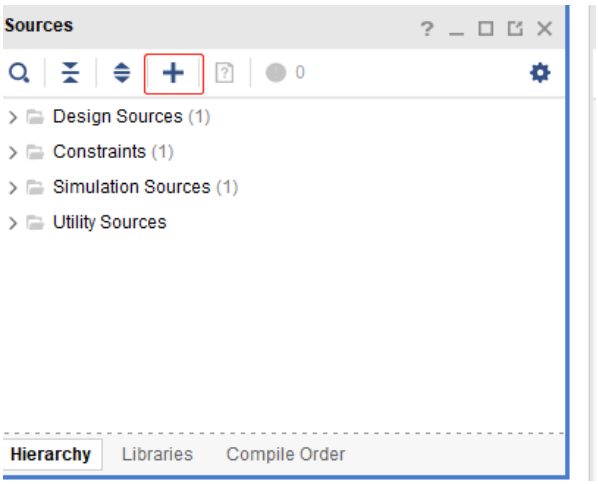
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



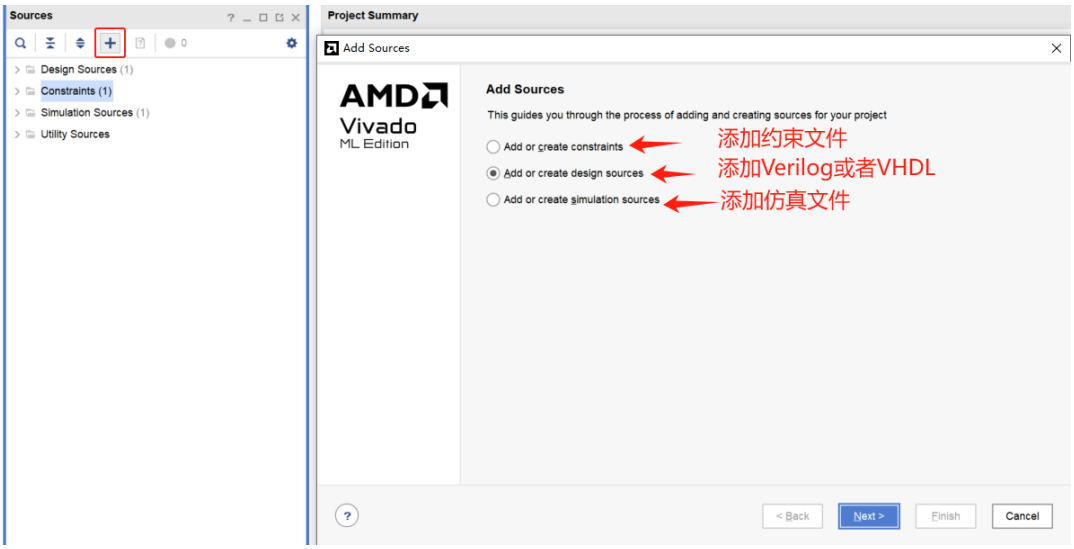
单击 Finish 完成工程创建

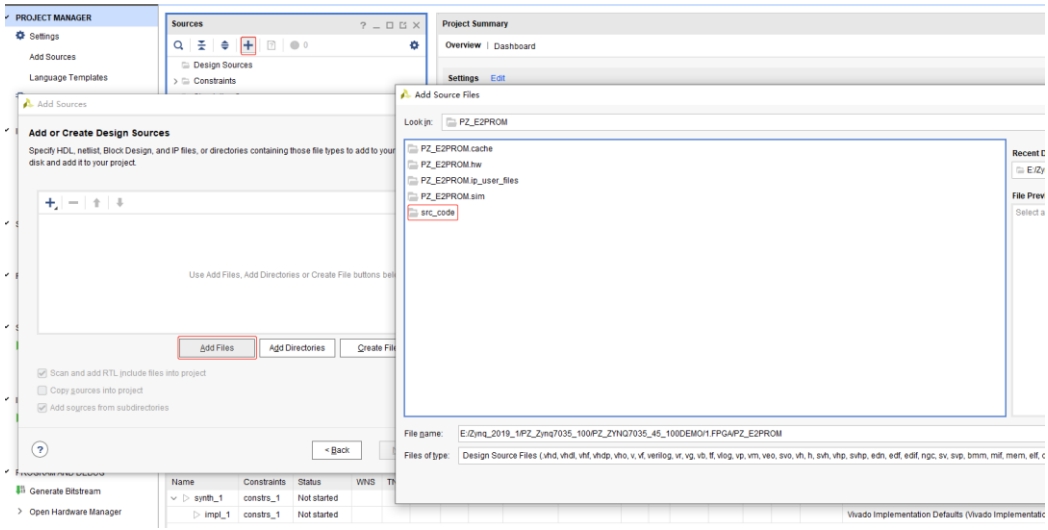


添加代码，显示界面如下



添加代码





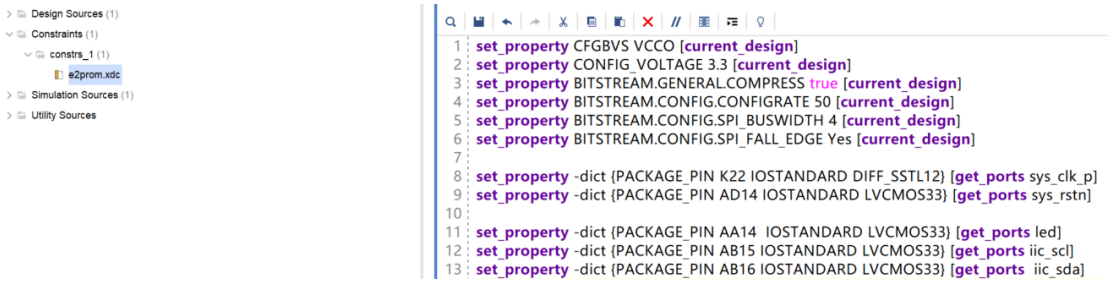
添加代码文件（代码文件在工程路径下）



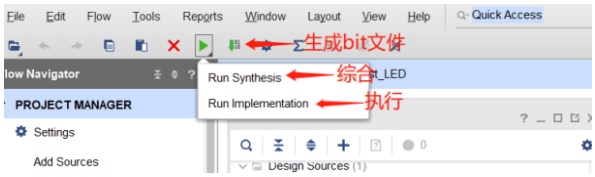
添加完成后，可以看到 Design Sources 文件夹中有四个**.v 文件，这几个文件就是操作 E2PROM 的代码文件（具体代码，可以点开代码文件查看）



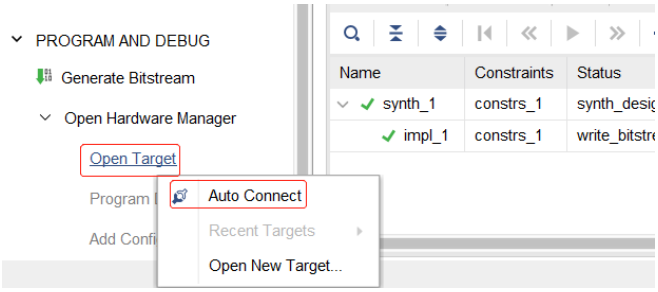
添加管脚约束文件



编译并且产生 bit 文件
1:单击综合
2:单击执行
3:单击产生 bit



下载程序
1：给开发板通电，并且连接下载器
2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

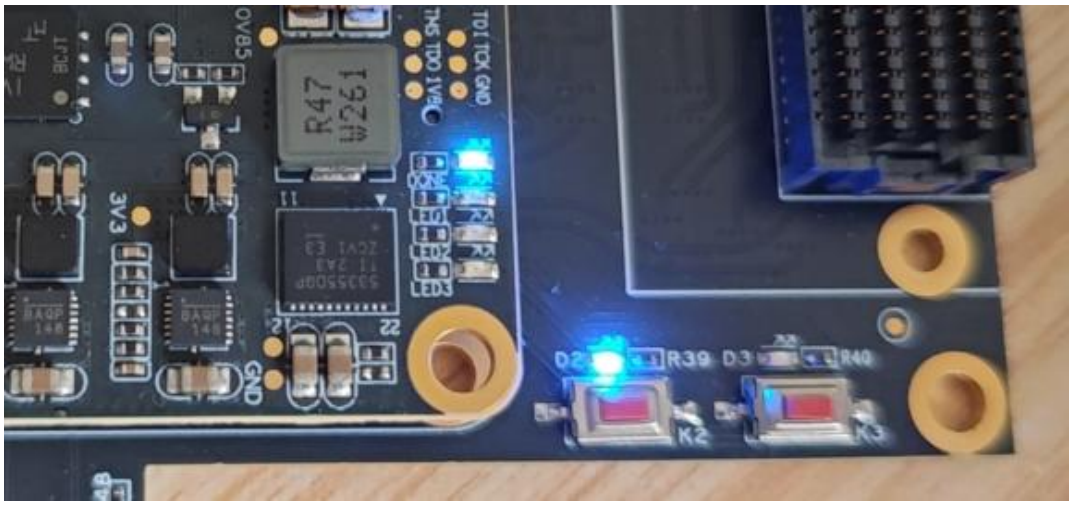


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.9.4 运行结果

下载完成后，数据对比正确，LED 灯常亮



3.10 串口收发例程

3.10.1 功能概述

串口通信（Serial Communications）的概念非常简单，串口按位（bit）发送和接收字节。尽管比按字节（byte）的并行通信慢，但是串口可以在使用一根线发送数据的同时用另一根线接收数据。

它很简单并且能够实现远距离通信。比如 IEEE488 定义并行通行状态时，规定设备线总长不得超过 20 米，并且任意两个设备间的长度不得超过 2 米；而对于串口而言，长度可达 1200 米。

典型的串口用于 ASCII 码字符的传输，通信使用 3 根线完成，分别是地线、发送、接收。由于串口通信是异步的，端口能够在一根线上发送数据同时在另一根线上接收数据。串口通信最重要的参数是波特率、数据位、停止位和奇偶校验。对于两个进行通信的端口，这些参数必须匹配。

本实例通过串口实现定时 1 秒输出固定字符串“Hello PuZhi”同时可以把接受到的字节进行回传。

3.10.2 管脚配置

管脚约束如下：

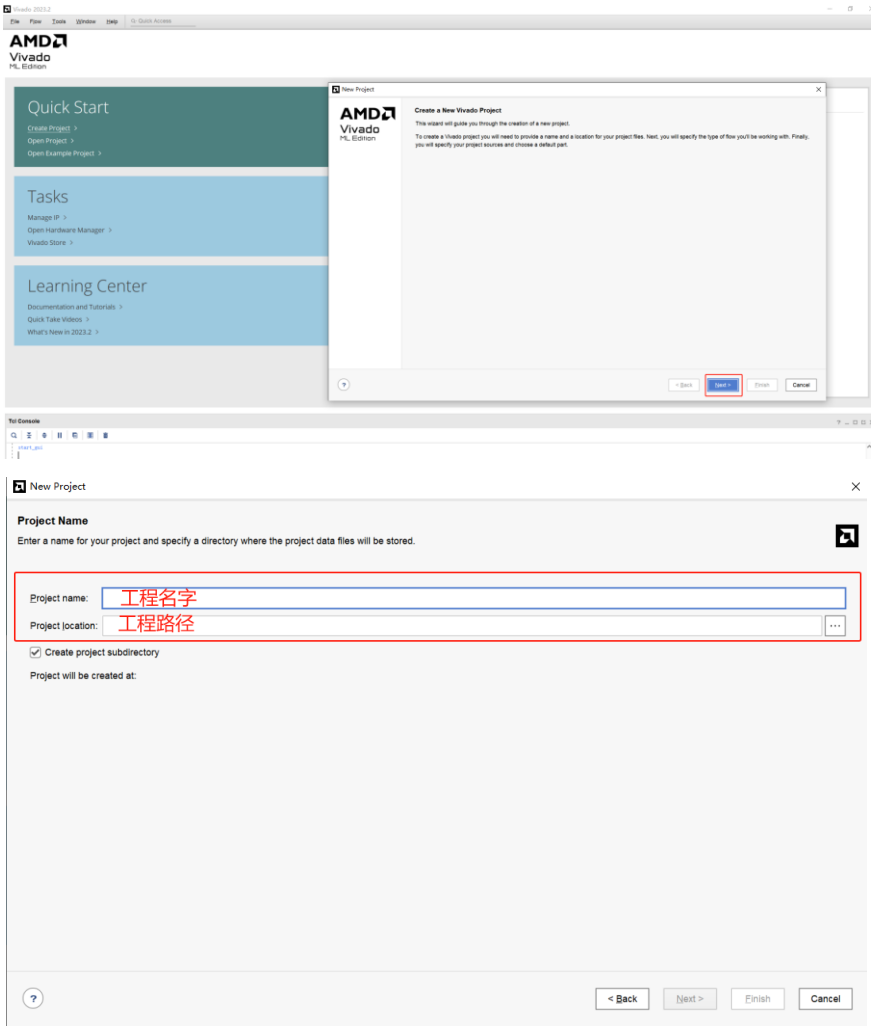
```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]

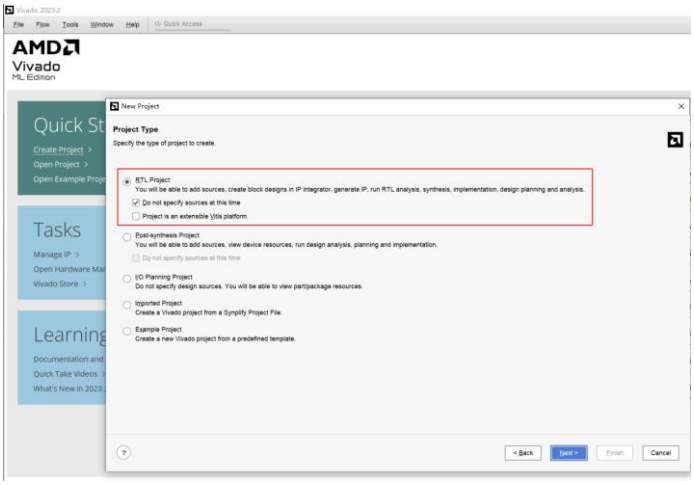
set_property -dict {PACKAGE_PIN AC14 IOSTANDARD LVCMOS33} [get_ports
uart_tx]
set_property -dict {PACKAGE_PIN AC13 IOSTANDARD LVCMOS33} [get_ports
uart_rx]
```

3.10.3 工程搭建

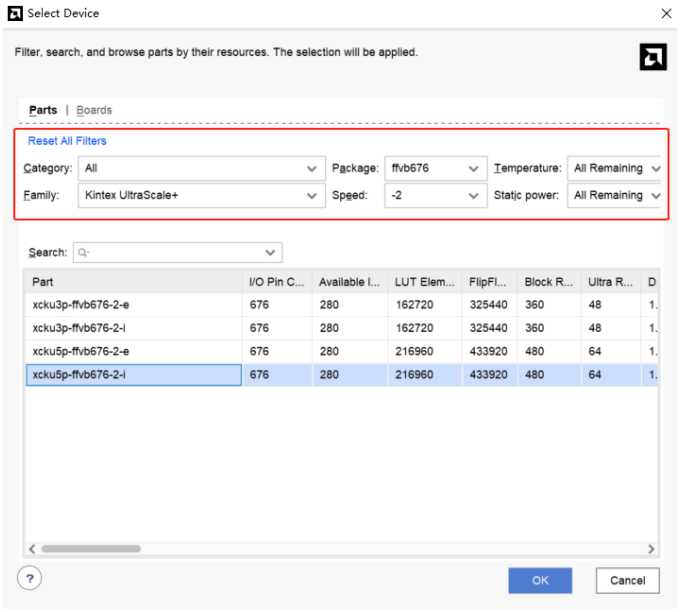
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



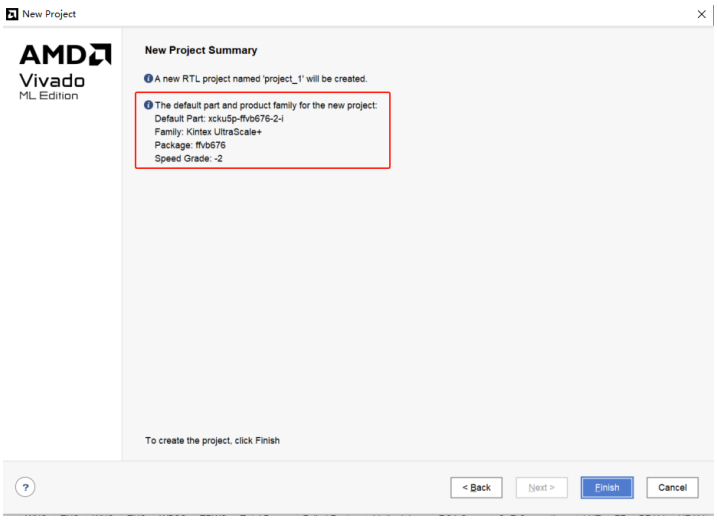
选择 RTL Project 并且勾选复选框，之后单击 NEXT



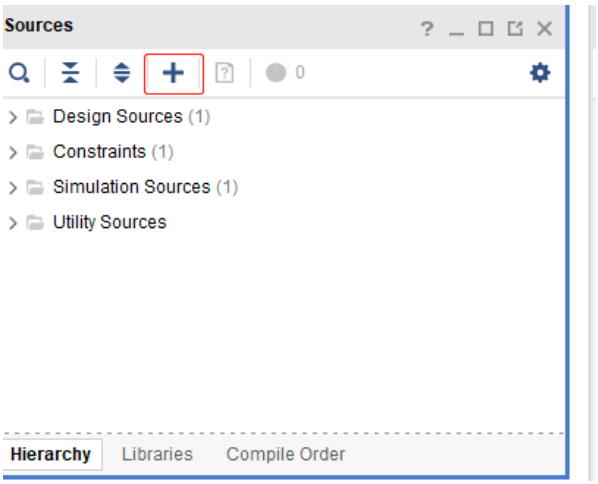
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



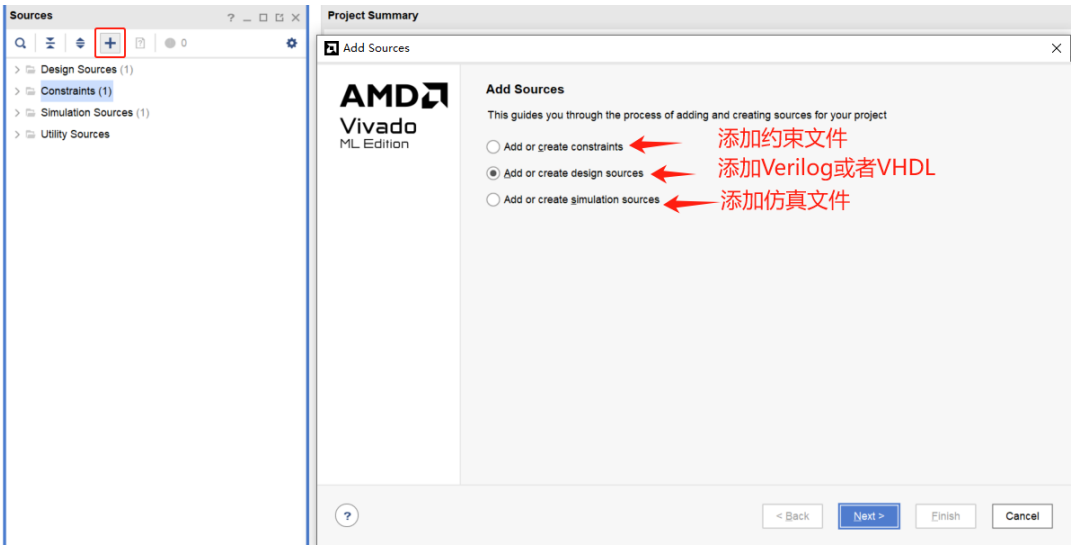
单击 Finish 完成工程创建

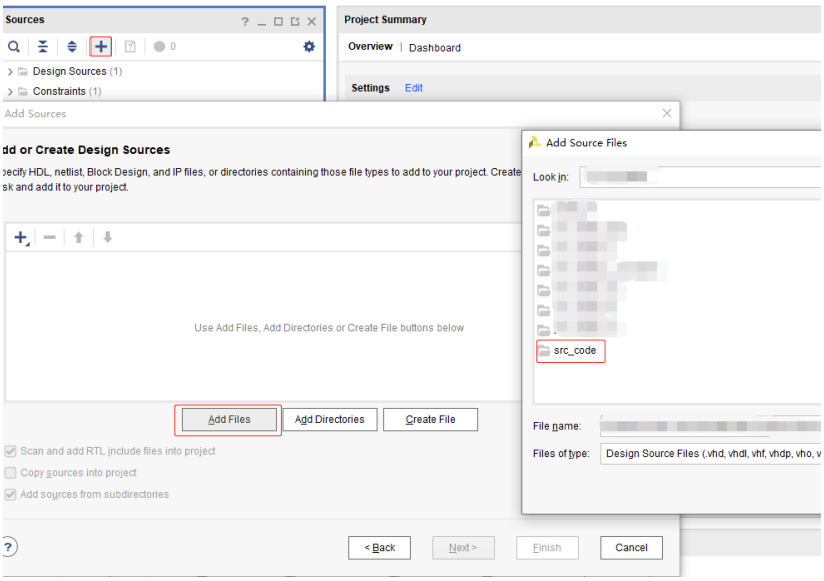


添加代码，显示界面如下

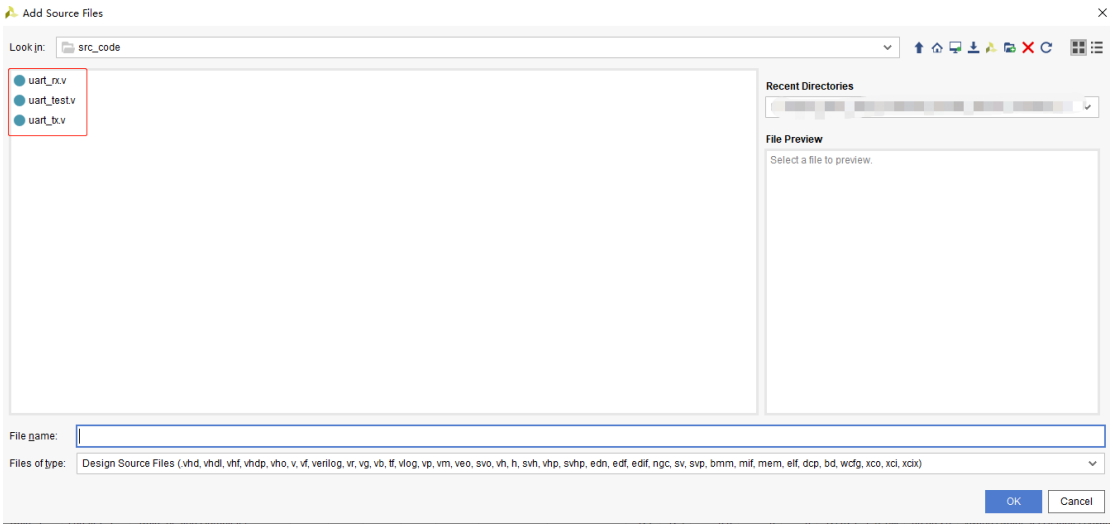


添加代码

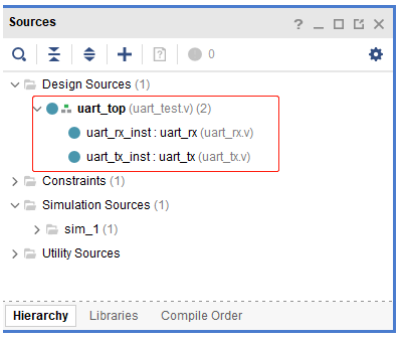




添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有三个**.v 文件，这几个文件就是操作串口的代码文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

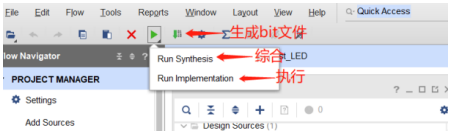



```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

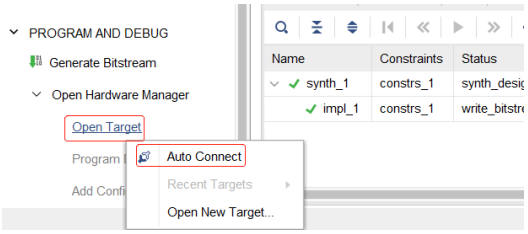
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]

set_property -dict {PACKAGE_PIN AC14 IOSTANDARD LVCMOS33} [get_ports uart_tx]
set_property -dict {PACKAGE_PIN AC13 IOSTANDARD LVCMOS33} [get_ports uart_rx]
```

编译并且产生 bit 文件
1:单击综合
2:单击执行
3:单击产生 bit



下载程序
1：给开发板通电，并且连接下载器
2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

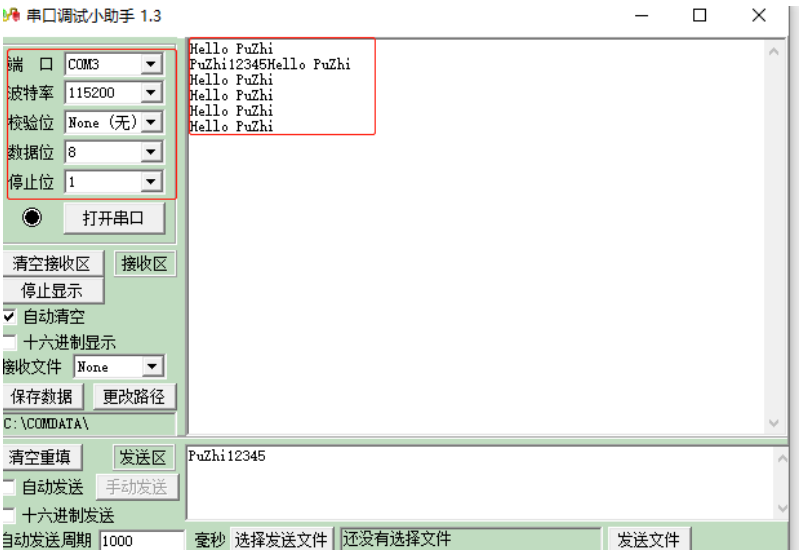


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.10.4 运行结果

下载完成后，结果如下图：



3.11 SD 卡读写例程

3.11.1 功能概述

SD 卡的英文全称是 Secure Digital Card，即安全数字卡（又叫安全数码卡），是在 MMC 卡 （Multimedia Card，多媒体卡）的基础上发展而来，主要增加了两个特色：更高的安全性和更快的读写速度。

SD 卡的初始化

由于市面上大多采用 SD2.0 版本协议的 SD 卡，接下来我们仅介绍 SD2.0 版本协议的初始化流程，以下提到的 SD 卡均代表基于 SD2.0 版本协议的 SDHC 卡，其详细初始化步骤如下：

1、 SD 卡完成上电后，主机 FPGA 先对从机 SD 卡发送至少 74 个以上的同步时钟，在上电同 步期间，片选 CS 引脚和 MOSI 引脚必须为高电平（MOSI 引脚除发送命令或数据外，其 余时刻都为高电平）。

2、 拉低片选 CS 引脚，发送命令 CMD0（0x40）复位 SD 卡，命令发送完成后等待 SD 卡返回响应数据。

3、 SD 卡返回响应数据后，先等待 8 个时钟周期再拉高片选 CS 信号，此时判断返回的响应数据。如果返回的数据为复位完成信号 0x01，在接收返回信息期间片选 CS 为低电平，此时 SD 卡进入 SPI 模式，并开始进行下一步，如果返回的值为其它值，则重新执行第 2 步。

4、 拉低片选 CS 引脚，发送命令 CMD8（0x48）查询 SD 卡的版本号，只有 SD2.0 版本的卡才支持此命令，命令发送完成后等待 SD 卡返回响应数据。

5、 SD 卡返回响应数据后，先等待 8 个时钟周期再拉高片选 CS 信号，此时判断返回的响应数据。如果返回的电压范围为 4’b0001 即 2.7V~3.6V，说明此 SD 卡为 2.0 版本，进行下一步，否则重新执行第 4 步。

6、 拉低片选 CS 引脚，发送命令 CMD55（0x77）告诉 SD 卡下一次发送的命令是应用相关命令，命令发送完成后等待 SD 卡返回响应数据。

7、 SD 卡返回响应数据后，先等待 8 个时钟周期再拉高片选 CS 信号，此时判断返回的响应数据。如果返回的数据为空闲信号 0x01，开始进行下一步，否则重新执行第 6 步。

8、 拉低片选 CS 引脚，发送命令 ACMD41（0x69）查询 SD 卡是否初始化完成，命令发送完成后等待 SD 卡返回响应数据。

9、 SD 卡返回响应数据后，先等待 8 个时钟周期再拉高片选 CS 信号，此时判断返回的响应数据。如果返回的数据为 0x00，此时初始化完成，否则重新执行第 6 步。

SD 卡的写操作流程

1、 拉低片选 CS 引脚，发送命令 CMD24（0x58）读取单个数据块，命令发送完成后等待 SD 卡返回响应数据。

2、 SD 卡返回正确响应数据 0x00 后，等待至少 8 个时钟周期，开始发送数据头 0xfe。

3、 发送完数据头 0xfe 后，接下来开始发送 512 个字节的数据。

4、 数据发送完成后，发送 2 个字节的 CRC 校验数据。由于 SPI 模式下不对数据进行 CRC 校验，直接发送两个字节的 0xff 即可。

5、 校验数据发送完成后，等待 SD 卡响应。

6、 SD 卡返回响应数据后会进入写忙状态（MISO 引脚为低电平），即此时不允许其它操 作。当检测到 MISO 引脚为高电平时，SD 卡此时退出写忙状态。

7、 拉高 CS 引脚，等待 8 个时钟周期后允许进行其它操作。

SD 卡的读操作

1、 拉低片选 CS 引脚，发送命令 CMD17（0x51）读取单个数据块，命令发送完成后等待 SD 卡返回响应数据。

2、SD 卡返回正确响应数据 0x00 后，准备开始解析 SD 卡返回的数据头 0xfe。

3、 解析到数据头 0xfe 后，接下来接收 SD 卡返回的 512 个字节的数据。

4、 数据解析完成后，接下来接收两个字节的 CRC 校验值。由于 SPI 模式下不对数据进行 CRC 校验，可直接忽略这两个字节。

5、 校验数据接收完成后，等待 8 个时钟周期。

6、 拉高片选 CS 引脚，等待 8 个时钟周期后允许进行其它操作。

本实例把 K2 按键按下的累计次数值写到 SD 卡中然后读取出来通过 ILA 进行查看同时 LED 灯按低五位二进制的值进行显示。

3.11.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]

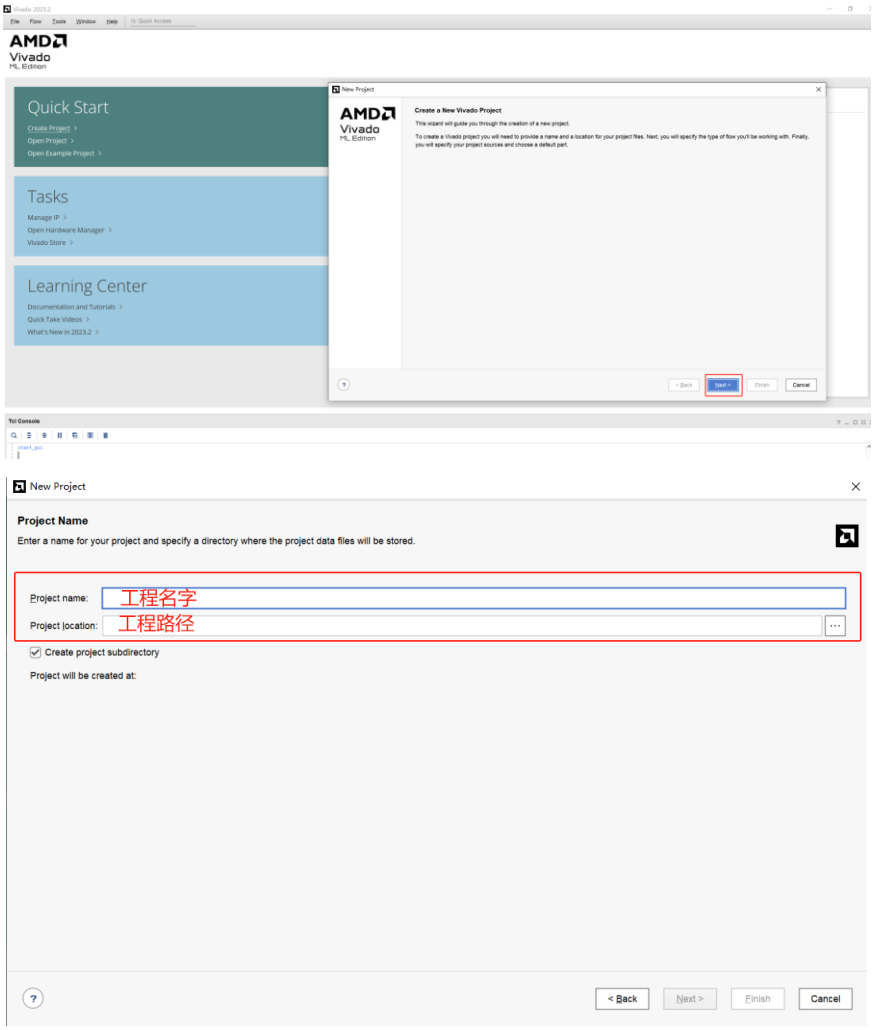
set_property -dict {PACKAGE_PIN C12 IOSTANDARD LVCMOS33} [get_ports
sd_dclk]
set_property -dict {PACKAGE_PIN F13 IOSTANDARD LVCMOS33} [get_ports
sd_ncs]
set_property -dict {PACKAGE_PIN B12 IOSTANDARD LVCMOS33} [get_ports
sd_mosi]
set_property -dict {PACKAGE_PIN G12 IOSTANDARD LVCMOS33} [get_ports
sd_miso]

set_property -dict {PACKAGE_PIN AA14 IOSTANDARD LVCMOS33} [get_ports
{led[0]}]
set_property -dict {PACKAGE_PIN AB14 IOSTANDARD LVCMOS33} [get_ports
{led[1]}]

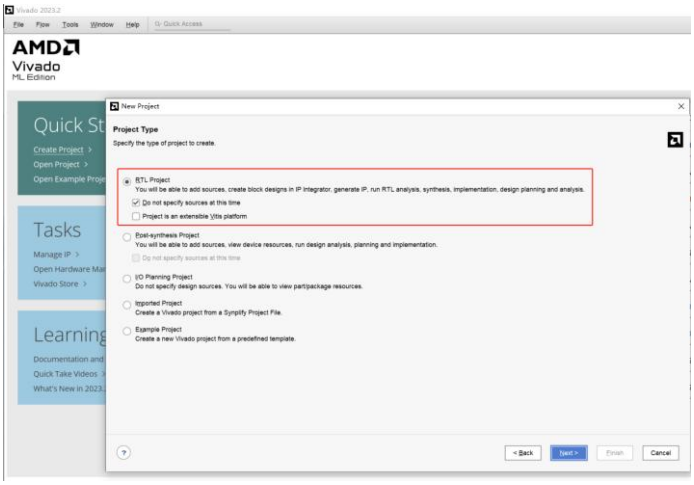
set_property -dict {PACKAGE_PIN AE13 IOSTANDARD LVCMOS33} [get_ports
key]
```

3.11.3 工程搭建

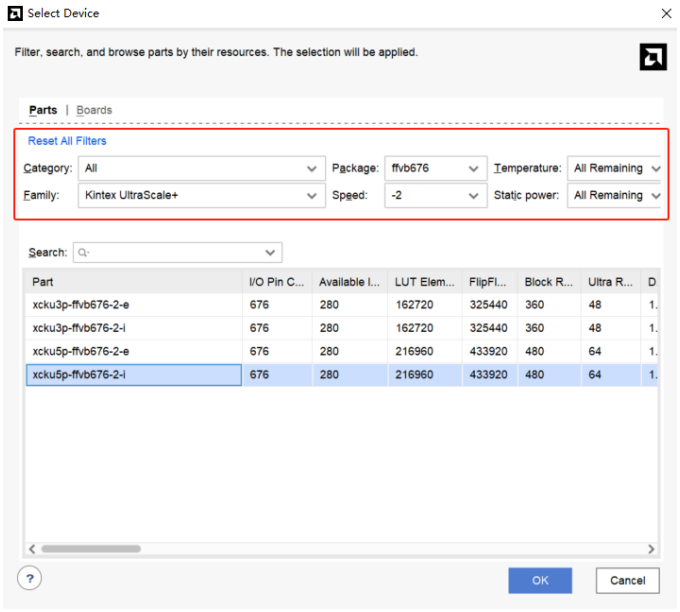
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



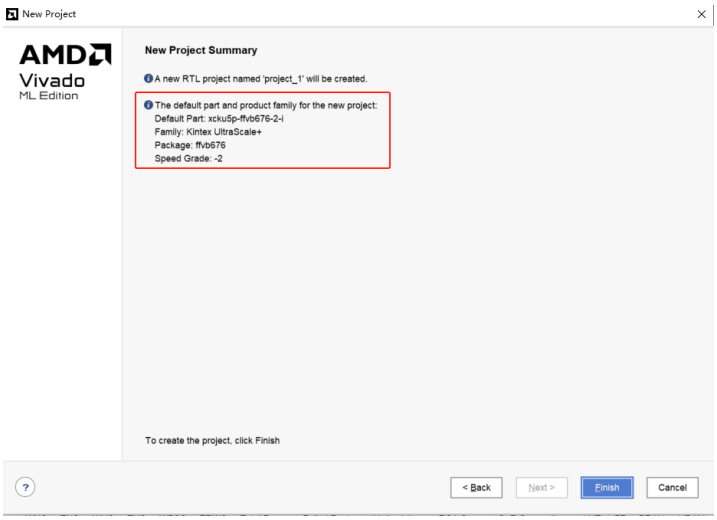
选择 RTL Project 并且勾选复选框，之后单击 NEXT



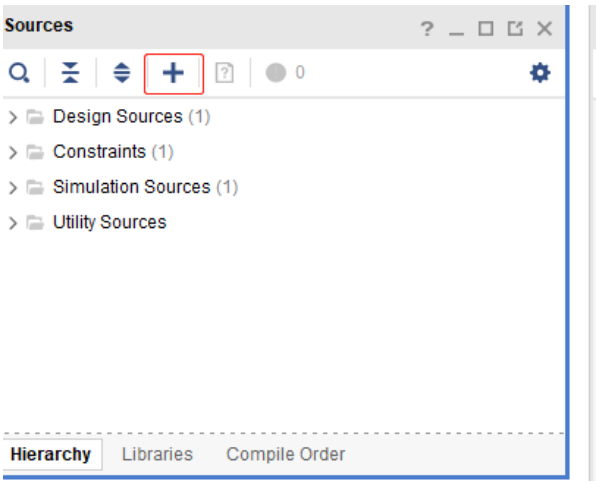
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



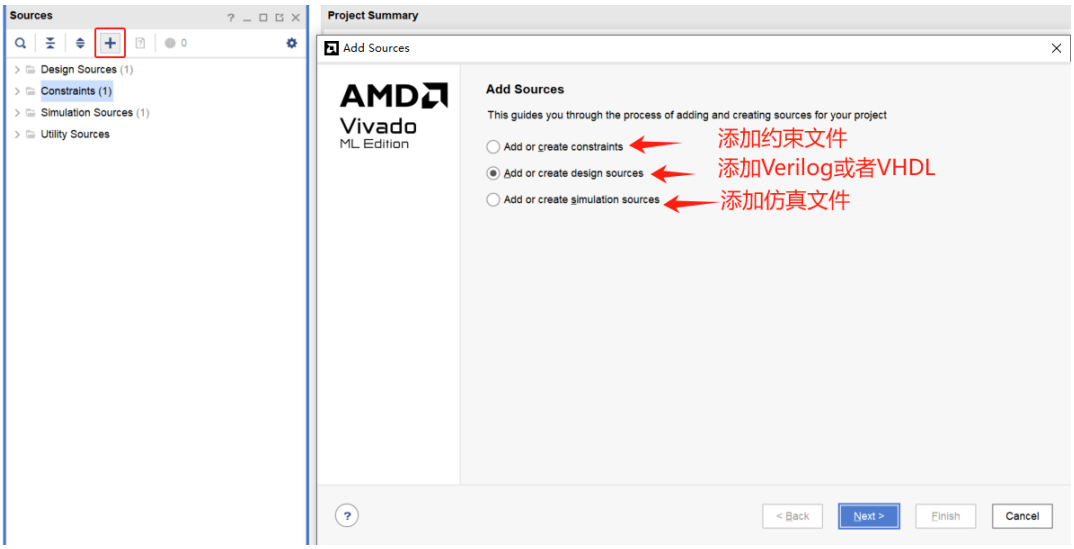
单击 Finish 完成工程创建

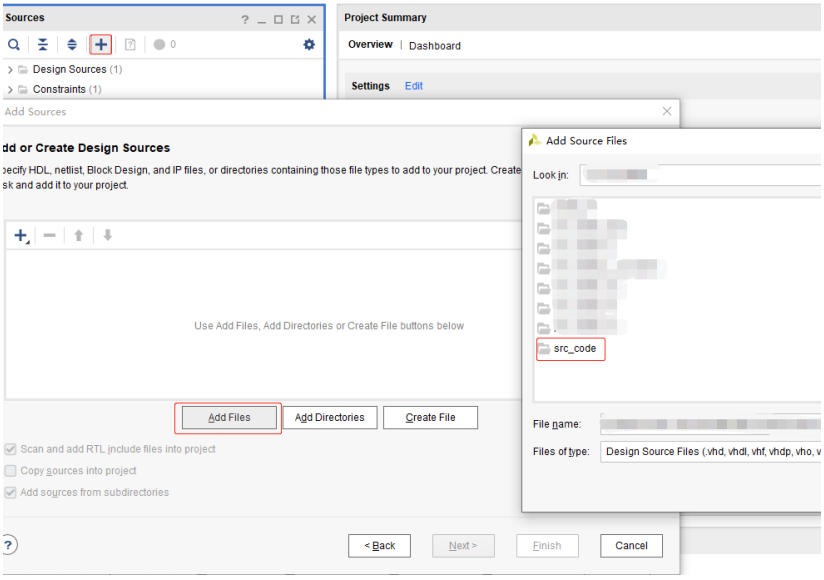


添加代码，显示界面如下

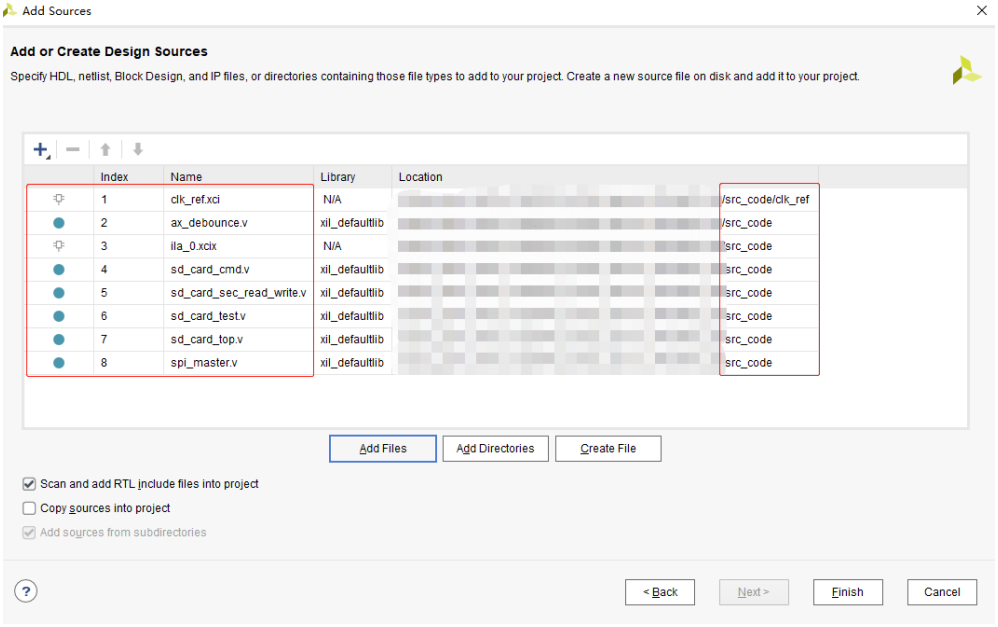


添加代码

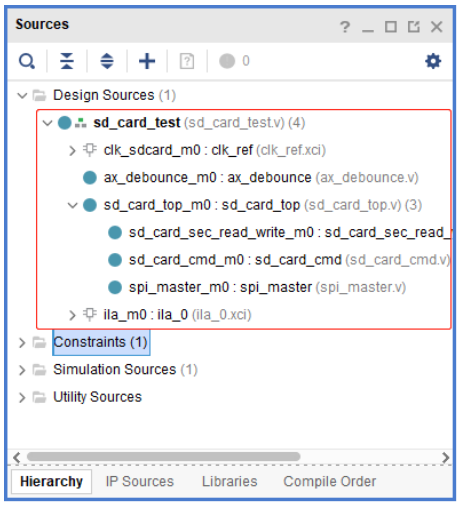




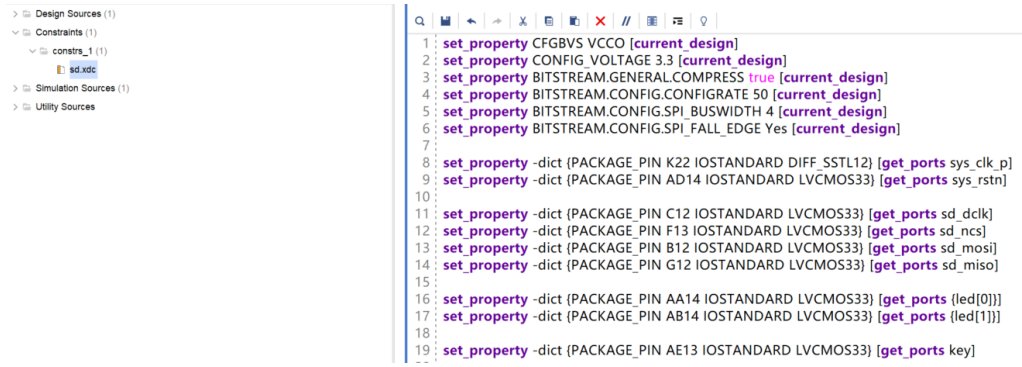
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件，这几个文件就是操作 SD 卡的代码文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

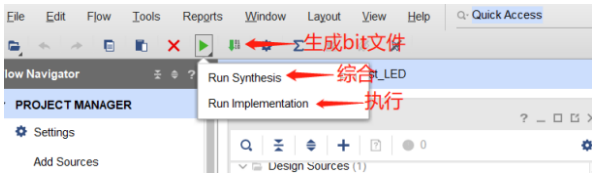


编译并且产生 bit 文件

1:单击综合

2:单击执行

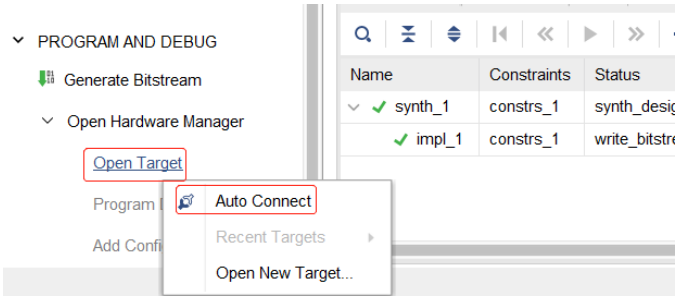
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

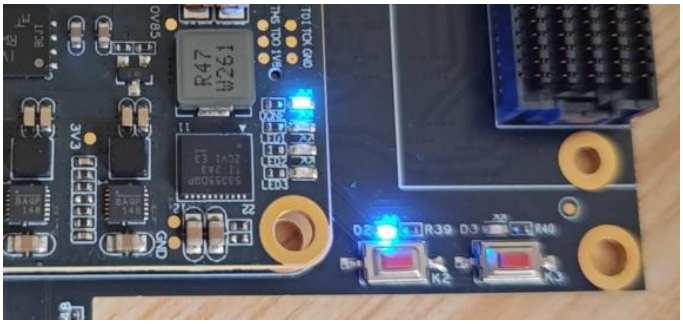


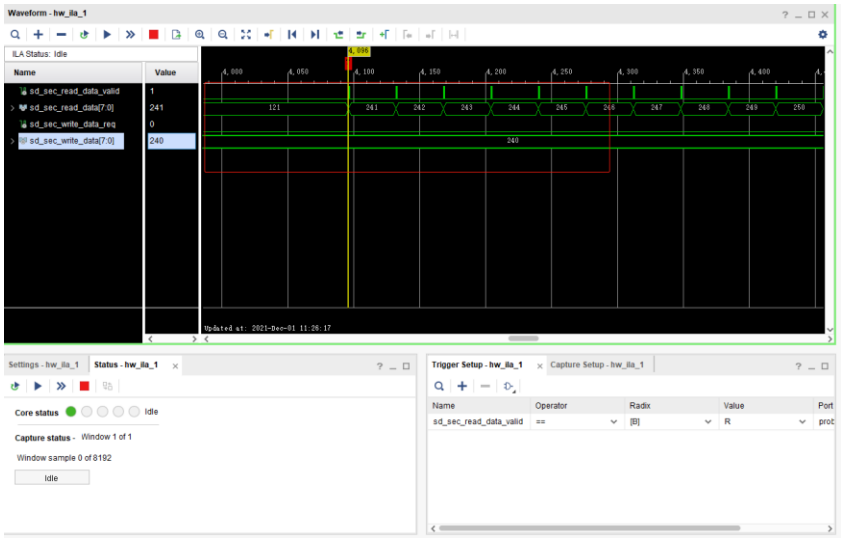
单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.11.4 运行结果

下载完成后，LED 灯对应点亮，ILA 如下图





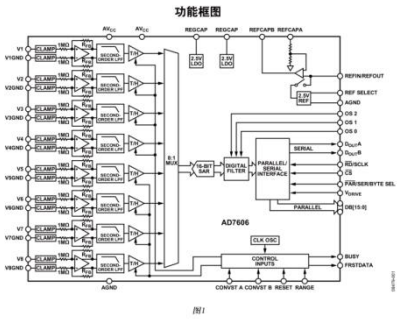
3.12 AD_DA 转换例程

3.12.1 功能概述

AD7606 是一款集成式 8 通道同步采样数据采集系统，片内集成输入放大器、过压保护电路、二阶模拟抗混叠滤波器、模拟多路复用器、16 位 200 kSPS SAR ADC 和一个数字滤波器，2.5 V 基准电压源、基准电压缓冲以及高速串行和并行接口。

AD7606 采用+5V 单电源供电，可以处理±5V 和±10V 真双极性输入信号，同时所有通道均以高达 200KSPS 的吞吐速率采样，输入钳位保护电路可以耐受最高达±16.5V 的电压。

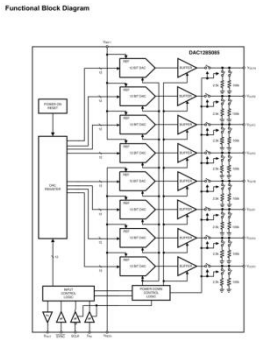
AD7606 抗混叠滤波器的 3dB 截至频率为 22kHz；当采样速率为 200kSPS 时，它具有 40dB 抗混叠抑制特性。灵活的数字滤波器采用引脚驱动，可以改善信噪比(SNR)，并降低 3dB 带宽。



DAC128S085 是一款功能齐全的通用 12 位电压输出数模转换器（DAC），可以在 2.7V 至 5.5V 的单电源下工作，在 3V 时的功耗为 1.95mW。在 5 V 时的功耗为 4.85mW。

DAC128S085 有两个参考电压。一个参考输入电压提供给通道 A 到 D，而另一个参考输入电压提供给通道 E 到 H。每个参考可以在 0.5 V 和 VA 之间独立设置，从而提供最大的输出动态范围。

DAC128S085 具有一个 16 位输入移位寄存器，该寄存器控制工作模式、掉电条件以及 DAC 通道的寄存器输出值。八路通道输出均可同时或单独更新。



本实例通过控制 DAC128S085 输出正弦波，然后再通过控制 AD7606 采集正弦波，在 ILA 界面进行查看。

3.12.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]
```

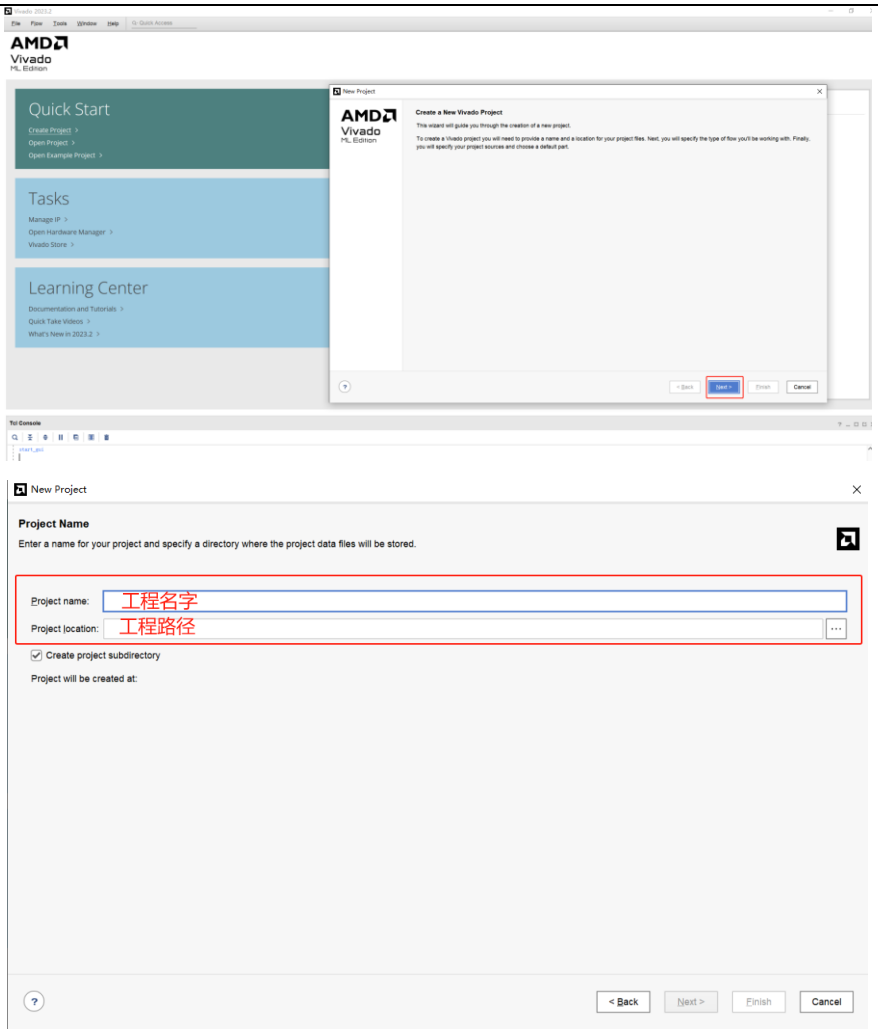
```
set_property   -dict {PACKAGE_PIN D10 IOSTANDARD LVCMOS33}  [get_ports
{ad_data[0]}]
set_property   -dict {PACKAGE_PIN B9 IOSTANDARD LVCMOS33}  [get_ports
{ad_data[1]}]
set_property   -dict {PACKAGE_PIN C11 IOSTANDARD LVCMOS33}  [get_ports
{ad_data[2]}]
set_property   -dict {PACKAGE_PIN A9 IOSTANDARD LVCMOS33}  [get_ports
{ad_data[3]}]
set_property   -dict {PACKAGE_PIN B11 IOSTANDARD LVCMOS33}  [get_ports
{ad_data[4]}]
set_property   -dict {PACKAGE_PIN B10 IOSTANDARD LVCMOS33}  [get_ports
{ad_data[5]}]
set_property   -dict {PACKAGE_PIN  D9 IOSTANDARD LVCMOS33} [get_ports
{ad_data[6]}]
set_property       -dict   {PACKAGE_PIN   A10   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[7]}]
set_property       -dict   {PACKAGE_PIN   C9   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[8]}]
set_property       -dict   {PACKAGE_PIN   H11    IOSTANDARD   LVCMOS33}
[get_ports {ad_data[9]}]
set_property       -dict   {PACKAGE_PIN    J11   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[10]}]
set_property       -dict   {PACKAGE_PIN    G11   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[11]}]
set_property       -dict   {PACKAGE_PIN    J10   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[12]}]
set_property       -dict   {PACKAGE_PIN    J9   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[13]}]
set_property       -dict   {PACKAGE_PIN    K10   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[14]}]
set_property       -dict   {PACKAGE_PIN    H13   IOSTANDARD   LVCMOS33}
[get_ports {ad_data[15]}]
set_property       -dict   {PACKAGE_PIN   J15    IOSTANDARD   LVCMOS33}
[get_ports {ad_os[2]}]
set_property       -dict   {PACKAGE_PIN   J13    IOSTANDARD   LVCMOS33}
[get_ports {ad_os[1]}]
set_property       -dict   {PACKAGE_PIN    J14   IOSTANDARD   LVCMOS33}
[get_ports {ad_os[0]}]
set_property       -dict   {PACKAGE_PIN    H9   IOSTANDARD   LVCMOS33}
[get_ports ad_range]
set_property       -dict   {PACKAGE_PIN    E11   IOSTANDARD   LVCMOS33}
[get_ports ad_convst]
set_property   -dict {PACKAGE_PIN  G9 IOSTANDARD LVCMOS33} [get_ports
```

```
ad_reset]
set_property       -dict   {PACKAGE_PIN     F10   IOSTANDARD   LVCMOS33}
[get_ports ad_cs]
set_property       -dict   {PACKAGE_PIN     D11   IOSTANDARD   LVCMOS33}
[get_ports ad_busy]
set_property       -dict   {PACKAGE_PIN     E10   IOSTANDARD   LVCMOS33}
[get_ports ad_rd]
set_property       -dict   {PACKAGE_PIN     F9   IOSTANDARD   LVCMOS33}
[get_ports first_data]
set_property       -dict   {PACKAGE_PIN     K9   IOSTANDARD   LVCMOS33}
[get_ports ad_par_mode]

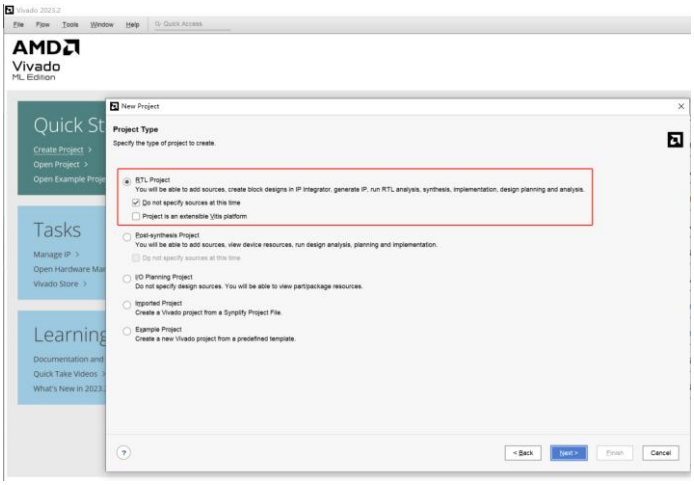

set_property       -dict   {PACKAGE_PIN   H14     IOSTANDARD   LVCMOS33}
[get_ports da_spi_clk_o]
set_property       -dict   {PACKAGE_PIN   C14     IOSTANDARD   LVCMOS33}
[get_ports da_nsync    ]
set_property       -dict   {PACKAGE_PIN   G14     IOSTANDARD   LVCMOS33}
[get_ports da_spi_mosi ]
set_property       -dict   {PACKAGE_PIN     C13   IOSTANDARD   LVCMOS33}
[get_ports da_spi_miso ]
```

3.12.3 工程搭建

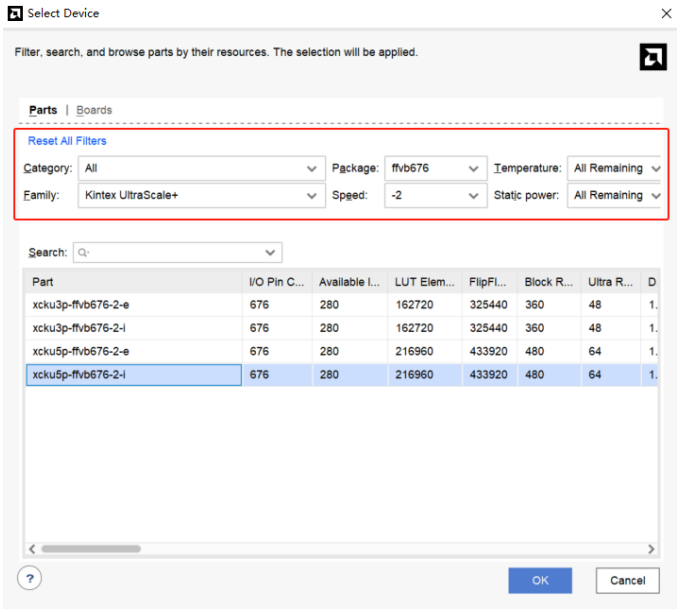
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



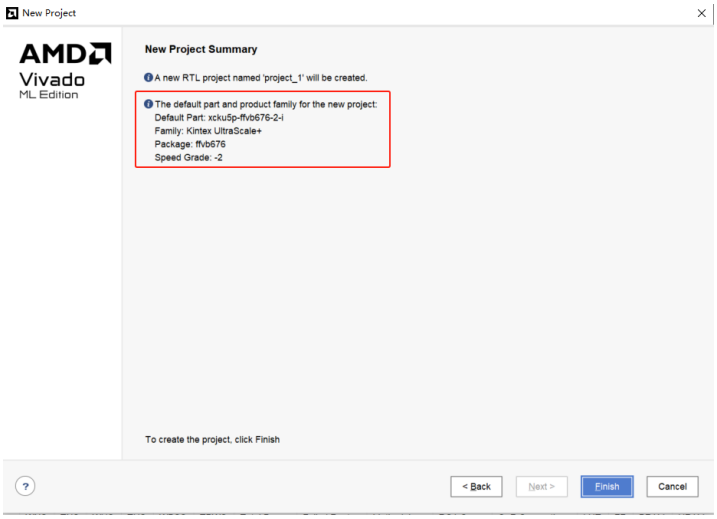
选择 RTL Project 并且勾选复选框，之后单击 NEXT



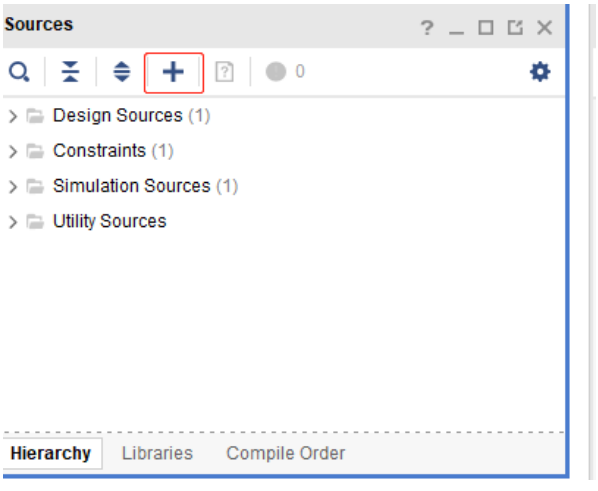
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

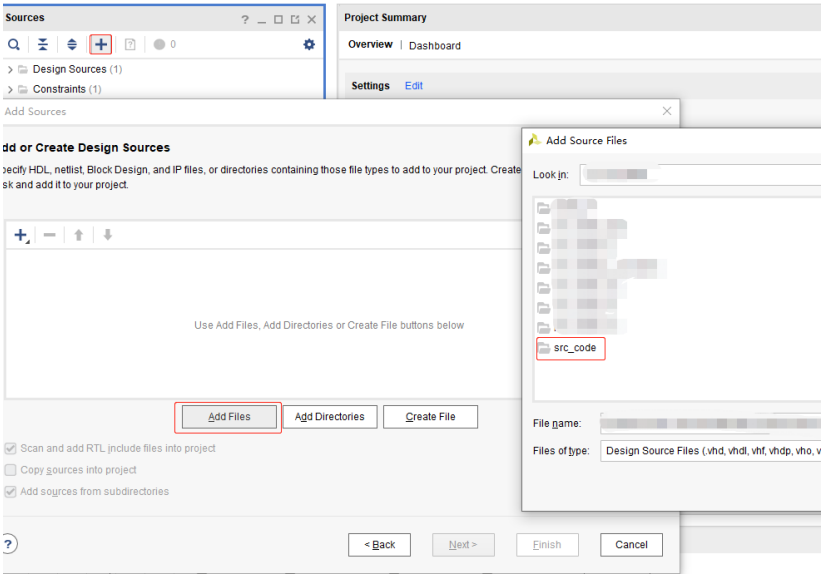


单击 Finish 完成工程创建

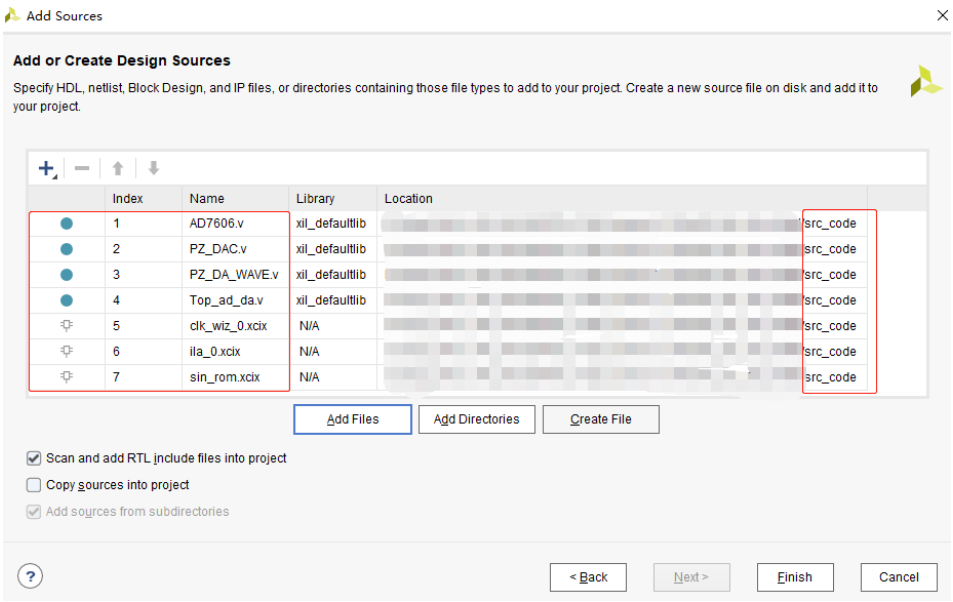


添加代码，显示界面如下

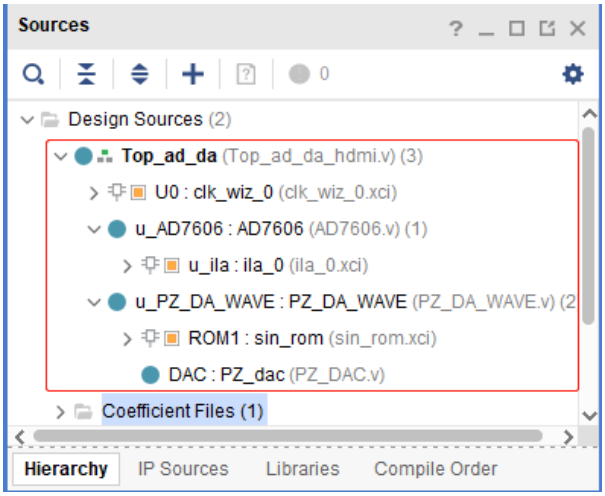




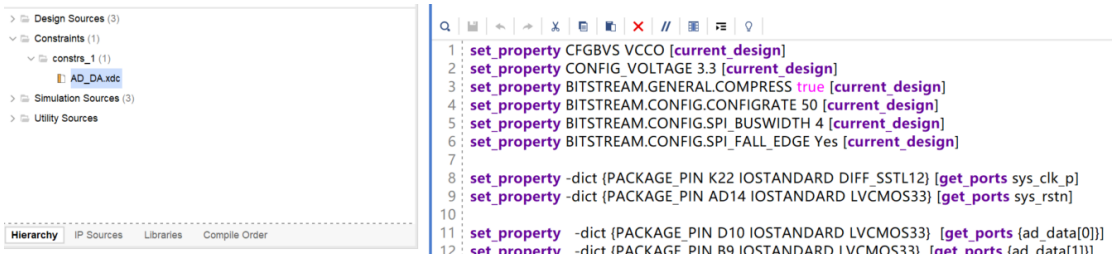
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

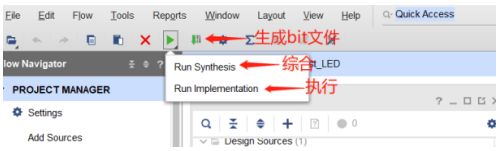


编译并且产生 bit 文件

1:单击综合

2:单击执行

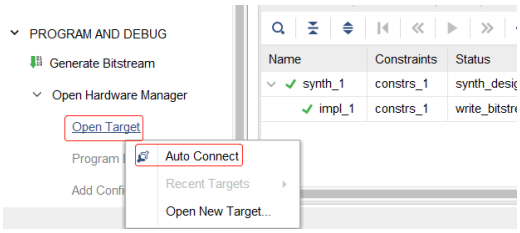
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

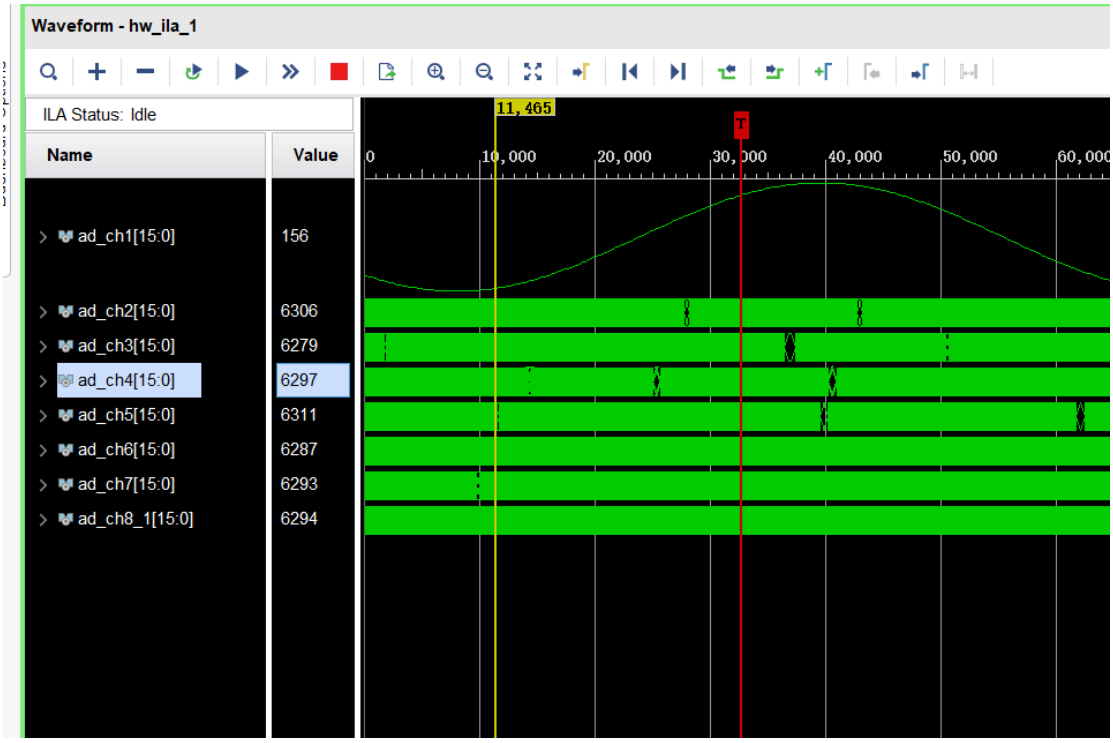


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.12.4 运行结果

下载过程下载完成后，ILA 显示如下（我们把 DA 的通道 1 接到了 AD 的通道 1）：



3.13 LCD 彩条显示例程

3.13.1 功能概述

LCD （ Liquid Crystal Display 的简称）液晶显示器。LCD 的构造是在两片平行的玻璃基板当中放置液晶盒，下基板玻璃上设置 TFT（薄膜晶体管），上基板玻璃上设置彩色滤光片，通过 TFT 上的信号与电压改变来控制液晶分子的转动方向，从而达到控制每个像素点偏振光出射与否而达到显示目的。

本实例实现在 LCD 显示屏上显示彩色条。

3.13.2 管脚配置

管脚配置如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

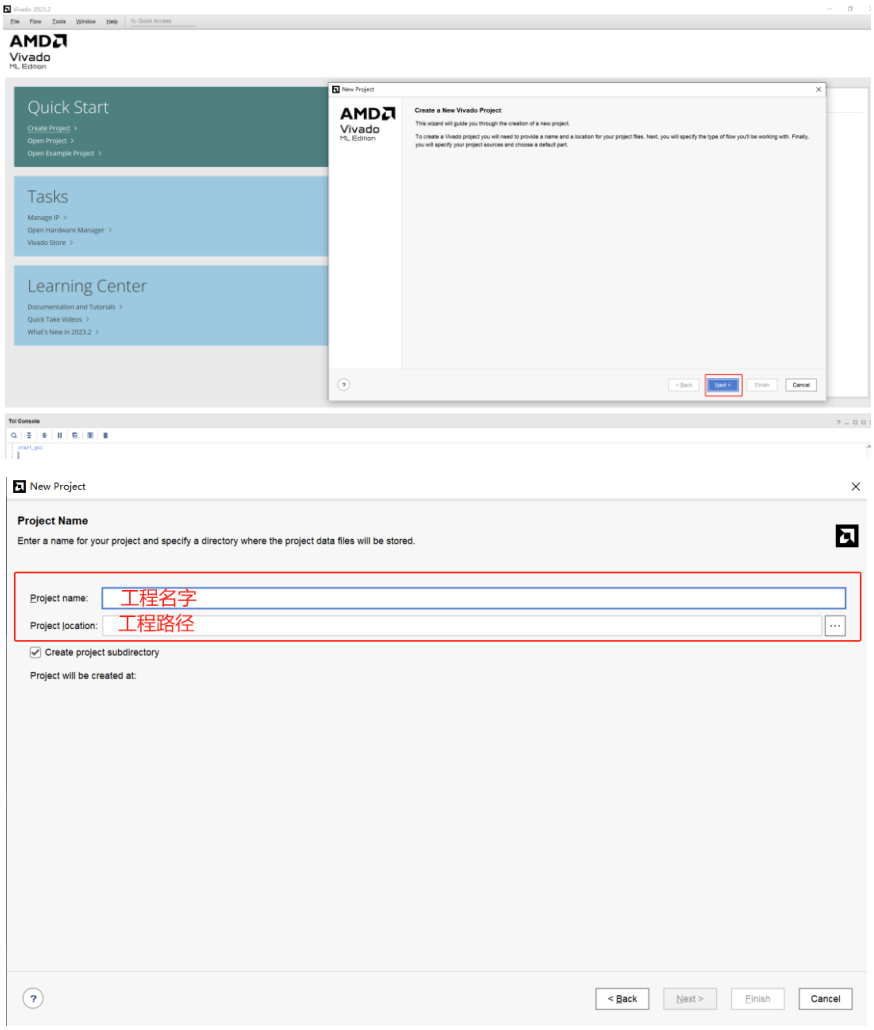
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
rst_n]


set_property -dict {PACKAGE_PIN E11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[0]}]
set_property -dict {PACKAGE_PIN G10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[1]}]
set_property -dict {PACKAGE_PIN  E10 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[2]}]
set_property -dict {PACKAGE_PIN  G9  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[3]}]
set_property -dict {PACKAGE_PIN  D11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[4]}]
set_property  -dict  {PACKAGE_PIN    F10        IOSTANDARD  LVCMOS33}
[get_ports {lcd_rgb[5]}]
```

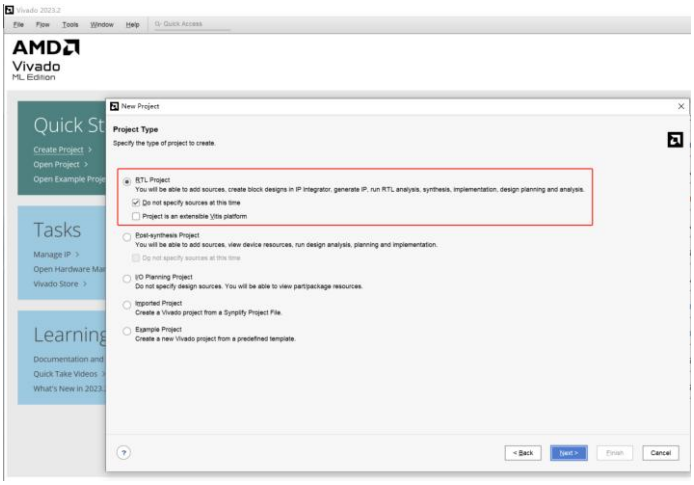
```
set_property -dict {PACKAGE_PIN  D10  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[6]}]
set_property -dict {PACKAGE_PIN  F9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[7]}]
set_property -dict {PACKAGE_PIN  C11  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[8]}]
set_property -dict {PACKAGE_PIN  B9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[9]}]
set_property -dict {PACKAGE_PIN  B11  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[10]}]
set_property -dict {PACKAGE_PIN  A9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[11]}]
set_property -dict {PACKAGE_PIN  D9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[12]}]
set_property -dict {PACKAGE_PIN  B10  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[13]}]
set_property -dict {PACKAGE_PIN  C9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[14]}]
set_property -dict {PACKAGE_PIN  A10  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[15]}]
set_property -dict {PACKAGE_PIN  J11  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[16]}]
set_property -dict {PACKAGE_PIN  H11  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[17]}]
set_property -dict {PACKAGE_PIN  J10  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[18]}]
set_property -dict {PACKAGE_PIN  G11  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[19]}]
set_property -dict {PACKAGE_PIN  K10  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[20]}]
set_property -dict {PACKAGE_PIN  J9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[21]}]
set_property -dict {PACKAGE_PIN  K9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[22]}]
set_property -dict {PACKAGE_PIN  H9   IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[23]}]
set_property -dict {PACKAGE_PIN  J13  IOSTANDARD LVCMOS33} [get_ports lcd_clk]
set_property -dict {PACKAGE_PIN  J15  IOSTANDARD LVCMOS33} [get_ports lcd_hs]
set_property -dict {PACKAGE_PIN  H13  IOSTANDARD LVCMOS33} [get_ports lcd_vs]
```

3.13.3 工程搭建

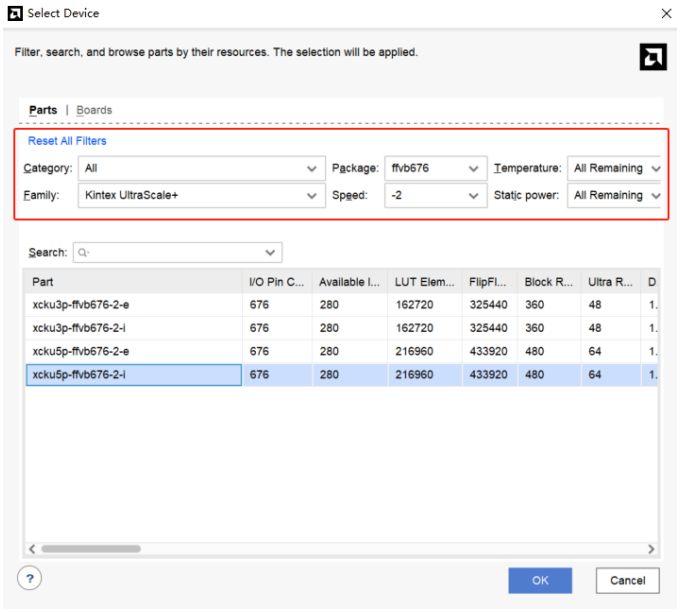
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



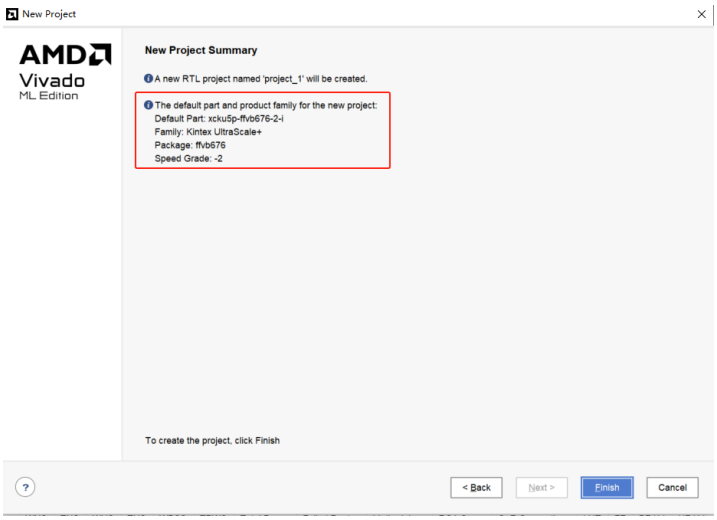
选择 RTL Project 并且勾选复选框，之后单击 NEXT



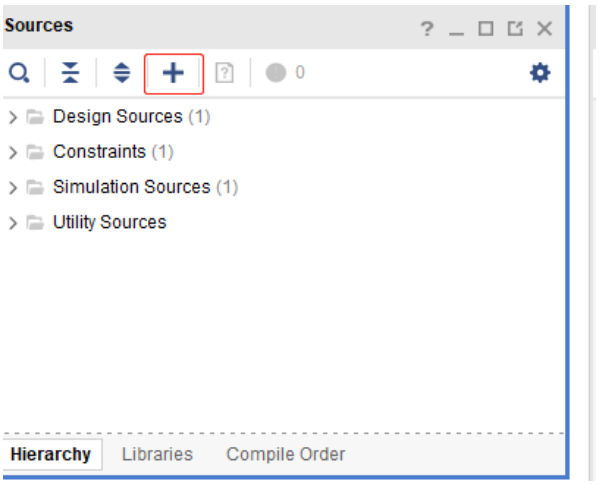
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

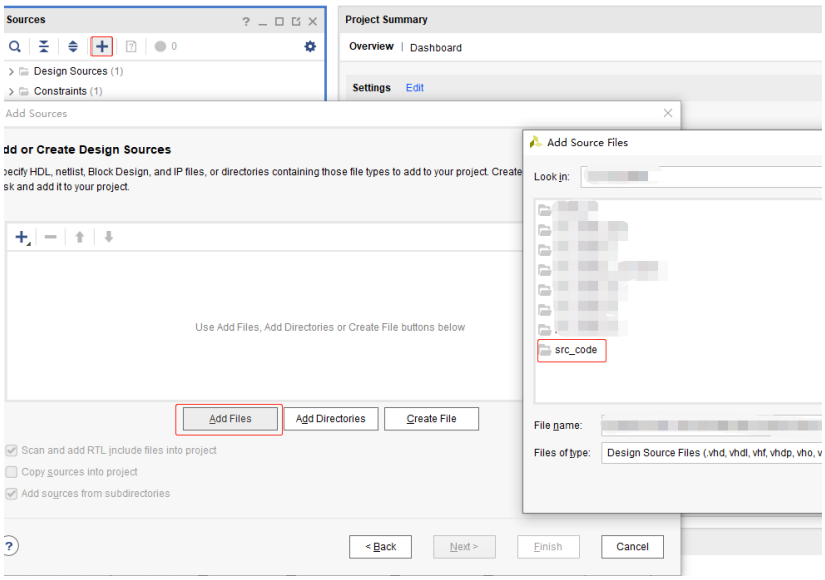


单击 Finish 完成工程创建

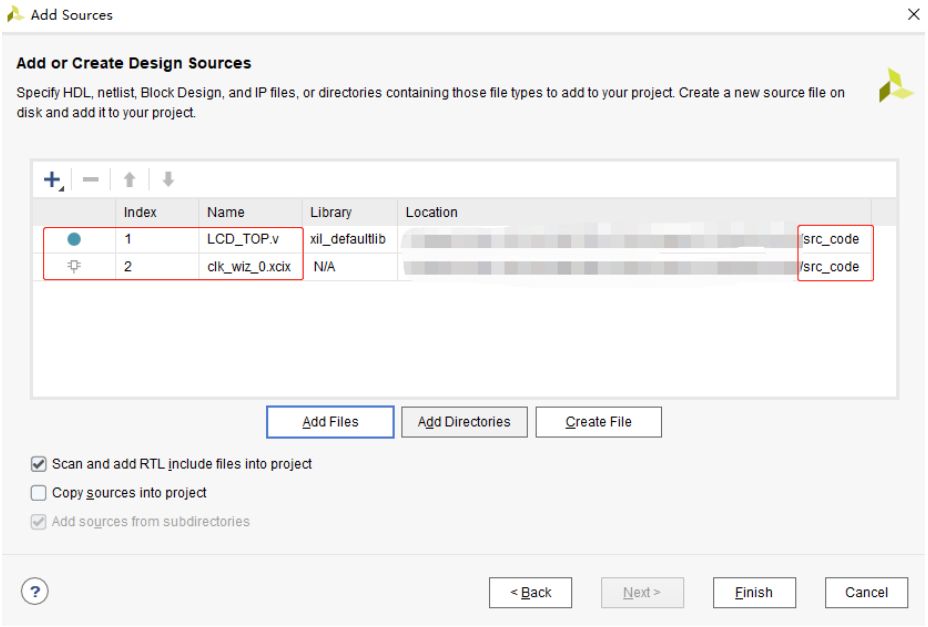


添加代码，显示界面如下





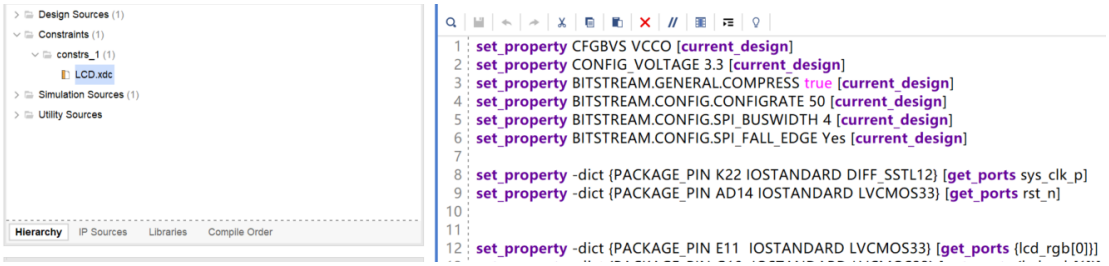
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

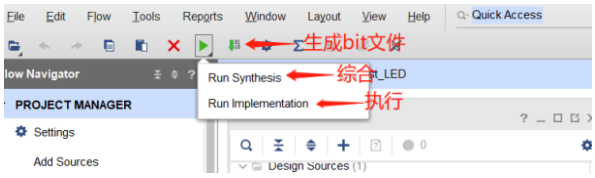


编译并且产生 bit 文件

1:单击综合

2:单击执行

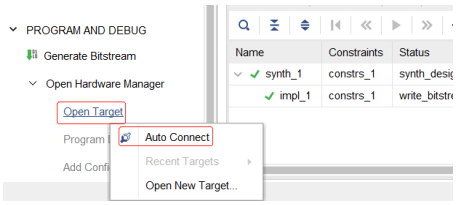
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



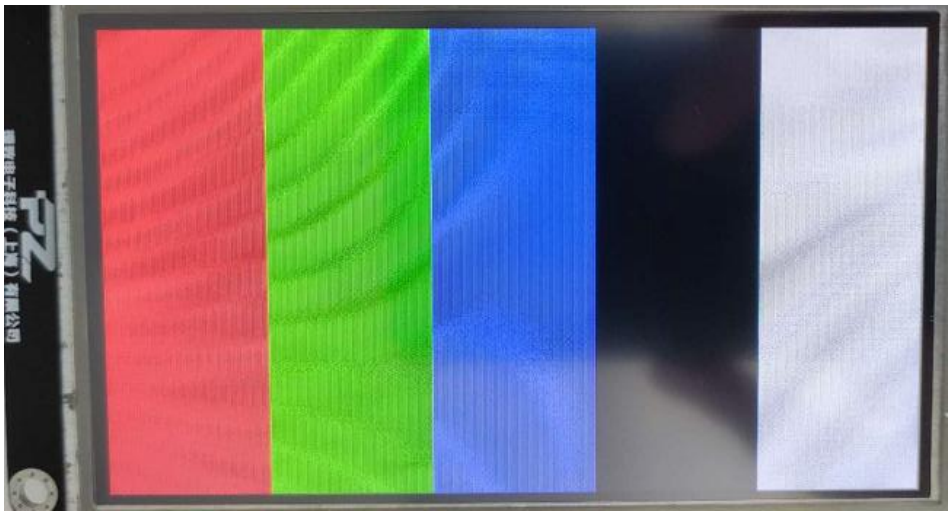
单击 Program Device（也可以从顶部单击 Program device）

弹出的对话框中有我们要下载的 Bit 文件



3.13.4 运行结果

下载完成后，可以看到 LCD 屏幕显示彩色条，如下图：



3.14 LCD 输出图片例程

3.14.1 功能概述

本实例通过从 FPGA 自带的 ROM 中读取图片数据，然后驱动 LCD 屏幕输出测试图片。

因为自带 ROM 的资源有限，所以选择用分辨率为 480*272 的图片。

3.14.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
rst_n]

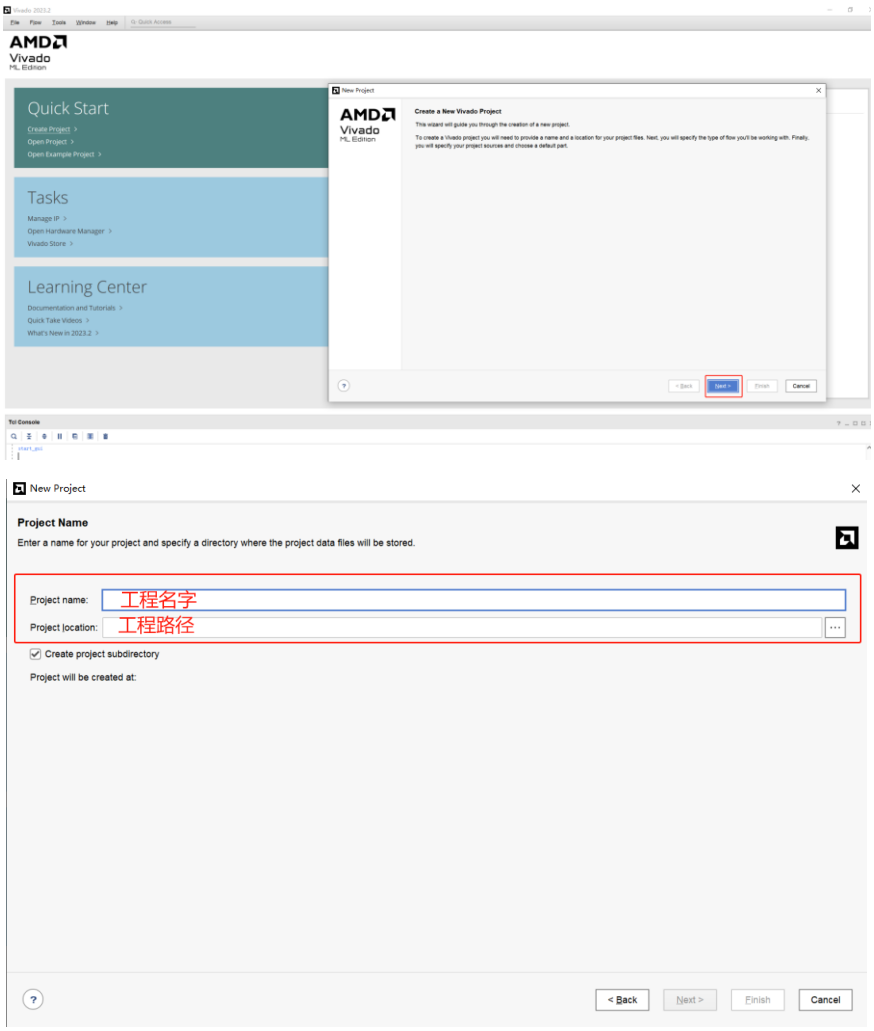

set_property -dict {PACKAGE_PIN E11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[0]}]
set_property -dict {PACKAGE_PIN G10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[1]}]
```

```
set_property -dict {PACKAGE_PIN  E10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[2]}]
set_property -dict {PACKAGE_PIN  G9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[3]}]
set_property -dict {PACKAGE_PIN  D11 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[4]}]
set_property -dict {PACKAGE_PIN  F10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[5]}]
set_property -dict {PACKAGE_PIN  D10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[6]}]
set_property -dict {PACKAGE_PIN  F9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[7]}]
set_property -dict {PACKAGE_PIN  C11 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[8]}]
set_property -dict {PACKAGE_PIN  B9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[9]}]
set_property -dict {PACKAGE_PIN  B11 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[10]}]
set_property -dict {PACKAGE_PIN  A9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[11]}]
set_property -dict {PACKAGE_PIN  D9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[12]}]
set_property -dict {PACKAGE_PIN  B10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[13]}]
set_property -dict {PACKAGE_PIN  C9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[14]}]
set_property -dict {PACKAGE_PIN  A10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[15]}]
set_property -dict {PACKAGE_PIN  J11 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[16]}]
set_property -dict {PACKAGE_PIN  H11 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[17]}]
set_property -dict {PACKAGE_PIN  J10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[18]}]
set_property -dict {PACKAGE_PIN  G11 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[19]}]
set_property -dict {PACKAGE_PIN  K10 IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[20]}]
set_property -dict {PACKAGE_PIN  J9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[21]}]
set_property -dict {PACKAGE_PIN  K9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[22]}]
set_property -dict {PACKAGE_PIN  H9  IOSTANDARD LVCMOS33} [get_ports {lcd_rgb[23]}]
```

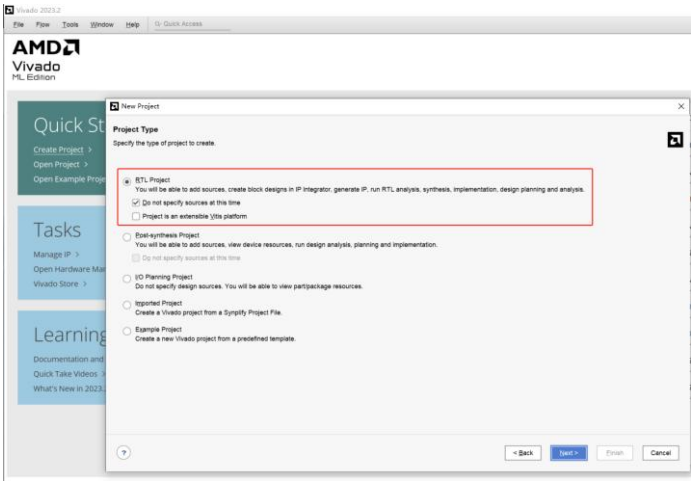
```
set_property -dict {PACKAGE_PIN  J13 IOSTANDARD LVCMOS33} [get_ports lcd_clk]
set_property -dict {PACKAGE_PIN  J15 IOSTANDARD LVCMOS33} [get_ports lcd_hs]
set_property -dict {PACKAGE_PIN  H13 IOSTANDARD LVCMOS33} [get_ports lcd_vs]
```

3.14.3 工程搭建

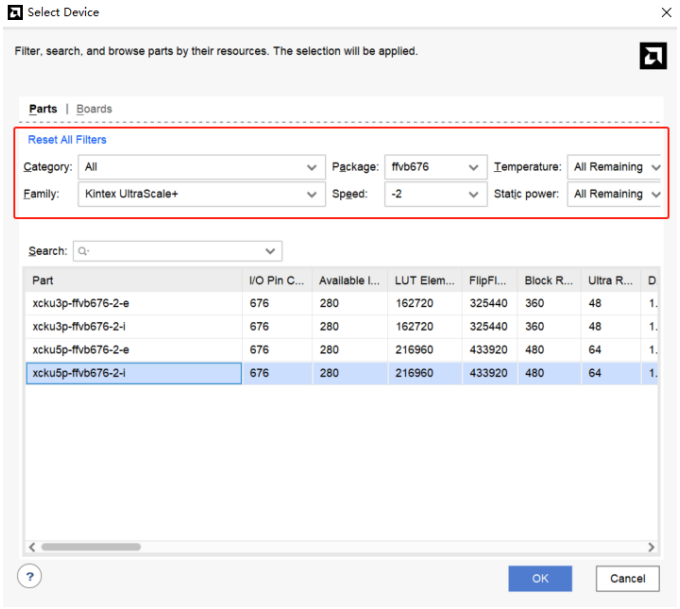
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



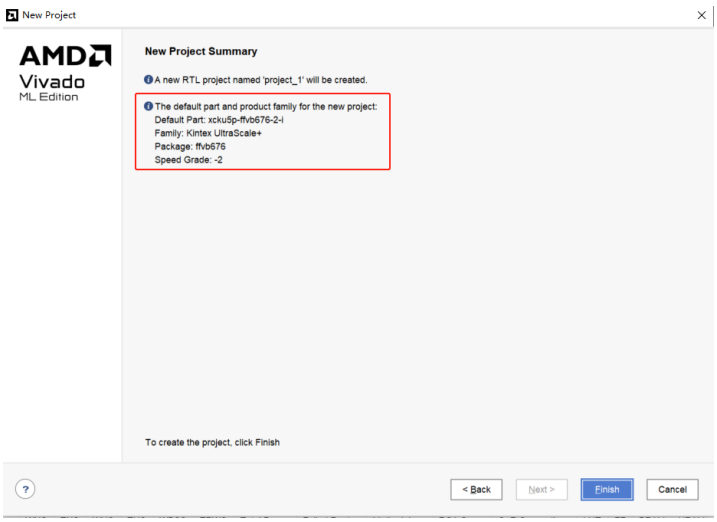
选择 RTL Project 并且勾选复选框，之后单击 NEXT



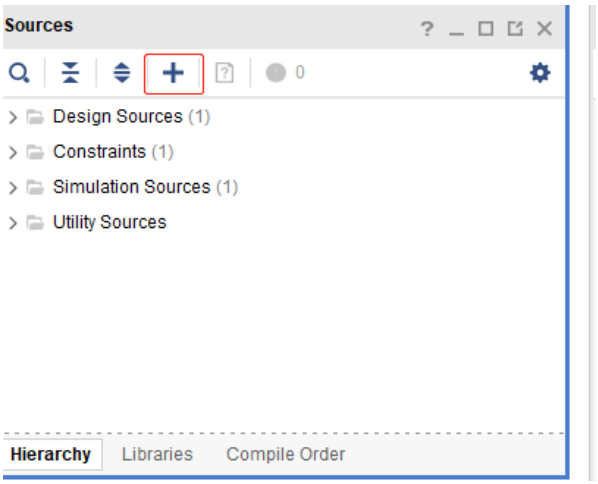
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

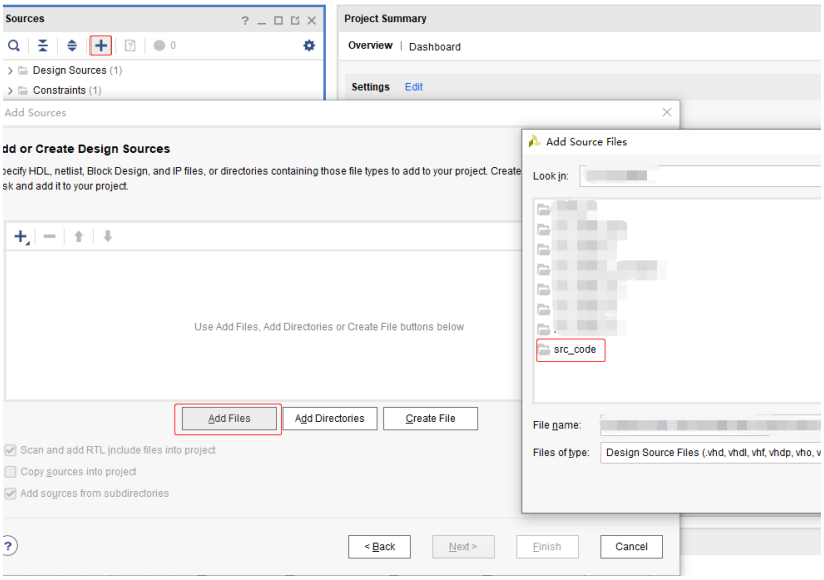


单击 Finish 完成工程创建

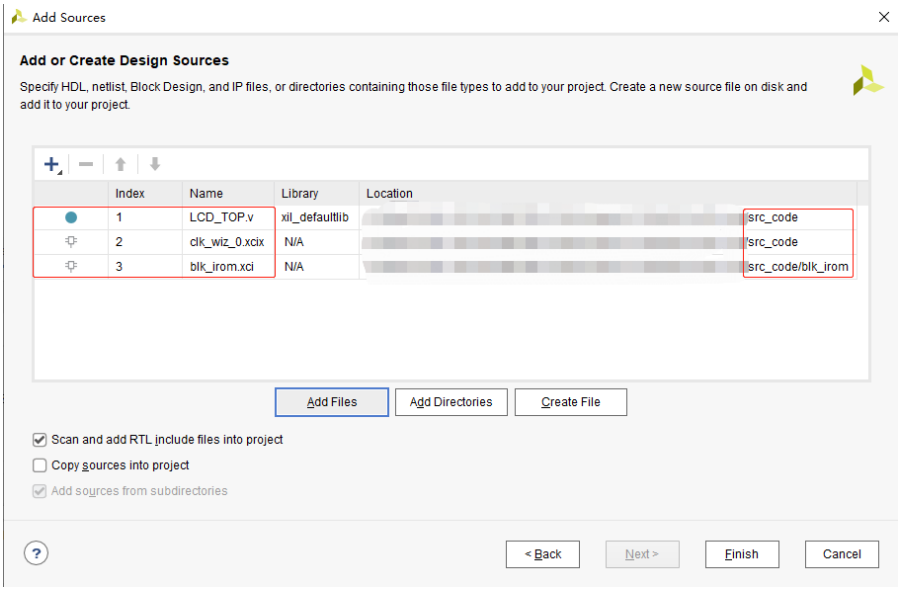


添加代码，显示界面如下

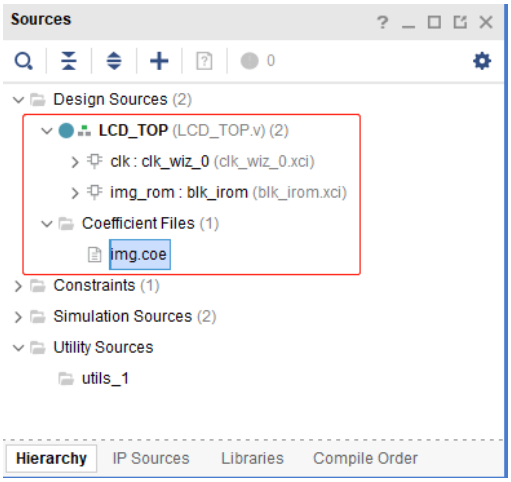




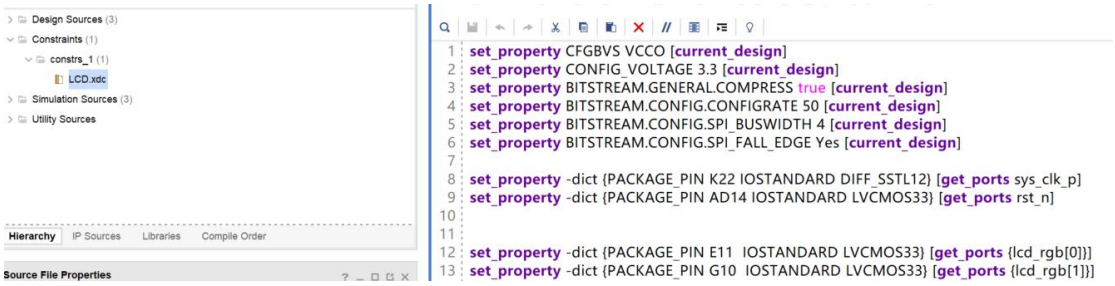
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

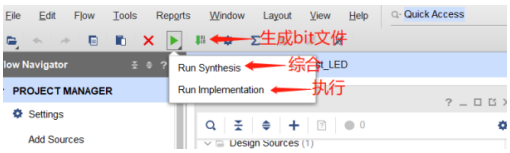


编译并且产生 bit 文件

1:单击综合

2:单击执行

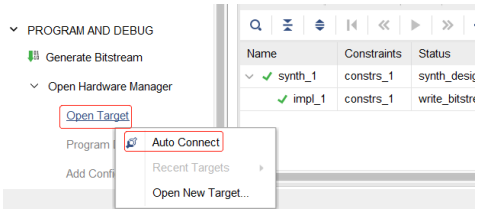
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.14.4 运行结果

下载过程下载完成后，LCD 屏显示图片如下图：



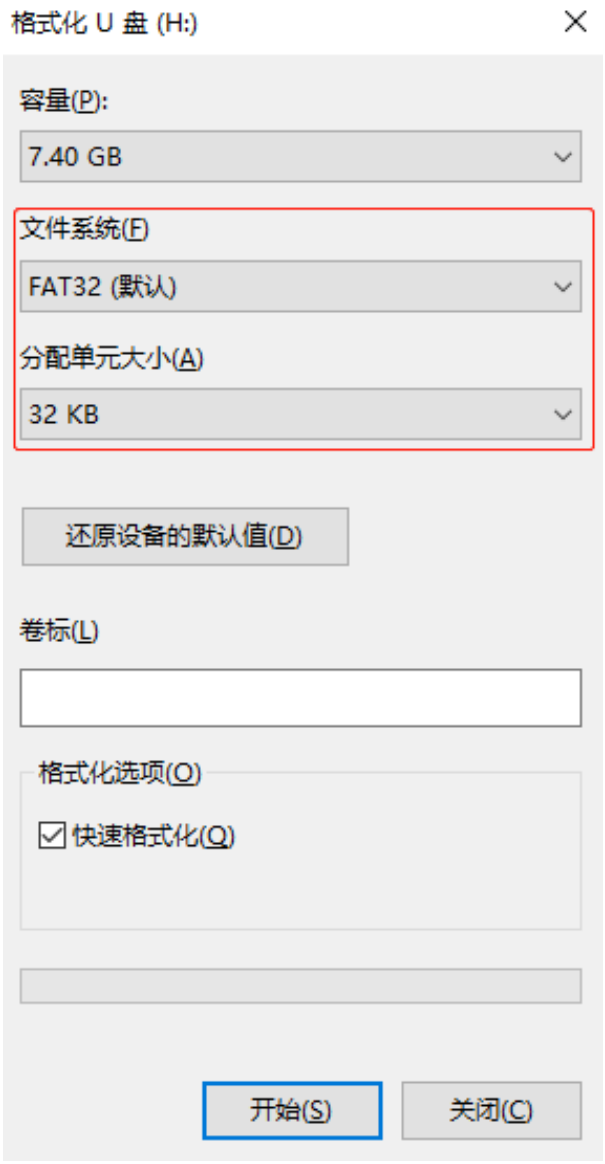
3.15 SD 卡读 BMP 图片 LCD 显示例程

3.15.1 功能概述

本实例通过从 SD 卡中读取图片数据，缓存到 DDR 中，然后读取 DDR 中的图片数据，驱动 LCD 屏幕输出显示 SD 卡中的图片。

因为 LCD 屏幕分辨率不高，所以选择用分辨率为 480*272 的 24 位 BMP 图片，图片在工程文件下有提供。

测试工程时，SD 卡需要先如下格式化之后，再拷贝 BMP 图片。



3.15.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
```

```
sys_rst_n]

set_property -dict {PACKAGE_PIN E11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[0]}]
set_property -dict {PACKAGE_PIN G10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[1]}]
set_property -dict {PACKAGE_PIN  E10 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[2]}]
set_property -dict {PACKAGE_PIN  G9  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[3]}]
set_property -dict {PACKAGE_PIN  D11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[4]}]
set_property  -dict  {PACKAGE_PIN    F10        IOSTANDARD  LVCMOS33}
[get_ports {lcd_rgb[5]}]
set_property -dict {PACKAGE_PIN  D10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[6]}]
set_property -dict {PACKAGE_PIN  F9    IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[7]}]
set_property -dict {PACKAGE_PIN  C11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[8]}]
set_property -dict {PACKAGE_PIN  B9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[9]}]
set_property -dict {PACKAGE_PIN  B11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[10]}]
set_property -dict {PACKAGE_PIN  A9  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[11]}]
set_property -dict {PACKAGE_PIN  D9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[12]}]
set_property -dict {PACKAGE_PIN  B10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[13]}]
set_property -dict {PACKAGE_PIN  C9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[14]}]
set_property -dict {PACKAGE_PIN  A10 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[15]}]
set_property -dict {PACKAGE_PIN  J11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[16]}]
set_property -dict {PACKAGE_PIN  H11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[17]}]
set_property -dict {PACKAGE_PIN  J10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[18]}]
set_property -dict {PACKAGE_PIN  G11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[19]}]
set_property -dict {PACKAGE_PIN  K10 IOSTANDARD LVCMOS33} [get_ports
```

```
{lcd_rgb[20]}]
set_property -dict {PACKAGE_PIN  J9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[21]}]
set_property -dict {PACKAGE_PIN  K9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[22]}]
set_property  -dict  {PACKAGE_PIN    H9          IOSTANDARD  LVCMOS33}
[get_ports {lcd_rgb[23]}]
set_property -dict {PACKAGE_PIN  J13 IOSTANDARD LVCMOS33} [get_ports
lcd_clk]
set_property -dict {PACKAGE_PIN  J15 IOSTANDARD LVCMOS33} [get_ports
lcd_hs]
set_property -dict {PACKAGE_PIN  H13 IOSTANDARD LVCMOS33} [get_ports
lcd_vs]
#----------------------------------SD 卡-----------------------------
------------
set_property -dict {PACKAGE_PIN C12 IOSTANDARD LVCMOS33} [get_ports
sd_clk]
set_property -dict {PACKAGE_PIN F13 IOSTANDARD LVCMOS33} [get_ports
sd_cs]
set_property -dict {PACKAGE_PIN B12 IOSTANDARD LVCMOS33} [get_ports
sd_mosi]
set_property -dict {PACKAGE_PIN G12 IOSTANDARD LVCMOS33} [get_ports
sd_miso]
#----------------------------------DDR4----------------------------
-----------
set_property PACKAGE_PIN H23 [get_ports {c0_ddr4_cs_n[0]}]
set_property PACKAGE_PIN G24 [get_ports {c0_ddr4_ck_t[0]}]
set_property PACKAGE_PIN G25 [get_ports {c0_ddr4_ck_c[0]}]
set_property PACKAGE_PIN H24 [get_ports {c0_ddr4_cke[0]}]
set_property PACKAGE_PIN L25 [get_ports c0_ddr4_reset_n]
set_property PACKAGE_PIN L24 [get_ports c0_ddr4_act_n]
set_property PACKAGE_PIN H21 [get_ports {c0_ddr4_ba[1]}]
set_property PACKAGE_PIN J25 [get_ports {c0_ddr4_ba[0]}]
set_property PACKAGE_PIN M24 [get_ports {c0_ddr4_odt[0]}]
set_property PACKAGE_PIN G26 [get_ports {c0_ddr4_bg[0]}]

set_property PACKAGE_PIN  F23 [get_ports {c0_ddr4_adr[0]}]
set_property PACKAGE_PIN  D25 [get_ports {c0_ddr4_adr[1]}]
set_property PACKAGE_PIN  D23 [get_ports {c0_ddr4_adr[2]}]
set_property PACKAGE_PIN  B26 [get_ports {c0_ddr4_adr[3]}]
set_property PACKAGE_PIN  E23 [get_ports {c0_ddr4_adr[4]}]
set_property PACKAGE_PIN  C26 [get_ports {c0_ddr4_adr[5]}]
set_property PACKAGE_PIN  F24 [get_ports {c0_ddr4_adr[6]}]
set_property PACKAGE_PIN  D26 [get_ports {c0_ddr4_adr[7]}]
```

```
set_property PACKAGE_PIN  H26 [get_ports {c0_ddr4_adr[8]}]
set_property PACKAGE_PIN  E25 [get_ports {c0_ddr4_adr[9]}]
set_property PACKAGE_PIN  C24 [get_ports {c0_ddr4_adr[10]}]
set_property PACKAGE_PIN  F22 [get_ports {c0_ddr4_adr[11]}]
set_property PACKAGE_PIN  B25 [get_ports {c0_ddr4_adr[12]}]
set_property PACKAGE_PIN  E26 [get_ports {c0_ddr4_adr[13]}]
set_property PACKAGE_PIN  D24 [get_ports {c0_ddr4_adr[14]}]
set_property PACKAGE_PIN  J23 [get_ports {c0_ddr4_adr[15]}]
set_property PACKAGE_PIN  G22 [get_ports {c0_ddr4_adr[16]}]


set_property PACKAGE_PIN C21 [get_ports {c0_ddr4_dqs_t[0]}]
set_property PACKAGE_PIN B21 [get_ports {c0_ddr4_dqs_c[0]}]
set_property PACKAGE_PIN A17 [get_ports {c0_ddr4_dqs_t[1]}]
set_property PACKAGE_PIN A18 [get_ports {c0_ddr4_dqs_c[1]}]
set_property PACKAGE_PIN F20 [get_ports {c0_ddr4_dqs_t[2]}]
set_property PACKAGE_PIN E20 [get_ports {c0_ddr4_dqs_c[2]}]
set_property PACKAGE_PIN E16 [get_ports {c0_ddr4_dqs_t[3]}]
set_property PACKAGE_PIN E17 [get_ports {c0_ddr4_dqs_c[3]}]

set_property PACKAGE_PIN  A22 [get_ports {c0_ddr4_dm_dbi_n[0]}]
set_property PACKAGE_PIN  C18 [get_ports {c0_ddr4_dm_dbi_n[1]}]
set_property PACKAGE_PIN  H18 [get_ports {c0_ddr4_dm_dbi_n[2]}]
set_property PACKAGE_PIN  G15 [get_ports {c0_ddr4_dm_dbi_n[3]}]



set_property PACKAGE_PIN C23  [get_ports {c0_ddr4_dq[0]}]
set_property PACKAGE_PIN B22  [get_ports {c0_ddr4_dq[1]}]
set_property PACKAGE_PIN A24  [get_ports {c0_ddr4_dq[2]}]
set_property PACKAGE_PIN C22  [get_ports {c0_ddr4_dq[3]}]
set_property PACKAGE_PIN B24  [get_ports {c0_ddr4_dq[4]}]
set_property PACKAGE_PIN D21  [get_ports {c0_ddr4_dq[5]}]
set_property PACKAGE_PIN A25  [get_ports {c0_ddr4_dq[6]}]
set_property PACKAGE_PIN E21  [get_ports {c0_ddr4_dq[7]}]

set_property PACKAGE_PIN A19  [get_ports {c0_ddr4_dq[8]}]
set_property PACKAGE_PIN B17  [get_ports {c0_ddr4_dq[9]}]
set_property PACKAGE_PIN A20  [get_ports {c0_ddr4_dq[10]}]
set_property PACKAGE_PIN C17  [get_ports {c0_ddr4_dq[11]}]
set_property PACKAGE_PIN B20  [get_ports {c0_ddr4_dq[12]}]
set_property PACKAGE_PIN B15  [get_ports {c0_ddr4_dq[13]}]
set_property PACKAGE_PIN B19  [get_ports {c0_ddr4_dq[14]}]
set_property PACKAGE_PIN A15  [get_ports {c0_ddr4_dq[15]}]
```

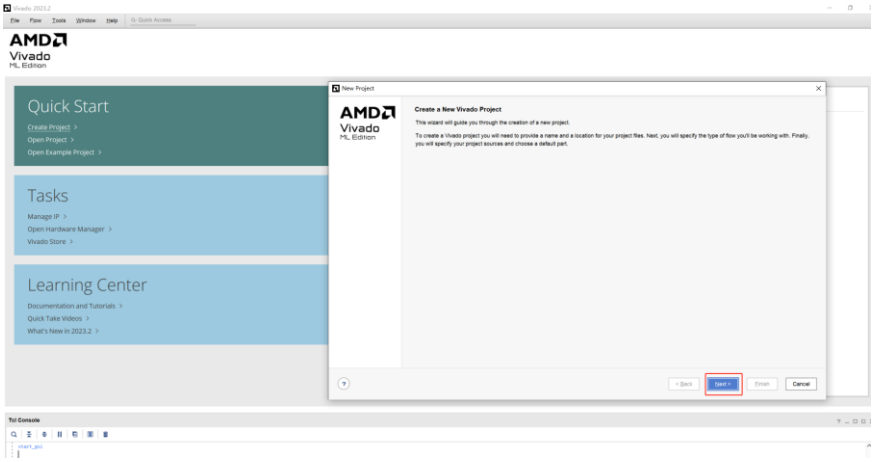
```
set_property PACKAGE_PIN D19 [get_ports {c0_ddr4_dq[16]}]
set_property PACKAGE_PIN F19 [get_ports {c0_ddr4_dq[17]}]
set_property PACKAGE_PIN D20 [get_ports {c0_ddr4_dq[18]}]
set_property PACKAGE_PIN G20 [get_ports {c0_ddr4_dq[19]}]
set_property PACKAGE_PIN G21 [get_ports {c0_ddr4_dq[20]}]
set_property PACKAGE_PIN F18 [get_ports {c0_ddr4_dq[21]}]
set_property PACKAGE_PIN D18 [get_ports {c0_ddr4_dq[22]}]
set_property PACKAGE_PIN E18 [get_ports {c0_ddr4_dq[23]}]

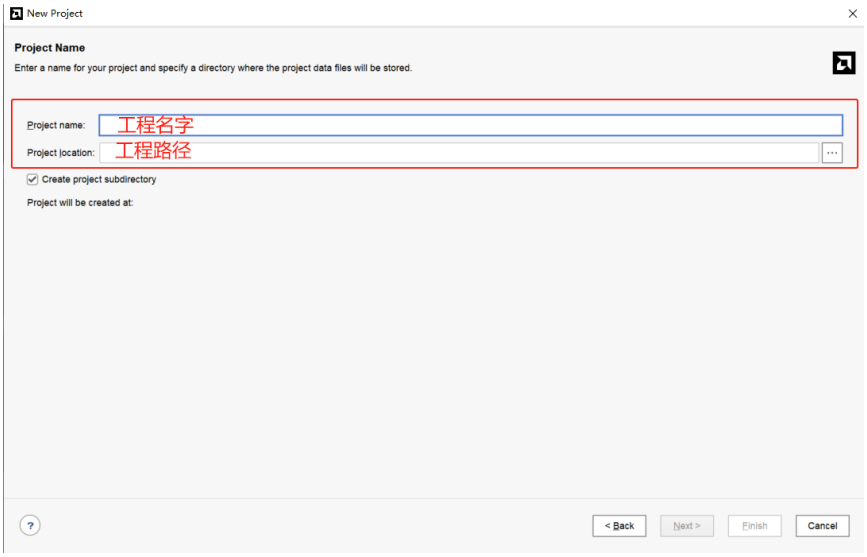
set_property PACKAGE_PIN E15 [get_ports {c0_ddr4_dq[24]}]
set_property PACKAGE_PIN G16 [get_ports {c0_ddr4_dq[25]}]
set_property PACKAGE_PIN D15 [get_ports {c0_ddr4_dq[26]}]
set_property PACKAGE_PIN G17 [get_ports {c0_ddr4_dq[27]}]
set_property PACKAGE_PIN D16 [get_ports {c0_ddr4_dq[28]}]
set_property PACKAGE_PIN H17 [get_ports {c0_ddr4_dq[29]}]
set_property PACKAGE_PIN C16 [get_ports {c0_ddr4_dq[30]}]
set_property PACKAGE_PIN H16 [get_ports {c0_ddr4_dq[31]}]


set_property INTERNAL_VREF 0.6 [get_iobanks 64]
```

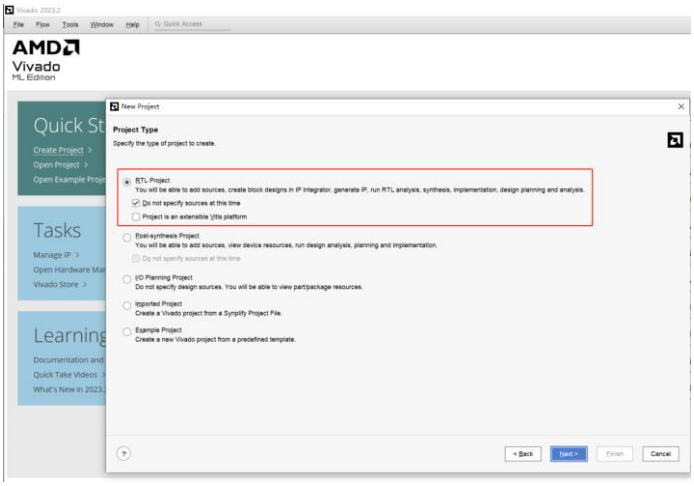
3.15.3 工程搭建

先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择

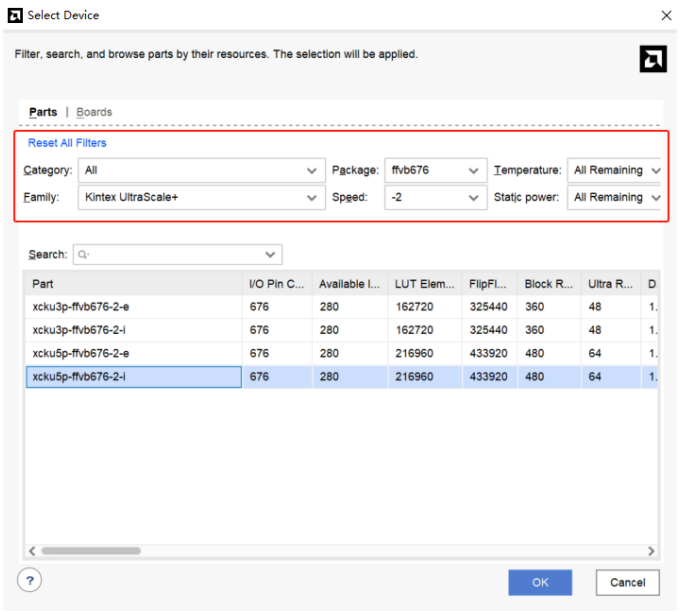




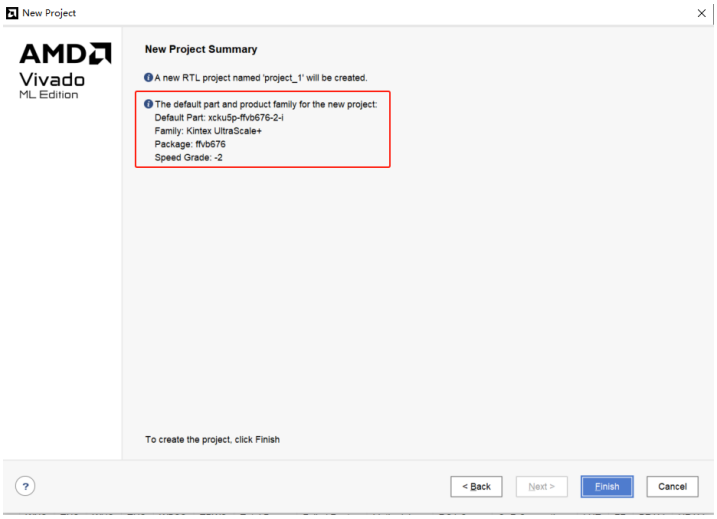
选择 RTL Project 并且勾选复选框，之后单击 NEXT



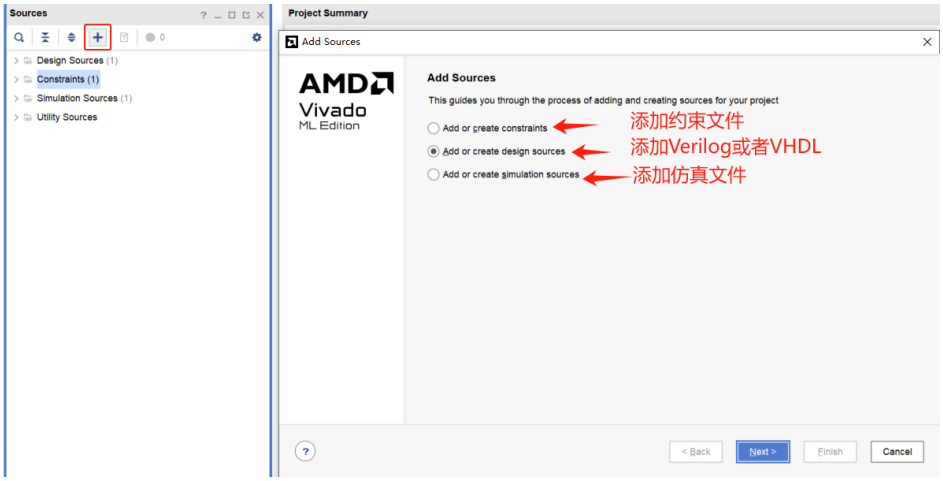
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

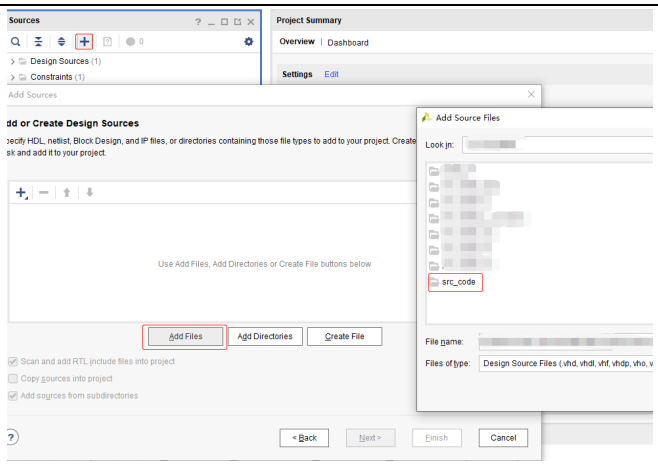


单击 Finish 完成工程创建

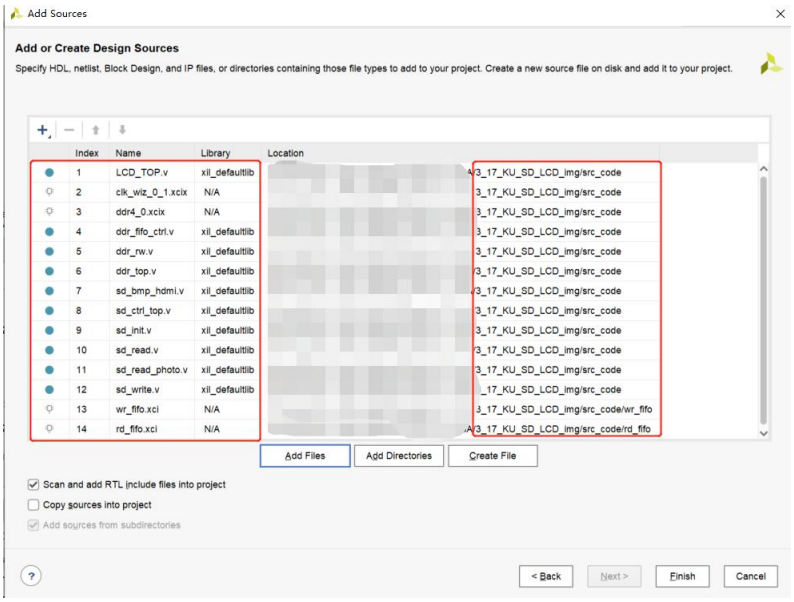


添加代码，显示界面如下

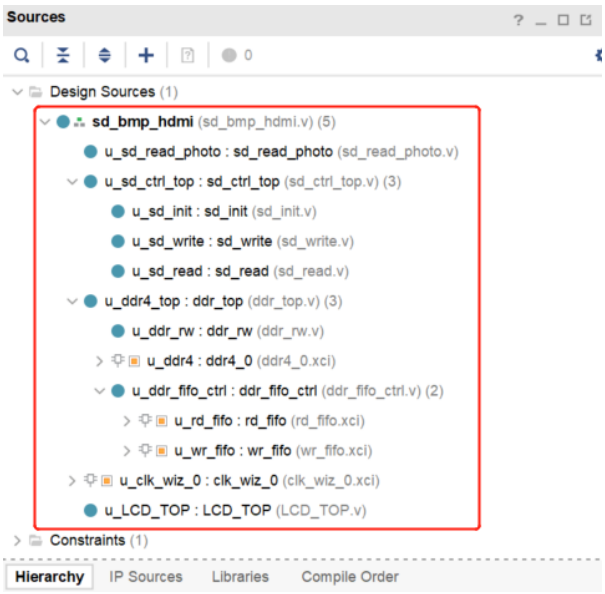




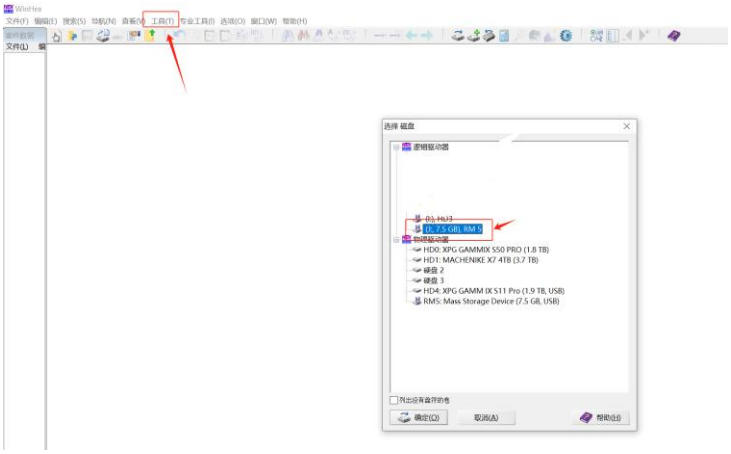
添加代码文件（代码文件在工程路径下）



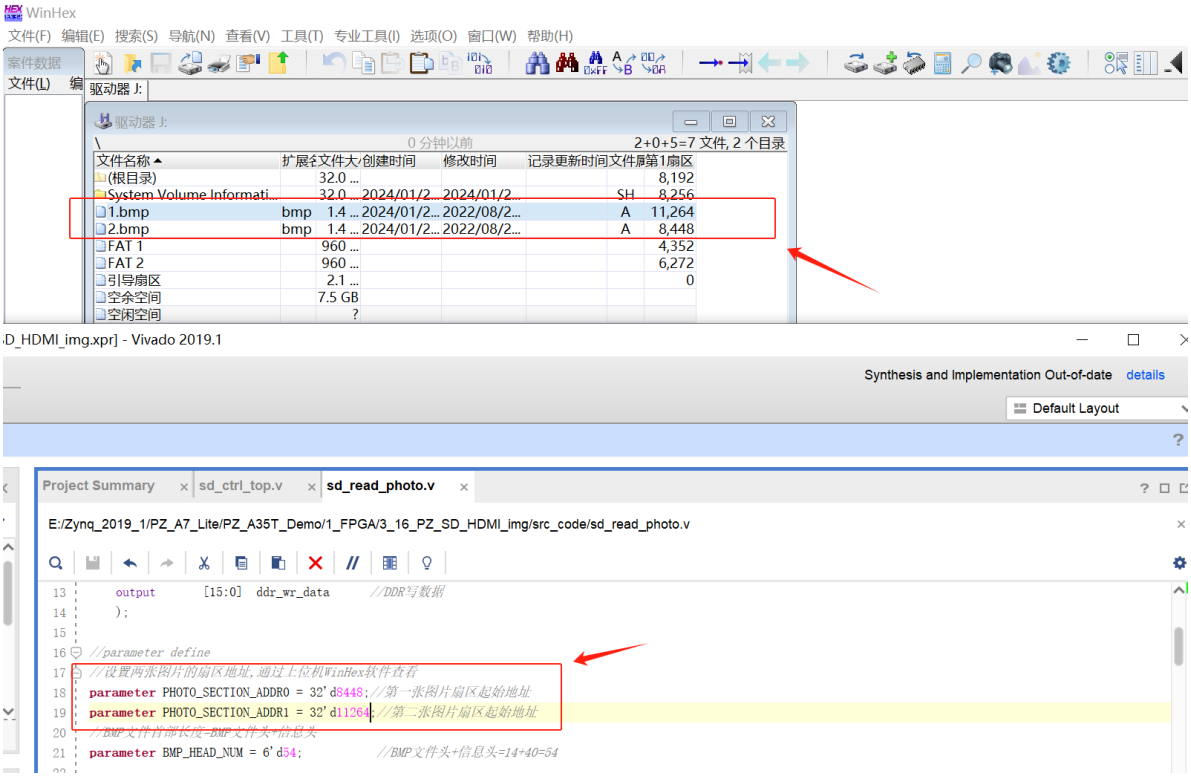
添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



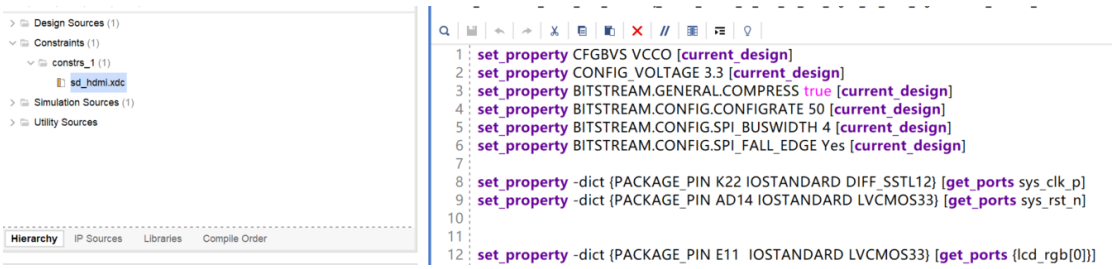
我们用 WinHex 工具（在工程文件夹下），点开工具选择 SD，看下图片在 SD 里面的位置



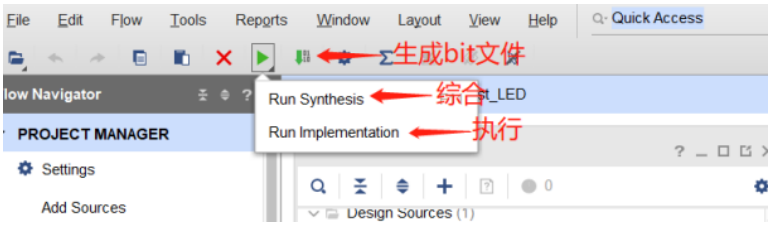
我们对比下工具读取的扇区与代码里面的扇区地址是否一致，如果一致的话，代码不需要修改，不一致的话，需要把代码里面的扇区地址改成与工具读取到的一致，不然显示会乱（SD 卡不一样，地址可能会不一样）



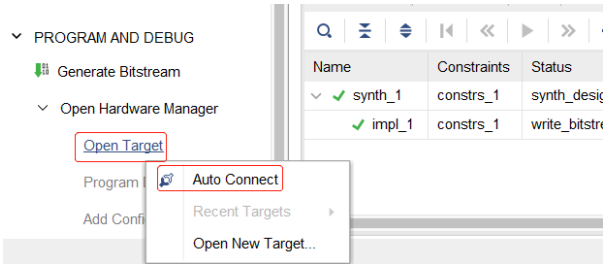
添加管脚约束文件



编译并且产生 bit 文件
1:单击综合
2:单击执行
3:单击产生 bit



下载程序
1：给开发板通电，并且连接下载器
2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



单击 Program Device（也可以从顶部单击 Program device）
弹出的对话框中有我们要下载的 Bit 文件



3.15.4 运行结果

下载过程下载完成后，显示屏显示图片如下图：





3.16 基于 RTL8211 实现千兆 UDP 例程

3.16.1 千兆以太网、UDP 协议简介及功能概述

3.16.1.1 千兆以太网简介

千兆以太网是建立在基础以太网标准之上的技术。千兆以太网和大量使用的以太网与快速以太网完全兼容，并利用了原以太网标准所规定的全部技术规范，其中包括 CSMA/CD 协议、以太网帧、全双工、流量控制以及 IEEE803.16.3 标准中所定义的管理对象。作为以太网的一个组成部分，千兆以太网也支持流量管理技术，它保证在以太网上的服务质量，这些技术包括 IEEE 803.18.1P 第二层优先级、第三层优先级的 QoS 编码位、特别服务和资源预留协议（RSVP）。

千兆以太网还利用 IEEE 803.18.1QVLAN 支持、第四层过滤、千兆位的第三层交换。千兆以太网原先是作为一种交换技术设计的，采用光纤作为上行链路，用于楼宇之间的连接。之后，在服务器的连接和骨干网中，千兆以太网获得广泛应用，由于 IEEE803.18.3ab 标准（采用 5 类及以上非屏蔽双绞线的千兆以太网标准）的出台，千兆以太网可适用于任何大中小型企事业单位。

千兆以太网已经发展成为主流网络技术。大到成千上万人的大型企业，小到几十人的中小型企业，在建设企业局域网时都会把千兆以太网技术作为首选的高速网络技术。千兆以太网技术甚至正在取代 ATM 技术，成为城域网建设的主力军。

3.16.1.2 UDP 协议简介

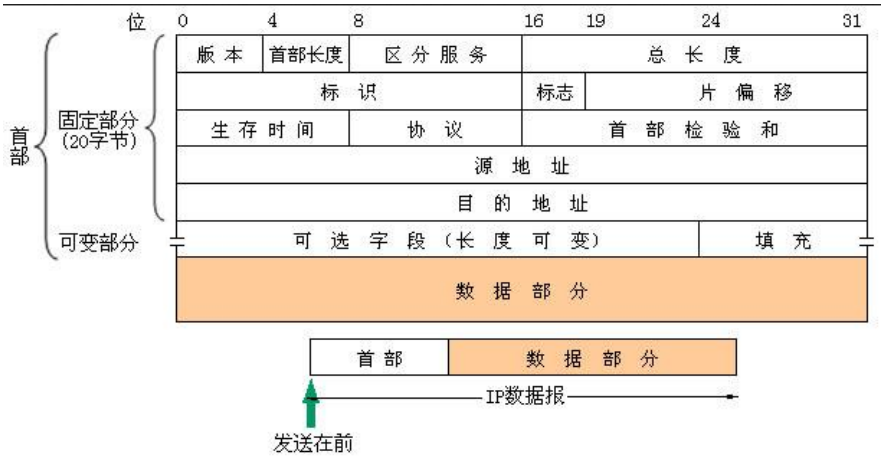
UDP 是传输层的协议，功能即为在 IP 的数据报服务之上增加了最基本的服务：复用和分用以及差错检测。

TCP 是面向连接的，需要三次握手建立连接之后再传输数据，而是 UDP 面向无连接的，它并不能保证信息交付，也不能保证按序交互，也不跟踪连接状态，也不需要拥塞控制。

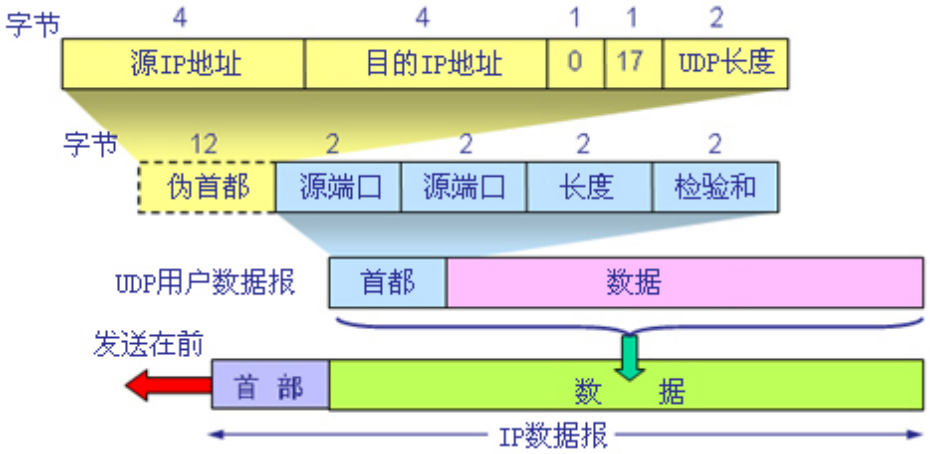
要了解 UDP 和为什么它通常被称为“空协议”，我们首先需要了解一下互联网协议（IP），它位于 TCP 和 UDP 协议层下面。IP 层主要任务就是基于地址将数据报从源主机发送到目的主机。要做到这一点，消息都封装在一个 IP 包，标识源和目的地址，以及一些其他路由参数。

我们再次强调一下上面提到的数据报这个术语的含义：IP 层提供了不可靠的数据传输，既没有消息确认，也没有丢失通知， IP 层直接把这一层的不可靠性暴露给上层。如果一个数据报在传输过程中因为某个路由节点拥塞，高负荷，或因其他原因丢失，那么由 IP 上层的协议来检测、恢复并重传数据 ，当然这是在上层有这个需求的时候！

IPv4 的首部结构如下：



UDP 协议会用自己的分组结构封装用户信息，其数据格式如下：



如上图所示，我们在 UDP 数据报里增加了源端口和目标端口，这样就使得当 IP 分组被送到接收端后，接收端就可以拆开 UDP 分组，根据目标端口找到对应的应用程序，然后再把数据传递给应用程序。

从 IP 和 UDP 的数据格式可以看到，它们的首部都带有校验和，都可以用来校验数据，那么应用程序即使忽略 UDP 的校验和也不影响数据完整性，校验和字段是可选的。这意味着 UDP 层所有的错误检测和纠错，可以委托给上述应用层校验。说到底，UDP 仅仅是在 IP 层上通过嵌入应用程序的源端口和目标端口，提供了一个“应用程序多路复用”机制。

由此可以得到 UDP 的特征如下：

不保证消息交付：不确认、不重传、无超时；

不保证交付顺序：不设置包序号、不重排、不发生队首阻塞；

不跟踪连接状态：不必建立连接或重启状态机；

不需要拥塞控制：不内置客户端或网络反馈机。

3.16.1.3 功能概述

本例程基于 RTL8211 芯片，通过纯逻辑完成 UDP 协议，实现跟上位机软件进行 UDP 数据 LOOP 回环。

3.16.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

create_clock -period 8.000 -name eth_rxc [get_ports net_rxc]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rst_n]

set_property -dict {PACKAGE_PIN A13  IOSTANDARD LVCMOS33} [get_ports
eth1_mdc]
set_property -dict {PACKAGE_PIN A12  IOSTANDARD LVCMOS33} [get_ports
eth1_mdio]
set_property -dict {PACKAGE_PIN V23 IOSTANDARD LVCMOS18} [get_ports
net1_rxc]
```

```
set_property -dict {PACKAGE_PIN W23 IOSTANDARD LVCMOS18} [get_ports net1_rx_ctl]
set_property -dict {PACKAGE_PIN R25 IOSTANDARD LVCMOS18} [get_ports {net1_rxd[0]}]
set_property -dict {PACKAGE_PIN R26 IOSTANDARD LVCMOS18} [get_ports {net1_rxd[1]}]
set_property -dict {PACKAGE_PIN N24 IOSTANDARD LVCMOS18} [get_ports {net1_rxd[2]}]
set_property -dict {PACKAGE_PIN P24 IOSTANDARD LVCMOS18} [get_ports {net1_rxd[3]}]

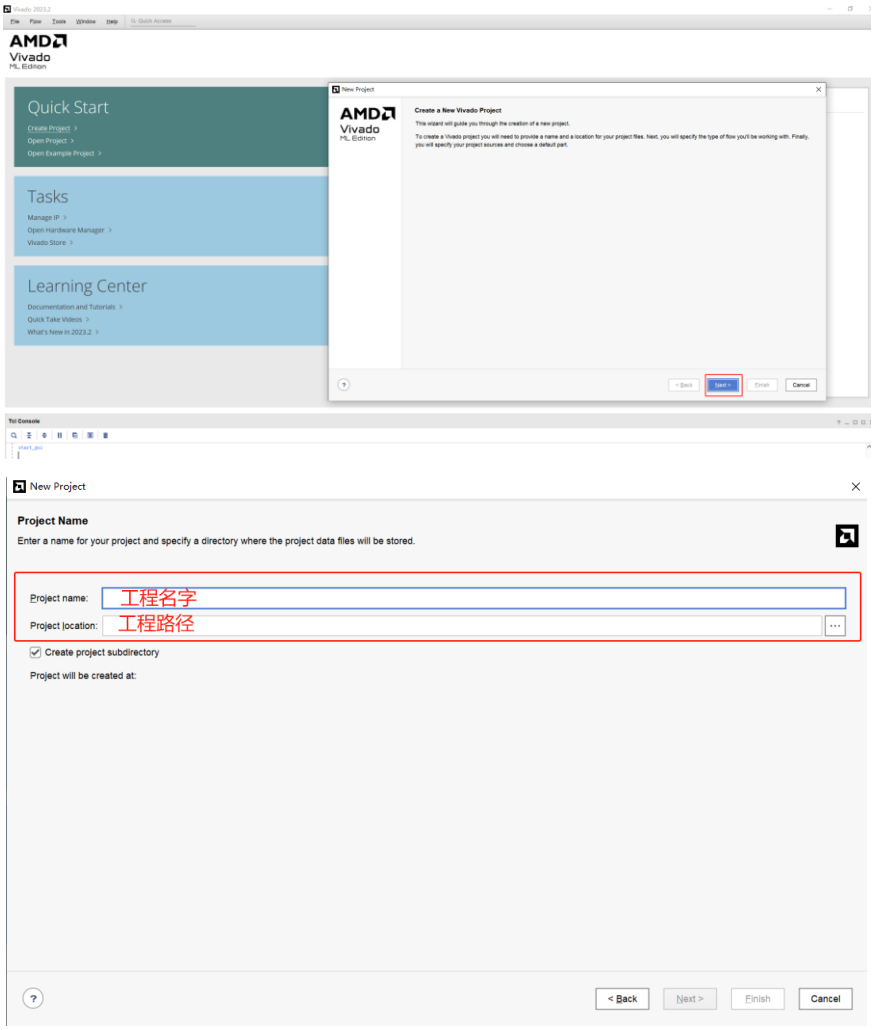
set_property -dict {PACKAGE_PIN N21 IOSTANDARD LVCMOS18} [get_ports net1_txc]
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS18} [get_ports net1_tx_ctl]
set_property -dict {PACKAGE_PIN R20 IOSTANDARD LVCMOS18} [get_ports {net1_txd[0]}]
set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS18} [get_ports {net1_txd[1]}]
set_property -dict {PACKAGE_PIN W19 IOSTANDARD LVCMOS18} [get_ports {net1_txd[2]}]
set_property -dict {PACKAGE_PIN W20 IOSTANDARD LVCMOS18} [get_ports {net1_txd[3]}]



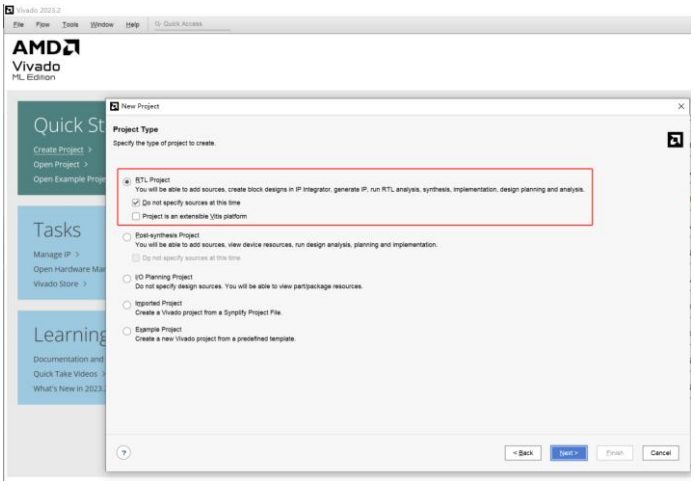
set_property SLEW FAST [get_ports  net1_txc]
set_property SLEW FAST [get_ports  net1_tx_ctl]
set_property SLEW FAST [get_ports {net1_txd[*]}]
```

3.16.3 工程搭建

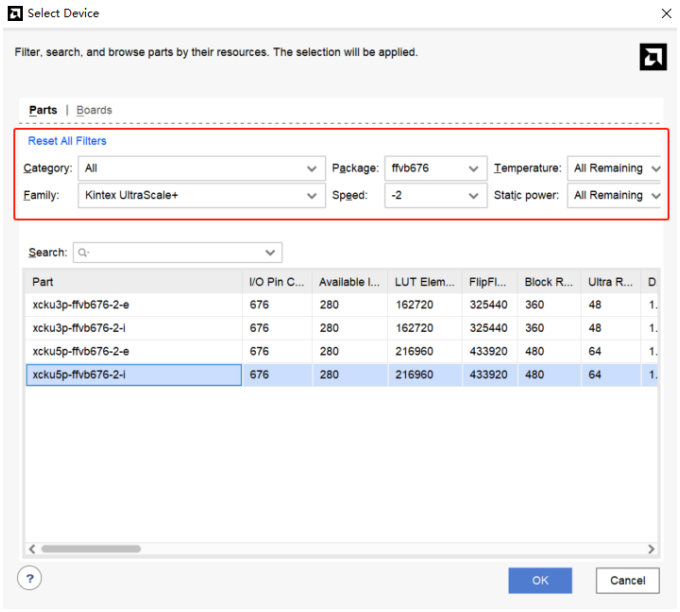
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



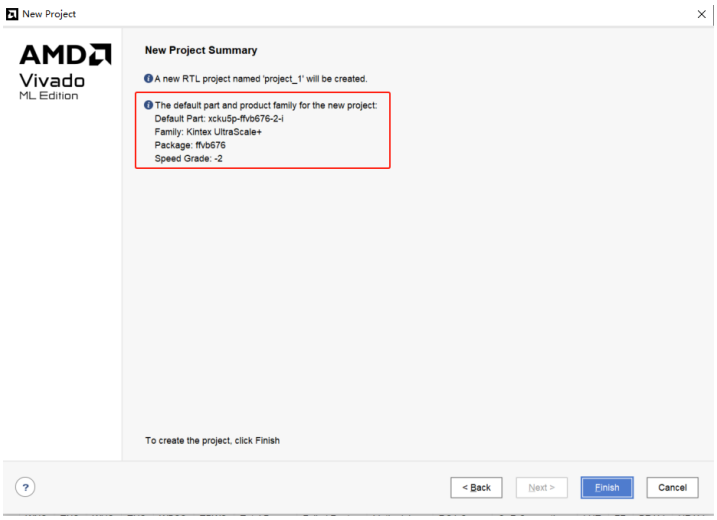
选择 RTL Project 并且勾选复选框，之后单击 NEXT



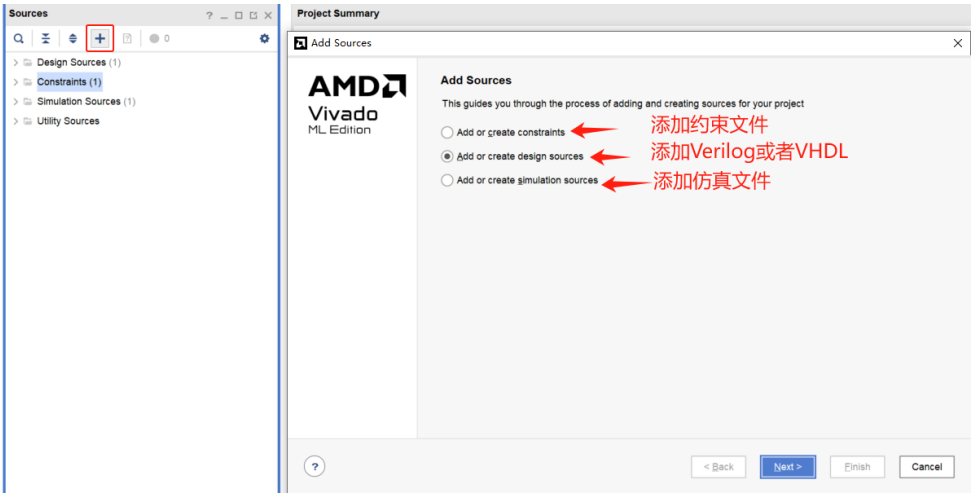
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

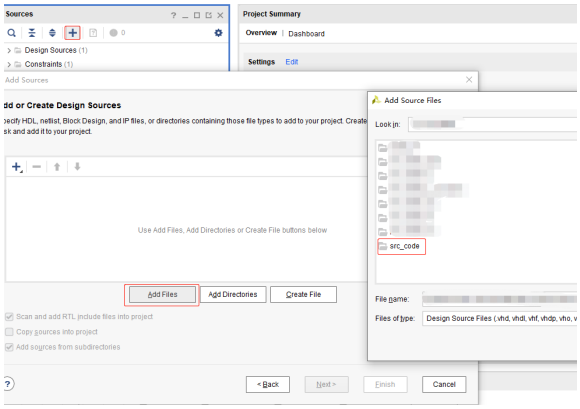


单击 Finish 完成工程创建

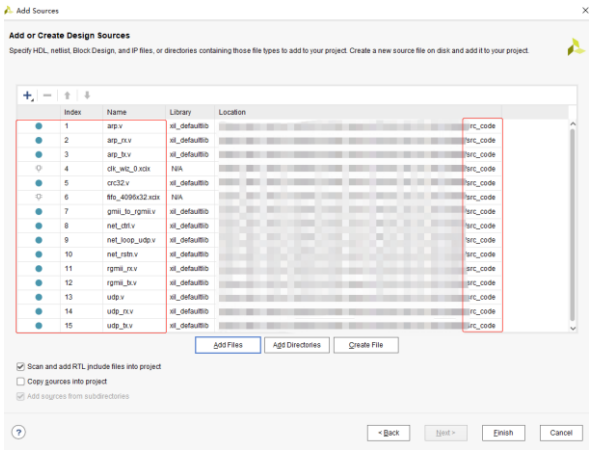


添加代码，显示界面如下



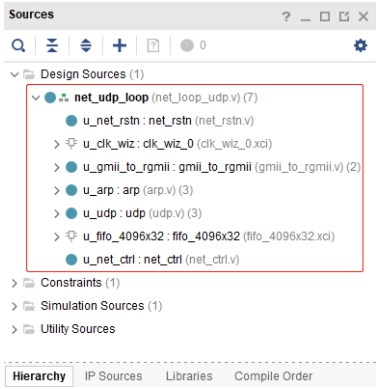


添加代码文件（代码文件在工程路径下）

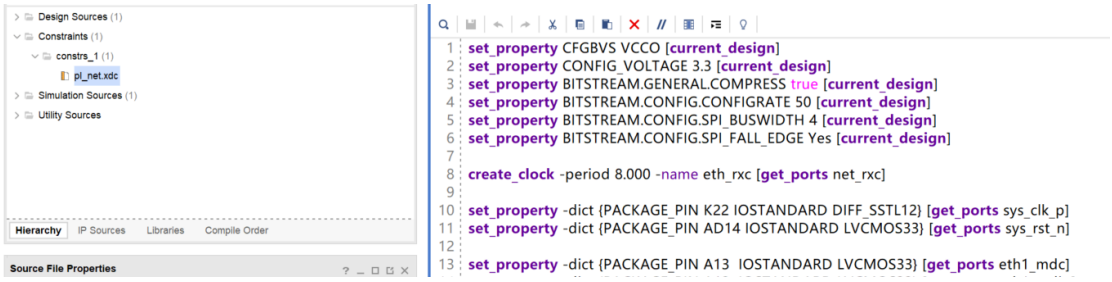




添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

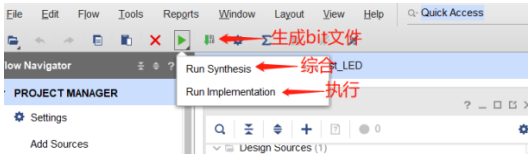


编译并且产生 bit 文件

1:单击综合

2:单击执行

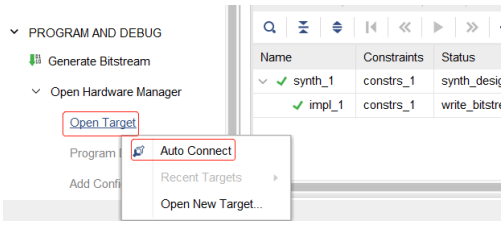
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.16.4 运行结果

下载完成后，我们打开上位机软件，设置好参数，进行数据收发，如下图：



如上图，开发板跟上位机软件进行 UDP 数据 LOOP 回环正常。

3.17 QSFP 误码率例程

3.17.1 功能概述

本章节通过两个 QSFP 光口实现环路，测试在 25G 下 GT 收发器误码率测试。

3.17.2 约束配置

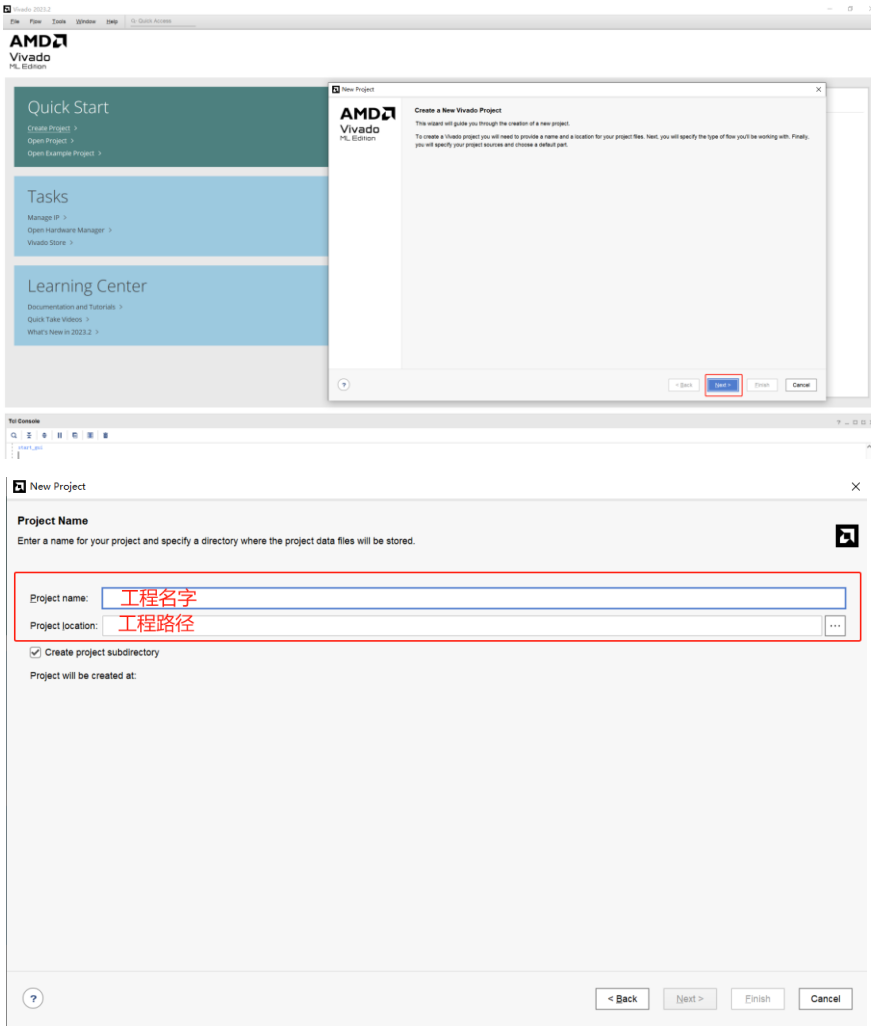
约束配置如下：

```
set_property C_CLK_INPUT_FREQ_HZ 125000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER true [get_debug_cores dbg_hub]
##
##gtrefclk lock constraints
##
set_property PACKAGE_PIN P7 [get_ports gty_refclk0p_i[0]]
set_property PACKAGE_PIN P6 [get_ports gty_refclk0n_i[0]]
set_property PACKAGE_PIN M7 [get_ports gty_refclk1p_i[0]]
set_property PACKAGE_PIN M6 [get_ports gty_refclk1n_i[0]]
##
## Refclk constraints
##
create_clock -name gtrefclk0_2 -period 8.0 [get_ports
gty_refclk0p_i[0]]
create_clock -name gtrefclk1_2 -period 8.0 [get_ports
gty_refclk1p_i[0]]
set_clock_groups -group [get_clocks gtrefclk0_2 -
include_generated_clocks] -asynchronous
set_clock_groups -group [get_clocks gtrefclk1_2 -
include_generated_clocks] -asynchronous
create_waiver -internal -quiet -type CDC -id {CDC-1} -user
ibert_ultrascale_gty -tags "1165692" -description "CDC-1 waiver for
CPLL Calibration logic" \
                    -scope -from [get_ports {gty_refclk*p_i[*]}] \
                        -to [get_pins -quiet -filter
```

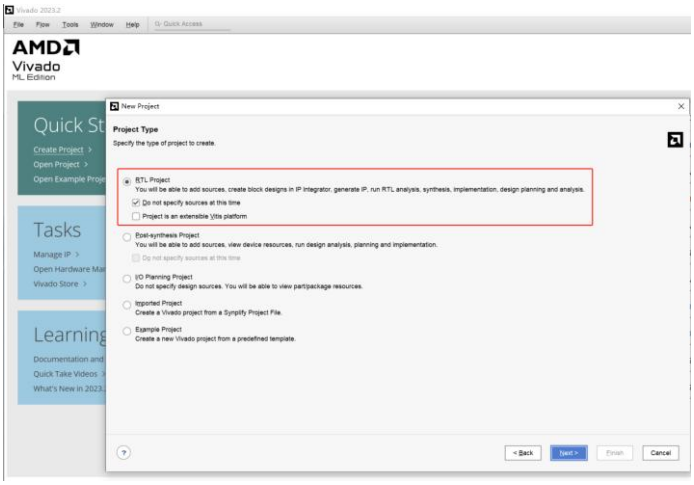
```
{REF_PIN_NAME=~*D} -of_objects [get_cells -hierarchical -filter {NAME
=~*QUAD*.u_q/u_common/U_COMMON_REGS/reg_*/I_EN_STAT_EQ1.U_STAT/xsdb_r
eg_reg[*]}]]
# GT X0Y8
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[0].u_ch/u_gtye4_channel/RXOUTCLK}]
-include_generated_clocks]
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[0].u_ch/u_gtye4_channel/TXOUTCLK}]
-include_generated_clocks]
# GT X0Y9
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[1].u_ch/u_gtye4_channel/RXOUTCLK}]
-include_generated_clocks]
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[1].u_ch/u_gtye4_channel/TXOUTCLK}]
-include_generated_clocks]
# GT X0Y10
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[2].u_ch/u_gtye4_channel/RXOUTCLK}]
-include_generated_clocks]
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[2].u_ch/u_gtye4_channel/TXOUTCLK}]
-include_generated_clocks]
# GT X0Y11
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[3].u_ch/u_gtye4_channel/RXOUTCLK}]
-include_generated_clocks]
set_clock_groups   -asynchronous   -group   [get_clocks   -of_objects
[get_pins
{u_ibert_gty_core/inst/QUAD0.u_q/CH[3].u_ch/u_gtye4_channel/TXOUTCLK}]
-include_generated_clocks]
```

3.17.3 工程搭建

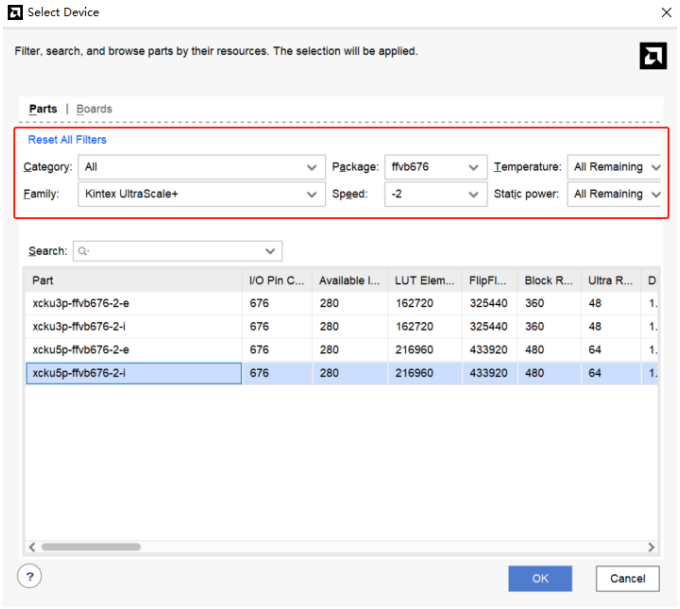
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



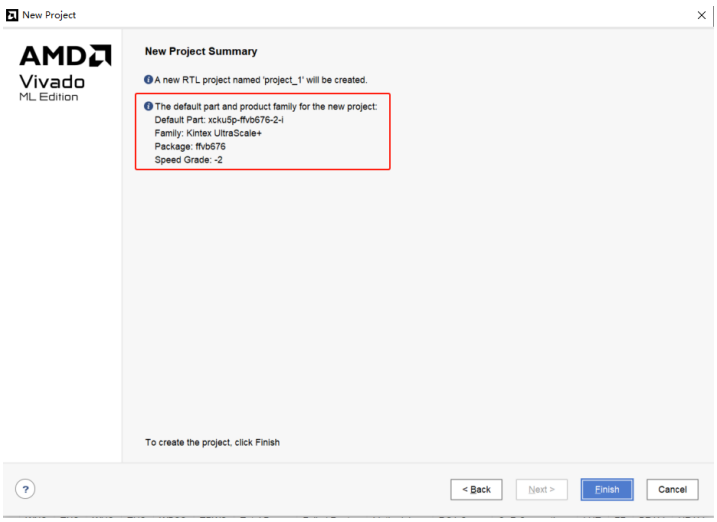
选择 RTL Project 并且勾选复选框，之后单击 NEXT



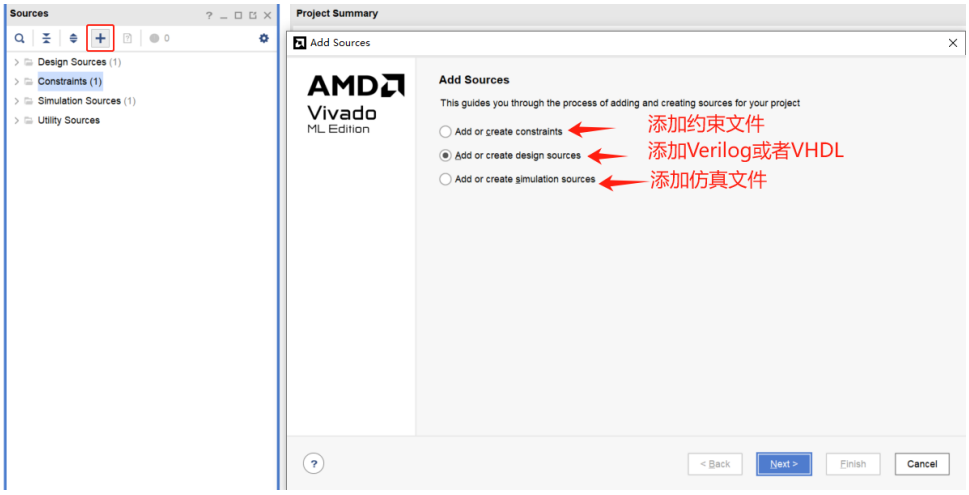
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

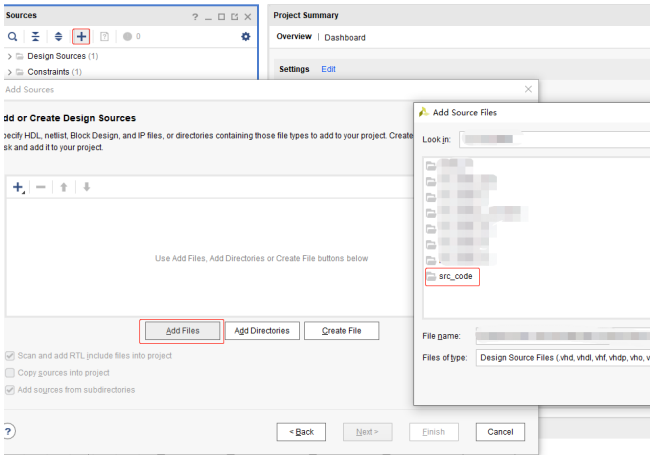


单击 Finish 完成工程创建

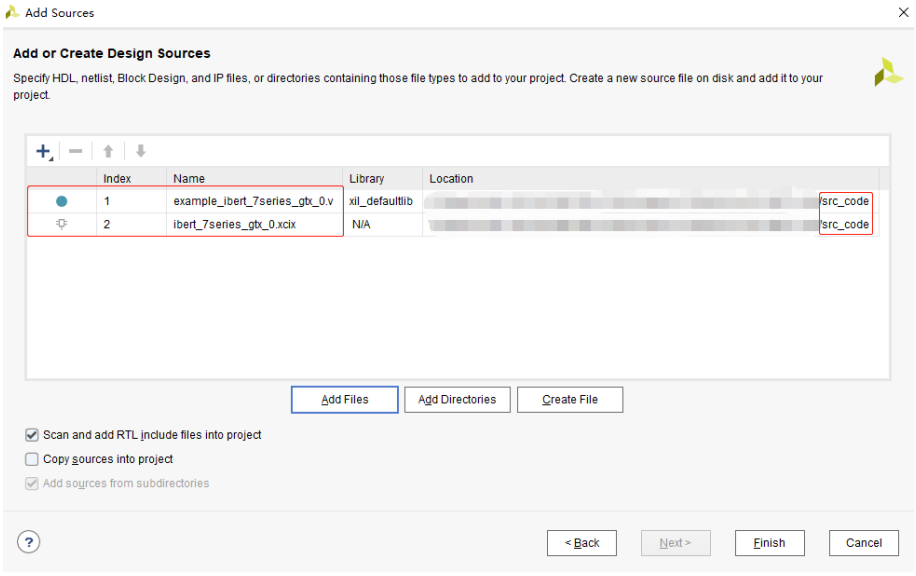


添加代码，显示界面如下

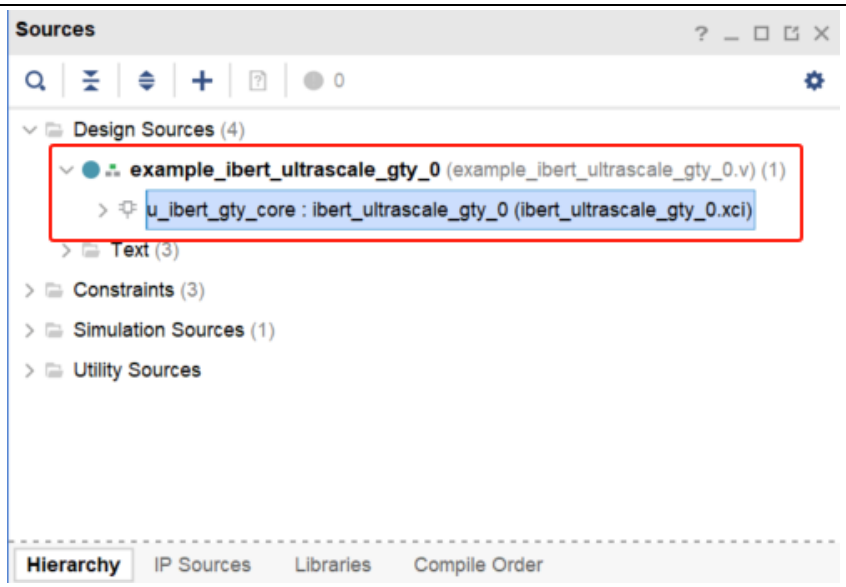




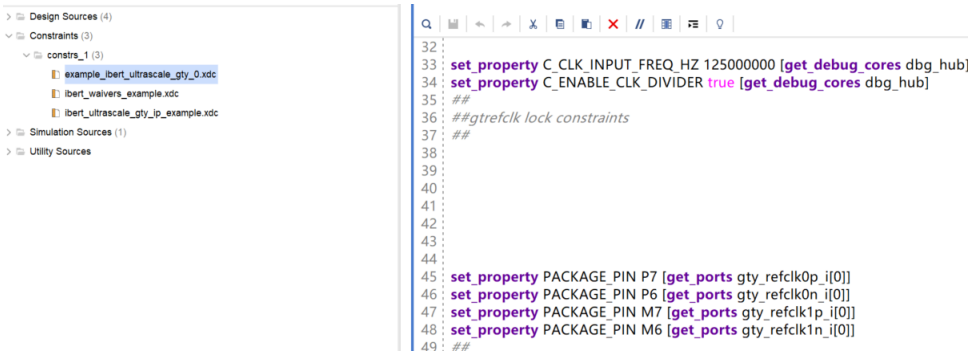
添加代码文件（代码文件在工程路径下）



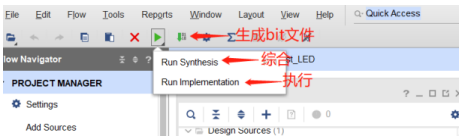
添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



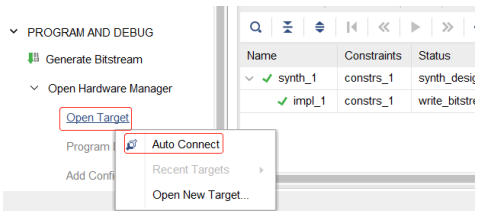
添加管脚约束文件



编译并且产生 bit 文件
1:单击综合
2:单击执行
3:单击产生 bit



下载程序
1：给开发板通电，并且连接下载器
2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



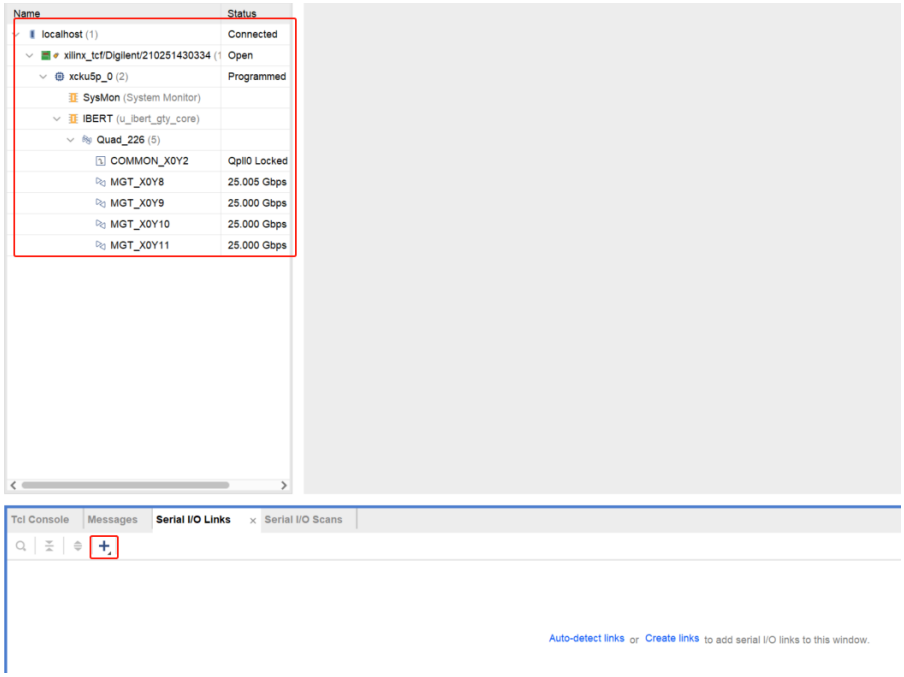
单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



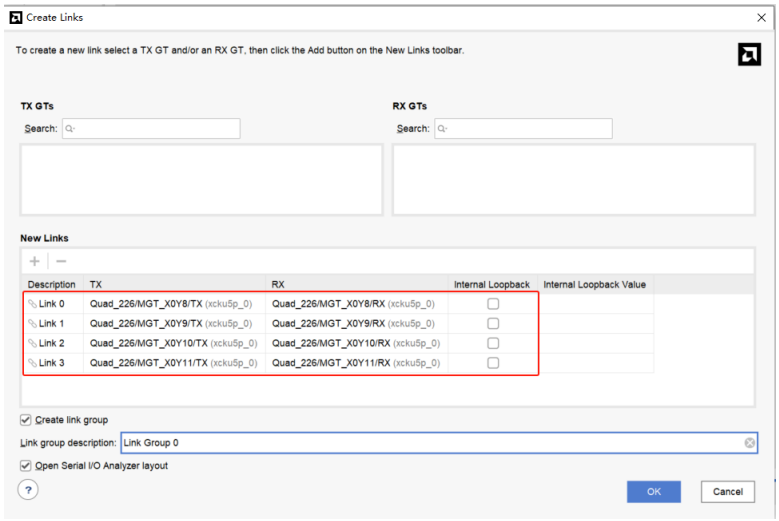
3.17.4 运行结果

下载完成后，创建链接（测试模块需要为 100G 模块，短接回环测试）





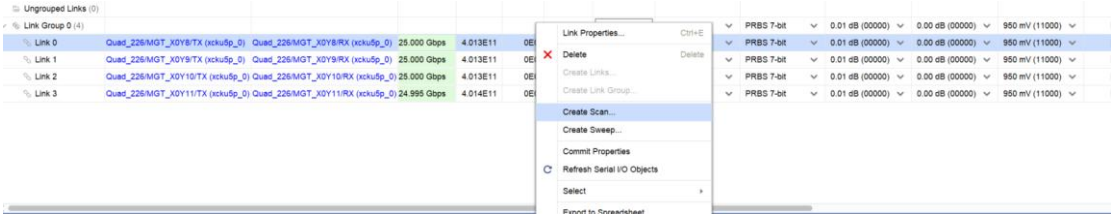
四个通道链接依次全部添加，点击 OK

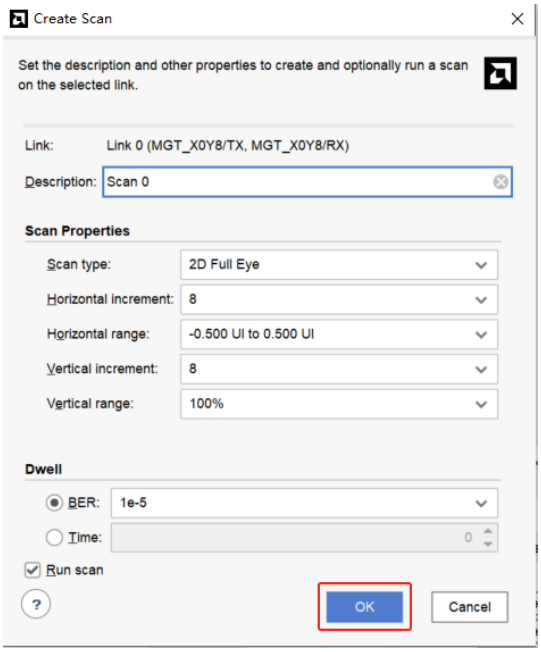


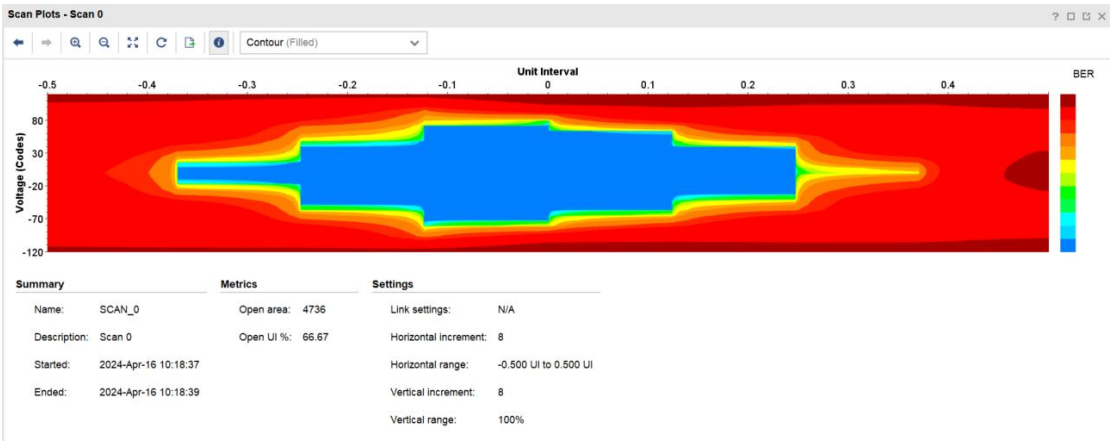
下图可以看到有四个链接



创建眼图如下







SFP 光口测试误码率，到此完成。

3.18 PCIE 测试例程

3.18.1 功能概述

本实例测试 8Line、8GT/S 等配置下，PCIE 接口的读写速度。

3.18.2 管脚配置

本工程管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12}
[get_ports diff_clock_clk_p]

##################------------ddr4------------##################
set_property PACKAGE_PIN H23 [get_ports {c0_ddr4_cs_n[0]}]
set_property PACKAGE_PIN G24 [get_ports {c0_ddr4_ck_t[0]}]
set_property PACKAGE_PIN G25 [get_ports {c0_ddr4_ck_c[0]}]
set_property PACKAGE_PIN H24 [get_ports {c0_ddr4_cke[0]}]
set_property PACKAGE_PIN L25 [get_ports c0_ddr4_reset_n]
set_property PACKAGE_PIN L24 [get_ports c0_ddr4_act_n]
set_property PACKAGE_PIN H21 [get_ports {c0_ddr4_ba[1]}]
set_property PACKAGE_PIN J25 [get_ports {c0_ddr4_ba[0]}]
set_property PACKAGE_PIN M24 [get_ports {c0_ddr4_odt[0]}]
set_property PACKAGE_PIN G26 [get_ports {c0_ddr4_bg[0]}]

set_property PACKAGE_PIN  F23 [get_ports {c0_ddr4_adr[0]}]
set_property PACKAGE_PIN  D25 [get_ports {c0_ddr4_adr[1]}]
set_property PACKAGE_PIN  D23 [get_ports {c0_ddr4_adr[2]}]
set_property PACKAGE_PIN  B26 [get_ports {c0_ddr4_adr[3]}]
set_property PACKAGE_PIN  E23 [get_ports {c0_ddr4_adr[4]}]
set_property PACKAGE_PIN  C26 [get_ports {c0_ddr4_adr[5]}]
set_property PACKAGE_PIN  F24 [get_ports {c0_ddr4_adr[6]}]
set_property PACKAGE_PIN  D26 [get_ports {c0_ddr4_adr[7]}]
set_property PACKAGE_PIN  H26 [get_ports {c0_ddr4_adr[8]}]
set_property PACKAGE_PIN  E25 [get_ports {c0_ddr4_adr[9]}]
set_property PACKAGE_PIN  C24 [get_ports {c0_ddr4_adr[10]}]
set_property PACKAGE_PIN  F22 [get_ports {c0_ddr4_adr[11]}]
set_property PACKAGE_PIN  B25 [get_ports {c0_ddr4_adr[12]}]
set_property PACKAGE_PIN  E26 [get_ports {c0_ddr4_adr[13]}]
set_property PACKAGE_PIN  D24 [get_ports {c0_ddr4_adr[14]}]
set_property PACKAGE_PIN  J23 [get_ports {c0_ddr4_adr[15]}]
set_property PACKAGE_PIN  G22 [get_ports {c0_ddr4_adr[16]}]
```

```
set_property PACKAGE_PIN C21 [get_ports {c0_ddr4_dqs_t[0]}]
set_property PACKAGE_PIN B21 [get_ports {c0_ddr4_dqs_c[0]}]
set_property PACKAGE_PIN A17 [get_ports {c0_ddr4_dqs_t[1]}]
set_property PACKAGE_PIN A18 [get_ports {c0_ddr4_dqs_c[1]}]
set_property PACKAGE_PIN F20 [get_ports {c0_ddr4_dqs_t[2]}]
set_property PACKAGE_PIN E20 [get_ports {c0_ddr4_dqs_c[2]}]
set_property PACKAGE_PIN E16 [get_ports {c0_ddr4_dqs_t[3]}]
set_property PACKAGE_PIN E17 [get_ports {c0_ddr4_dqs_c[3]}]

set_property PACKAGE_PIN  A22 [get_ports {c0_ddr4_dm_n[0]}]
set_property PACKAGE_PIN  C18 [get_ports {c0_ddr4_dm_n[1]}]
set_property PACKAGE_PIN  H18 [get_ports {c0_ddr4_dm_n[2]}]
set_property PACKAGE_PIN  G15 [get_ports {c0_ddr4_dm_n[3]}]

set_property PACKAGE_PIN C23  [get_ports {c0_ddr4_dq[0]}]
set_property PACKAGE_PIN B22  [get_ports {c0_ddr4_dq[1]}]
set_property PACKAGE_PIN A24  [get_ports {c0_ddr4_dq[2]}]
set_property PACKAGE_PIN C22  [get_ports {c0_ddr4_dq[3]}]
set_property PACKAGE_PIN B24  [get_ports {c0_ddr4_dq[4]}]
set_property PACKAGE_PIN D21  [get_ports {c0_ddr4_dq[5]}]
set_property PACKAGE_PIN A25  [get_ports {c0_ddr4_dq[6]}]
set_property PACKAGE_PIN E21  [get_ports {c0_ddr4_dq[7]}]
set_property PACKAGE_PIN A19  [get_ports {c0_ddr4_dq[8]}]
set_property PACKAGE_PIN B17  [get_ports {c0_ddr4_dq[9]}]
set_property PACKAGE_PIN A20  [get_ports {c0_ddr4_dq[10]}]
set_property PACKAGE_PIN C17  [get_ports {c0_ddr4_dq[11]}]
set_property PACKAGE_PIN B20  [get_ports {c0_ddr4_dq[12]}]
set_property PACKAGE_PIN B15  [get_ports {c0_ddr4_dq[13]}]
set_property PACKAGE_PIN B19  [get_ports {c0_ddr4_dq[14]}]
set_property PACKAGE_PIN A15  [get_ports {c0_ddr4_dq[15]}]
set_property PACKAGE_PIN D19 [get_ports {c0_ddr4_dq[16]}]
set_property PACKAGE_PIN F19 [get_ports {c0_ddr4_dq[17]}]
set_property PACKAGE_PIN D20 [get_ports {c0_ddr4_dq[18]}]
set_property PACKAGE_PIN G20 [get_ports {c0_ddr4_dq[19]}]
set_property PACKAGE_PIN G21 [get_ports {c0_ddr4_dq[20]}]
set_property PACKAGE_PIN F18 [get_ports {c0_ddr4_dq[21]}]
set_property PACKAGE_PIN D18 [get_ports {c0_ddr4_dq[22]}]
set_property PACKAGE_PIN E18 [get_ports {c0_ddr4_dq[23]}]

set_property PACKAGE_PIN E15 [get_ports {c0_ddr4_dq[24]}]
set_property PACKAGE_PIN G16 [get_ports {c0_ddr4_dq[25]}]
set_property PACKAGE_PIN D15 [get_ports {c0_ddr4_dq[26]}]
set_property PACKAGE_PIN G17 [get_ports {c0_ddr4_dq[27]}]
```

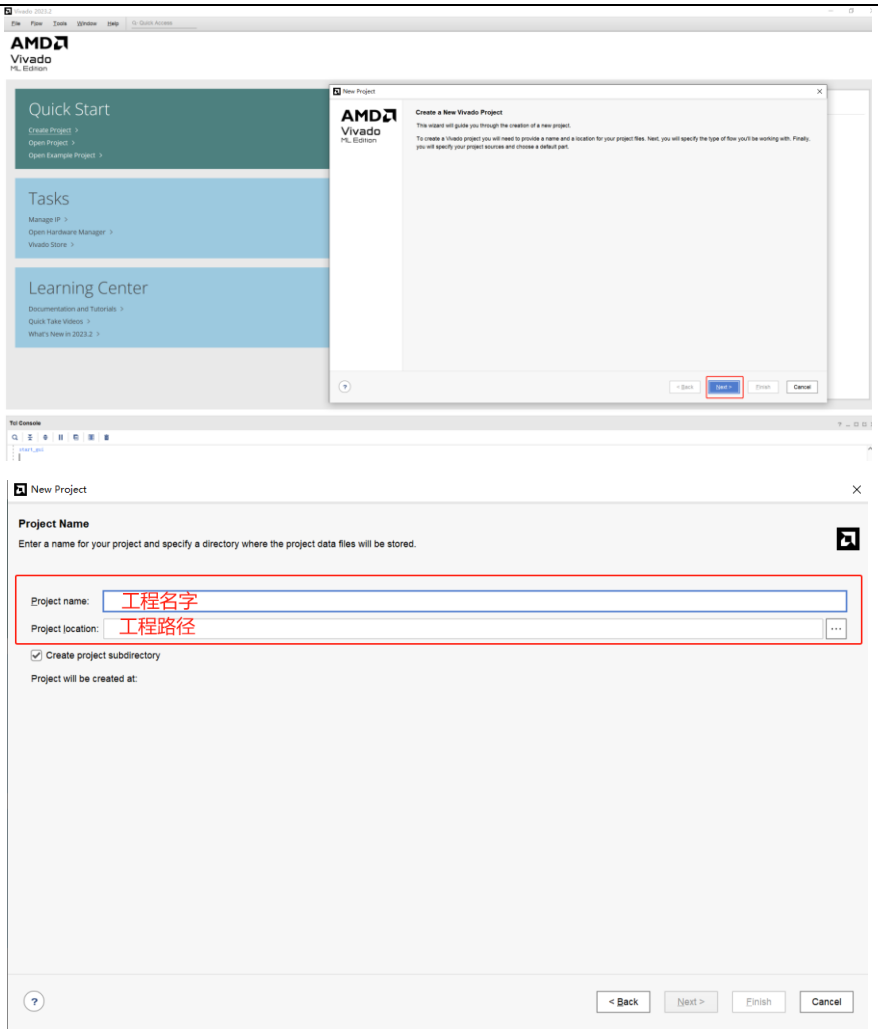
```
set_property PACKAGE_PIN D16 [get_ports {c0_ddr4_dq[28]}]
set_property PACKAGE_PIN H17 [get_ports {c0_ddr4_dq[29]}]
set_property PACKAGE_PIN C16 [get_ports {c0_ddr4_dq[30]}]
set_property PACKAGE_PIN H16 [get_ports {c0_ddr4_dq[31]}]


set_property INTERNAL_VREF 0.6 [get_iobanks 64]
# PCI Express reference clock 100MHz
set_property PACKAGE_PIN AD13   [get_ports pcie_rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports pcie_rst_n]
# PCI Express reference clock 100MHz
create_clock -add  -name pcie_ref_clk_p[0] -period 10 [get_ports
pcie_ref_clk_p[0]]
set_property PACKAGE_PIN AB7  [get_ports {pcie_ref_clk_p[0]}]
# MGT locations
set_property PACKAGE_PIN AF7  [get_ports {pcie_mgt_txp[0]}]
set_property PACKAGE_PIN AE9  [get_ports {pcie_mgt_txp[1]}]
set_property PACKAGE_PIN AD7 [get_ports {pcie_mgt_txp[2]}]
set_property PACKAGE_PIN AC5  [get_ports {pcie_mgt_txp[3]}]
set_property PACKAGE_PIN AA5   [get_ports {pcie_mgt_txp[4]}]
set_property PACKAGE_PIN W5    [get_ports {pcie_mgt_txp[5]}]
set_property PACKAGE_PIN U5    [get_ports {pcie_mgt_txp[6]}]
set_property PACKAGE_PIN R5    [get_ports {pcie_mgt_txp[7]}]

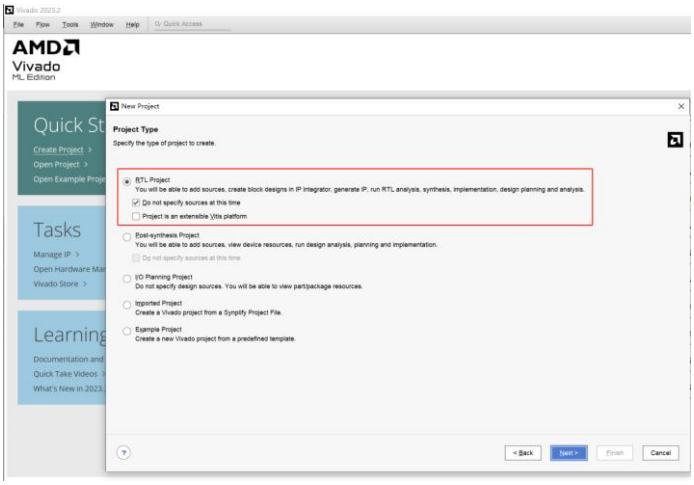
set_property PACKAGE_PIN AF2 [get_ports {pcie_mgt_rxp[0]}]
set_property PACKAGE_PIN AE4 [get_ports {pcie_mgt_rxp[1]}]
set_property PACKAGE_PIN AD2 [get_ports {pcie_mgt_rxp[2]}]
set_property PACKAGE_PIN AB2 [get_ports {pcie_mgt_rxp[3]}]
set_property PACKAGE_PIN Y2 [get_ports {pcie_mgt_rxp[4]}]
set_property PACKAGE_PIN V2 [get_ports {pcie_mgt_rxp[5]}]
set_property PACKAGE_PIN T2 [get_ports {pcie_mgt_rxp[6]}]
set_property PACKAGE_PIN P2 [get_ports {pcie_mgt_rxp[7]}]
```

3.18.3 工程搭建

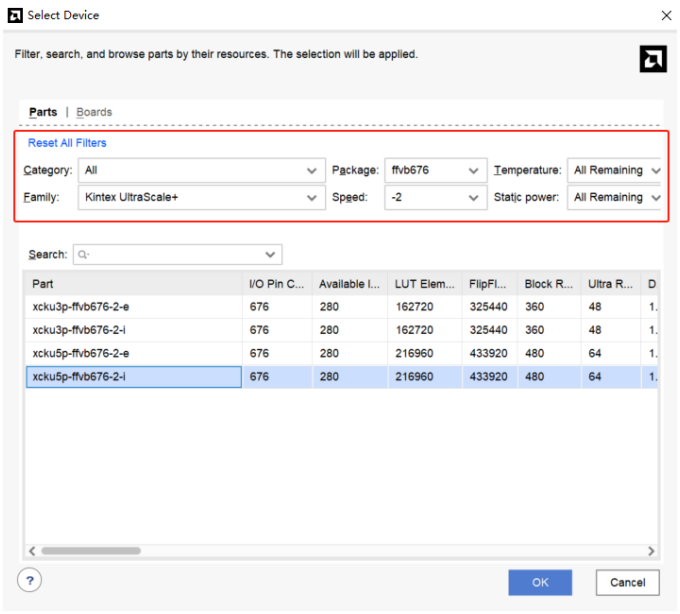
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



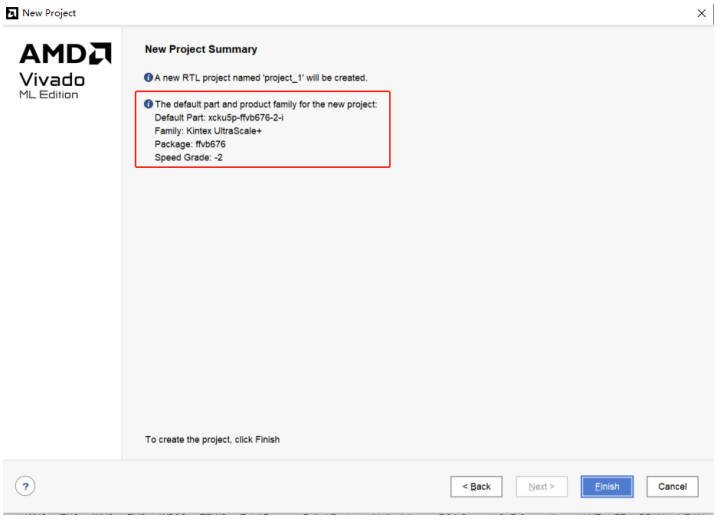
选择 RTL Project 并且勾选复选框，之后单击 NEXT



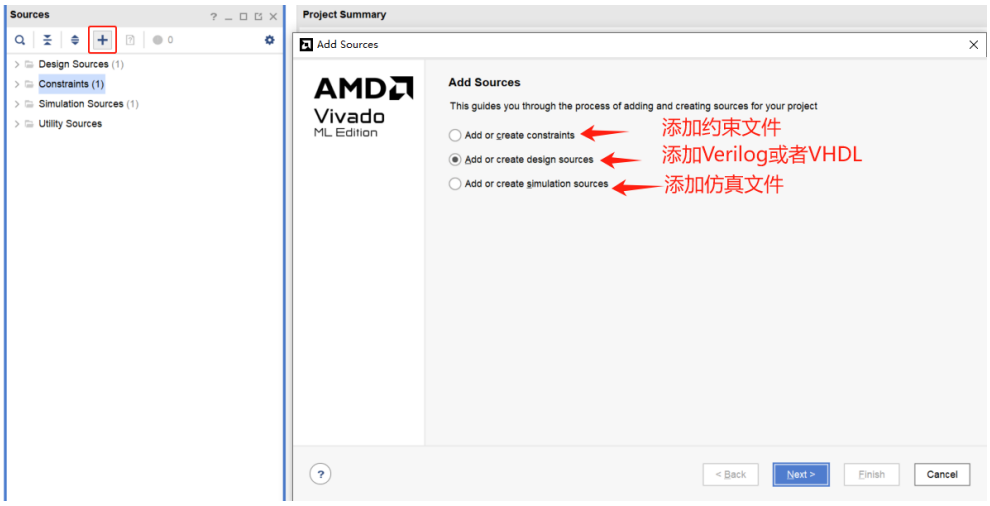
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例



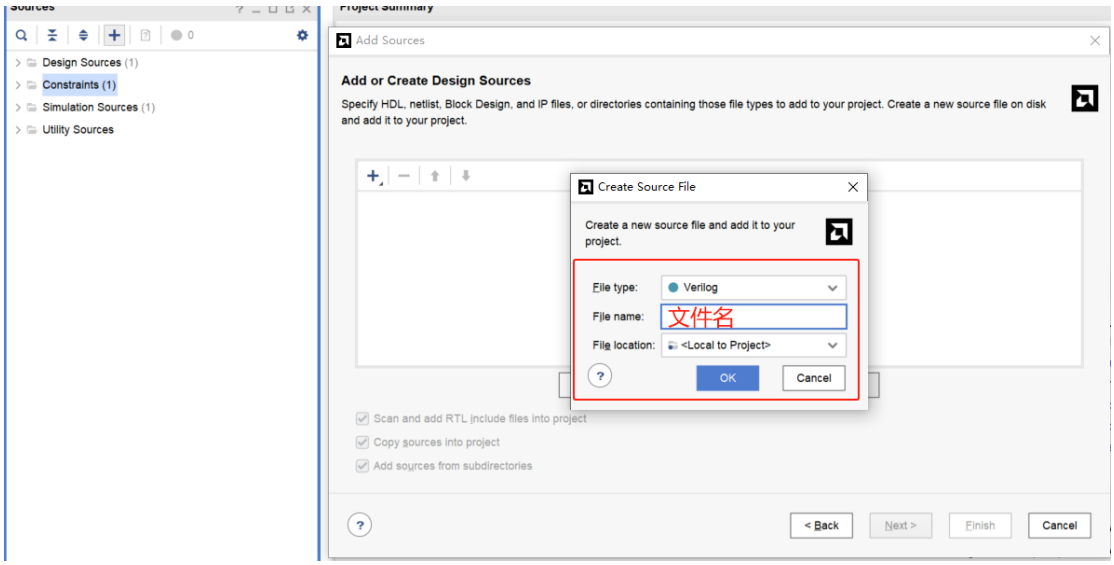
单击 Finish 完成工程创建



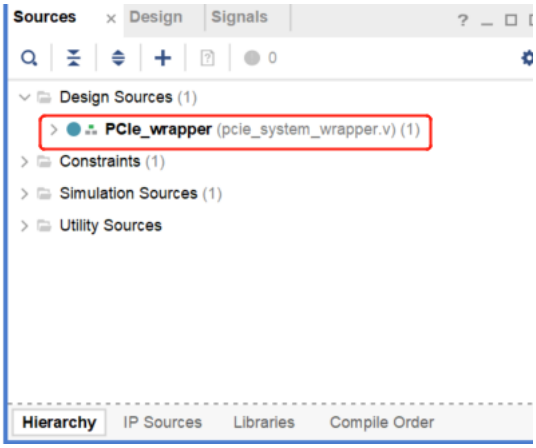
添加代码，显示界面如下



创建一个 PCIe_wrapper 文件，并且选择 Verilog 格式



创建完成后，可以看到 Design Sources 文件夹中有 PCIe_wrapper.v 文件，这个文件就是我们可以编写 verilog 程序的文件



双击 PCIe_wrapper 文件，添加代码，代码如下：

```
`timescale 1 ps / 1 ps

module pcie_system_wrapper
   (c0_ddr4_act_n,
    c0_ddr4_adr,
    c0_ddr4_ba,
    c0_ddr4_bg,
    c0_ddr4_ck_c,
    c0_ddr4_ck_t,
    c0_ddr4_cke,
    c0_ddr4_cs_n,
    c0_ddr4_dm_n,
    c0_ddr4_dq,
    c0_ddr4_dqs_c,
    c0_ddr4_dqs_t,
    c0_ddr4_odt,
```

```
    c0_ddr4_reset_n,
    diff_clock_clk_n,
    diff_clock_clk_p,
    pcie_mgt_rxn,
    pcie_mgt_rxp,
    pcie_mgt_txn,
    pcie_mgt_txp,
    pcie_ref_clk_n,
    pcie_ref_clk_p,
    pcie_rst_n);
  output c0_ddr4_act_n;
  output [16:0]c0_ddr4_adr;
  output [1:0]c0_ddr4_ba;
  output [0:0]c0_ddr4_bg;
  output [0:0]c0_ddr4_ck_c;
  output [0:0]c0_ddr4_ck_t;
  output [0:0]c0_ddr4_cke;
  output [0:0]c0_ddr4_cs_n;
  inout [3:0]c0_ddr4_dm_n;
  inout [31:0]c0_ddr4_dq;
  inout [3:0]c0_ddr4_dqs_c;
  inout [3:0]c0_ddr4_dqs_t;
  output [0:0]c0_ddr4_odt;
  output c0_ddr4_reset_n;
  input diff_clock_clk_n;
  input diff_clock_clk_p;
  input [7:0]pcie_mgt_rxn;
  input [7:0]pcie_mgt_rxp;
  output [7:0]pcie_mgt_txn;
  output [7:0]pcie_mgt_txp;
  input [0:0]pcie_ref_clk_n;
  input [0:0]pcie_ref_clk_p;
  input pcie_rst_n;

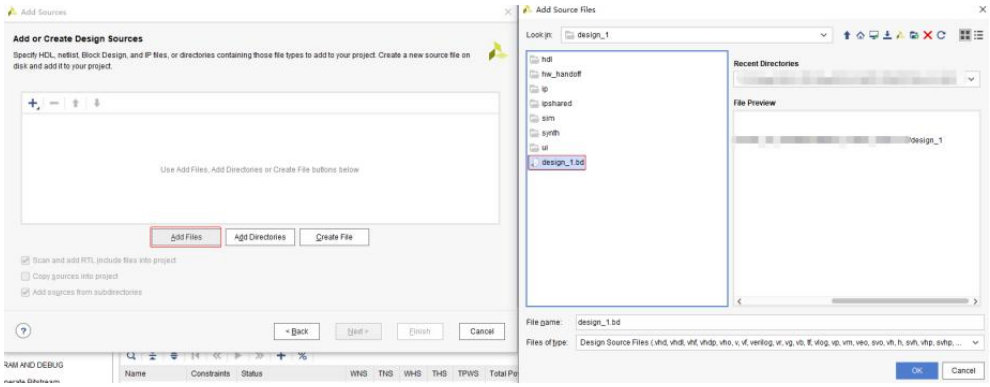
  wire c0_ddr4_act_n;
  wire [16:0]c0_ddr4_adr;
  wire [1:0]c0_ddr4_ba;
  wire [0:0]c0_ddr4_bg;
  wire [0:0]c0_ddr4_ck_c;
  wire [0:0]c0_ddr4_ck_t;
  wire [0:0]c0_ddr4_cke;
  wire [0:0]c0_ddr4_cs_n;
  wire [3:0]c0_ddr4_dm_n;
  wire [31:0]c0_ddr4_dq;
```

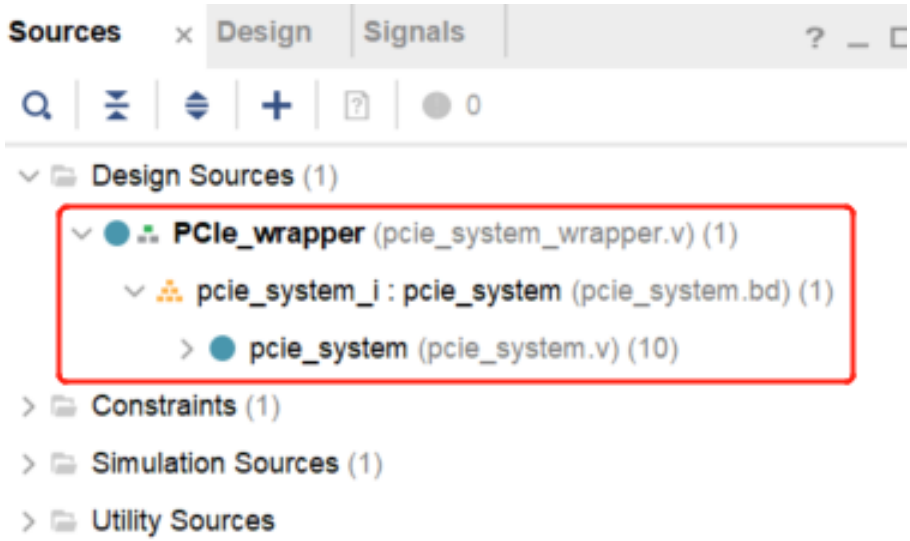
```
    wire [3:0]c0_ddr4_dqs_c;
    wire [3:0]c0_ddr4_dqs_t;
    wire [0:0]c0_ddr4_odt;
    wire c0_ddr4_reset_n;
    wire diff_clock_clk_n;
    wire diff_clock_clk_p;
    wire [7:0]pcie_mgt_rxn;
    wire [7:0]pcie_mgt_rxp;
    wire [7:0]pcie_mgt_txn;
    wire [7:0]pcie_mgt_txp;
    wire [0:0]pcie_ref_clk_n;
    wire [0:0]pcie_ref_clk_p;
    wire pcie_rst_n;

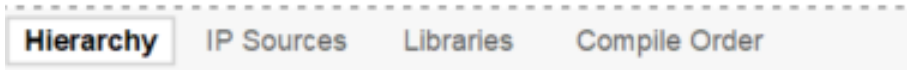
    pcie_system pcie_system_i
         (.c0_ddr4_act_n(c0_ddr4_act_n),
          .c0_ddr4_adr(c0_ddr4_adr),
          .c0_ddr4_ba(c0_ddr4_ba),
          .c0_ddr4_bg(c0_ddr4_bg),
          .c0_ddr4_ck_c(c0_ddr4_ck_c),
          .c0_ddr4_ck_t(c0_ddr4_ck_t),
          .c0_ddr4_cke(c0_ddr4_cke),
          .c0_ddr4_cs_n(c0_ddr4_cs_n),
          .c0_ddr4_dm_n(c0_ddr4_dm_n),
          .c0_ddr4_dq(c0_ddr4_dq),
          .c0_ddr4_dqs_c(c0_ddr4_dqs_c),
          .c0_ddr4_dqs_t(c0_ddr4_dqs_t),
          .c0_ddr4_odt(c0_ddr4_odt),
          .c0_ddr4_reset_n(c0_ddr4_reset_n),
          .diff_clock_clk_n(diff_clock_clk_n),
          .diff_clock_clk_p(diff_clock_clk_p),
          .pcie_mgt_rxn(pcie_mgt_rxn),
          .pcie_mgt_rxp(pcie_mgt_rxp),
          .pcie_mgt_txn(pcie_mgt_txn),
          .pcie_mgt_txp(pcie_mgt_txp),
          .pcie_ref_clk_n(pcie_ref_clk_n),
          .pcie_ref_clk_p(pcie_ref_clk_p),
          .pcie_rst_n(pcie_rst_n));
endmodule
```

继续添加.bd 模块（模块路径在该工程文件夹下）

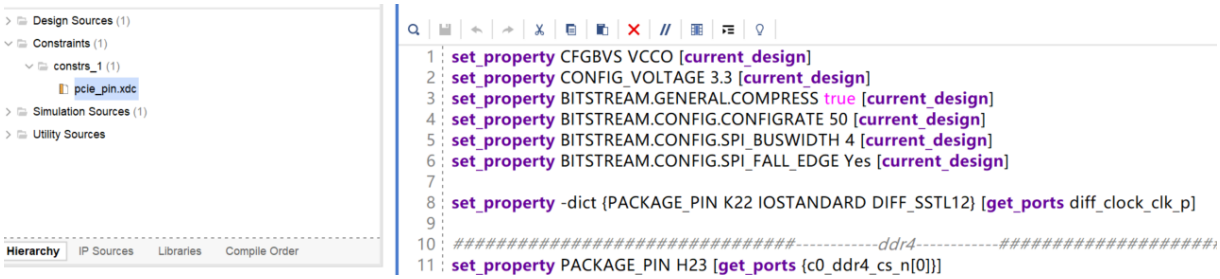
路径：E:..\\1_FPGA\3_20_KUP_PCIE\pcie_system.srcs\sources_1\bd





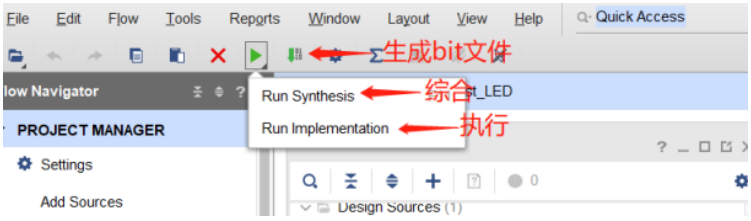


添加管脚约束文件

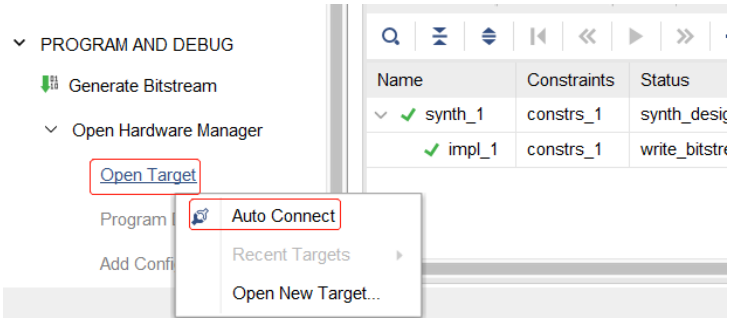


编译并且产生 bit 文件

1:单击综合
2:单击执行
3:单击产生 bit



下载程序
1：给开发板通电，并且连接下载器
2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



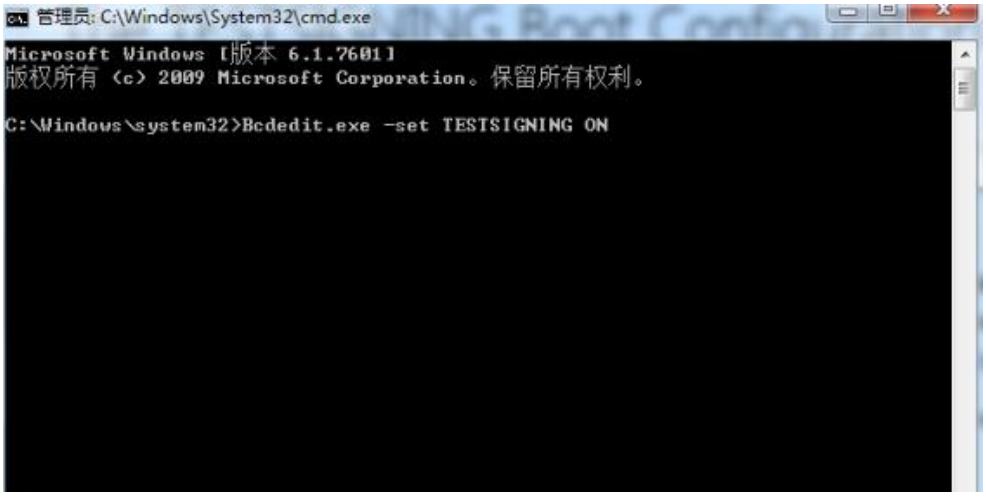
单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.18.4 运行结果

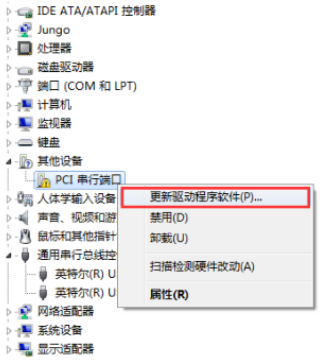
我们先把 WIN10/7 切换到测试模式，右键点击开始按钮，选择命令提示符（管

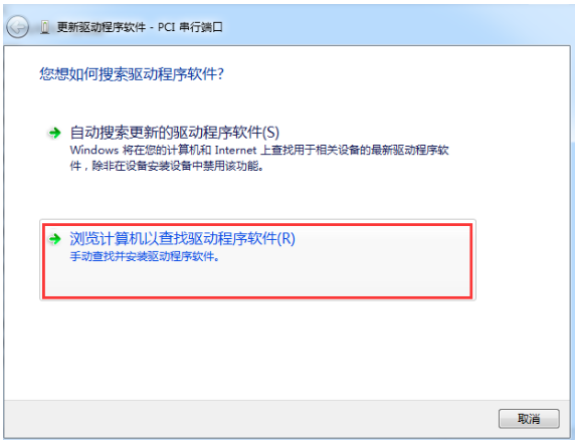
理员身份运行），然后打开命令提示符后，输入：bcdedit /set testsigning on 回车，重新启动即可（退出命令 bcdedit /set testsigning off）



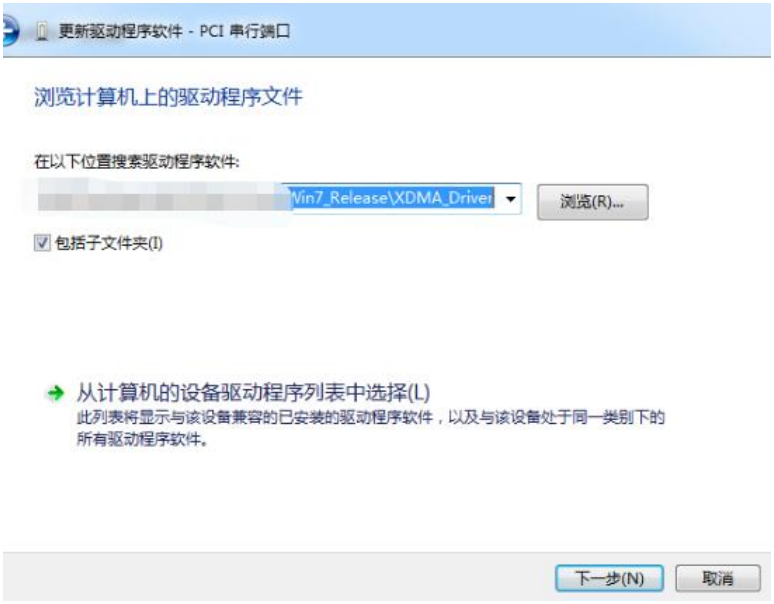


重启之后，我们开始安装驱动，如下：





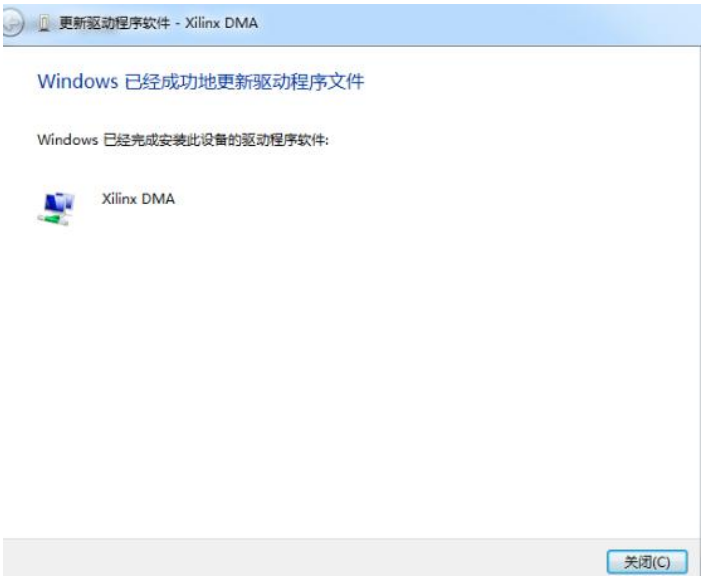
选择驱动路径



始终安装



安装完成

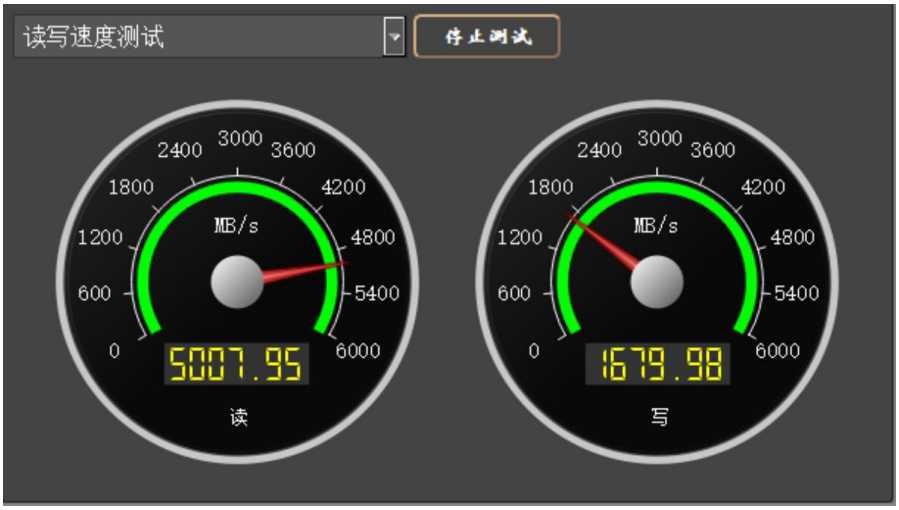


运行我们提供的上位机软件，进行读写测试，受限于我们的主机，理论上性能

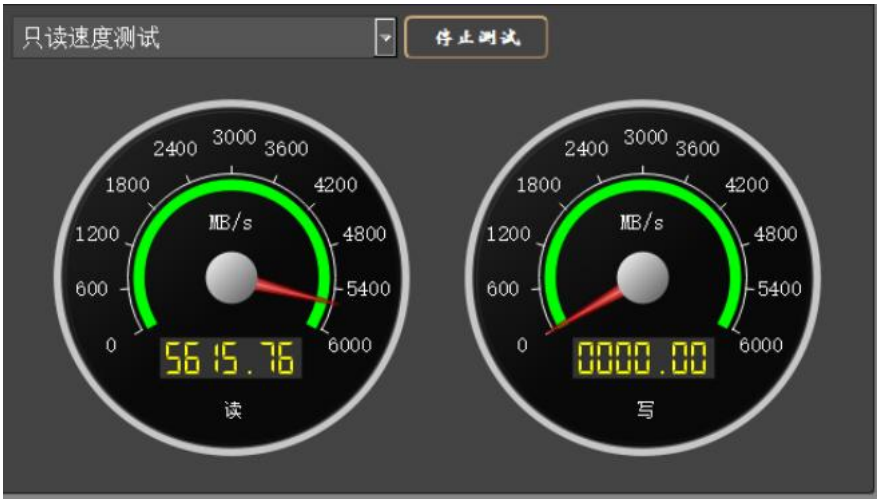
应该更好

结果如下：

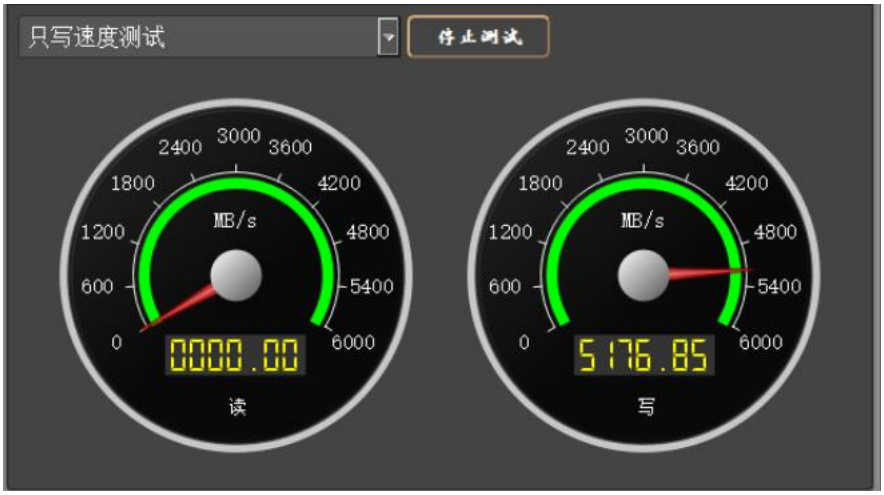
读写测试



只写测试



只读测试



3.19 Aurora 通信例程

3.19.1 功能概述

Aurora 是一个用于在点对点串行链路间移动数据的可扩展轻量级链路层协议。这为物理层提供透明接口，让专有协议或业界标准协议上层能方便地使用高速收发器。虽然使用的逻辑资源非常少，但 Aurora 能提供低延迟高带宽和高度可配置的特性集。在 Xilinx FPGA 上使用是免费的。

主要特性：
高带宽，仅受限于收发器的数据速率
支持大量键合线路，实现较高的总带宽
支持全双工和单工通道
无限帧尺寸/灵活组帧
小型逻辑封装，采用标准的 AXI-ST 接口。
内置流程控制和热插拔支持

本例程通过两个光口各自进行 Aurora loop 通信，在 ila 界面查看 Aurora loop 通信结果。

3.19.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]

set_property -dict {PACKAGE_PIN W15 IOSTANDARD LVCMOS33} [get_ports
LPMode]

set_property package_pin M6 [get_ports mgtrefclk1_x0y2_n]
set_property package_pin M7 [get_ports mgtrefclk1_x0y2_p]

set_property package_pin M2 [get_ports ch0_gtyrxp_in]
set_property package_pin N5 [get_ports ch0_gtytxp_out]

set_property package_pin K2 [get_ports ch1_gtyrxp_in]
set_property package_pin L5 [get_ports ch1_gtytxp_out]

set_property package_pin H2 [get_ports ch2_gtyrxp_in]
set_property package_pin J5 [get_ports ch2_gtytxp_out]

set_property package_pin F2 [get_ports ch3_gtyrxp_in]
set_property package_pin G5 [get_ports ch3_gtytxp_out]# -------------
-------------------------------------------------------------------
-------------------------------------
create_clock    -name    clk_freerun    -period    10.0    [get_ports
hb_gtwiz_reset_clk_freerun_in]
create_clock  -name  clk_mgtrefclk1_x0y2_p  -period  8.0  [get_ports
mgtrefclk1_x0y2_p]

# False path constraints
# -------------------------------------------------------------------
-------------------------------------------------
set_false_path  -to  [get_cells  -hierarchical  -filter  {NAME  =~
```

```
*bit_synchronizer*inst/i_in_meta_reg}] -quiet
##set_false_path  -to  [get_cells  -hierarchical  -filter  {NAME  =~
*reset_synchronizer*inst/rst_in_*_reg}] -quiet
set_false_path -to [get_pins -filter {REF_PIN_NAME=~*D} -of_objects
[get_cells       -hierarchical       -filter       {NAME       =~
*reset_synchronizer*inst/rst_in_meta*}]] -quiet
set_false_path -to [get_pins -filter {REF_PIN_NAME=~*PRE} -of_objects
[get_cells       -hierarchical       -filter       {NAME       =~
*reset_synchronizer*inst/rst_in_meta*}]] -quiet
set_false_path -to [get_pins -filter {REF_PIN_NAME=~*PRE} -of_objects
[get_cells       -hierarchical       -filter       {NAME       =~
*reset_synchronizer*inst/rst_in_sync1*}]] -quiet
set_false_path -to [get_pins -filter {REF_PIN_NAME=~*PRE} -of_objects
[get_cells       -hierarchical       -filter       {NAME       =~
*reset_synchronizer*inst/rst_in_sync2*}]] -quiet
set_false_path -to [get_pins -filter {REF_PIN_NAME=~*PRE} -of_objects
[get_cells       -hierarchical       -filter       {NAME       =~
*reset_synchronizer*inst/rst_in_sync3*}]] -quiet
set_false_path -to [get_pins -filter {REF_PIN_NAME=~*PRE} -of_objects
[get_cells       -hierarchical       -filter       {NAME       =~
*reset_synchronizer*inst/rst_in_out*}]] -quiet


set_false_path  -to  [get_cells  -hierarchical  -filter  {NAME  =~
*gtwiz_userclk_tx_inst/*gtwiz_userclk_tx_active_*_reg}] -quiet
set_false_path  -to  [get_cells  -hierarchical  -filter  {NAME  =~
*gtwiz_userclk_rx_inst/*gtwiz_userclk_rx_active_*_reg}] -quiet

###  for exdes
create_waiver  -internal  -quiet  -type  CDC  -id  {CDC-11}  -user
gtwizard_ultrascale -tags "1165536" -description "CDC-11 waiver for
CPLL  Calibration  logic"  -scope  -from  [get_pins  -quiet  -filter
{REF_PIN_NAME=~*C}  -of_objects  [get_cells  -hierarchical  -filter
{NAME=~*reset_synchronizer_rx_done_inst/rst_in_out_reg}]]        -to
[get_pins -quiet -filter {REF_PIN_NAME=~*D} -of_objects [get_cells -
hierarchical -filter {NAME =~*i_in_meta_reg}]]
create_waiver  -internal  -quiet  -type  CDC  -id  {CDC-11}  -user
gtwizard_ultrascale -tags "1165536" -description "CDC-11 waiver for
CPLL  Calibration  logic"  -scope  -from  [get_pins  -quiet  -filter
{REF_PIN_NAME=~*C}  -of_objects  [get_cells  -hierarchical  -filter
{NAME=~*reset_synchronizer_tx_done_inst/rst_in_out_reg}]]        -to
[get_pins -quiet -filter {REF_PIN_NAME=~*D} -of_objects [get_cells -
hierarchical -filter {NAME =~*i_in_meta_reg}]]
```

```
create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*PROBE_OUT0_INST/Probe_out_reg*}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*example_checking_reset_synchronizer_inst/rst_in_meta_reg}]]
create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*PROBE_OUT0_INST/Probe_out_reg*}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*example_stimulus_reset_synchronizer_inst/rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]
create_waiver -internal -quiet -type CDC -id {CDC-11} -user gtwizard_ultrascale -tags "1165536" -description "CDC-11 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]
##create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*example_stimulus_reset_synchronizer_inst/rst_in_meta_reg}]]
##create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between
```

```
support blocks like vio and exdes inst" -scope -from [get_pins -quiet
-filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -
filter
{NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*
x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -
of_objects [get_cells -hierarchical -filter {NAME
=~*example_checking_reset_synchronizer_inst/rst_in_meta_reg}]]
##create_waiver -internal -quiet -type CDC -id {CDC-10} -user
gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between
support blocks like vio and exdes inst" -scope -from [get_pins -quiet
-filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -
filter
{NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*
x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -
of_objects [get_cells -hierarchical -filter {NAME
=~*reset_synchronizer_gtwiz_buffbypass_rx_reset_inst/rst_in_meta_reg}
]]
create_waiver -internal -quiet -type CDC -id {CDC-10} -user
gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between
support blocks like vio and exdes inst" -scope -from [get_pins -quiet
-filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -
filter
{NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*
x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*D} -
of_objects [get_cells -hierarchical -filter {NAME
=~*bit_synchronizer_*x_init_done_inst/i_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user
gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between
support blocks like vio and exdes inst" -scope -from [get_pins -quiet
-filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -
filter
{NAME=~*gen_gtwiz_buffbypass_*x_main.gen_auto_mode.gtwiz_buffbypass_*
x_done_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -
of_objects [get_cells -hierarchical -filter {NAME
=~*gtwiz_buffbypass_rx_reset*inst/rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user
gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between
support blocks like vio and exdes inst" -scope -from [get_pins -quiet
-filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -
filter
{NAME=~*gen_single_instance.gtwiz_reset_inst/gtwiz_reset_*x_done_int_
reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*CLR} -of_objects
```

```
[get_cells -hierarchical -filter {NAME =~*reset_synchronizer_*x_done_inst/rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*reset_synchronizer_*x_done_inst/rst_in_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~**example_*_reset_synchronizer_inst/rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*reset_all_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*example_*_reset_synchronizer_inst/rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*gen_cal_*x_en.mask_user_in_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*CLR} -of_objects [get_cells -hierarchical -filter {NAME =~*gtwiz_userclk_*x_active_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*USER_*XPRGDIVRESETDONE_OUT_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*CLR} -of_objects [get_cells -hierarchical -filter {NAME =~*gtwiz_userclk_*x_active_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*prbs_match_out_reg}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]
```

```
create_waiver -internal -quiet -type CDC -id {CDC-11} -user gtwizard_ultrascale -tags "1165536" -description "CDC-11 In exdes path" -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*prbs_match_out_reg*}]] -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-11} -user gtwizard_ultrascale -tags "1165536" -description "CDC-11 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*Probe_out_reg*}]] \
                              -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]


create_waiver -internal -quiet -type CDC -id {CDC-12} -user gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*Probe_out_reg*}]] \
                              -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-12} -user gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*gtwiz_buffbypass_tx_done_out_reg*}]] \
                              -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-12} -user gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter
```

```
{NAME=~*gtwiz_buffbypass_rx_done_out_reg*}]] \
                            -to     [get_pins     -quiet     -filter
{REF_PIN_NAME=~*PRE}  -of_objects  [get_cells  -hierarchical  -filter
{NAME =~*rst_in_meta_reg}]]

create_waiver  -internal  -quiet  -type  CDC  -id  {CDC-12}  -user
gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and
vio path" \
                        -scope   -from   [get_pins   -quiet   -filter
{REF_PIN_NAME=~*C}  -of_objects  [get_cells  -hierarchical  -filter
{NAME=~*gtwiz_buffbypass_rx_done_out_reg*}]] \
                            -to     [get_pins     -quiet     -filter
{REF_PIN_NAME=~*PRE}  -of_objects  [get_cells  -hierarchical  -filter
{NAME =~*i_in_meta_reg}]]
create_waiver  -internal  -quiet  -type  CDC  -id  {CDC-12}  -user
gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and
vio path" \
                        -scope   -from   [get_pins   -quiet   -filter
{REF_PIN_NAME=~*C}  -of_objects  [get_cells  -hierarchical  -filter
{NAME=~*gtwiz_buffbypass_rx_done_out_reg*}]] \
                            -to     [get_pins     -quiet     -filter
{REF_PIN_NAME=~*D} -of_objects [get_cells -hierarchical -filter {NAME
=~*i_in_meta_reg}]]

create_waiver  -internal  -quiet  -type  CDC  -id  {CDC-10}  -user
gtwizard_ultrascale -tags "1165536" -description "CDC-10 In exdes and
vio path" \
                        -scope   -from   [get_pins   -quiet   -filter
{REF_PIN_NAME=~*C}  -of_objects  [get_cells  -hierarchical  -filter
{NAME=~*rst_in_out_reg}]] \
                            -to     [get_pins     -quiet     -filter
{REF_PIN_NAME=~*D} -of_objects [get_cells -hierarchical -filter {NAME
=~*i_in_meta_reg}]]

create_waiver  -internal  -quiet  -type  CDC  -id  {CDC-12}  -user
gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and
vio path" \
                        -scope   -from   [get_pins   -quiet   -filter
{REF_PIN_NAME=~*C}  -of_objects  [get_cells  -hierarchical  -filter
{NAME=~*rst_in_out_reg}]] \
                            -to     [get_pins     -quiet     -filter
{REF_PIN_NAME=~*D} -of_objects [get_cells -hierarchical -filter {NAME
=~*i_in_meta_reg}]]
```

```
create_waiver -internal -quiet -type CDC -id {CDC-12} -user gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*reset_all_out_reg*}]] \
                              -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]

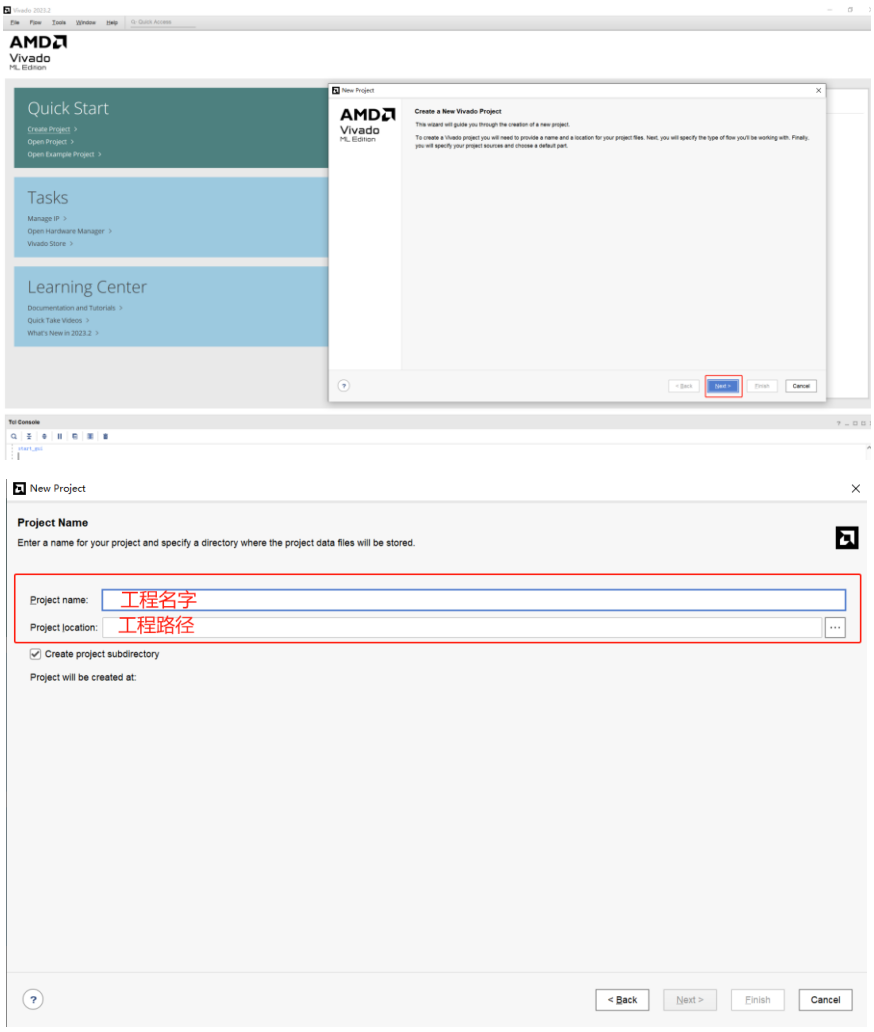
create_waiver -internal -quiet -type CDC -id {CDC-12} -user gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*rst_in_out_reg*}]] \
                              -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]

create_waiver -internal -quiet -type CDC -id {CDC-12} -user gtwizard_ultrascale -tags "1165536" -description "CDC-12 In exdes and vio path" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*prbs_match_out_reg*}]] \
                              -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]

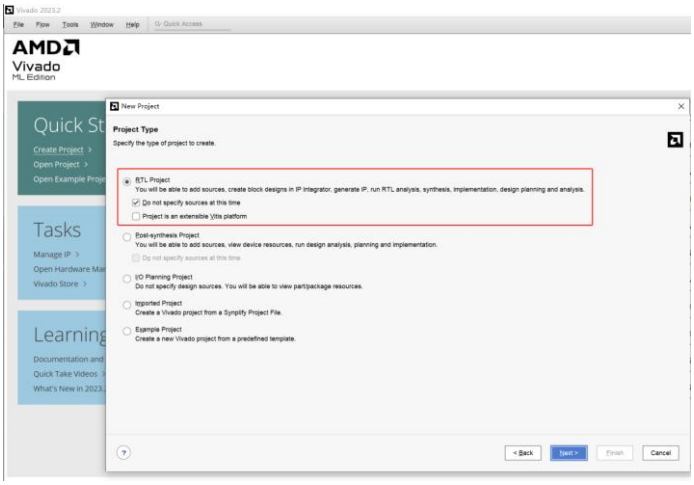
create_waiver -internal -quiet -type CDC -id {CDC-10} -user gtwizard_ultrascale -tags "1165536" -description "CDC-10 Paths between support blocks like vio and exdes inst" \
                        -scope -from [get_pins -quiet -filter {REF_PIN_NAME=~*C} -of_objects [get_cells -hierarchical -filter {NAME=~*prbs_match_out_reg_inv}]] \
                    -to [get_pins -quiet -filter {REF_PIN_NAME=~*PRE} -of_objects [get_cells -hierarchical -filter {NAME =~*rst_in_meta_reg}]]
```

3.19.3 工程搭建

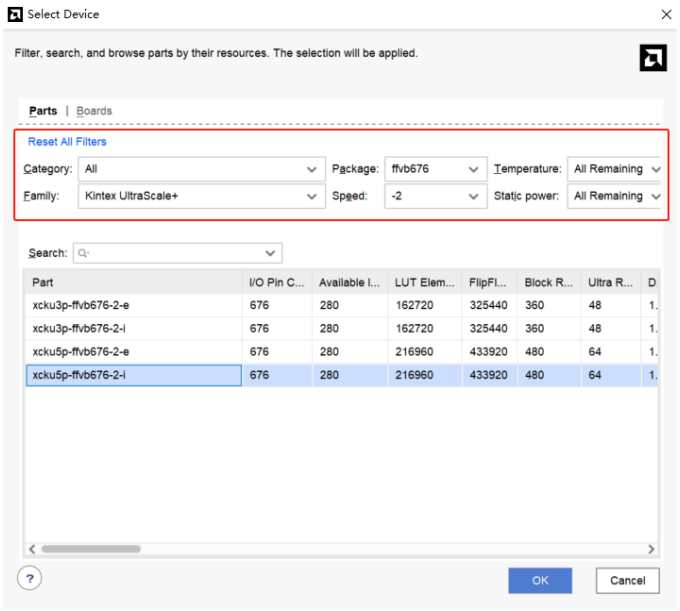
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



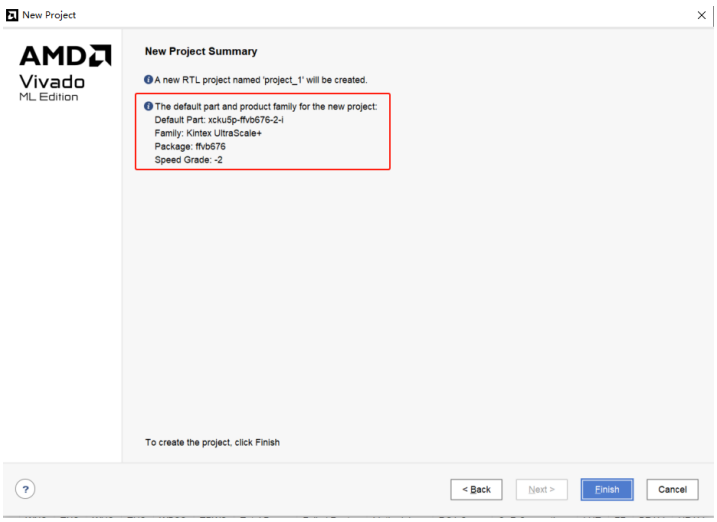
选择 RTL Project 并且勾选复选框，之后单击 NEXT



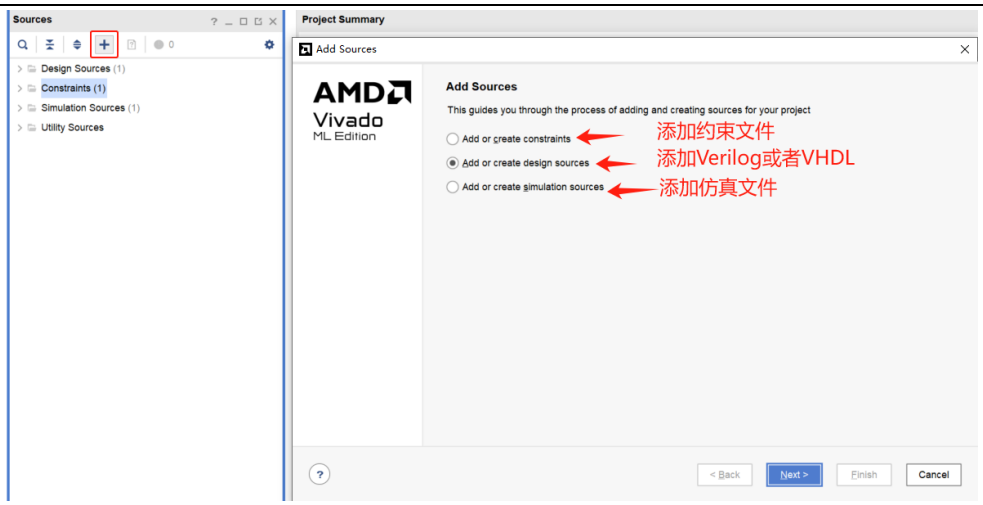
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

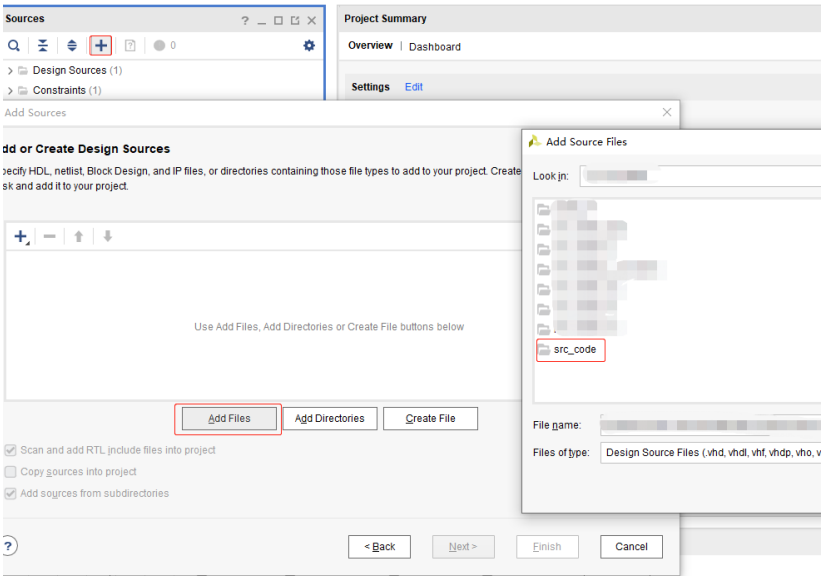


单击 Finish 完成工程创建

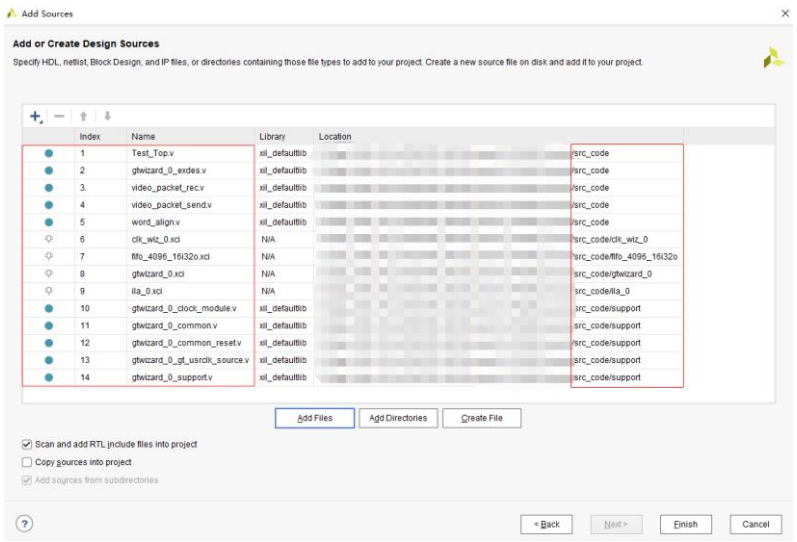


添加代码，显示界面如下

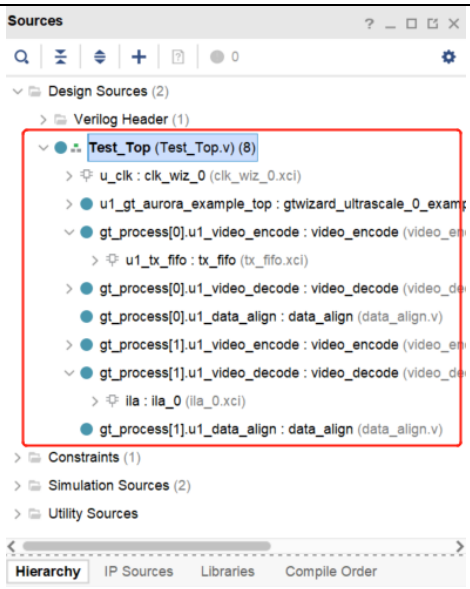




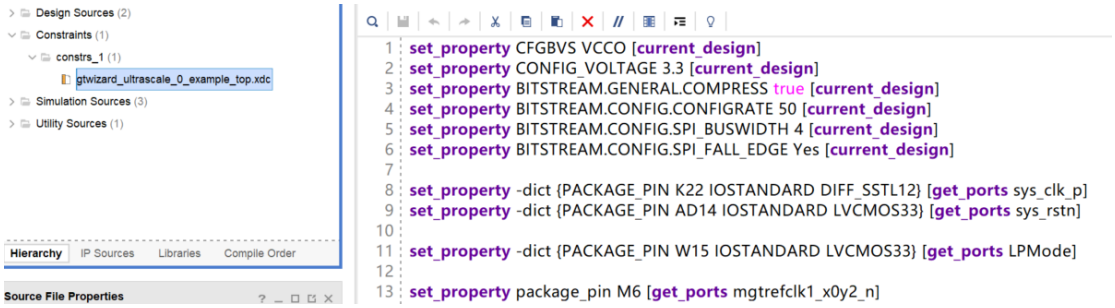
添加代码文件（代码文件在工程路径下）



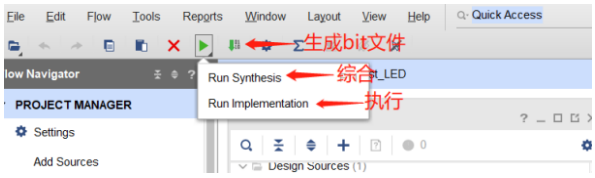
添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



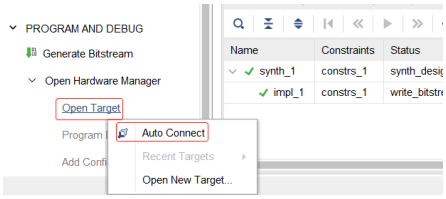
添加管脚约束文件



编译并且产生 bit 文件
1:单击综合
2:单击执行
3:单击产生 bit



下载程序
1：给开发板通电，并且连接下载器
2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



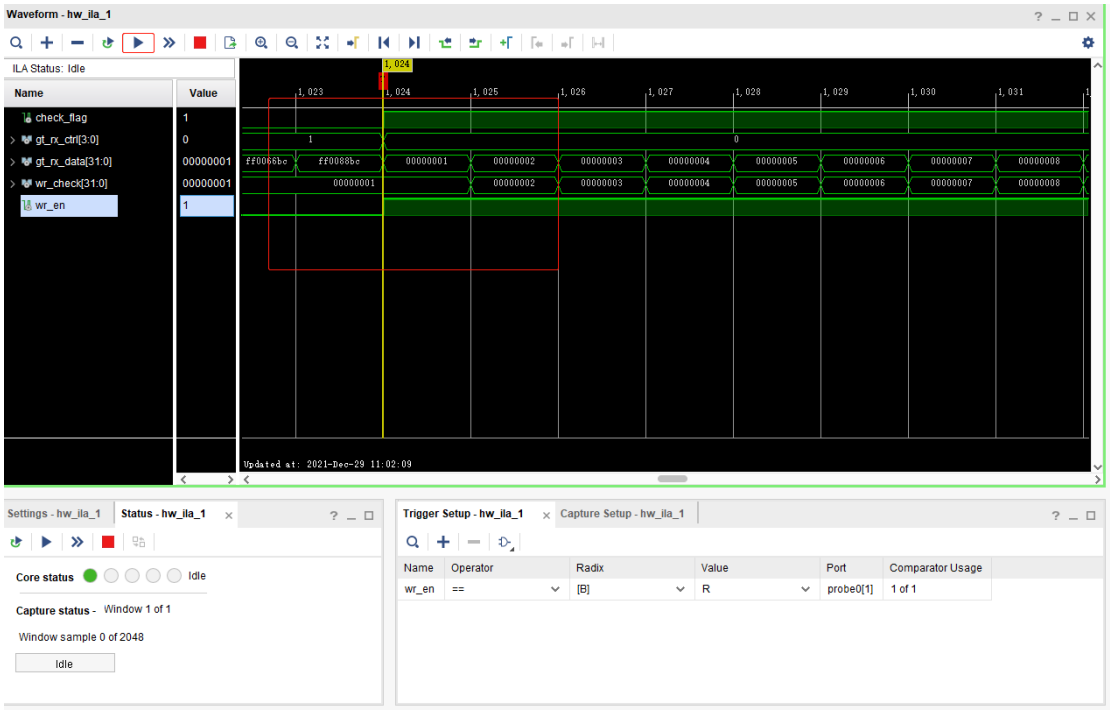
单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.19.4 运行结果

下载完成后，ILA 自动运行，我们点击红色方框抓取数据（测试模块需要为 40G 模块，短接回环测试，因为我们使用的速率为 10G，4Line 即 40G）
如下图：





3.20 高速 ADC—AD9643 采集例程(250MSPS)

3.20.1 功能概述

AD9643 是一款双通道、14 位、采样速率最高达 250 MSPS 的模数转换器(ADC)，旨在为低成本、小尺寸、宽带宽、多功能通信应用提供解决方案。

这款双通道 ADC 采用多级、差分流水线架构，并集成了输出纠错逻辑。每个 ADC 均具有宽带宽输入，支持用户可选的各种输入范围。集成基准电压源可简化设计。占空比稳定器可用来补偿 ADC 时钟占空比的波动，使转换器保持出色的性能。

ADC 输出数据直接路由至两个外部 14 位 LVDS 输出端口，格式化为交错式或通道复用式。

需要时，灵活的关断选项可以明显降低功耗。

设置与控制编程利用三线式 SPI 兼容型串行接口来完成。

AD9643 采用 64 引脚 LFCSP 封装，额定温度范围为−40° C 至+85° C 工业温度范围。

本例程通过 ILA 抓取 AD9643 采集的数据，然后转成模拟显示（需要高速 AD9643 子模块，然后接插在开发板 J23 FMC 扩展口上）。

3.20.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

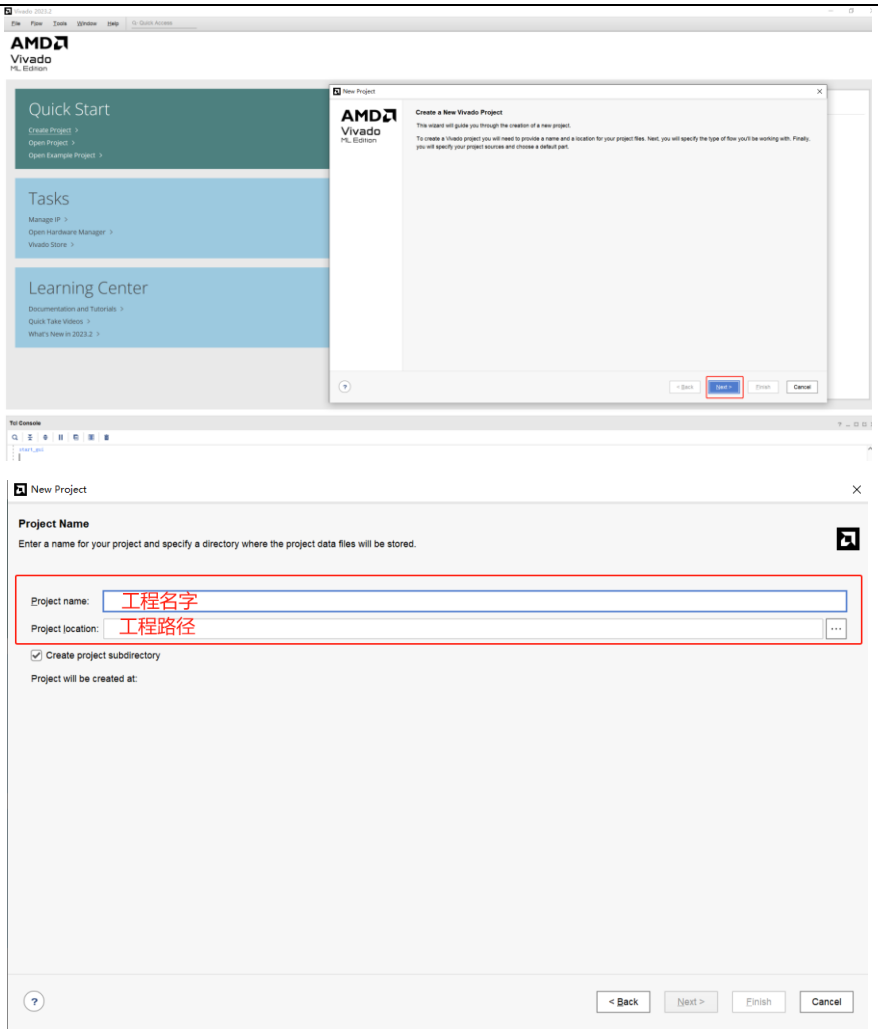
set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports sys_rstn]
```

```
##########################################################################
set_property PACKAGE_PIN AE17  [get_ports {adc_data_p[0]}]
set_property PACKAGE_PIN AF24  [get_ports {adc_data_p[1]}]
set_property PACKAGE_PIN AD16  [get_ports {adc_data_p[2]}]
set_property PACKAGE_PIN AF18  [get_ports {adc_data_p[3]}]
set_property PACKAGE_PIN AE25  [get_ports {adc_data_p[4]}]
set_property PACKAGE_PIN AE22  [get_ports {adc_data_p[5]}]
set_property PACKAGE_PIN AC18  [get_ports {adc_data_p[6]}]
set_property PACKAGE_PIN AB25  [get_ports {adc_data_p[7]}]
set_property PACKAGE_PIN AD23  [get_ports {adc_data_p[8]}]
set_property PACKAGE_PIN AB17  [get_ports {adc_data_p[9]}]
set_property PACKAGE_PIN AB24  [get_ports {adc_data_p[10]}]
set_property PACKAGE_PIN AD24  [get_ports {adc_data_p[11]}]
set_property PACKAGE_PIN AA19  [get_ports {adc_data_p[12]}]
set_property PACKAGE_PIN Y18   [get_ports {adc_data_p[13]}]
set_property PACKAGE_PIN AC19  [get_ports {adc_clk_p}]
set_property PACKAGE_PIN AC21  [get_ports adc_clk_oeb]
set_property PACKAGE_PIN AB21  [get_ports adc_clk_pwn]
set_property PACKAGE_PIN AD21  [get_ports {adc_clk_op}]

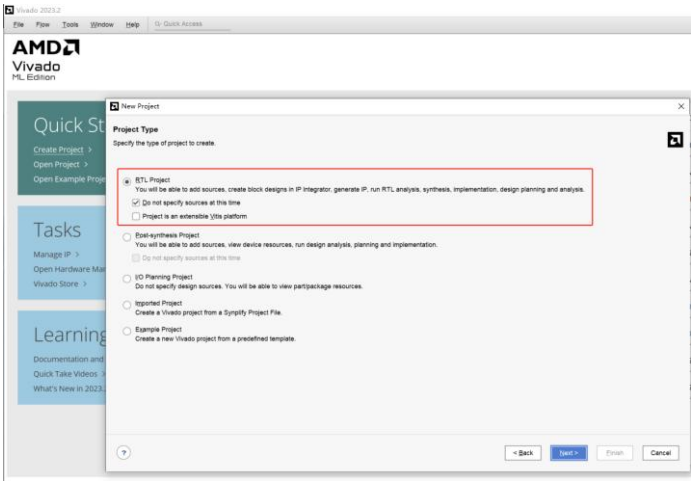
set_property  -dict  {IOSTANDARD  LVDS  DIFF_TERM  1}  [get_ports
{adc_data_p[*]}]
set_property -dict {IOSTANDARD LVDS DIFF_TERM 1} [get_ports adc_clk_p]
set_property IOSTANDARD LVCMOS18 [get_ports adc_clk_oeb]
set_property IOSTANDARD LVCMOS18 [get_ports adc_clk_pwn]
set_property -dict {IOSTANDARD LVDS} [get_ports adc_clk_op]
```

3.20.3 工程搭建

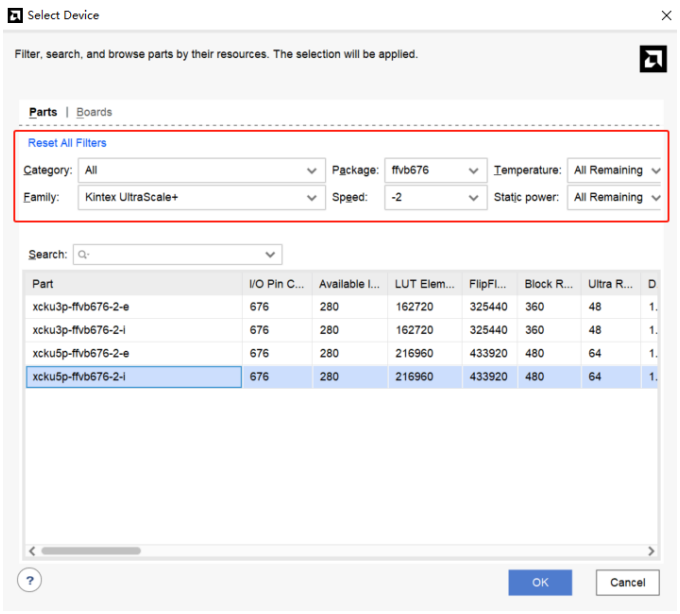
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



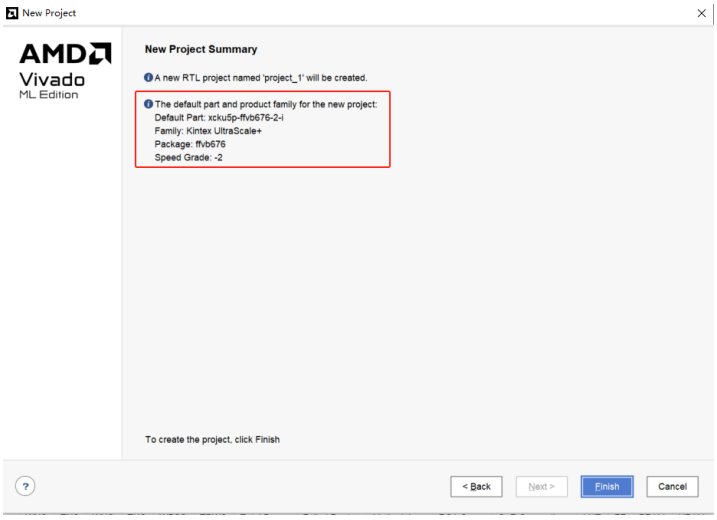
选择 RTL Project 并且勾选复选框，之后单击 NEXT



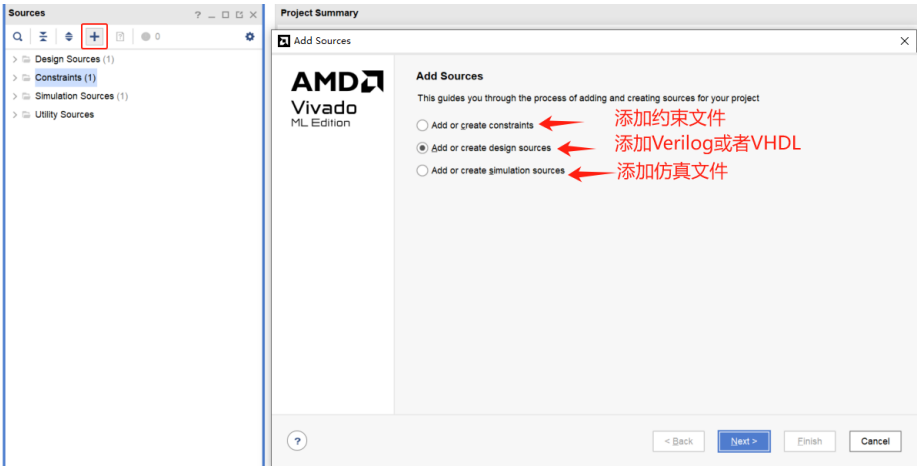
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

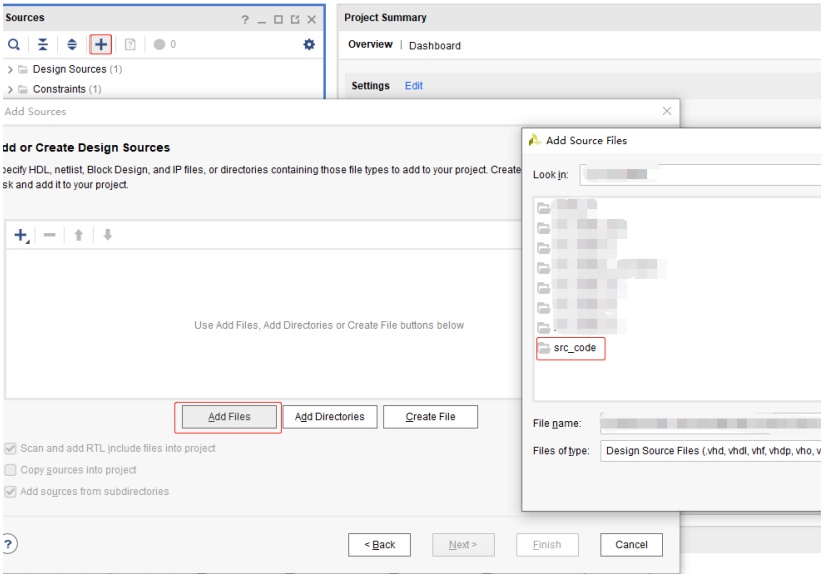


单击 Finish 完成工程创建

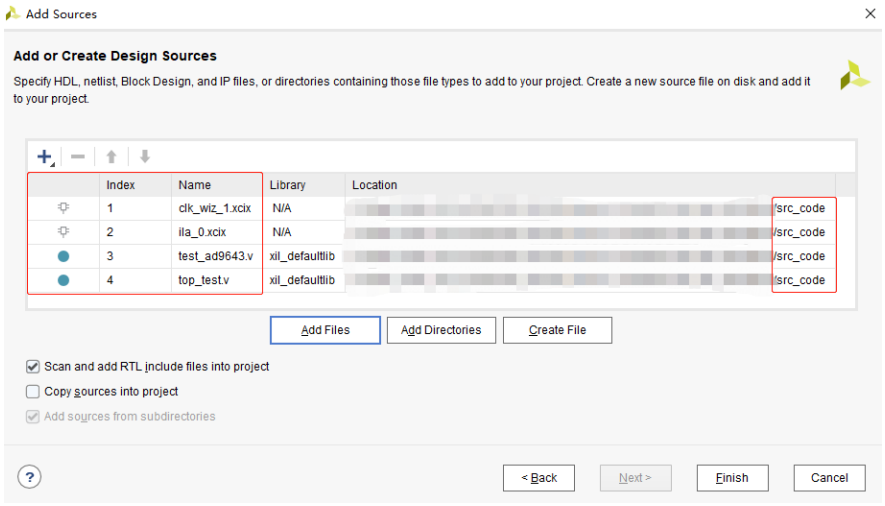


添加代码，显示界面如下

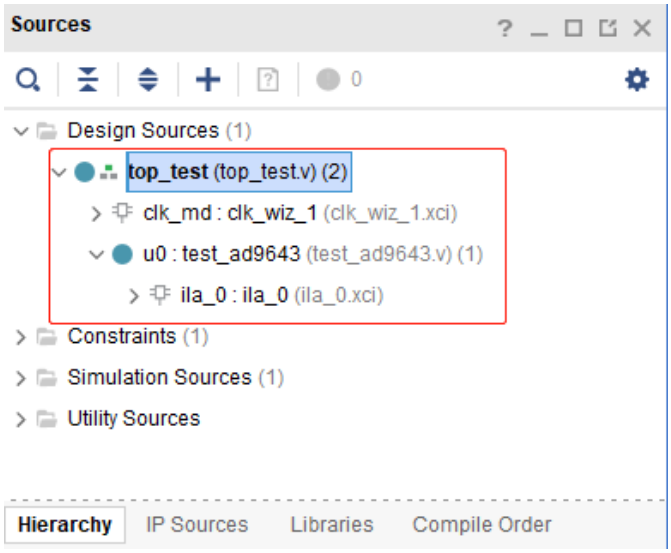




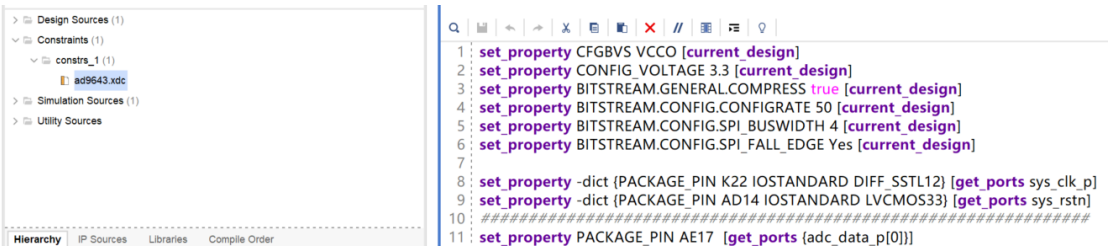
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

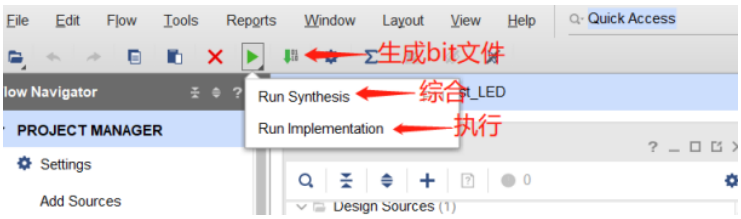


编译并且产生 bit 文件

1:单击综合

2:单击执行

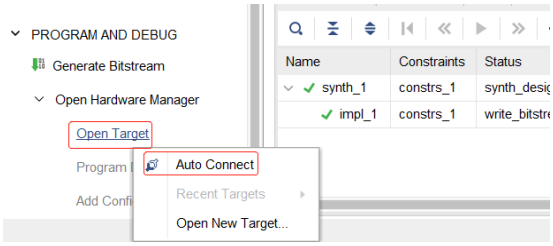
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

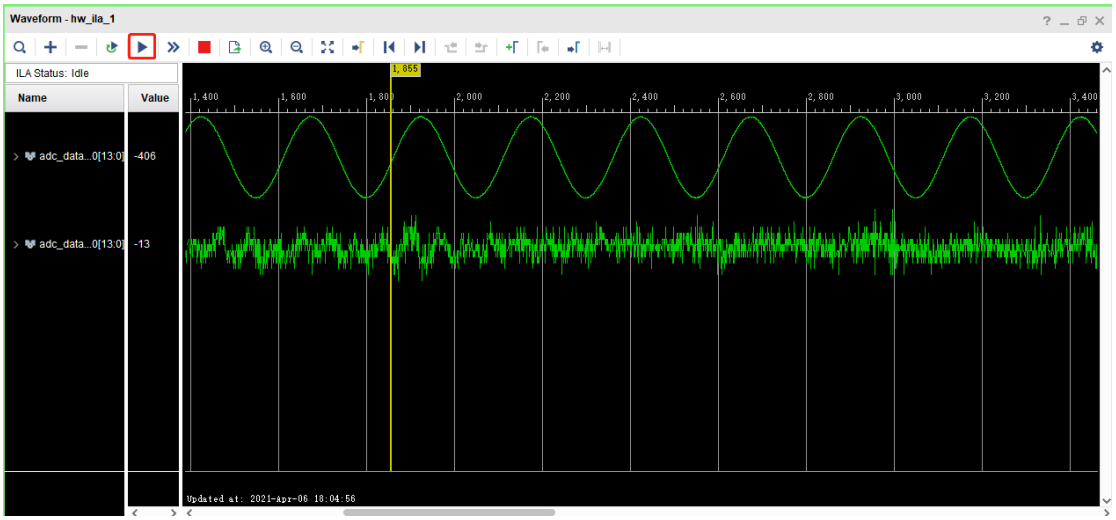


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.20.4 运行结果

下载完成后，ILA 自动运行，我们点击红色方框抓取数据，然后切换到模拟显示，如下图：



如上图，ILA 可以显示信号发生器输入的波形。

3.21 UDP 传输高速 ADC 采集数据例程

3.21.1 功能概述

AD9643 是一款双通道、14 位、采样速率最高达 250 MSPS 的模数转换器(ADC)，旨在为低成本、小尺寸、宽带宽、多功能通信应用提供解决方案。

这款双通道 ADC 采用多级、差分流水线架构，并集成了输出纠错逻辑。每个 ADC 均具有宽带宽输入，支持用户可选的各种输入范围。集成基准电压源可简化设计。占空比稳定器可用来补偿 ADC 时钟占空比的波动，使转换器保持出色的性能。

ADC 输出数据直接路由至两个外部 14 位 LVDS 输出端口，格式化为交错式或通道复用式。

需要时，灵活的关断选项可以明显降低功耗。

设置与控制编程利用三线式 SPI 兼容型串行接口来完成。

AD9643 采用 64 引脚 LFCSP 封装，额定温度范围为−40° C 至+85° C 工业温度范围。

本例程先获取 AD9643 采集的数据，然后通过 FPGA 侧的网口 LAN0，实现 UDP 协议把数据传输到 PC 主机上，用上位机软件可以实时查看采集的波形。考虑到 PC 主机及千兆网口等因素，网口 LAN0 的传输速率控制在 350Mb/s 左右（需要高速 AD9643 子模块，然后接插在开发板 J19 FMC 扩展口上）。

3.21.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
rst_n]
##########################################################################
```

```
set_property PACKAGE_PIN AE17  [get_ports {adc_data_p[0]}]
set_property PACKAGE_PIN AF24  [get_ports {adc_data_p[1]}]
set_property PACKAGE_PIN AD16  [get_ports {adc_data_p[2]}]
set_property PACKAGE_PIN AF18  [get_ports {adc_data_p[3]}]
set_property PACKAGE_PIN AE25  [get_ports {adc_data_p[4]}]
set_property PACKAGE_PIN AE22  [get_ports {adc_data_p[5]}]
set_property PACKAGE_PIN AC18  [get_ports {adc_data_p[6]}]
set_property PACKAGE_PIN AB25  [get_ports {adc_data_p[7]}]
set_property PACKAGE_PIN AD23  [get_ports {adc_data_p[8]}]
set_property PACKAGE_PIN AB17  [get_ports {adc_data_p[9]}]
set_property PACKAGE_PIN AB24  [get_ports {adc_data_p[10]}]
set_property PACKAGE_PIN AD24  [get_ports {adc_data_p[11]}]
set_property PACKAGE_PIN AA19  [get_ports {adc_data_p[12]}]
set_property PACKAGE_PIN Y18   [get_ports {adc_data_p[13]}]
set_property PACKAGE_PIN AC19  [get_ports {adc_clk_p}]
set_property PACKAGE_PIN AC21  [get_ports adc_clk_oeb]
set_property PACKAGE_PIN AB21  [get_ports adc_clk_pwn]
set_property PACKAGE_PIN AD21  [get_ports {adc_clk_op}]

set_property  -dict  {IOSTANDARD  LVDS  DIFF_TERM  1}  [get_ports
{adc_data_p[*]}]
set_property -dict {IOSTANDARD LVDS DIFF_TERM 1} [get_ports adc_clk_p]
set_property IOSTANDARD LVCMOS18 [get_ports adc_clk_oeb]
set_property IOSTANDARD LVCMOS18 [get_ports adc_clk_pwn]
set_property -dict {IOSTANDARD LVDS} [get_ports adc_clk_op]

create_clock -period 4.000 -name adc_clk_p -waveform {0.000 2.000}
[get_ports adc_clk_p]


set_property -dict {PACKAGE_PIN A13  IOSTANDARD LVCMOS33} [get_ports
eth_mdc]
set_property -dict {PACKAGE_PIN A12  IOSTANDARD LVCMOS33} [get_ports
eth_mdio]
set_property -dict {PACKAGE_PIN V23 IOSTANDARD LVCMOS18} [get_ports
net_rxc]
set_property -dict {PACKAGE_PIN W23 IOSTANDARD LVCMOS18} [get_ports
net_rx_ctl]
set_property -dict {PACKAGE_PIN R25 IOSTANDARD LVCMOS18} [get_ports
{net_rxd[0]}]
set_property -dict {PACKAGE_PIN R26 IOSTANDARD LVCMOS18} [get_ports
{net_rxd[1]}]
set_property -dict {PACKAGE_PIN N24 IOSTANDARD LVCMOS18} [get_ports
{net_rxd[2]}]
```

```
set_property -dict {PACKAGE_PIN P24 IOSTANDARD LVCMOS18} [get_ports {net_rxd[3]}]

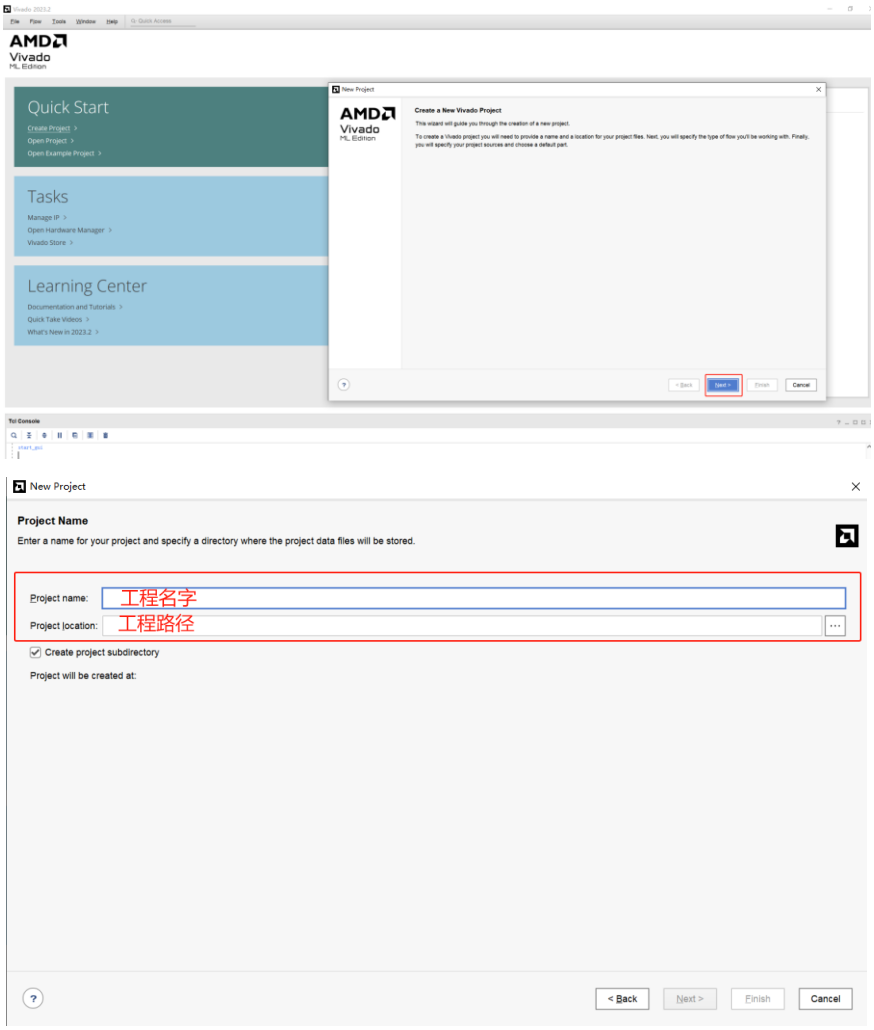
set_property -dict {PACKAGE_PIN N21 IOSTANDARD LVCMOS18} [get_ports net_txc]
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS18} [get_ports net_tx_ctl]
set_property -dict {PACKAGE_PIN R20 IOSTANDARD LVCMOS18} [get_ports {net_txd[0]}]
set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS18} [get_ports {net_txd[1]}]
set_property -dict {PACKAGE_PIN W19 IOSTANDARD LVCMOS18} [get_ports {net_txd[2]}]
set_property -dict {PACKAGE_PIN W20 IOSTANDARD LVCMOS18} [get_ports {net_txd[3]}]



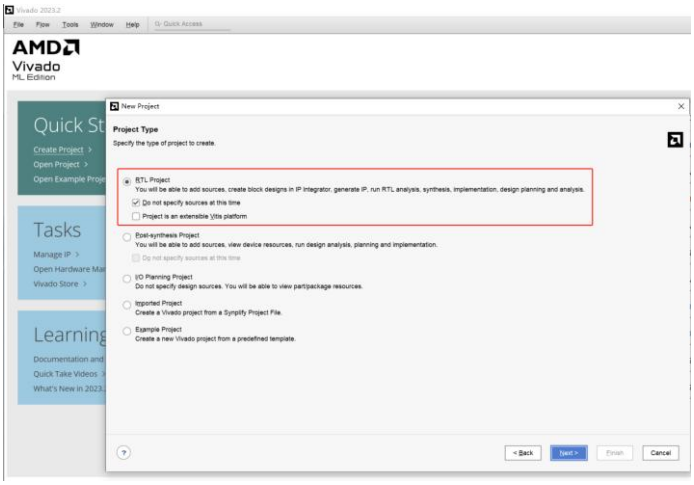
set_property SLEW FAST [get_ports  net_txc]
set_property SLEW FAST [get_ports  net_tx_ctl]
set_property SLEW FAST [get_ports {net_txd[*]}]
```

3.21.3 工程搭建

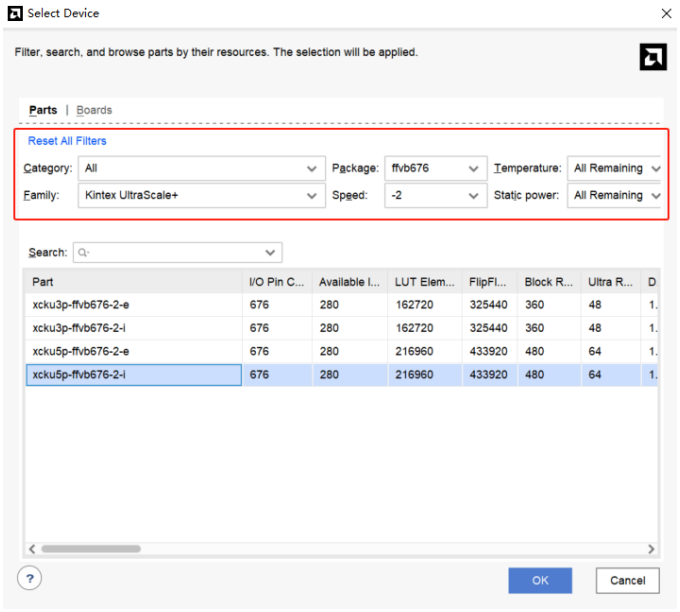
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



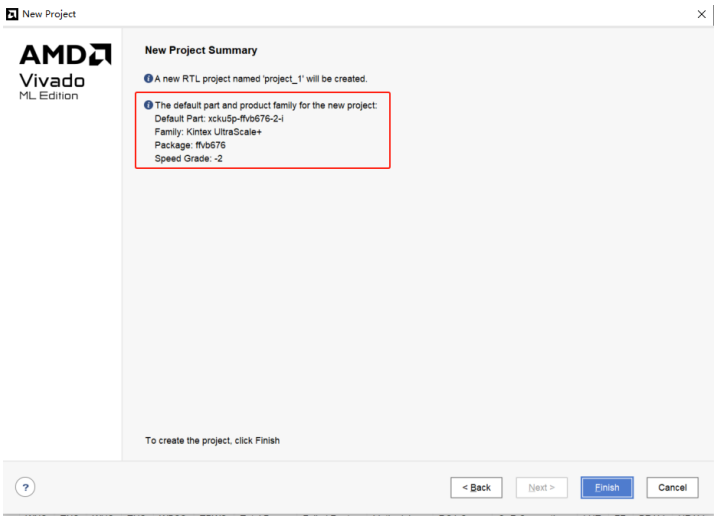
选择 RTL Project 并且勾选复选框，之后单击 NEXT



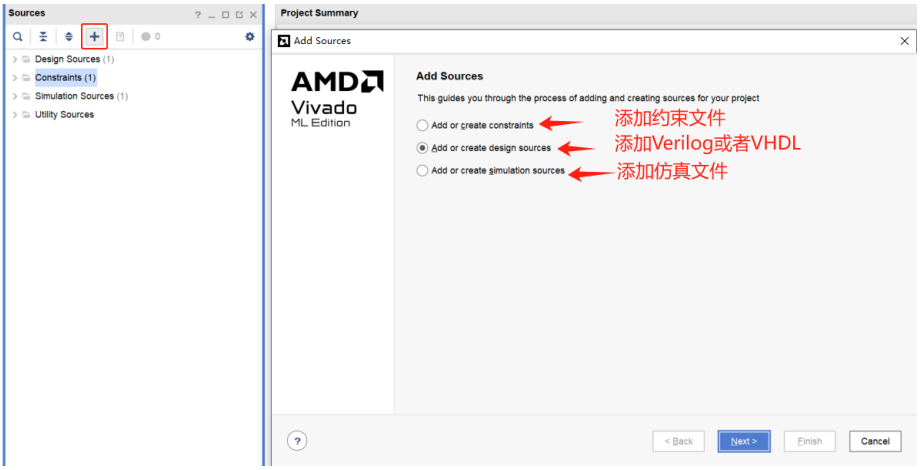
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

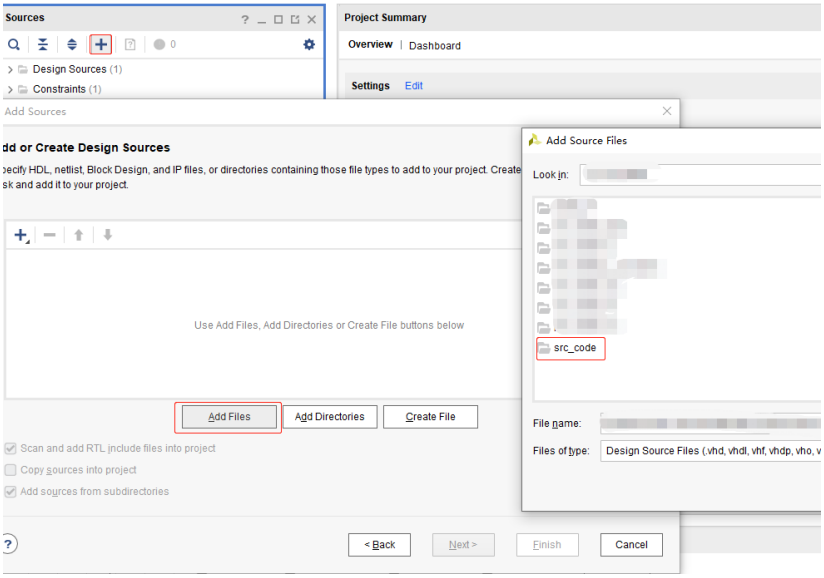


单击 Finish 完成工程创建

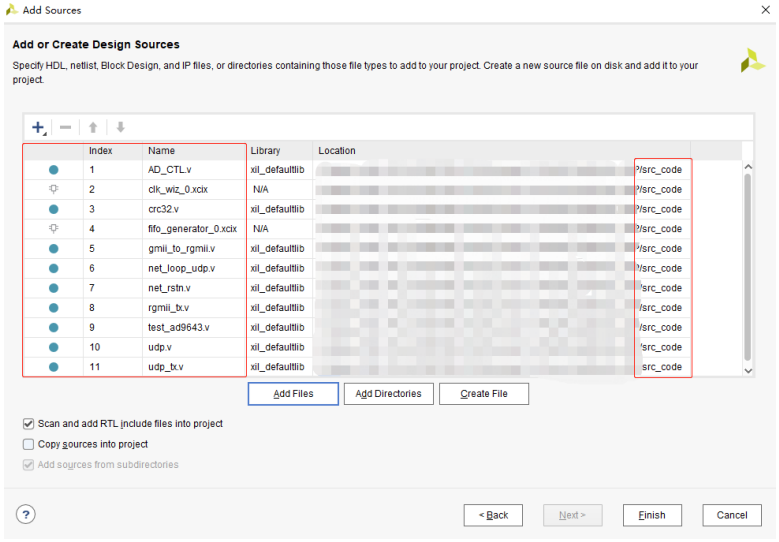


添加代码，显示界面如下





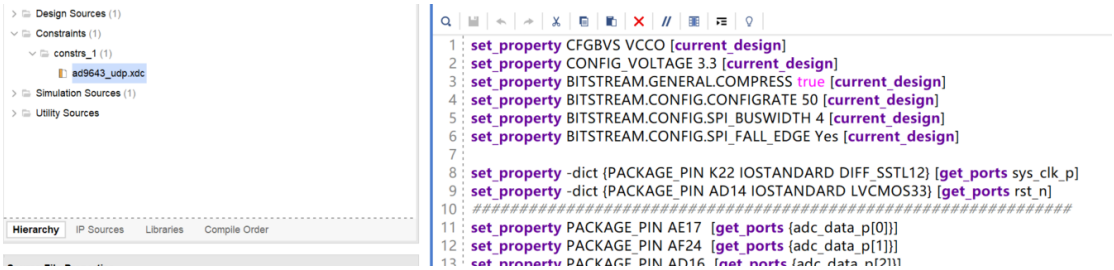
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

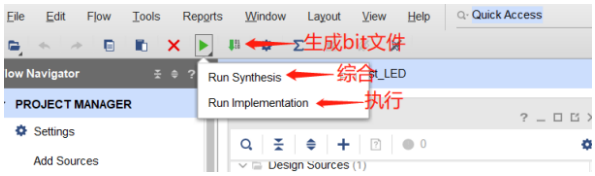


编译并且产生 bit 文件

1:单击综合

2:单击执行

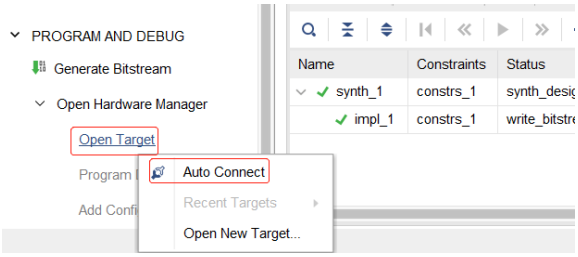
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

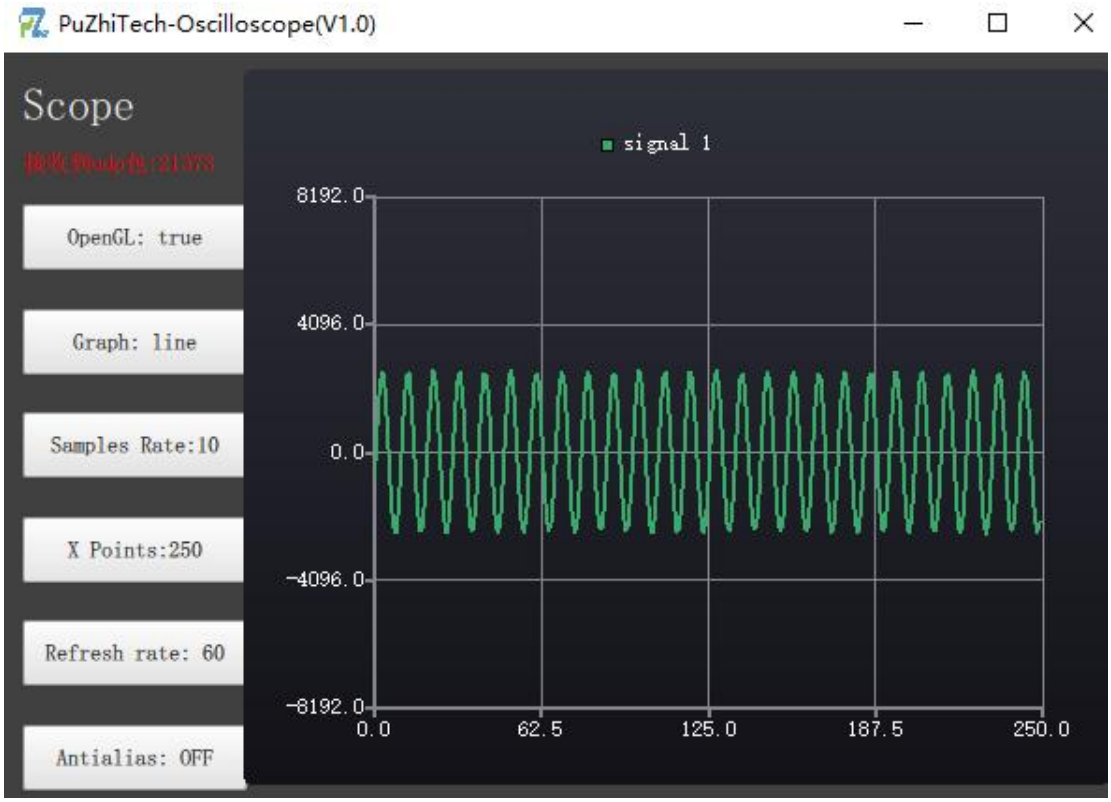


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.21.4 运行结果

下载完成后，网口 LAN 接网线到电脑端，子卡(注意：是 J19 FMC 接口)接入信号源，打开 QT 示波器软件(QT 软件包在 Vivado 工程下)，如下图：
左侧功能分别如下:
1.接收到的 UDP 包数量:用于指示当前 UDP 是否在工作
2.是否使用 GPU 的 OpenGL(显示更高效)
3.曲线显示方式:折线图(Line)还是散点图(Scatter)
4.数据抽样率:一个 UDP 包每次传输几百个数据，从里面按照这个间隔抽取数据
5.X 轴上显示的数据个数:对应 X 轴大小
6.刷新率:曲线显示实际就是把一个从 UDP 包中拿过来的数组刷新到界面上，这个参数控制每秒刷新多少次，一般 60 即可
7.是否打开抗锯齿功能:打开后，曲线显示更加平滑



3.22 高速 ADC—AD9643 的 LCD 显示例程

3.22.1 功能概述

AD9643 是一款双通道、14 位、采样速率最高达 250 MSPS 的模数转换器(ADC)，旨在为低成本、小尺寸、宽带宽、多功能通信应用提供解决方案。

这款双通道 ADC 采用多级、差分流水线架构，并集成了输出纠错逻辑。每个 ADC 均具有宽带宽输入，支持用户可选的各种输入范围。集成基准电压源可简化设计。占空比稳定器可用来补偿 ADC 时钟占空比的波动，使转换器保持出色的性能。

ADC 输出数据直接路由至两个外部 14 位 LVDS 输出端口，格式化为交错式或通道复用式。

需要时，灵活的关断选项可以明显降低功耗。

设置与控制编程利用三线式 SPI 兼容型串行接口来完成。

AD9643 采用 64 引脚 LFCSP 封装，额定温度范围为−40° C 至+85° C 工业温度范围。

本例程先获取 AD9643 采集的数据，然后通过 LCD 显示屏输出显示实时波形。

3.22.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rstn]
##########################################################################
set_property PACKAGE_PIN AE17  [get_ports {adc_data_p[0]}]
set_property PACKAGE_PIN AF24  [get_ports {adc_data_p[1]}]
set_property PACKAGE_PIN AD16  [get_ports {adc_data_p[2]}]
set_property PACKAGE_PIN AF18  [get_ports {adc_data_p[3]}]
set_property PACKAGE_PIN AE25  [get_ports {adc_data_p[4]}]
```

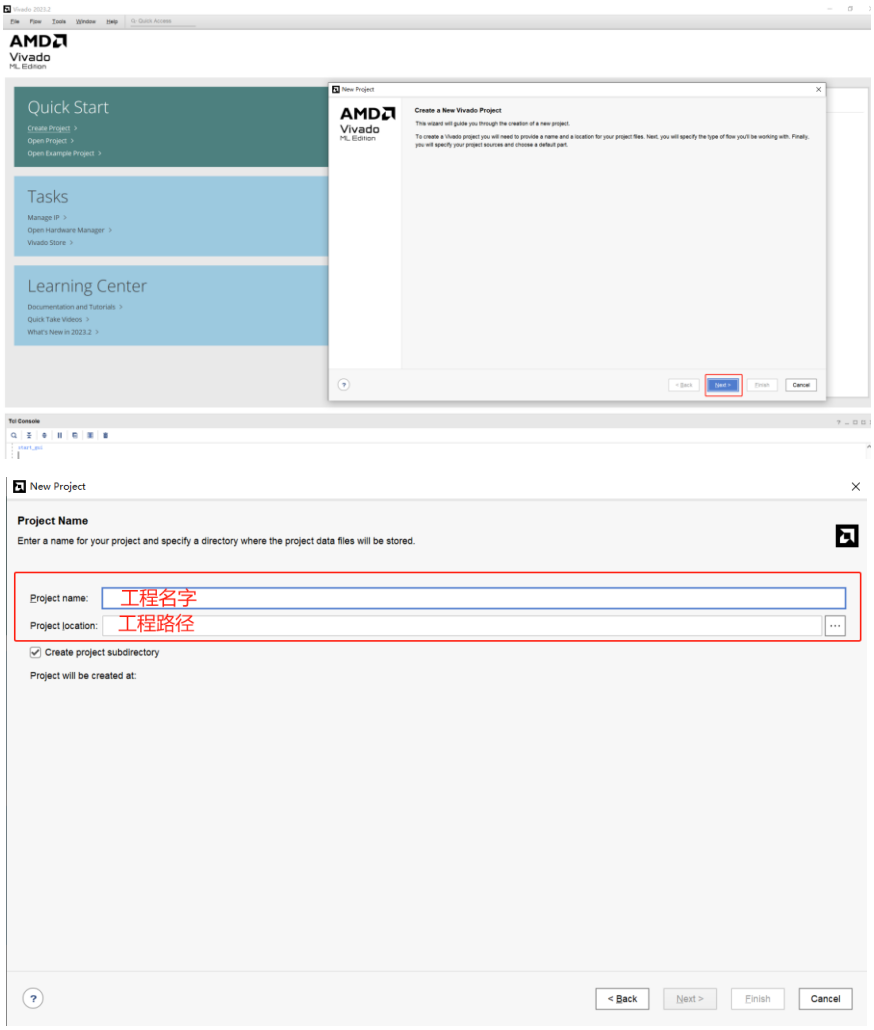
```
set_property PACKAGE_PIN AE22  [get_ports {adc_data_p[5]}]
set_property PACKAGE_PIN AC18  [get_ports {adc_data_p[6]}]
set_property PACKAGE_PIN AB25  [get_ports {adc_data_p[7]}]
set_property PACKAGE_PIN AD23  [get_ports {adc_data_p[8]}]
set_property PACKAGE_PIN AB17  [get_ports {adc_data_p[9]}]
set_property PACKAGE_PIN AB24  [get_ports {adc_data_p[10]}]
set_property PACKAGE_PIN AD24  [get_ports {adc_data_p[11]}]
set_property PACKAGE_PIN AA19  [get_ports {adc_data_p[12]}]
set_property PACKAGE_PIN Y18   [get_ports {adc_data_p[13]}]
set_property PACKAGE_PIN AC19  [get_ports {adc_clk_p}]
set_property PACKAGE_PIN AC21  [get_ports adc_clk_oeb]
set_property PACKAGE_PIN AB21  [get_ports adc_clk_pwn]
set_property PACKAGE_PIN AD21  [get_ports {adc_clk_op}]

set_property  -dict  {IOSTANDARD  LVDS  DIFF_TERM  1}  [get_ports
{adc_data_p[*]}]
set_property -dict {IOSTANDARD LVDS DIFF_TERM 1} [get_ports adc_clk_p]
set_property IOSTANDARD LVCMOS18 [get_ports adc_clk_oeb]
set_property IOSTANDARD LVCMOS18 [get_ports adc_clk_pwn]
set_property -dict {IOSTANDARD LVDS} [get_ports adc_clk_op]
##########################################################################
set_property -dict {PACKAGE_PIN E11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[0]}]
set_property -dict {PACKAGE_PIN G10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[1]}]
set_property -dict {PACKAGE_PIN  E10 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[2]}]
set_property -dict {PACKAGE_PIN  G9  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[3]}]
set_property -dict {PACKAGE_PIN  D11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[4]}]
set_property  -dict  {PACKAGE_PIN    F10        IOSTANDARD  LVCMOS33}
[get_ports {lcd_rgb[5]}]
set_property -dict {PACKAGE_PIN  D10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[6]}]
set_property -dict {PACKAGE_PIN  F9    IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[7]}]
set_property -dict {PACKAGE_PIN  C11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[8]}]
set_property -dict {PACKAGE_PIN  B9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[9]}]
set_property -dict {PACKAGE_PIN  B11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[10]}]
set_property -dict {PACKAGE_PIN  A9  IOSTANDARD LVCMOS33} [get_ports
```

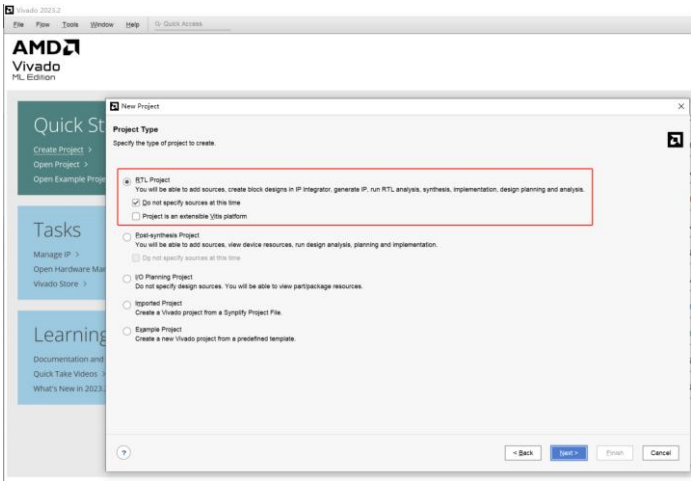
```
{lcd_rgb[11]}]
set_property -dict {PACKAGE_PIN  D9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[12]}]
set_property -dict {PACKAGE_PIN  B10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[13]}]
set_property -dict {PACKAGE_PIN  C9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[14]}]
set_property -dict {PACKAGE_PIN  A10 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[15]}]
set_property -dict {PACKAGE_PIN  J11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[16]}]
set_property -dict {PACKAGE_PIN  H11 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[17]}]
set_property -dict {PACKAGE_PIN  J10  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[18]}]
set_property -dict {PACKAGE_PIN  G11  IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[19]}]
set_property -dict {PACKAGE_PIN  K10 IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[20]}]
set_property -dict {PACKAGE_PIN  J9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[21]}]
set_property -dict {PACKAGE_PIN  K9   IOSTANDARD LVCMOS33} [get_ports
{lcd_rgb[22]}]
set_property  -dict  {PACKAGE_PIN    H9          IOSTANDARD  LVCMOS33}
[get_ports {lcd_rgb[23]}]
set_property -dict {PACKAGE_PIN  J13 IOSTANDARD LVCMOS33} [get_ports
lcd_clk]
set_property -dict {PACKAGE_PIN  J15 IOSTANDARD LVCMOS33} [get_ports
lcd_hs]
set_property -dict {PACKAGE_PIN  H13 IOSTANDARD LVCMOS33} [get_ports
lcd_vs]
```

3.22.3 工程搭建

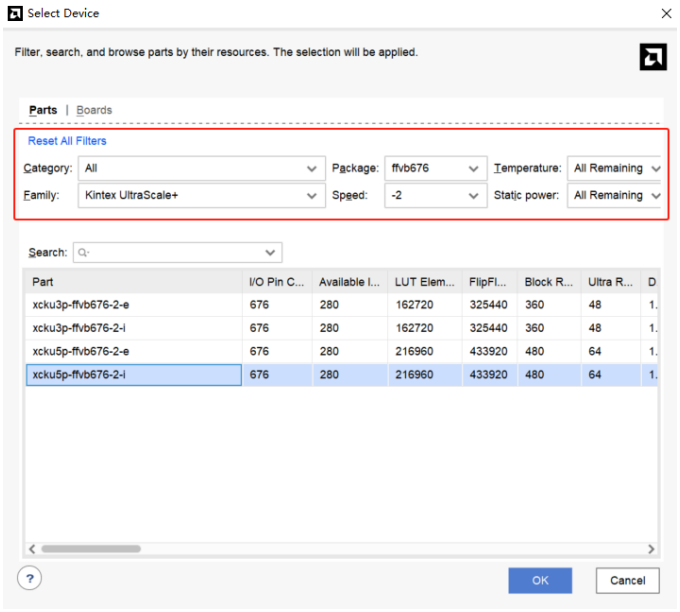
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



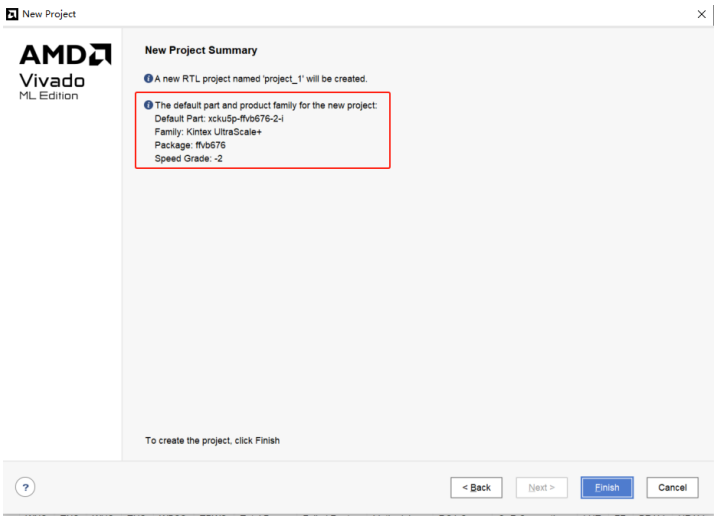
选择 RTL Project 并且勾选复选框，之后单击 NEXT



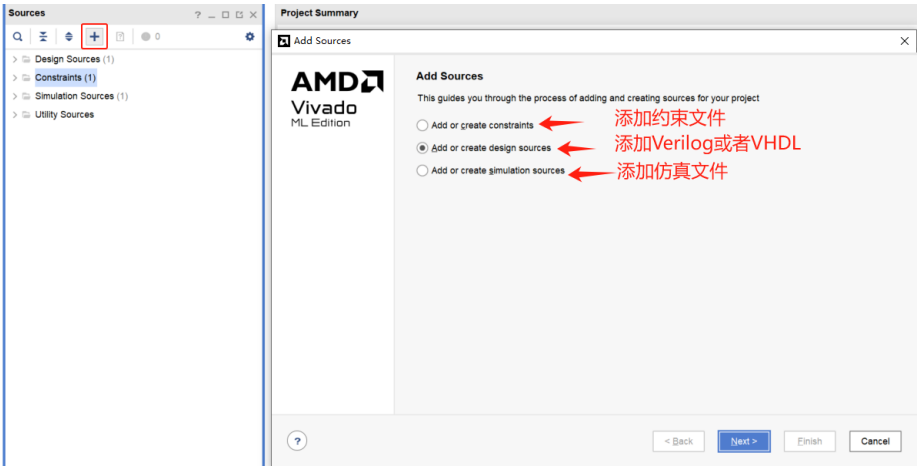
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

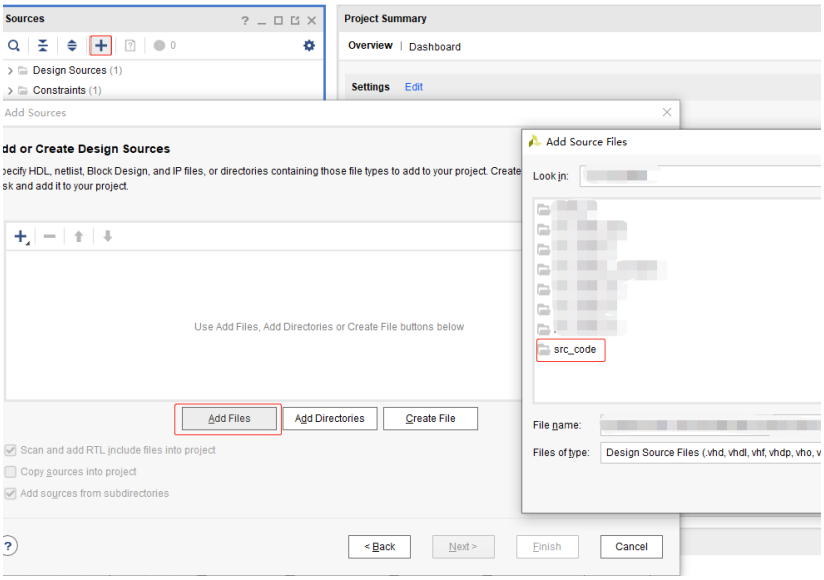


单击 Finish 完成工程创建

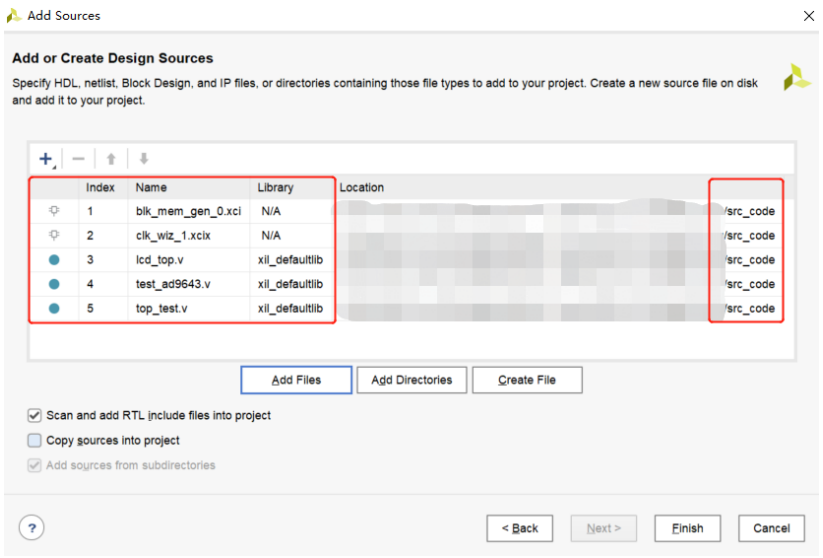


添加代码，显示界面如下

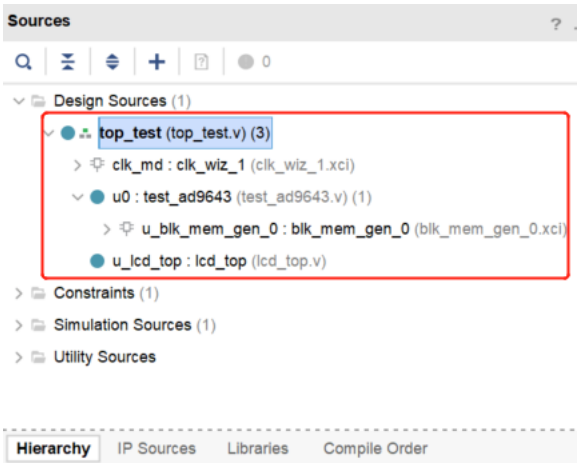




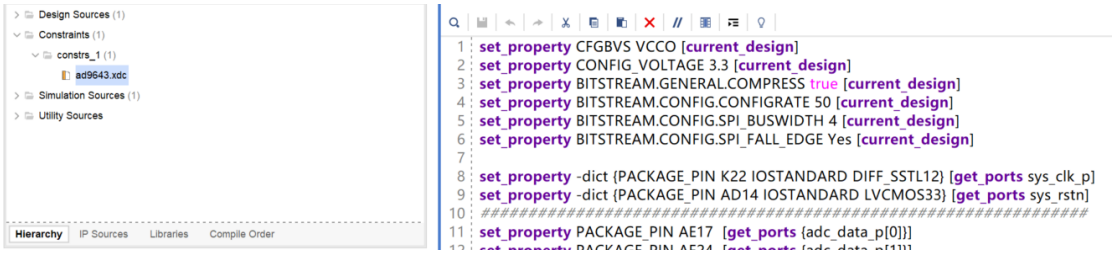
添加代码文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

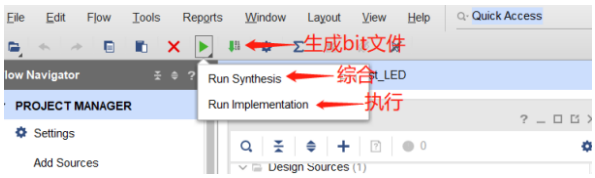


编译并且产生 bit 文件

1:单击综合

2:单击执行

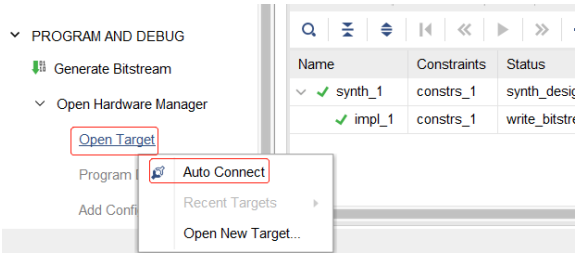
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：

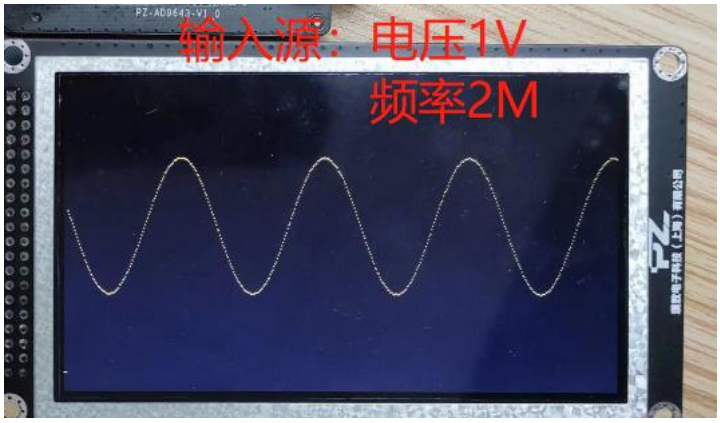


单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.22.4 运行结果

下载完成后，子卡(注意：是 J19 FMC 接口)接入信号源，然后查看 LCD 屏幕，建议输入电压 1V 以内，频率 10M 以内，显示效果较好，如下图：



3.23 OV5640 图像的 UDP 传输电脑端显示

3.23.1 功能概述

本实例通过 UDP 传输图像数据给上位机，在上位机软件中显示双 OV5640 摄像头采集的图像。每个摄像头的分辨率被裁剪为 320*480，然后合并成分辨率为 640*480 的图像，再一起同步显示，帧率为 26 帧左右。

说明：本实例需要双目摄像头子卡配合

3.23.2 管脚配置

管脚约束如下：

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]

create_clock -period 8.000 -name eth_rxc [get_ports net_rxc]

set_property -dict {PACKAGE_PIN K22 IOSTANDARD DIFF_SSTL12} [get_ports
sys_clk_p]
set_property -dict {PACKAGE_PIN AD14 IOSTANDARD LVCMOS33} [get_ports
sys_rst_n]

#CAMERA1
set_property -dict {PACKAGE_PIN C9  IOSTANDARD LVCMOS33} [get_ports
cam_clk]
set_property -dict {PACKAGE_PIN G14  IOSTANDARD LVCMOS33} [get_ports
cam_rst_n_1]
create_clock -period 40.000 -name cmos_pclk [get_ports cam_pclk_1]
set_property -dict {PACKAGE_PIN C14  IOSTANDARD LVCMOS33} [get_ports
cam_pclk_1]
 set_property       CLOCK_DEDICATED_ROUTE       FALSE       [get_nets
cam_pclk_1_IBUF_inst/O]
set_property -dict {PACKAGE_PIN  H11  IOSTANDARD LVCMOS33} [get_ports
cam_scl_1]
```

```
set_property -dict {PACKAGE_PIN  J11   IOSTANDARD LVCMOS33} [get_ports cam_sda_1]
set_property -dict {PACKAGE_PIN  H14  IOSTANDARD LVCMOS33} [get_ports cam_vsync_1]
set_property -dict {PACKAGE_PIN  C13  IOSTANDARD LVCMOS33} [get_ports cam_href_1]
set_property -dict {PACKAGE_PIN  H13   IOSTANDARD LVCMOS33} [get_ports {cam_data_1[0]}]
set_property -dict {PACKAGE_PIN  G11  IOSTANDARD LVCMOS33} [get_ports {cam_data_1[1]}]
set_property -dict {PACKAGE_PIN  J13  IOSTANDARD LVCMOS33} [get_ports {cam_data_1[2]}]
set_property -dict {PACKAGE_PIN  K9   IOSTANDARD LVCMOS33} [get_ports {cam_data_1[3]}]
set_property -dict {PACKAGE_PIN  H9  IOSTANDARD LVCMOS33} [get_ports {cam_data_1[4]}]
set_property -dict {PACKAGE_PIN  K10  IOSTANDARD LVCMOS33} [get_ports {cam_data_1[5]}]
set_property -dict {PACKAGE_PIN  J9  IOSTANDARD LVCMOS33} [get_ports {cam_data_1[6]}]
set_property -dict {PACKAGE_PIN  J10  IOSTANDARD LVCMOS33} [get_ports {cam_data_1[7]}]


#CAMERA2
create_clock -period 40.000 -name cmos_pclk [get_ports cam_pclk_2]
set_property       CLOCK_DEDICATED_ROUTE       FALSE       [get_nets cam_pclk_2_IBUF_inst/O]
set_property -dict {PACKAGE_PIN B10 IOSTANDARD LVCMOS33} [get_ports cam_pclk_2]
set_property -dict {PACKAGE_PIN G10 IOSTANDARD LVCMOS33} [get_ports cam_scl_2]
set_property -dict {PACKAGE_PIN E11 IOSTANDARD LVCMOS33} [get_ports cam_sda_2]
set_property -dict {PACKAGE_PIN D9 IOSTANDARD LVCMOS33} [get_ports cam_vsync_2]
set_property -dict {PACKAGE_PIN A10 IOSTANDARD LVCMOS33} [get_ports cam_href_2]
set_property -dict {PACKAGE_PIN B11 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[0]}]
set_property -dict {PACKAGE_PIN G9 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[1]}]
set_property -dict {PACKAGE_PIN C11 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[2]}]
```

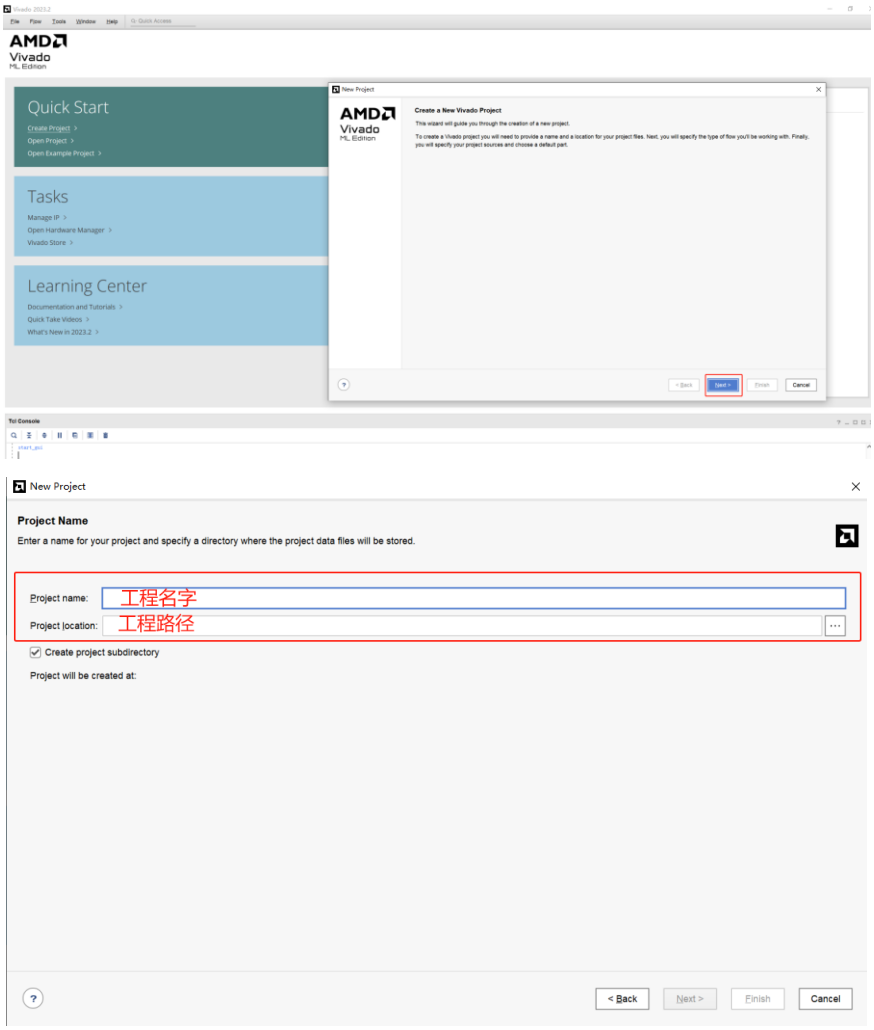
```
set_property -dict {PACKAGE_PIN D10 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[3]}]
set_property -dict {PACKAGE_PIN F9 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[4]}]
set_property -dict {PACKAGE_PIN D11 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[5]}]
set_property -dict {PACKAGE_PIN F10 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[6]}]
set_property -dict {PACKAGE_PIN E10 IOSTANDARD LVCMOS33} [get_ports {cam_data_2[7]}]

set_property -dict {PACKAGE_PIN A13  IOSTANDARD LVCMOS33} [get_ports eth_mdc]
set_property -dict {PACKAGE_PIN A12  IOSTANDARD LVCMOS33} [get_ports eth_mdio]
set_property -dict {PACKAGE_PIN V23 IOSTANDARD LVCMOS18} [get_ports net_rxc]
set_property -dict {PACKAGE_PIN W23 IOSTANDARD LVCMOS18} [get_ports net_rx_ctl]
set_property -dict {PACKAGE_PIN R25 IOSTANDARD LVCMOS18} [get_ports {net_rxd[0]}]
set_property -dict {PACKAGE_PIN R26 IOSTANDARD LVCMOS18} [get_ports {net_rxd[1]}]
set_property -dict {PACKAGE_PIN N24 IOSTANDARD LVCMOS18} [get_ports {net_rxd[2]}]
set_property -dict {PACKAGE_PIN P24 IOSTANDARD LVCMOS18} [get_ports {net_rxd[3]}]

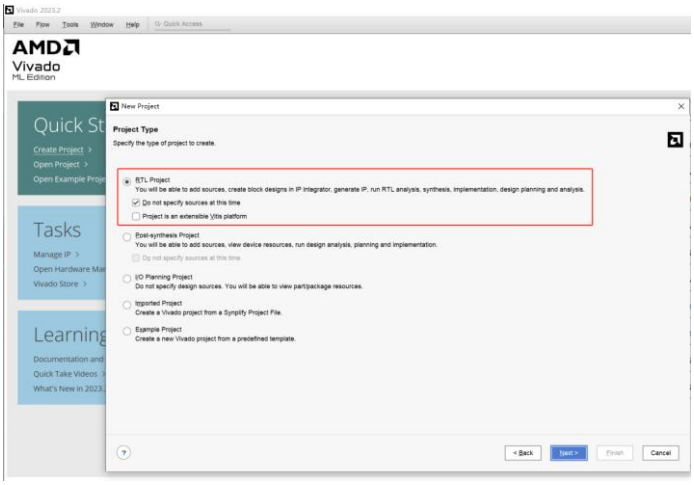
set_property -dict {PACKAGE_PIN N21 IOSTANDARD LVCMOS18} [get_ports net_txc]
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS18} [get_ports net_tx_ctl]
set_property -dict {PACKAGE_PIN R20 IOSTANDARD LVCMOS18} [get_ports {net_txd[0]}]
set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS18} [get_ports {net_txd[1]}]
set_property -dict {PACKAGE_PIN W19 IOSTANDARD LVCMOS18} [get_ports {net_txd[2]}]
set_property -dict {PACKAGE_PIN W20 IOSTANDARD LVCMOS18} [get_ports {net_txd[3]}]
set_property SLEW FAST [get_ports  net_txc]
set_property SLEW FAST [get_ports  net_tx_ctl]
set_property SLEW FAST [get_ports {net_txd[*]}]
```

3.23.3 工程搭建

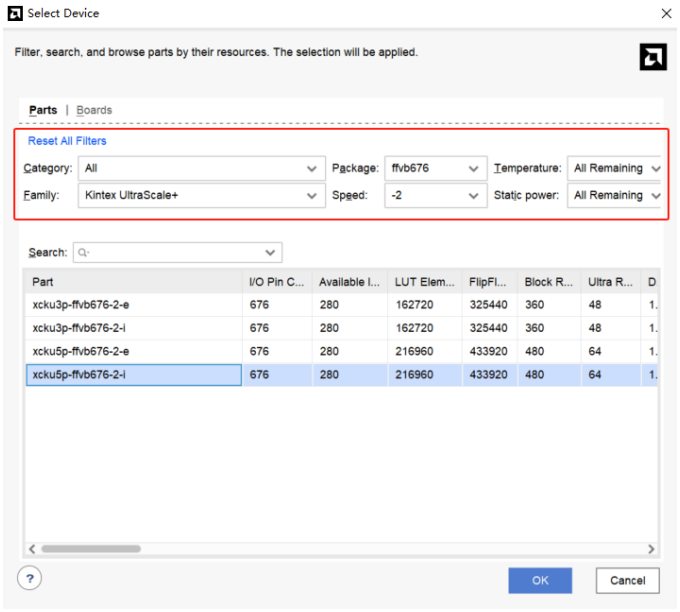
先启动 Vivado，创建工程，输入工程名字和工程路径，工程路径可以通过浏览文件夹选择



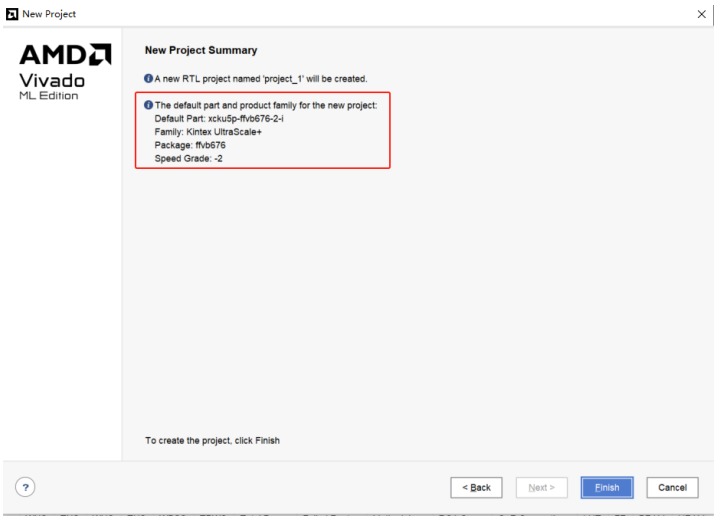
选择 RTL Project 并且勾选复选框，之后单击 NEXT



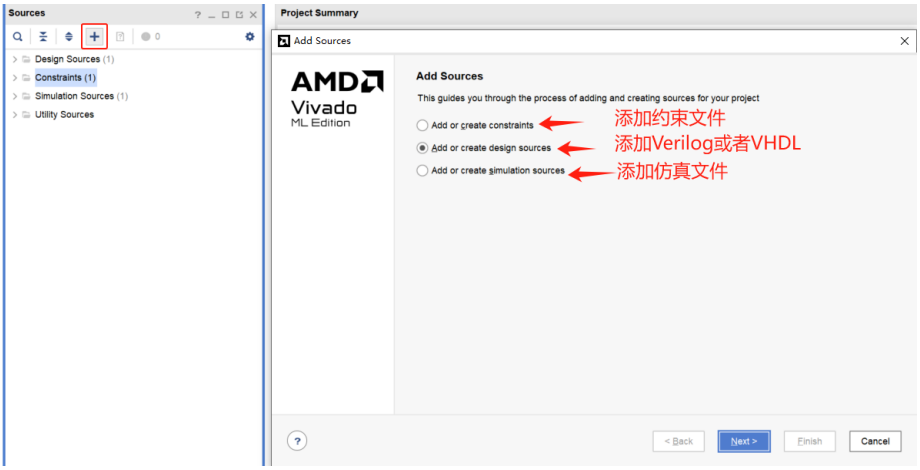
选择芯片的型号和封装速度等级，我们选择 xcku5p-ffvb676-2-i，如下本例程以 xcku5p-ffvb676-2-i 为例

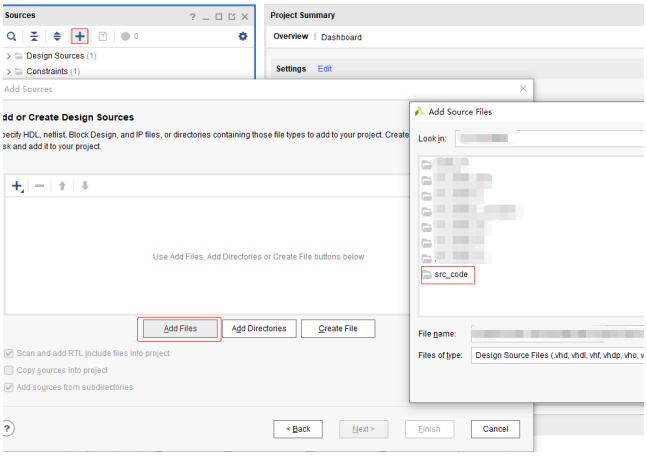


单击 Finish 完成工程创建

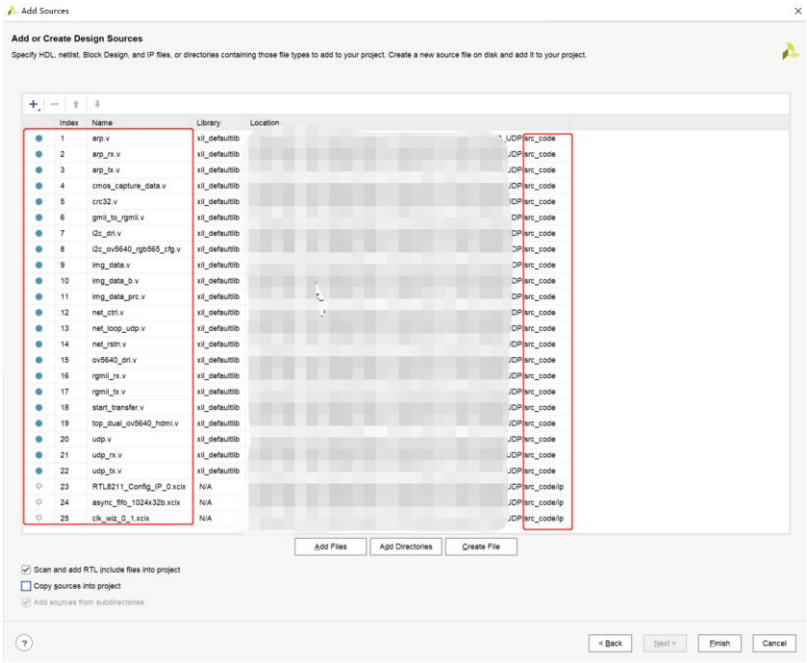


添加代码，显示界面如下

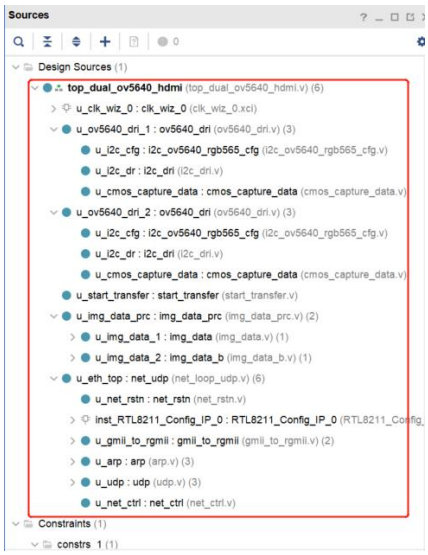




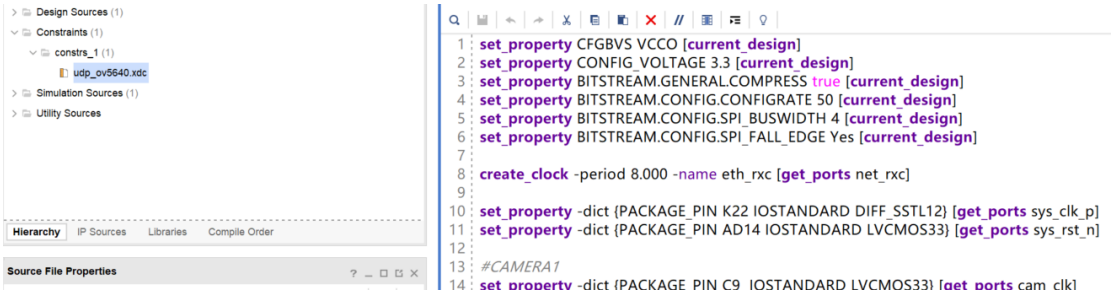
添加代码和 IP 文件（代码文件在工程路径下）



添加完成后，可以看到 Design Sources 文件夹中有相关文件（具体代码，可以点开代码文件查看）



添加管脚约束文件

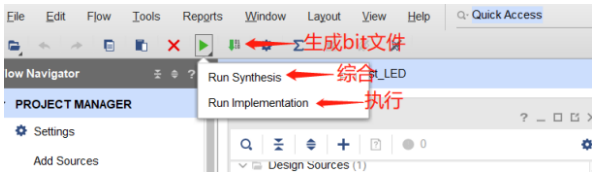


编译并且产生 bit 文件

1:单击综合

2:单击执行

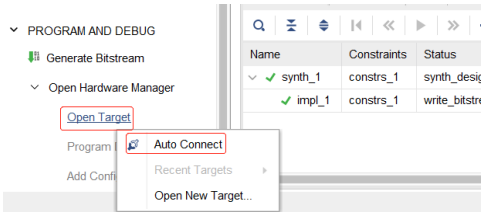
3:单击产生 bit



下载程序

1：给开发板通电，并且连接下载器

2：单击 OpenTarget 然后单击 Auto Connect



连接成功如下：



单击 Program Device（也可以从顶部单击 Program device）弹出的对话框中有我们要下载的 Bit 文件



3.23.4 运行结果

下载完成后，网口 LAN 接网线到电脑端（注意：上位机端需要是千兆网口）选择对应的摄像头，然后点击打开按钮，效果如下图



